CSE433 · DEEP LEARNING · EMERGENCY REFERENCE

One night, seven lectures. Here's how you get through it.

For the midterm on **Monday 9 November, 13:30** · Official scope: CMU 11-785 L1 (p75–end), L2 (p1–136), L3, L4 (p1–47), L6 (p36–45, p81–end), L8 (p1–46), and Stanford CS231n Lecture 4 (backprop) in place of CMU L5 · written for someone starting from zero.

Can you finish all of this before 13:30 tomorrow?

Yes — if you stop trying to "watch the lectures". The seven decks are 1,000+ slides and roughly 10 hours of video. That road ends at 4 a.m. with half of Lecture 5 and nothing retained. This document is the other road: every idea the exam can test, explained from scratch, in the order that builds on itself, with the hand-calculations marked so you know exactly when to pick up the pen.

With the official scope it adds up to about **11½ hours of study**, plus **5½ hours of sleep** that are part of the plan, not a reward at the end of it. Sleep is when your brain files what you learned tonight; an all-nighter before a maths exam costs more marks than it buys.

Two choices shape the plan. It doesn't skip the lectures that look "conceptual" (they carry easy marks and class polls), and it puts backpropagation (the hardest maths) first thing in the morning, when you're fresh, right after a real night's sleep. Everything outside the official exam scope has been cut.

A suggested plan for the night before (exam Monday 13:30)

When	What	Tier
18:15–19:20	Lecture 1 (p75→end) — the perceptron as a logic gate, XOR, decision regions	B
19:20–21:30	Lecture 2 + Sheet 1 — what networks can represent (heavy on pen & paper)	A
21:30–22:00	*Dinner, prayer, walk away from the screen*	
22:00–00:15	Lecture 3 + Assignment 3 — how a network learns: perceptron, ADALINE, risk	A
00:15–01:25	Lecture 4 (p1–47) — derivatives, gradients, Hessian, gradient descent	A
01:25–07:00	**Sleep (5½ h).** Don't trade this for more cards. If you wake for Fajr, go back to sleep after.	
07:00–07:15	*Breakfast while skimming your own notes*	
07:15–09:10	Lecture 5 = Stanford CS231n L4 — backprop on computational graphs, by hand	A
09:10–09:15	*Break*	
09:15–10:45	Lecture 6 (p36–45, p81→end) — convergence, Newton, RProp, QuickProp, momentum	A
10:45–10:50	*Break*	
10:50–11:50	Lecture 8 (p1–46) — L2 vs KL, covariate shift, batch normalisation	A
11:50–12:30	Formula sheet from memory, then the 🔁 drills and ⚠️ trap boxes	A
12:30–13:15	*Eat something real. Walk to the exam. No new material after 12:30.*	

Running late? Don't push sleep back. Skip the ⑧ "Try alone" extras first, then Tier C cards, then Tier B. Each lecture opens with an **"Only 20 minutes?"** line telling you which cards to keep.

How to read this document

📖 Read & understand
No pen. Read it slowly once; say the idea back to yourself in one sentence.

✏️ Pen & paper
Close the screen, solve it on paper, then check the worked solution at the end of the lecture. This is where the marks are.

🔁 Recall drill
Class polls and quick questions. CMU's quizzes reuse the polls. Answer before you look.

🎥 Watch
Optional. Only where a moving picture helps; the exact part is named.

Tier A must do · Tier B should do · Tier C only if you're ahead. Minutes (⏱) are for a tired student, not a fresh one.

Links. Every ↗ Understand more link opens the exact step of the interactive study guide (the one with the labs and guided solves), the CMU video, or a short outside explanation. Click them when a card didn't click; don't click them by default — tonight the PDF is the spine.

🧭 **Analogies** connect each idea to something you already know — Claude, Spotify, Google Maps, or a Nokia 3310. Each one says where it stops being true.

Where solutions are. Pen-and-paper solutions and drill answers sit at the end of each lecture, so you can't see them while you work. The card tells you the solution number.

Contents

1. **Lecture 1** · Where neural networks come from, the perceptron, gates, MLPs and decision regions · 8 cards · ⏱ 62 min
2. **Lecture 2** · What a network of perceptrons can compute: gates, depth, DNF, parity, regions, approximation (+ Sheet 1) · 13 cards · ⏱ 131 min
3. **Lecture 3** · Learning the network: ERM, the perceptron algorithm, ADALINE, MADALINE, smooth activations · 11 cards · ⏱ 137 min
4. **Lecture 4** · Derivatives, gradients, the Hessian and gradient descent · 8 cards · ⏱ 68 min
5. **Lecture 5 · Stanford CS231n L4** · Neural networks and backpropagation on computational graphs · 9 cards · ⏱ 115 min
6. **Lecture 6** · Optimization part 1: does gradient descent get there, and how fast? · 8 cards · ⏱ 92 min
7. **Lecture 8** · Choosing the divergence, and batch normalisation · 7 cards · ⏱ 61 min

LECTURE 1

Where neural networks come from, the perceptron, gates, MLPs and decision regions

Tier B · should (A for perceptron-as-gate) ⏱ 62 min Slides: lec1.intro.pdf (116 p) CMU video Guide tutorial 2

Lecture 1 introduces the one object the whole course is built from: the **perceptron**, a unit that adds up weighted inputs and fires if the total is big enough. It shows what one unit can do (AND, OR, NOT, cut the plane with a straight line) and what it can't do (XOR). Then it shows how stacking units into layers (an MLP) fixes that. Lecture 2 picks up exactly here, so this is your foundation.

What the exam can ask from this lecture

- The two in-scope class polls (units for a hexagon region, sine neurons for cos 2x). They are in card 8.
- Give weights and a threshold for AND, OR, NOT; prove one perceptron can't do XOR; build the 3-unit XOR network.
- Count the units for a polygon region, and say why a non-convex region needs a third layer.
- Threshold vs bias (b = −T), "fires at ≥", linear vs affine, "a perceptron is a linear classifier".
- One step of the perceptron learning rule $w \leftarrow w + \eta(d - y)x$.
- Two names from in-scope slides: Rosenblatt (1958, perceptron + learning rule that converges on separable classes, p75–77), Minsky–Papert (1969, one unit can't do XOR, p80).

⏳ **Only 20 minutes?** Do card 2 (6 min), then card 3 (14 min). If you get 3 more, do the drill in card 8; with 12 more after that, card 5.

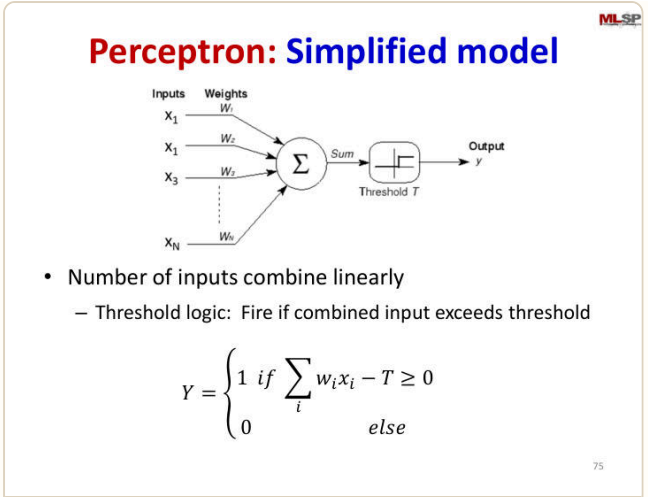
📖 READ & UNDERSTAND · ⏱ 2 min · Tier C

1 · Rosenblatt's perceptron and the 1958 hype (skim)

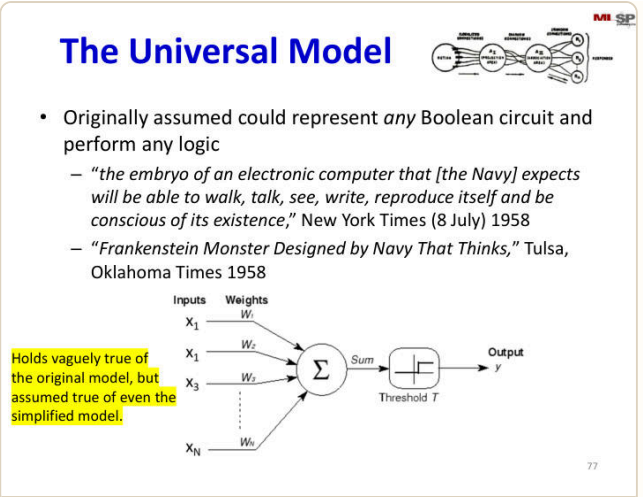
Your scope starts at p75, so the history before it (Bain, Turing, McCulloch–Pitts, Hebb) is **out**. Background, not on the exam: Frank Rosenblatt built the perceptron in 1958, and in his "simplified model" only the last layer of weights learns.

p75 defines the **simplified perceptron**: inputs are combined linearly and the unit fires if the combined input reaches a threshold (card 2 unpacks this). p76 adds a **learning rule** that updates the weights whenever the output is wrong, and Rosenblatt **proved it converges** when the classes can be split by a straight line (card 7). p77 is the hype: the New York Times (1958) called it "the embryo of an electronic computer" that would walk, talk and be conscious.

🧭 **Like the 2023 ChatGPT headlines.** A real but limited breakthrough was sold as a thinking machine. *Where it breaks:* the perceptron's limit was hit almost at once (XOR, card 3); today's models are far more capable.



L1 slide p75



L1 slide p77

↗ Understand more: the simplified perceptron and its learning rule

📖 READ & UNDERSTAND · ⏱ 6 min · Tier A

2 · The perceptron: add up, compare, fire

A perceptron takes N input numbers $x_1, x_2, \ldots, x_N$. Written as one list, $\mathbf{x} = [x_1, \ldots, x_N]$, that is called a **vector**. Each input has its own **weight** w_i, a number saying how much that input counts (negative weights count against firing).

The unit multiplies each input by its weight, adds them up, and **fires (outputs 1) if the total reaches a threshold** T; otherwise it outputs 0 (L1 p85):

$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i - T \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Note the $\geq$: when the sum **equals** T, the unit fires. The slides say "matches or exceeds" (p85–86). Keep this; several exam answers depend on it. E.g. with $w = (0.5, 2)$ and $T = 1$: input $(1, 0)$ gives $0.5 - 1 < 0$, output 0; input $(0, 1)$ gives $2 - 1 = 1 \geq 0$, output 1.

The same unit, written the modern way (p86): move T inside and call $b = -T$ the **bias**. Then

$$z = \sum_i w_i x_i + b, \qquad y = \theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

z is the unit's total input and θ (theta) is the **threshold activation**, the step function that turns z into 0 or 1. An **activation function** is just whatever function turns z into the output; p88 shows that you can swap the step for smooth ones like the sigmoid $\sigma(z) = 1/(1 + e^{-z})$ or ReLU, which later lectures need.

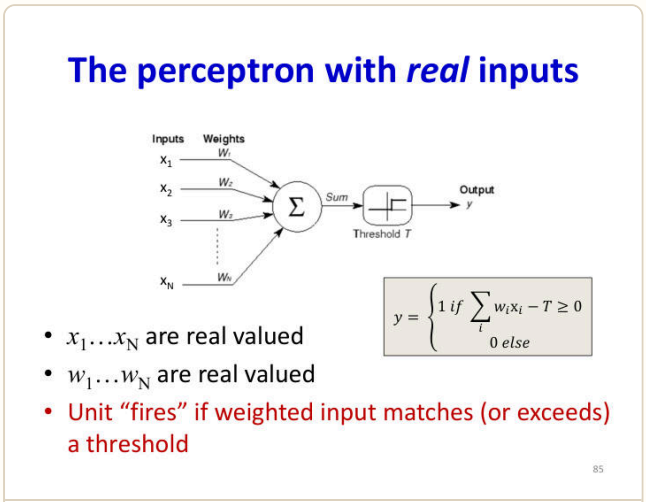
The slide asks "what is the difference between linear and affine?" (p87). **Linear** means $\mathbf{w}^\top \mathbf{x} = \sum_i w_i x_i$: zero in, zero out, no offset. **Affine** means linear **plus a constant**, $\mathbf{w}^\top \mathbf{x} + b$. So z is an affine function of the inputs, because of the bias.

🧭 **Like Celsius to Fahrenheit.** $F = 1.8\,C + 32$ is affine, not linear: 0 °C is 32 °F, not 0 °F. The 1.8 plays the weight, the 32 plays the bias.

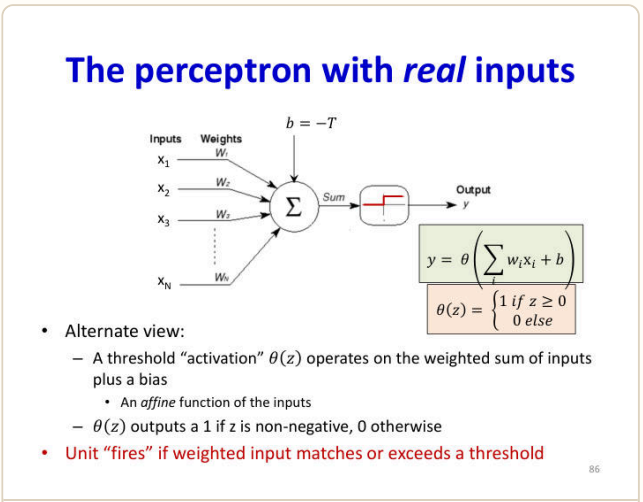
On real-valued inputs a perceptron is a linear classifier (p89). With two inputs, the unit switches exactly on the line $w_1 x_1 + w_2 x_2 = T$. Every point on one side gives 1 and every point on the

other side gives 0. The weight vector $\mathbf{w} = [w_1, w_2]$ points into the side that fires. A **classifier** is anything that sorts inputs into classes; "linear" because its boundary is a straight line (a flat plane, or "hyperplane", in more dimensions).

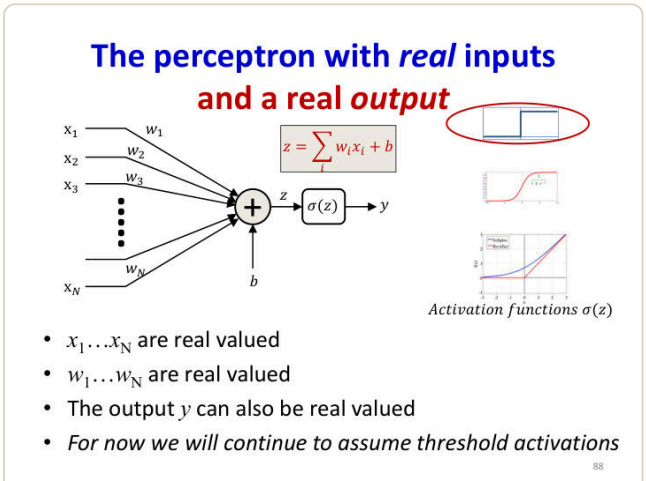
🧭 **Like an old-school spam filter.** Each word has a score ("free" +3, "winner" +4, your boss's name −5); add up the scores of the words in the email and call it spam if the total reaches 6. That is a perceptron: weights, sum, threshold. *Where it breaks:* modern filters output a probability rather than a hard yes/no, which is the smooth-activation version you meet in Lecture 3.



L1 slide p85



L1 slide p86



L1 slide p88

↗ Understand more: threshold T vs bias b ↗ Linear vs affine ↗ A perceptron is a linear classifier ↗ Wikipedia: Perceptron

🖍 **PEN & PAPER · ⏱ 14 min · Tier A**

3 · One perceptron makes AND, OR and NOT, but never XOR

① **WHAT IS BEING ASKED, IN PLAIN WORDS**

A **Boolean** input or output is just 0 (false) or 1 (true). A **gate** is a rule from Boolean inputs to a Boolean output: AND (1 only if all are 1), OR (1 if at least one is 1), NOT (flips 0 ↔ 1), NOR ("neither": 1 only if all are 0), XOR, "exclusive or" (1 if the two inputs **differ**), XNOR (1 if they are **equal**). A perceptron with weights $w_1, w_2, \ldots$ and threshold T fires when $w_1 X + w_2 Y + \cdots \geq T$. The slide (p78) draws gates as circles: numbers on the arrows are weights, the number in the circle is the threshold. The exam asks for

numbers for some gates and a proof that **no** numbers give another. We learn it on a sibling: 3-input AND and OR, the 2-input NOR, and the proof for **XNOR**.

② WARM-UP BY HAND, NO FORMULAS

(sibling). Three inputs X, Y, Z, weights 1, 1, 1, threshold 3. Run a few inputs: 000 → sum 0; 011 → sum 2; 101 → sum 2; 111 → sum 3. Which reach 3? Only 111. That is the **3-input AND**, found by trying. Lower the threshold to 1 and every input with at least one 1 fires: the **3-input OR**.

③ THE ONE IDEA

With all weights 1 the sum **counts how many inputs are on**, and the threshold says how many are needed. A negative weight makes an input count *against* firing. To prove a gate is **impossible**, write the inequality each input row demands, add two of them, and show the result contradicts another row.

④ BUILD IT, STEP BY STEP

(sibling).

1. AND of X, Y, Z: $w = (1, 1, 1)$, $T = 3$.
   Why: the sum reaches 3 only when all three are 1.
2. OR of X, Y, Z: $w = (1, 1, 1)$, $T = 1$.
   Why: one input on is already enough.
3. NOR of X, Y: $w = (-1, -1)$, $T = 0$. 00: $0 \geq 0$ → fires (1). 01 or 10: $-1 \geq 0$ is false → 0. 11: $-2 \rightarrow$ 0.
   Why: the minus signs make every "on" input push against firing; the $\geq$ makes 00 fire.
4. XNOR, suppose some w_1, w_2, T worked. Input 00 must fire: $0 \geq T$, so $T \leq 0$.
   Why: with no inputs on, the sum is 0.
5. Inputs 10 and 01 must not fire: $w_1 < T$ and $w_2 < T$. Add them: $w_1 + w_2 < 2T$.
   Why: adding two true inequalities gives a true one.
6. Since $T \leq 0$, $2T \leq T$, so $w_1 + w_2 < T$: input 11 **does not fire**. But XNOR needs it on.
   Why: contradiction, so no such numbers exist. ■

⑤ CHECK IT

(sibling). NOR with $w = (-1, -1)$, $T = 0$ on all four inputs: outputs 1, 0, 0, 0 ✓. Try a "best effort" XNOR with that NOR unit: it gets 00, 01, 10 right and 11 wrong, exactly the row step 6 says must fail. Edge case: with a strict ">" instead of "≥", NOR at 00 would give $0 > 0$ = false → 0, wrong. The $\geq$ matters.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Problem. (a) Give weights and a threshold for AND, OR and NOT, and check every input. (b) Prove that no single perceptron computes XOR(X, Y).

Do it the same way: steps ④–⑤ with two inputs for AND and OR, one input for NOT, and XOR's four rows for the proof.

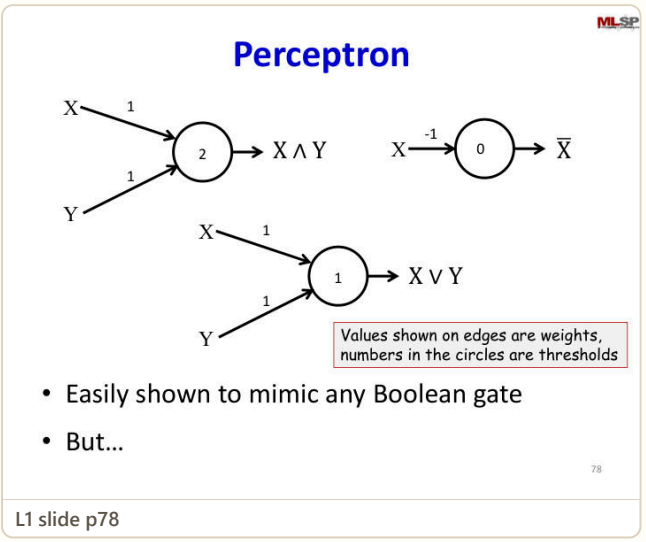
✍ Full worked solution: S1.1 (end of this lecture)

⑦ TRY ALONE

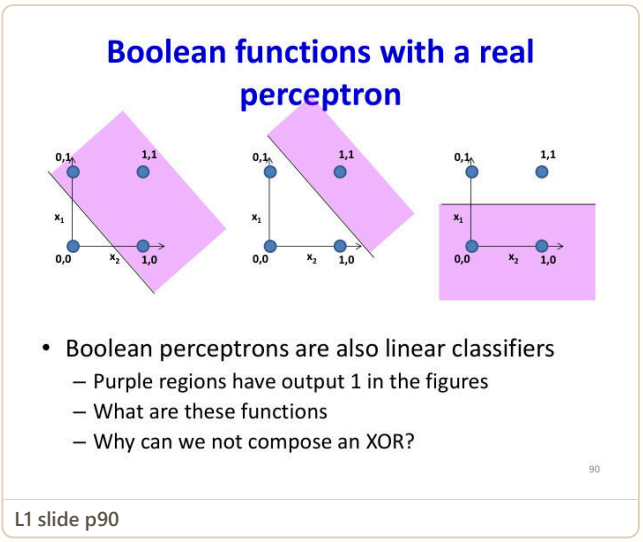
Give weights and a threshold for: (i) NAND ("not both": 0 only on 11); (ii) majority of three ("at least 2 of X, Y, Z"); (iii) "X AND NOT Y" (1 only on X = 1, Y = 0).

✍ Answers: S1.2 (end of this lecture)

🧭 **Like the two light switches on a staircase.** The light is on when the switches are in different positions: that is XOR. You can't wire it as "light on if the switches' combined push is big enough", because both-up pushes hardest yet must be off. *Where it breaks:* real staircase wiring isn't a threshold circuit at all; the point is only that XOR isn't "enough votes".



L1 slide p78



L1 slide p90

↗ Understand more: one perceptron, three gates ↗ Gates as half-planes (p90) ↗ The XOR proof

🖍 PEN & PAPER · ⏱ 11 min · Tier A

4 · The fix: a hidden layer computes XOR

① WHAT IS BEING ASKED, IN PLAIN WORDS

"A single neuron is not enough; networked elements are required" (p80, citing **Minsky and Papert, 1969**, who showed one perceptron can't do XOR). A **multi-layer perceptron (MLP)** feeds the outputs of some perceptrons into others. The units between the inputs and the final output form the **hidden layer**: hidden because you never see their outputs directly. The exam gives you all the weights of a small network; the task is to say what each unit does and run it by hand. We learn it on a sibling network for **XNOR** (1 when X = Y): h_1 has weights $1, 1$ and threshold 2; h_2 has weights $-1, -1$ and threshold 0; the output has weights $1, 1$ from h_1, h_2 and threshold 1. All fire at $\geq$.

② WARM-UP BY HAND, NO FORMULAS

(sibling). Feed in $X = 0, Y = 0$. h_1: $0 + 0 = 0 \geq 2$? No → 0. h_2: $-0 - 0 = 0 \geq 0$? Yes → 1. Output: $0 + 1 = 1 \geq 1$? Yes → **1**. Correct for XNOR (the inputs are equal).

③ THE ONE IDEA

Read each unit on its own as a gate, then read the output as a gate on the hidden units. With weights 1, 1 the sum counts the inputs that are on; with weights $-1, -1$, multiply the inequality by -1 (which flips it) to turn "at least" into "at most".

④ BUILD IT, STEP BY STEP

(sibling).

1. h_1 fires if $X + Y \geq 2$: both inputs on, so $h_1 = X \wedge Y$ (AND).
   Why: the sum counts the inputs that are on.
2. h_2 fires if $-X - Y \geq 0$. Multiply by -1, which flips the inequality: $X + Y \leq 0$, "neither on". So $h_2 = \neg(X \vee Y)$ (NOR).
   Why: negative weights turn "at least" into "at most".
3. The output fires if $h_1 + h_2 \geq 1$, i.e. at least one is on: output $= h_1 \vee h_2$ (OR).
   Why: with weights 1, 1, one hidden unit on reaches 1.
4. Put it together: "both on" OR "both off" = "equal" = XNOR. Then run all four inputs:

X	Y	h_1: X+Y ≥ 2	h_2: −X−Y ≥ 0	out: $h_1 + h_2 \geq 1$
0	0	0 ≥ 2? → 0	0 ≥ 0? → 1	1 ≥ 1? → **1**
0	1	1 ≥ 2? → 0	−1 ≥ 0? → 0	0 ≥ 1? → **0**
1	0	1 ≥ 2? → 0	−1 ≥ 0? → 0	0 ≥ 1? → **0**
1	1	2 ≥ 2? → 1	−2 ≥ 0? → 0	1 ≥ 1? → **1**

Why: the exam's "show the output is ..." means exactly this table, one row per input.

⑤ CHECK IT

(sibling). Outputs 1, 0, 0, 1 for 00, 01, 10, 11: XNOR ✓. Edge case: h_1 and h_2 never fire together (both on and both off can't happen at once), so the output's sum is never 2; any output threshold between 0.5 and 1 would also work.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Problem (the network on L1 p81). Hidden unit h_1: weights 1, 1 from X, Y, threshold 1. Hidden unit h_2: weights $-1, -1$ from X, Y, threshold -1. Output unit: weights 1, 1 from h_1, h_2, threshold 2. All fire at $\geq$. (a) Say in words what each unit computes. (b) Run all four inputs and show the output is XOR.

Do it the same way: steps ④–⑤ with these numbers (watch the $\geq$ on the rows where a sum equals its threshold).

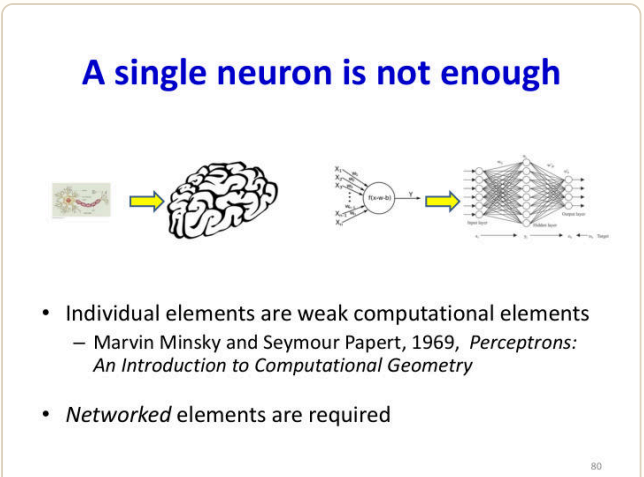
✍ Full worked solution: S1.3 (end of this lecture)

⑦ TRY ALONE

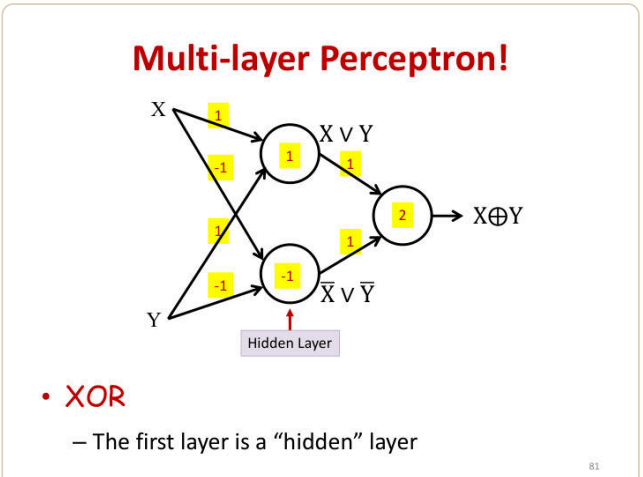
(i) In the exam network, change only the output threshold from 2 to 1. What does the network compute? (ii) Keep the exam's h_1, h_2 but make the output unit weights $-1, -1$ with threshold -1 (a NAND of h_1, h_2). Run all four inputs. (iii) What does the exam's h_2 compute if its threshold is -2 instead of -1?

✍ Answers: S1.4 (end of this lecture)

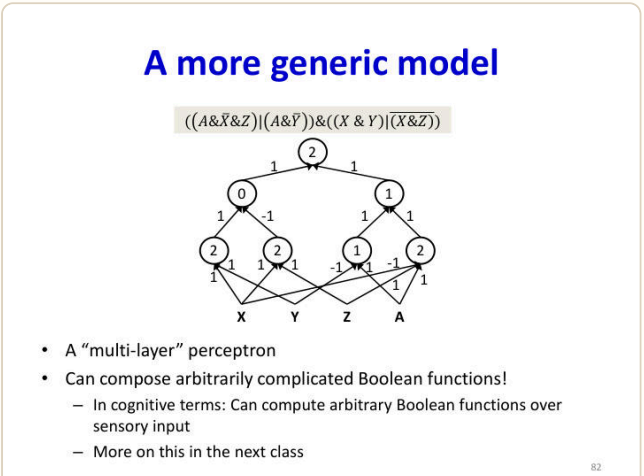
The bigger point (p82–83): by stacking gates, an MLP can compute **any Boolean function** at all ("MLPs are universal Boolean functions"). p82 shows a 3-layer net for a 4-input formula; Lecture 2 shows how to build such nets systematically, so don't study p82 here.



L1 slide p80



L1 slide p81



L1 slide p82

↗ Understand more: the three-perceptron XOR ↗ MLPs are universal Boolean functions (p82) ↗ Try XOR in TensorFlow Playground

🥓 PEN & PAPER · ⏱ 12 min · Tier A

5 · Decision regions: one unit per edge, then AND (and OR)

① WHAT IS BEING ASKED, IN PLAIN WORDS

Now the inputs are real numbers, so each input is a point (x_1, x_2) in the plane. A **decision region** is the area where the network should output 1. From card 2, one perceptron fires on one side of one straight line: a **half-plane**. The exam asks how many perceptrons it takes to fire exactly inside a polygon (a shape with straight edges), what threshold the last one uses, what happens to a point just outside, and what changes when the region is two separate pieces. We learn it on a sibling: the **square** $0 \le x_1 \le 2,\ 0 \le x_2 \le 2$.

② WARM-UP BY HAND, NO FORMULAS

(sibling). The square's four edges give four units: y_1 fires if $x_1 \geq 0$, y_2 if $x_1 \leq 2$, y_3 if $x_2 \geq 0$, y_4 if $x_2 \leq 2$. Test three points and add up the units that fire:

point	y_1	y_2	y_3	y_4	$\sum y_i$	inside?
$(1, 1)$	1	1	1	1	**4**	yes
$(3, 1)$	1	0	1	1	3	no (outside one edge)
$(3, 3)$	1	0	1	0	2	no (beyond a corner)

Only the inside point reaches 4, the number of edges.

③ THE ONE IDEA

A shape is **convex** if the straight segment between any two of its points stays inside it (no dents, no holes, one piece). For a convex polygon, **inside = on the inner side of every edge**, so one output unit that needs all edge units on (an AND: weights 1, threshold = number of edges) does the job. Several separate pieces: one AND per piece, then an OR on top.

④ BUILD IT, STEP BY STEP

(sibling).

1. One perceptron per edge, each firing on the square's side of its edge: 4 edges → 4 units. E.g. "$x_1 \leq 2$" is weights $(-1, 0)$, threshold -2.
   Why: each unit can only draw one straight line.
2. Output unit: all weights 1, threshold 4, i.e. fire iff $\sum y_i \geq 4$.
   Why: the count reaches 4 only if every edge agrees (p91–97).
3. A point outside exactly one edge: three edge units fire, $\sum y_i = 3 < 4$, no fire.
   Why: the AND needs all of them; one "no" is enough to reject.
4. Total for one convex polygon: $4 + 1 = 5$ units, in 2 layers.
   Why: 4 edge units plus the AND.
5. Two separate squares: layer 1 = $4 + 4 = 8$ edge units; layer 2 = 2 AND units (threshold 4 each), one per square; layer 3 = 1 OR (weights 1, threshold 1). Total $8 + 2 + 1 = 11$ units in **3 layers**.
   Why: a point is in the region iff it is in at least one piece (p98), and that OR is a third layer of units.

⑤ CHECK IT

(sibling). Output threshold 4: $(1, 1) \to 4 \geq 4$, fires ✓; $(3, 1) \to 3 \geq 4$? No ✓. Edge case $(2, 1)$, exactly on the right edge: $2 \leq 2$ holds, so $y_2 = 1$, the sum is 4 and it fires: points on the boundary count as inside, because of the $\geq$.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Problem. (a) Poll 3 (p102): how many threshold perceptrons does an MLP need for a hexagonal region in 2-D? What threshold does the output unit use? (b) A point lies outside exactly one edge of that hexagon. What is $\sum y_i$, and does the network fire? (c) p98: the region

is the union of two separate pentagons. How many units, and how many layers of units? (d) Same question for two separate triangles.

Do it the same way: steps ④–⑤ with these numbers (count the edges first).

✍ Full worked solution: S1.5 (end of this lecture)

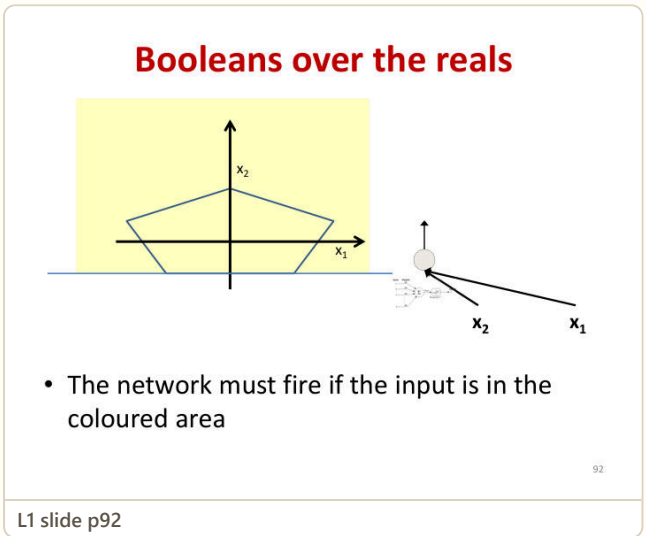
⑦ **TRY ALONE**

(i) An octagon region: how many units, and what output threshold? (ii) The union of a separate triangle and pentagon: how many units, how many layers? (iii) The pentagon's output threshold is lowered from 5 to 4. Which extra points now fire?

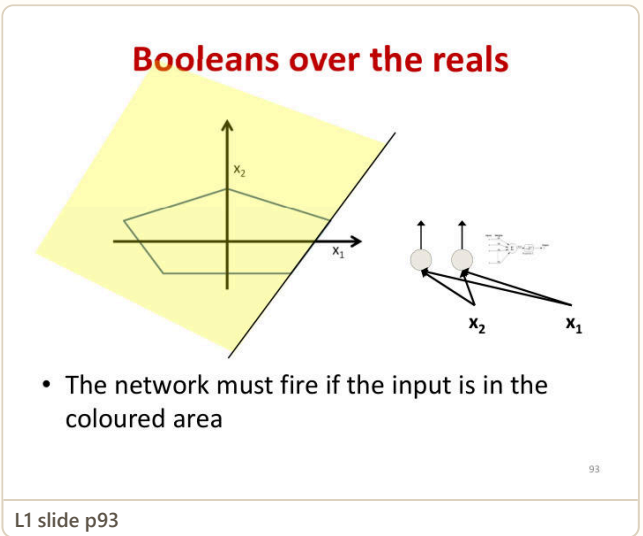
✍ Answers: S1.6 (end of this lecture)

🧭 **Like a delivery app checking if your address is in its zone.** For a convex zone it checks each border street: "is this address on the inner side?" Deliver only if every check says yes (the AND). If the company serves two separate neighbourhoods, it checks each zone that way and delivers if **either** zone says yes (the OR on top). *Where it breaks:* a real app uses curved map boundaries; the network approximates a curve with many short straight edges, so more edges means more units.

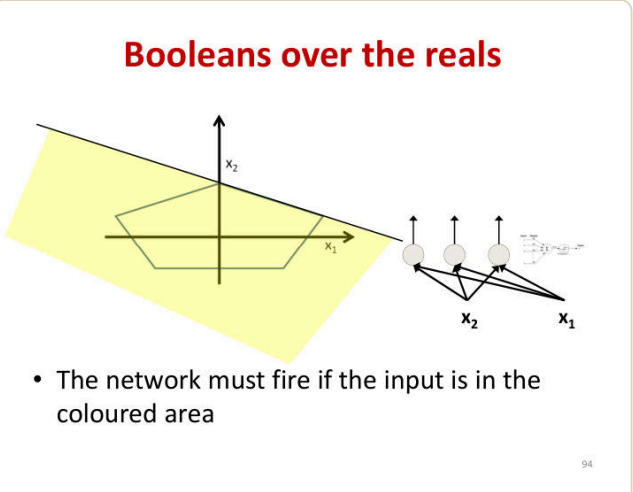
The punchline (p100–101): with enough units, an MLP can draw **any decision boundary** to any precision, even in 784 dimensions (a 28 × 28 MNIST digit image is 784 pixel values). MLPs are **universal classifiers**.



L1 slide p92



L1 slide p93



L1 slide p94

↗ Understand more: building the pentagon ↗ The AND on top (Σy ≥ 5) ↗ Non-convex shapes: OR of ANDs

📖 **READ & UNDERSTAND · ⏱ 4 min · Tier B**

6 · Real-valued outputs: a pulse from two units, any curve from many pulses

So far outputs are 0 or 1. Can a network output a real number, like a curve $f(x)$? Yes (p104–106). Take one input x and two threshold units: one fires when $x \geq T_1$, the other when $x \geq T_2$ (with $T_1 < T_2$). Feed them into a **summing** output unit (no threshold, it just adds) with weights $+1$ and -1:

$$f(x) = \theta(x - T_1) - \theta(x - T_2)$$

Check with $T_1 = 2$, $T_2 = 5$. At $x = 1$: $0 - 0 = 0$. At $x = 3$: $1 - 0 = 1$. At $x = 6$: $1 - 1 = 0$. So $f = 1$ **only between T_1 and T_2**: a square pulse, from 3 units (2 threshold + 1 summing output).

Now make many narrow pulses side by side, multiply pulse k by a height h_k (the curve's value there), and add them all up. You get a staircase that hugs any curve; narrower pulses hug it more tightly. That is why an MLP can approximate **any function of one input to any precision**, using $2n$ threshold units plus 1 summing unit for n pulses. Lecture 2 extends this to many inputs.

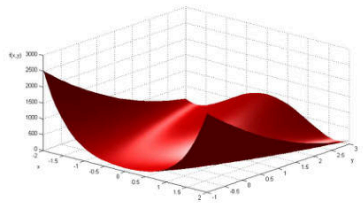
🧭 **Like drawing a curve on a Nokia 3310 screen.** The screen can only light rectangular pixels, yet a curve made of enough small pixels looks smooth. Each pulse is one column of pixels; more, thinner pulses = higher resolution. *Where it breaks:* the network's "pixels" can have any height (and be negative), not just on/off.

Poll 4 (p107–108) is a trick question: how many neurons with activation $y = \sin(z)$ does it take to model $y = \cos(2x)$ exactly? Since

$$\cos(2x) = \sin\left(2x + \tfrac{\pi}{2}\right)$$

a **single** sine neuron with weight 2 and bias $\pi/2$ does it. "1" is not an option, so the answer is **none of the above**. Lesson: the activation function matters; the pulse recipe is for threshold units, and a well-matched activation can need far fewer.

But what about continuous valued *outputs?*

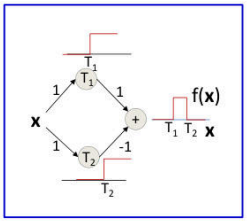


- Inputs may be real-valued
- Can outputs be continuous-valued too?

104

L1 slide p104

MLP as a continuous-valued regression

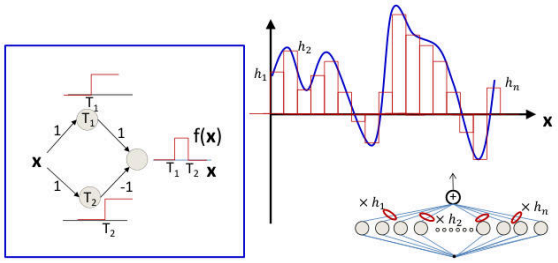


- A simple 3-unit MLP with a "summing" output unit can generate a "square pulse" over an input
  - Output is 1 only if the input lies between T_1 and T_2
  - T_1 and T_2 can be arbitrarily specified

105

L1 slide p105

MLP as a continuous-valued regression



- A simple 3-unit MLP can generate a "square pulse" over an input
- **An MLP with many units can model an arbitrary function over an input**
  - To arbitrary precision
    - Simply make the individual pulses narrower
- This generalizes to functions of any number of inputs (next class)

106

L1 slide p106

↗ Understand more: a square pulse from three units ↗ Sums of pulses: any 1-D function ↗ d2l.ai: Multilayer perceptrons

🖍 PEN & PAPER · ⏱ 10 min · Tier B

7 · The perceptron learning rule, by hand

① WHAT IS BEING ASKED, IN PLAIN WORDS

So far we chose the weights ourselves. Rosenblatt's rule (L1 p76) finds them from examples: show the unit an input, compare its output y with the desired output d (the right answer), and if it's wrong, nudge the weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(d - y)\,\mathbf{x}$$

"$\leftarrow$" means "replace by". η (eta, the **learning rate**) is a small positive step size. $d - y$ is the **error**: 0 if correct, $+1$ for a missed positive ($d = 1$, $y = 0$), -1 for a false alarm ($d = 0$, $y = 1$). To learn the bias too, append a constant input 1: $\tilde{\mathbf{x}} = [x_1, x_2, 1]$ and $\tilde{\mathbf{w}} = [w_1, w_2, b]$, so $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = w_1 x_1 + w_2 x_2 + b$ (a **dot product**: multiply matching entries and add). The exam asks you to run this rule by hand on a gate until it stops making mistakes. We learn it on the sibling: **train NOR** ($d = 1, 0, 0, 0$ for (0,0), (0,1), (1,0), (1,1)), $\eta = 1$, start $\tilde{\mathbf{w}} = [0, 0, 0]$.

🧭 **Like tuning an old radio dial.** Every time the station comes in wrong, nudge the dial a fixed step in the direction that fixes it; once it's clear, stop touching it. *Where it breaks:* here one nudge can fix one

input and break another, so you keep cycling through all inputs until a full pass has no mistakes.

② WARM-UP BY HAND, ONE UPDATE

(sibling). Show input $(0, 0)$, $d = 1$: $\tilde{\mathbf{x}} = [0, 0, 1]$, $z = 0$, and since $0 \geq 0$ the unit fires, $y = 1 = d$: correct, no change. Next, $(0, 1)$, $d = 0$: $\tilde{\mathbf{x}} = [0, 1, 1]$, $z = 0$, fires, $y = 1$. Wrong (a false alarm), $d - y = -1$. New weights: $[0, 0, 0] + 1 \cdot (-1) \cdot [0, 1, 1] = [0, -1, -1]$.

③ THE ONE IDEA

Each correction pushes z for that input the right way: adding $\tilde{\mathbf{x}}$ to the weights raises this input's z by $\tilde{\mathbf{x}}^\top \tilde{\mathbf{x}}$ (the sum of its squared entries, always > 0 because of the constant 1); subtracting lowers it by the same amount. Inputs that are 0 get no change in their weight: they didn't cause the mistake.

④ BUILD IT, STEP BY STEP

(sibling, NOR).

1. Fix the order: (0,0), (0,1), (1,0), (1,1), then repeat. One run through all four is a **pass**.
   Why: the rule looks at one example at a time.
2. For each input, compute z with the **current** weights and $y = 1$ if $z \geq 0$.
   Why: updates happen one by one, so later rows see earlier changes.
3. If $y = d$, do nothing. Otherwise add $(d - y)\,\tilde{\mathbf{x}}$.
   Why: that is the rule with $\eta = 1$.
4. Keep a table, one row per input shown (here, rows with no error in passes 2–3 left out):

pass	x	$\tilde{\mathbf{w}}$ before	z	y	d	d – y	$\tilde{\mathbf{w}}$ after
1	(0,0)	[0, 0, 0]	0	1	1	0	[0, 0, 0]
1	(0,1)	[0, 0, 0]	0	1	0	−1	**[0, −1, −1]**
1	(1,0)	[0, −1, −1]	−1	0	0	0	[0, −1, −1]
1	(1,1)	[0, −1, −1]	−2	0	0	0	[0, −1, −1]
2	(0,0)	[0, −1, −1]	−1	0	1	+1	**[0, −1, 0]**
2	(1,0)	[0, −1, 0]	0	1	0	−1	**[−1, −1, −1]**
3	(0,0)	[−1, −1, −1]	−1	0	1	+1	**[−1, −1, 0]**

   Why: the "before" column of each row is the "after" of the row above it.
5. Stop after a pass with no mistakes. Pass 4 with $[-1, -1, 0]$: $z = 0, -1, -1, -2$, so $y = 1, 0, 0, 0$. All correct; stop.
   Why: nothing will ever change again.

⑤ CHECK IT

(sibling). The final $\tilde{\mathbf{w}} = [-1, -1, 0]$ means weights $-1, -1$ and bias 0, i.e. threshold $T = 0$: exactly the NOR gate from card 3 ✓, found after 4 updates. Edge case, a correct answer: with $\tilde{\mathbf{w}} = [0, 1, 0]$ and input $(1, 1)$, $d = 1$: $z = 1$, $y = 1$, error 0, weights unchanged.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Problem. Train on OR: inputs (0,0), (0,1), (1,0), (1,1) with $d = 0, 1, 1, 1$, visited in that order, repeatedly. $\eta = 1$, start $\tilde{\mathbf{w}} = [0, 0, 0]$, fire if $z \geq 0$. Do pass 1 by hand, then find the final weights.

Do it the same way: steps ④–⑤ with these targets (careful with the very first row).

✍ Full worked solution: S1.7 (end of this lecture)

⑦ TRY ALONE

(i) $\tilde{\mathbf{w}} = [1, 0, 0]$, input $(0, 1)$, $d = 1$: update? (ii) Training AND with $\tilde{\mathbf{w}} = [1, 1, -1]$, input $(1, 0)$, $d = 0$: update? (iii) $\eta = 0.5$, $\tilde{\mathbf{w}} = [0, 0, 0]$, input $(0, 0)$, $d = 0$: new weights?

✍ Answers: S1.8 (end of this lecture)

Rosenblatt proved this rule always stops if the classes are **linearly separable** (one line can split them). On XOR it never stops. Lecture 3 proves it.

↗ Understand more: the simplified perceptron and its rule ↗ Practice: the rule on AND

🔁 RECALL DRILL · ⏱ 3 min · Tier A

8 · Recall drill: the two in-scope polls and quick checks

Poll 3 (p102–103). How many threshold perceptrons does an MLP need for a hexagonal region in 2-D? 6 · 7 · 12 · 13

✍ Answer: S1.9 (end of this lecture)

Poll 4 (p107–108). How many $\sin(z)$ neurons model $y = \cos(2x)$ exactly? 3 · floor($\pi/2$) or ceil($\pi/2$) · infinite · none of the above

✍ Answer: S1.10 (end of this lecture)

Quick checks

1. Perceptron with weights $(2, 1)$, threshold 3, input $(1, 1)$. Output?
2. Is $z = 2x_1 - x_2 + 4$ linear or affine?
3. $\eta = 1$, $\tilde{\mathbf{x}} = [1, 0, 1]$, $d = 0$, $y = 1$. What does the rule add to $\tilde{\mathbf{w}}$?
4. Who showed (1969) that a single perceptron can't compute XOR?

✍ Answers: S1.11 (end of this lecture)

⚠ Traps

- **z = 0 means fire.** The rule is $\geq$ ("matches or exceeds", p85–86), even though p75's bullet says "exceeds". NOT with threshold 0 only works because of this, and so does the first update in card 7.
- **Poll 3: don't forget the output unit.** N edges → N + 1 perceptrons (6 is the trap). A union of convex regions needs a third layer (OR of ANDs).
- **Slide error, p75 and p77:** the perceptron's inputs are labelled $x_1, x_1, x_3, \ldots$. The second one is x_2 (drawn correctly on p85).

- **Slide error, p90:** the axis labels (vertical x_1) disagree with the corner labels ("1,0" sits bottom-right, which needs x_1 horizontal). Either way, panel 3 is NOT of one variable, and panels 1–2 are OR and AND.
- **Slide typo, p83:** "McCullough and Pitt" → **McCulloch and Pitts**. Spell it right in a written answer.
- **The p76 rule only trains one unit's weights**; it has no way to train hidden weights. That is why XOR needed hand-built hidden units here, and backpropagation later.

Solutions and answers — Lecture 1

S1.1 · 3 · One perceptron makes AND, OR and NOT, but never XOR — Full worked solution

(a) Step 1, AND and OR (the slide's gates, L1 p78). With weights 1, 1 the sum X + Y counts the inputs that are on; compare it with the threshold, row by row:

X	Y	sum X + Y	AND: w = 1, 1, T = 2	OR: w = 1, 1, T = 1
0	0	0	0 ≥ 2? → 0	0 ≥ 1? → 0
0	1	1	1 ≥ 2? → 0	1 ≥ 1? → 1
1	0	1	1 ≥ 2? → 0	1 ≥ 1? → 1
1	1	2	2 ≥ 2? → 1	2 ≥ 1? → 1

Why: AND needs both on (count 2), OR needs at least one (count 1).

Step 2, NOT X: weight -1, threshold 0. For $X = 0$: $0 \geq 0$, fires, output 1. For $X = 1$: $-1 \geq 0$ is false, output 0.

Why: the minus sign flips the input. (This one only works because of the $\geq$: with a strict $>$, $X = 0$ would give 0.)

Same unit, same wiring: changing the threshold from 2 to 1 turns AND into OR. The "program" lives in the numbers: what a network computes is stored in its weights.

(b) Step 3, suppose some weights w_1, w_2 and threshold T did compute XOR. Write down what each of the four inputs demands:

$$(0,0) \to 0: \quad 0 < T$$
$$(1,0) \to 1: \quad w_1 \geq T$$
$$(0,1) \to 1: \quad w_2 \geq T$$
$$(1,1) \to 0: \quad w_1 + w_2 < T$$

Step 4, add the middle two lines (two true inequalities add to a true one):

$$w_1 + w_2 \geq 2T$$

Step 5, use the first line. It says $T > 0$, so $2T > T$, and therefore

$$w_1 + w_2 \geq 2T > T$$

That contradicts the last line ($w_1 + w_2 < T$): input 11 would fire. No choice of numbers works. ■

AND: $w = (1,1)$, $T = 2$. OR: $w = (1,1)$, $T = 1$. NOT: $w = -1$, $T = 0$. XOR: impossible for one unit, because the four inequalities contradict each other.

The picture behind (b) (p90): one perceptron's boundary is a single straight line across the unit square. OR's line cuts off only (0,0); AND's line cuts off only (1,1); a horizontal line gives NOT of one input. XOR needs (0,1) and (1,0) on one side and (0,0), (1,1) on the other: two diagonal pairs, and no single straight line separates them.

↩ back to the card

S1.2 · 3 · One perceptron makes AND, OR and NOT, but never XOR — Answers

(i) $w = (-1, -1)$, $T = -1$: sums $0, -1, -1, -2 \to$ outputs $1, 1, 1, 0$ ✓. (ii) $w = (1, 1, 1)$, $T = 2$: fires on 011, 101, 110, 111 ✓. (iii) $w = (1, -1)$, $T = 1$: sums for 00, 01, 10, 11 are $0, -1, 1, 0 \to$ only 10 fires ✓.

↩ back to the card

S1.3 · 4 · The fix: a hidden layer computes XOR — Full worked solution

(a) Step 1, h_1. It fires if $X + Y \geq 1$: at least one input is on, so $h_1 = X \vee Y$ (OR).
Why: with weights 1, 1 the sum counts the inputs that are on.

Step 2, h_2. It fires if $-X - Y \geq -1$. Multiply both sides by -1 (which flips the inequality):

$$-X - Y \geq -1 \iff X + Y \leq 1$$

"not both on". So $h_2 = \neg(X \wedge Y)$, which the slide writes $\bar{X} \vee \bar{Y}$ (NAND).
Why: negative weights turn "at least" into "at most".

Step 3, output. It fires if $h_1 + h_2 \geq 2$, i.e. both are on: output $= h_1 \wedge h_2$ (AND). Together: "at least one on" AND "not both on" = exactly one on = XOR.

(b) Step 4, one row per input:

X	Y	h_1: X+Y ≥ 1	h_2: −X−Y ≥ −1	out: $h_1 + h_2 \geq 2$
0	0	0 ≥ 1? → 0	0 ≥ −1? → 1	1 ≥ 2? → **0**
0	1	1 ≥ 1? → 1	−1 ≥ −1? → 1	2 ≥ 2? → **1**
1	0	1 ≥ 1? → 1	−1 ≥ −1? → 1	2 ≥ 2? → **1**
1	1	2 ≥ 1? → 1	−2 ≥ −1? → 0	1 ≥ 2? → **0**

Rows 01 and 10 fire only because of the $\geq$ (sums equal to the thresholds 1, -1 and 2).

$h_1 = X \vee Y$, $h_2 = \neg(X \wedge Y)$, output $= h_1 \wedge h_2$. Outputs 0, 1, 1, 0 for inputs 00, 01, 10, 11: exactly XOR, with 3 perceptrons (2 hidden + 1 output).

Geometrically: h_1's line cuts off (0,0), h_2's line cuts off (1,1), and the AND keeps the band between the two lines, which holds exactly (0,1) and (1,0).

🧭 **Like two security guards and a manager.** Guard 1 lets you in if at least one of your two badges is valid; guard 2 refuses anyone with both (they're suspicious). The manager approves only if both guards said yes. Each guard draws one simple line; together they carve out "exactly one badge". *Where it breaks:* nothing much; that is literally the network.

↩ back to the card

S1.4 · 4 · The fix: a hidden layer computes XOR — Answers

(i) The output becomes $h_1 \vee h_2$, and on every input at least one of them is on: **always 1**. (ii) (h_1, h_2) = (0,1), (1,1), (1,1), (1,0) → sums $-1, -2, -2, -1$ → outputs $1, 0, 0, 1$: **XNOR** (1 when the inputs are equal). (iii) $-X - Y \geq -2$ holds for every input (the smallest sum is -2): h_2 is **always 1**, and the network becomes plain OR.

↩ back to the card

S1.5 · 5 · Decision regions: one unit per edge, then AND (and OR) — Full worked solution

(a) Steps 1–2. A hexagon has 6 edges → 6 edge units, each firing on the inner side of its edge. Output: an AND with all weights 1 and **threshold 6** (fire iff $\sum y_i \geq 6$). **Step 4:** total $6 + 1 = 7$ units.
Why: inside = inner side of all six edges; don't forget the output unit (6 is the trap answer).

(b) Step 3. Outside exactly one edge: five of the six edge units fire, so $\sum y_i = 5$. Since $5 < 6$, the output does not fire. Correct: the point is outside. (p97 shows the same pattern for the pentagon: 5 inside, 4 outside one edge.)

(c) Step 5. Layer 1: $5 + 5 = 10$ edge units. Layer 2: 2 AND units (threshold 5 each), one per pentagon. Layer 3: 1 OR unit (weights 1, threshold 1). Total $10 + 2 + 1 = 13$, in 3 layers.
Why: two separate pieces are not convex together, so one AND can't do it; OR the two ANDs.

(d) Step 5 again. $3 + 3 = 6$ edge units, 2 AND units (threshold 3 each), 1 OR unit: $6 + 2 + 1 = 9$, in 3 layers.

(a) 7 units; output threshold 6 (Poll 3 answer: 7). (b) $\sum y_i = 5 < 6$: no fire. (c) 13 units, 3 layers. (d) 9 units, 3 layers.

↩ back to the card

S1.6 · 5 · Decision regions: one unit per edge, then AND (and OR) — Answers

(i) $8 + 1 = 9$ units, threshold 8. (ii) $3 + 5 = 8$ edge units, 2 ANDs (thresholds 3 and 5), 1 OR: $8 + 2 + 1 = 11$ units in 3 layers. (iii) Points just outside one edge (count 4) also fire, so the region gains a triangle-shaped tip beyond each of the five edges: a five-pointed **star**. Points beyond a corner (count 3) still don't.

↩ back to the card

S1.7 · 7 · The perceptron learning rule, by hand — Full worked solution

Steps 1–3. Order (0,0), (0,1), (1,0), (1,1); each row uses the weights *before* the update; $y = 1$ iff $z \geq 0$; only rows with an error change the weights, by $(d - y)\,\tilde{\mathbf{x}}$ with $\tilde{\mathbf{x}} = [x_1, x_2, 1]$.

Step 4, the table. Pass 1 in full, then only the rows with an error:

pass	x	w̃ before	z	y	d	d – y	w̃ after
1	(0,0)	[0, 0, 0]	0	1	0	−1	**[0, 0, −1]**
1	(0,1)	[0, 0, −1]	−1	0	1	+1	**[0, 1, 0]**
1	(1,0)	[0, 1, 0]	0	1	1	0	[0, 1, 0]
1	(1,1)	[0, 1, 0]	1	1	1	0	[0, 1, 0]
2	(0,0)	[0, 1, 0]	0	1	0	−1	**[0, 1, −1]**
2	(1,0)	[0, 1, −1]	−1	0	1	+1	**[1, 1, 0]**
3	(0,0)	[1, 1, 0]	0	1	0	−1	**[1, 1, −1]**

Row 1 is the classic trap: from all-zero weights $z = 0$, and with $\geq$ that means **fire**, so (0,0) is wrong straight away. The rows left out are correct: in pass 2, (0,1) with $[0, 1, -1]$ gives $z = 0$ (fires, $d = 1$) and (1,1) with $[1, 1, 0]$ gives $z = 2$; in pass 3, (0,1), (1,0), (1,1) with $[1, 1, -1]$ give $z = 0, 0, 1$.

Step 5, stop. Pass 4 with $[1, 1, -1]$: $z = -1, 0, 0, 1$, so $y = 0, 1, 1, 1$. All correct; stop.

$\tilde{\mathbf{w}} = [1, 1, -1]$ after 5 updates (pass 4 is clean): weights 1, 1 and bias -1, i.e. threshold $T = 1$. That is exactly the slide's OR gate.

↩ back to the card

S1.8 · 7 · The perceptron learning rule, by hand — Answers

(i) $z = 0 + 0 + 0 = 0 \to y = 1 = d$: no change. (ii) $z = 1 + 0 - 1 = 0 \to y = 1$, but $d = 0$: false alarm, $\tilde{\mathbf{w}} \leftarrow [1, 1, -1] - [1, 0, 1] = [0, 1, -2]$. (iii) $z = 0 \to y = 1$, error -1: $\tilde{\mathbf{w}} \leftarrow [0, 0, 0] - 0.5\,[0, 0, 1] = [0, 0, -0.5]$.

↩ back to the card

S1.9 · 8 · Recall drill: the two in-scope polls and quick checks — Answer

7: 6 edge units + 1 AND output.

↩ back to the card

S1.10 · 8 · Recall drill: the two in-scope polls and quick checks — Answer

None of the above: one neuron, $\sin(2x + \pi/2) = \cos(2x)$.

↩ back to the card

S1.11 · 8 · Recall drill: the two in-scope polls and quick checks — Answers

1. $2 + 1 = 3 \geq 3$: fires, **1**.
2. **Affine** (the $+4$ is an offset).
3. $1 \cdot (0 - 1) \cdot [1, 0, 1] = -[1, 0, 1]$: a false alarm lowers the weights on the active inputs.
4. **Minsky and Papert**, *Perceptrons*.

↩ back to the card

LECTURE 2

What a network of perceptrons can compute: gates, depth, DNF, parity, regions, approximation (+ Sheet 1)

Tier A · must ⏱ 131 min Slides: lec2.universal.pdf (156 p) 🎥 CMU video 📘 Guide tutorials 2–6

Lecture 1 showed one perceptron can be an AND, OR or NOT gate. Lecture 2 asks: **if we wire many of them together, what can the network compute?** The answer is "anything" (any Boolean function, any classification region, any real function, approximately), but the **size** can explode unless the network is **deep**. Sheet 1 is built entirely on this lecture, so this is where the exam's pen-and-paper marks live.

What the exam can ask from this lecture

- Give the weights and bias of a single perceptron for an AND / OR / "at least K of N literals" gate, and prove it works (Sheet 1 P1).
- Prove XOR is impossible for one perceptron using the four inequalities, then build and check the 2-hidden-unit XOR net (Sheet 1 P2).
- Truth table → DNF → K-map reduction → one-hidden-layer MLP with explicit weights (Sheet 1 P4).
- Count units: parity with one hidden layer versus a tree of XOR modules (modules, stages, growth) (Sheet 1 P3, Poll 2).
- Build a classifier for a non-convex region from half-planes, AND units and an OR unit (Sheet 1 P5).
- Staircase approximation of x^2: coefficients, the error bound, the K for a given error (Sheet 1 P6).
- True/False on depth, width, graded activations, output ranges, "exact vs approximate" (Sheet 1 P7, Polls 3–4; Poll 5 is on a skim slide).
- Skim only (p101–102 and p137 → end): the doctor showed them without explaining. Know the one-line claims in cards 8 and 12.

⏳ **Only 20 minutes?** Do card 3 (XOR proof + net, 16 min), then Polls 2–4 in the drill (card 13, 4 min). Those two cover the questions most likely to appear word for word. With 17 minutes more, add card 7 (parity counts).

📖 READ & UNDERSTAND · ⏱ 4 min · Tier A

1 · The perceptron as a gate: weights, bias, and "fires if z ≥ 0"

A **Boolean** variable is one that is either 0 (false) or 1 (true). A **perceptron** (threshold unit) takes inputs $x_1, \dots, x_N$, multiplies each by a **weight** w_i (how much that input counts, and in which direction), adds a **bias** b (a constant offset), and outputs 1 if the total is at least zero:

$$z = \sum_i w_i x_i + b, \qquad y = H(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

H is the **Heaviside step** ("threshold") function. Writing $b = -T$, the rule is the same as "fire if $\sum w_i x_i \geq T$": the slides switch between threshold T and bias b, so keep both in mind (L2 p9–10). E.g. with $\mathbf{w} = (2, -1)$, $b = -1$: input $(1, 0)$ gives $z = 2 - 0 - 1 = 1 \geq 0$, so $y = 1$; input $(0, 1)$ gives $z = 0 - 1 - 1 = -2$, so $y = 0$.

Linear vs affine (Poll 1). A **linear** function has no constant term: $3x + 7y$. An **affine** function is linear plus a constant: $3x + 7y + 4$. Every linear function is also affine (constant 0), but not the other

way round. The bias makes z affine, which lets the boundary line $z = 0$ sit anywhere instead of always passing through the origin.

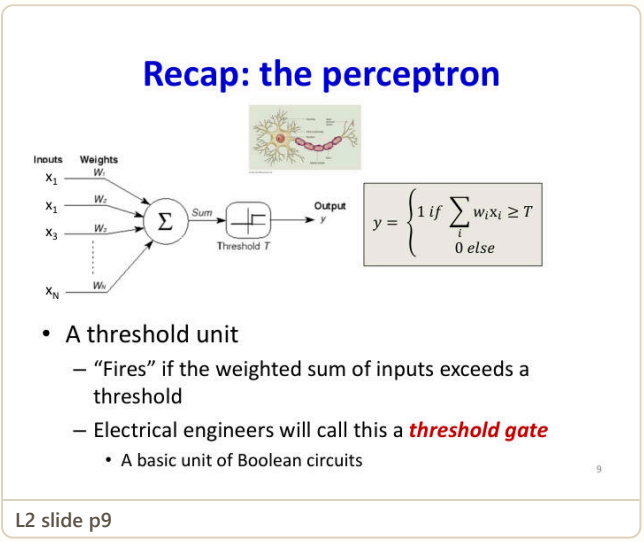
Like a Cairo taxi meter. The fare is (price per km) × km + flag-down fee. Without the flag-down fee the fare is linear (0 km costs 0); with it, it is affine. The bias is the flag-down fee. *Where it breaks:* a meter only adds; weights can be negative.

The gates you must be able to write in 10 seconds (L2 p25–30). A **literal** is a variable or its negation (x_i or $\bar{x}_i$, read "not x_i"). Give weight $+1$ to plain literals and -1 to negated ones. Let L = number of plain literals, N = total.

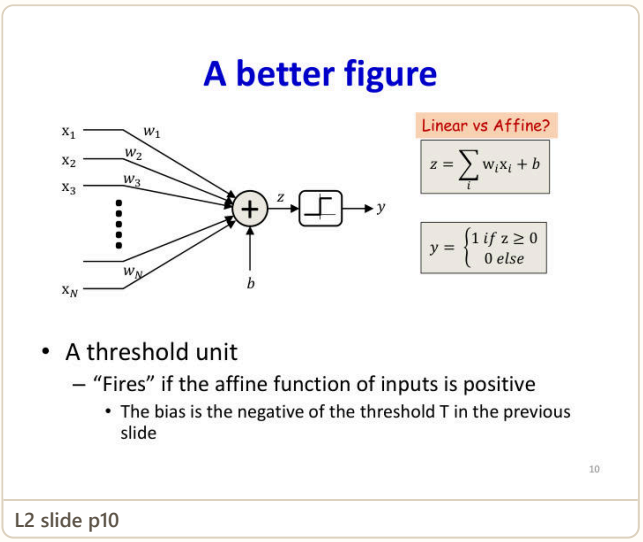
- **AND of all N literals:** fire iff $\sum w_i x_i \geq L$ (all plain ones on, all negated ones off).
- **OR of the literals:** fire iff $\sum w_i x_i \geq L - N + 1$.
- **At least K of the N literals:** fire iff $\sum w_i x_i \geq L - N + K$. (K = N gives AND, K = 1 gives OR, no negations gives the majority gate.)

E.g. the OR of $x_1, \bar{x}_2$: weights $(1, -1)$, $L = 1$, $N = 2$, so fire iff $x_1 - x_2 \geq 1 - 2 + 1 = 0$. On $(0, 1)$ both literals are false and $z = -1$: silent ✓. On $(0, 0)$, $\bar{x}_2$ is true and $z = 0$: fires ✓.

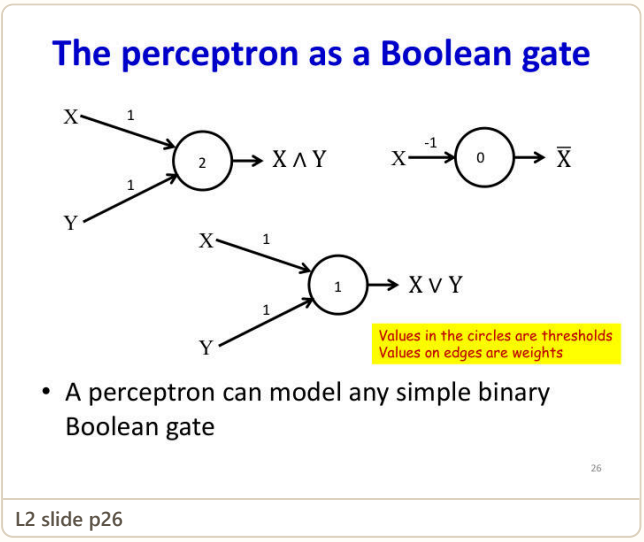
Like a password policy: "at least 3 of: uppercase, digit, symbol, 12+ characters". Each rule met adds a point; you pass if the count reaches 3. That is exactly an "at least K of N" perceptron.



L2 slide p9



L2 slide p10



L2 slide p26

↗ Understand more: linear vs affine (guide) ↗ Majority and the generalised gate (guide)

PEN & PAPER · ⏱ 16 min · Tier A

2 · Sheet 1 P1: one perceptron for "at least K of N literals"

① WHAT IS BEING ASKED, IN PLAIN WORDS

A perceptron does four things: multiply each input by its weight, add them up, add the bias b, and **fire (output 1) if the total z is ≥ 0**. The inputs are each 0 (off) or 1 (on).

A **literal** is a small true/false statement about one input. x_1 means "x_1 is on". $\bar{x}_2$ (read "not x_2") means "x_2 is NOT on", so it is true when $x_2 = 0$. The exam gives a list of literals and a number K, and asks for weights and a bias so the unit fires exactly when **at least K of the listed statements are true**, plus a proof and the general rule. We learn the method on a smaller sibling: **at least 2 of the 3 literals** $x_1,\ \bar{x}_2,\ x_3$.

② WARM-UP BY HAND, NO FORMULAS

(sibling). Take $x = (1, 1, 1)$, i.e. $x_1 = 1,\ x_2 = 1,\ x_3 = 1$. Go through the literals one at a time:

literal	x_1	$\bar{x}_2$	x_3
input value	$x_1 = 1$	$x_2 = 1$	$x_3 = 1$
literal true?	yes	no (it is on)	yes

Count the "yes": **2 true literals**. Two is at least two, so for this input the unit **must fire**.

③ THE ONE IDEA

If the unit could compute $z = \text{count} - K$, where count = the number of true literals, then $z \geq 0$ exactly when count $\geq K$. So the whole problem shrinks to: **write "count" as a weighted sum of the inputs plus a constant**.

④ BUILD IT, STEP BY STEP

(sibling, K = 2).

1. A plain literal as a number: x_1 is true (1) when $x_1 = 1$ and false (0) when $x_1 = 0$, so the literal **is just x_1**.
   Why: the input already is the 0/1 truth value.
2. A negated literal as a number: $\bar{x}_2 = 1 - x_2$. Check both values: $x_2 = 0$ gives $1 - 0 = 1$ (true ✓); $x_2 = 1$ gives $1 - 1 = 0$ (false ✓).
   Why: "not" swaps 0 and 1, and $1 - x$ does exactly that.
3. Add the literals to get the count:
   $$\text{count} = x_1 + (1 - x_2) + x_3$$
   Why: each true literal adds 1, each false one adds 0.
4. Collect the constant:
   $$\text{count} = x_1 - x_2 + x_3 + 1$$
   Why: each "not" brings a "1 −": the negated input gets a minus sign and the constant grows by 1 per negated literal (here one, so +1).

5. Subtract K = 2:

$$z = \text{count} - 2 = x_1 - x_2 + x_3 - 1$$

Why: this is the z from step ③.

6. Read off the numbers in front of each input and the constant: $\mathbf{w} = [1, -1, 1]$, $b = -1$.
Why: a perceptron computes $z = w_1x_1 + w_2x_2 + w_3x_3 + b$, and step 5 has exactly that shape.

7. The proof. Steps 3–5 are **algebra**: the identity $z = \text{count} - 2$ holds for every one of the 8 inputs at once. So $y = 1 \iff z \geq 0 \iff \text{count} \geq 2$.
Why: an identity proves all rows together; no truth table needed.

8. The pattern, for a general question: weight $+1$ on a plain literal, -1 on a negated one, and the constant is (number of negated literals) − K.
Why: step 4 gave $+1$ per negated literal, step 5 subtracted K. To **prove** it in general, redo steps 3–5 with letters: P = the plain indices, Q = the negated ones.

⑤ **CHECK IT**

(sibling). Warm-up input $(1, 1, 1)$: $z = 1 - 1 + 1 - 1 = 0 \geq 0$, **fires** ✓ (count 2). Edge case $(0, 1, 0)$, where no literal is true: $z = 0 - 1 + 0 - 1 = -2$, silent ✓ (count 0, and $0 - 2 = -2$). Edge case $(1, 0, 0)$: literals x_1 and $\bar{x}_2$ true, count 2; $z = 1 - 0 + 0 - 1 = 0$, fires ✓.

⑥ **NOW THE EXAM PROBLEM — YOUR TURN**

EXAM PROBLEM

Sheet 1 P1 (12 pts). $x_1, \ldots, x_6 \in \{0, 1\}$. Build a single perceptron that outputs 1 iff **at least four** of the literals $x_1,\ \bar{x}_2,\ x_3,\ \bar{x}_4,\ x_5,\ \bar{x}_6$ are true. (a) Give the six weights and the bias. (b) Prove it works. (c) Generalise to N inputs with a set Q of negated literals and "at least K".

Do it the same way: steps ④–⑤ with these numbers (six literals, K = 4), then step 8 with letters for (c).

✍ Hint: S2.1 (end of this lecture)

✍ Full worked solution: S2.2 (end of this lecture)

⑦ **TRY ALONE**

Give $\mathbf{w}$ and b, and check one input, for: (i) at least 2 of $\bar{x}_1,\ \bar{x}_2,\ x_3$; (ii) at least 3 of $x_1,\ x_2,\ \bar{x}_3,\ x_4,\ \bar{x}_5$; (iii) at least 1 of $\bar{x}_1,\ \bar{x}_2,\ \bar{x}_3,\ \bar{x}_4$ (what gate is this?).

✍ Answers: S2.3 (end of this lecture)

🧭 **Like a points-based visa check:** each requirement met is +1 point, and you get in at K points. A "must NOT have" requirement ($\bar{x}_i$) is worth a point when the box is unticked, which is why it gets weight −1 and adds +1 to the starting score (the bias).

↗ Understand more: Sheet 1 P1(a)(b) worked (guide) ↗ P1(c) general recipe (guide)

🖍 **PEN & PAPER · ⏱ 16 min · Tier A**

3 · Sheet 1 P2: XOR is impossible for one perceptron, easy for three

① WHAT IS BEING ASKED, IN PLAIN WORDS

XOR ("exclusive or", written $x_1 \oplus x_2$) takes two 0/1 inputs and outputs 1 when **exactly one** of them is 1. One perceptron computes $z = w_1x_1 + w_2x_2 + b$ and outputs $H(z)$: 1 if $z \geq 0$, else 0. The exam asks you to **prove** that no choice of the three numbers w_1, w_2, b gives XOR, then to build a small network of three perceptrons that does. We learn both moves on the sibling **XNOR** ("equal"): $00 \to 1,\ 01 \to 0,\ 10 \to 0,\ 11 \to 1$.

Like the two-way light switch on a staircase: the light is on when the top and bottom switches are in different positions (XOR). Wire it the other way round and it is on when they match (XNOR). Flip either switch and the light toggles.

② WARM-UP BY HAND, NO FORMULAS

(sibling, XNOR). Try the AND unit, $w = (1, 1)$, $b = -1.5$. Its z on 00, 01, 10, 11 is $-1.5,\ -0.5,\ -0.5,\ 0.5$, so it outputs $0, 0, 0, 1$: right on three rows, **wrong on 00**. To lift 00 above zero, raise the bias to $+0.5$: now $z = 0.5,\ 1.5,\ 1.5,\ 2.5$ and **01, 10 switch on too**. Try the NOR unit $w = (-1, -1)$, $b = 0.5$: outputs $1, 0, 0, 0$, now 11 is wrong. Every fix for one row breaks another: that tug-of-war is what the proof makes exact.

③ THE ONE IDEA

Write z at each corner of the square: $z_{00} = b$, $z_{10} = w_1 + b$, $z_{01} = w_2 + b$, $z_{11} = w_1 + w_2 + b$. Then, for **every** perceptron,

$$z_{11} = z_{10} + z_{01} - z_{00}$$

So the four signs can't be chosen freely: if a target function's demands on the four z's contradict this identity, no perceptron computes it. And to build a network, **split the target into gates you already know** (AND, OR, NAND, NOR), one hidden unit each, and combine them with one output gate.

④ BUILD IT, STEP BY STEP

(sibling, XNOR).

1. Check the identity: $z_{10} + z_{01} - z_{00} = (w_1 + b) + (w_2 + b) - b = w_1 + w_2 + b = z_{11}$ ✓.
   Why: it holds for any numbers, so it binds every possible perceptron.
2. Write what XNOR demands from each row, numbered: (1) $b \geq 0$ (00 → 1); (2) $w_1 + b < 0$ (10 → 0); (3) $w_2 + b < 0$ (01 → 0); (4) $w_1 + w_2 + b \geq 0$ (11 → 1).
   Why: output 1 needs $z \geq 0$, output 0 needs $z < 0$.
3. Add (2) and (3): $w_1 + w_2 + 2b < 0$, i.e. $w_1 + w_2 + b < -b$.
   Why: adding two true "< 0" facts gives a true "< 0" fact; then move one b across.
4. By (1), $b \geq 0$, so $-b \leq 0$, and so $w_1 + w_2 + b < 0$. That contradicts (4).
   Why: the four demands cannot all hold, so no perceptron does XNOR. ■
5. Split the target into known gates: XNOR = "both on" OR "both off" = (AND) OR (NOR).
   Why: the two 1-rows of XNOR are 11 (AND's only 1-row) and 00 (NOR's only 1-row).
6. Hidden unit 1 = AND: $h_1 = H(x_1 + x_2 - 1.5)$.

Why: the sum reaches 1.5 only when both are on.

7. Hidden unit 2 = NOR ("neither"): $h_2 = H(-x_1 - x_2 + 0.5)$.
   Why: $z_2 \geq 0 \iff x_1 + x_2 \leq 0.5$, true only when both are off.
8. Output = OR of the two: $y = H(h_1 + h_2 - 0.5)$.
   Why: $h_1 + h_2$ reaches 1 if either fires. (Half-way biases like $\pm 0.5,\ -1.5$ keep every z at least 0.5 away from 0, so a slip can't flip an answer.)

⑤ CHECK IT

(sibling), all four rows:

$x_1 x_2$	z_1	h_1	z_2	h_2	z_{out}	y	**XNOR**
00	−1.5	0	0.5	1	0.5	1	1 ✓
01	−0.5	0	−0.5	0	−0.5	0	0 ✓
10	−0.5	0	−0.5	0	−0.5	0	0 ✓
11	0.5	1	−1.5	0	0.5	1	1 ✓

Why geometry agrees: one perceptron's firing set is a half-plane (one side of one straight line). XNOR's 1-points (00 and 11) sit on one diagonal of the square and its 0-points on the other; no single line separates them, but two parallel lines do (L2 p79). In the exam a picture alone earns nothing: the inequalities are the proof.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Sheet 1 P2 (14 pts). (a) Prove no single threshold perceptron $y = H(w_1 x_1 + w_2 x_2 + b)$ computes XOR, **using inequalities from the four input points** (a picture alone gets no marks). (b)–(d) Build an MLP with one hidden layer of exactly two threshold units for XOR, give every weight and bias, and check all four inputs.

Do it the same way: steps ④–⑤ with XOR's four rows (write its four demands, add the two middle ones, compare with the other two; then split XOR into two known gates).

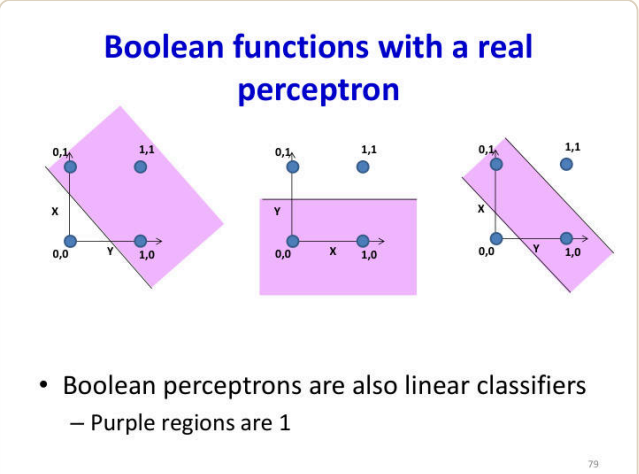
✍ Hint: S2.4 (end of this lecture)

✍ Full worked solution: S2.5 (end of this lecture)

⑦ TRY ALONE

(i) In the sibling XNOR net, change only the output bias from -0.5 to -1.5. What does it compute now? (ii) Run input 10 through the exam's XOR net: give h_1, h_2, y. (iii) The function $00 \to 1,\ 01 \to 1,\ 10 \to 0,\ 11 \to 1$ is **not** blocked by the identity. Find one perceptron for it.

✍ Answers: S2.6 (end of this lecture)



L2 slide p79

↗ Understand more: the XOR inequality proof (guide) ↗ P2(b)–(d) every weight (guide) ↗ Try XOR in TensorFlow Playground

📖 READ & UNDERSTAND · ⏱ 5 min · Tier A

4 · Depth and layers: what "deep" actually means

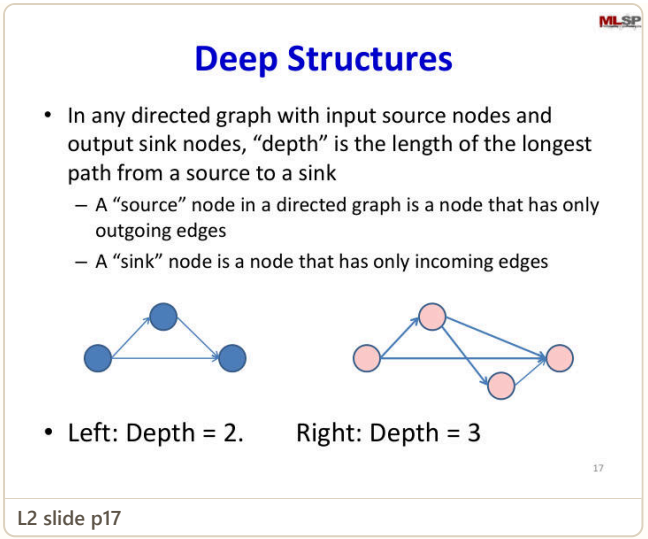
Draw the network as a **directed graph**: dots (inputs and neurons) joined by one-way arrows. A **source** has only outgoing arrows (the inputs); a **sink** has only incoming arrows (the outputs).

- **Depth** of the network = the length (number of arrows) of the **longest path** from a source to a sink (L2 p17).
- Depth of a neuron = longest path from any input to it. Inputs have depth 0.
- A **layer** = all neurons with the same depth (L2 p19). A **hidden layer** is any layer that is not the output.
- **"Deep"** = output depth greater than 2, i.e. at least 3 layers of neurons (L2 p18–19).

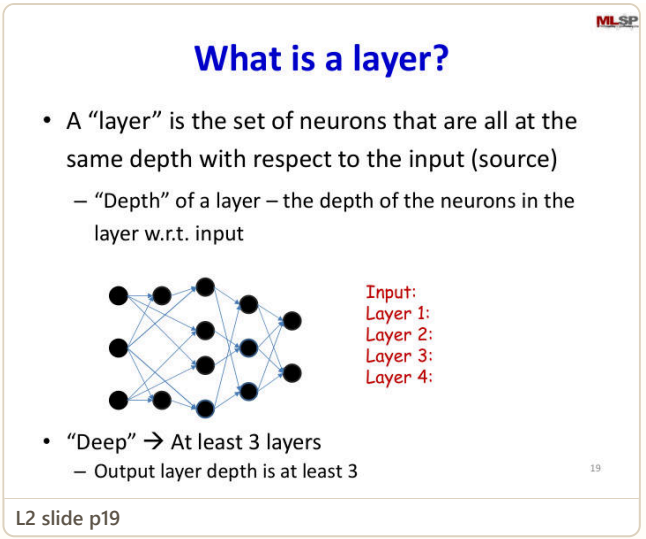
Longest, not shortest. If a neuron gets one arrow straight from the input and another through two neurons, its depth is 3, not 1. Quick check: edges $x_1 \to a,\ x_2 \to a,\ x_2 \to b,\ x_3 \to b,\ a \to c,\ x_1 \to c,\ b \to d,\ c \to d,\ x_3 \to d,\ d \to y,\ a \to y$ give depths $a = 1,\ b = 1,\ c = 2,\ d = 3,\ y = 4$. So the network has depth 4 even though $a \to y$ is a 2-arrow shortcut.

🧭 **Like a WhatsApp forward chain:** a message can reach you directly from your friend or through three cousins. "How deep is the family group?" is about the longest chain of forwards, because you can't act until the slowest version has arrived. *Where it breaks:* in a network every neuron waits for **all** its inputs, not just the first to arrive.

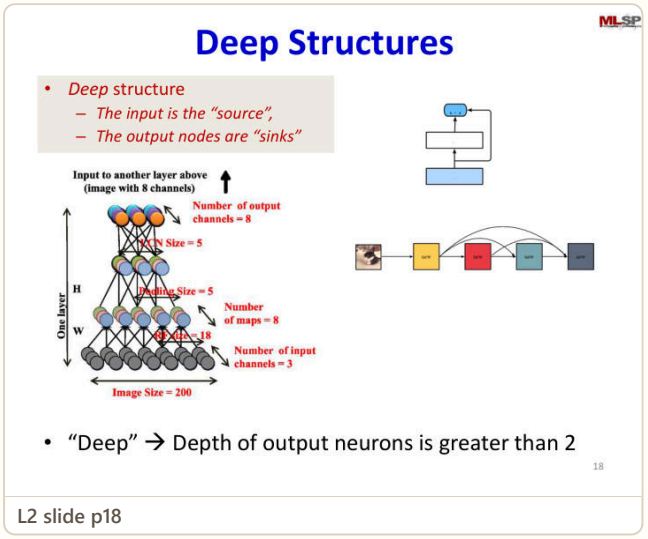
Skip connections and the 2-neuron XOR (L2 p33). If arrows may jump over a layer, XOR needs only 2 neurons: $h = H(x_1 + x_2 - 2)$ (the AND) and $y = H(x_1 + x_2 - 2h - 1)$. Check $(1,1)$: $h = 1$, $z = 1 + 1 - 2 - 1 = -1$, so $y = 0$ ✓; $(1,0)$: $h = 0$, $z = 0$, $y = 1$ ✓. The output has depth 2 (path $x \to h \to y$). This is why **Poll 3** is slippery: see the traps box.



L2 slide p17



L2 slide p19



L2 slide p18

↗ Understand more: depth = longest path (guide) ↗ The two-neuron XOR (guide) 🎥 CMU L2 video (slides p15–33)

📖 READ & UNDERSTAND · ⏱ 6 min · Tier A

5 · Any Boolean function with one hidden layer: truth table → DNF → network → K-map

A **truth table** lists the output for every input pattern; it *is* the function. A **minterm** is an AND of all N literals that is true on exactly one row (e.g. row 1011 → $x_1\bar{x}_2x_3x_4$). The **DNF** ("disjunctive normal form") is the OR of the minterms of all the rows where the output is 1 (L2 p36–37).

DNF → network (L2 p38–46). One hidden unit per term, one OR on top:

- **Term unit:** weight $+1$ on each plain literal, -1 on each barred one, 0 on variables not in the term; bias $= -(\text{number of plain literals})$. Example: $x_1\bar{x}_3x_4$ (4 inputs) → $\mathbf{w} = (1, 0, -1, 1)$, $b = -2$. On 1001: $z = 1 + 0 - 0 + 1 - 2 = 0$, fires ✓. On 1011: $z = -1$, silent ✓.
- **Output unit:** OR of the hidden units: all weights 1, bias -1 (fire if **any** term fires).

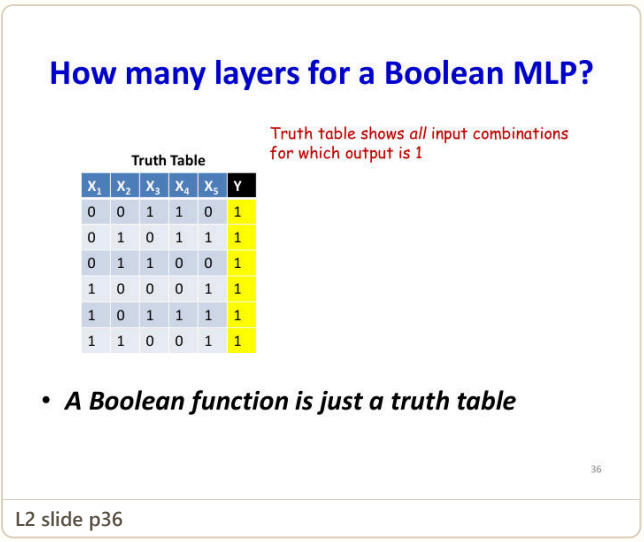
So **a one-hidden-layer MLP can compute every Boolean function**. The catch: the width can be huge, up to one unit per 1-row.

K-maps shrink the DNF (L2 p47–50). A **Karnaugh map** is the truth table folded into a grid, rows and columns in **Gray order** 00, 01, 11, 10 (each step flips one bit, **and the edges wrap**

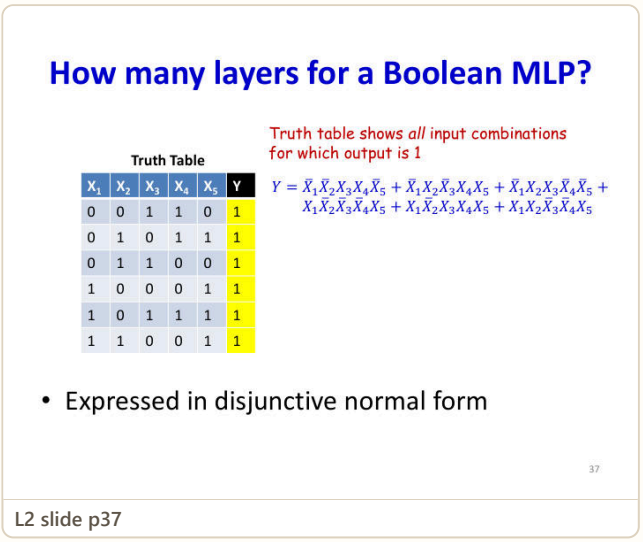
around). Neighbouring cells differ in one variable, so two neighbouring 1s merge into one term without that variable. Circle the 1s in rectangles of size 1, 2, 4, 8 (as big as possible, may overlap, may wrap); each rectangle is one term = one hidden unit.

> 🧭 **Like a café order list.** If the barista accepts "tea, sugar" and "tea, no sugar", you can write just "tea": sugar doesn't matter. Merging two neighbouring K-map cells is exactly this: the one variable that differs drops out. *Where it breaks:* you can only merge in blocks of 2, 4, 8, never 3.

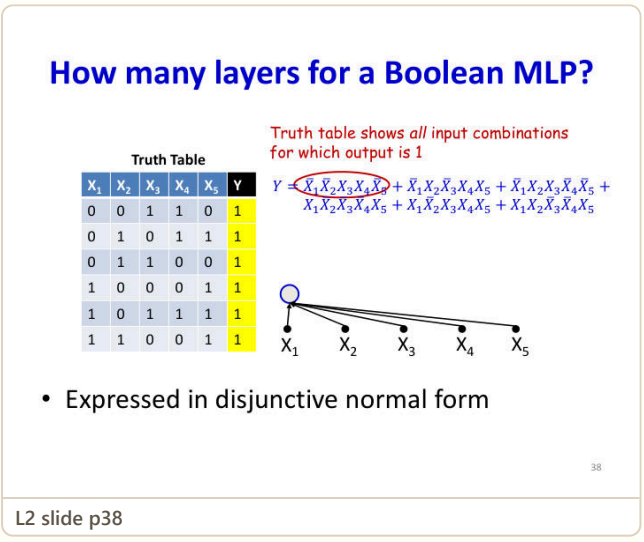
The worst case: the checkerboard (L2 p51–58, Poll 2). If flipping any single bit always flips the output (this is **parity**), no two 1s are neighbours, nothing merges, and you need one hidden unit per 1-row, which is half of all rows. For $N = 10$: $1024/2 = \mathbf{512}$ hidden neurons.



L2 slide p36



L2 slide p37



L2 slide p38

↗ Understand more: one hidden unit per term (guide) ↗ K-maps from zero (guide) ↗ K-map lab: group the ones yourself

🖍 **PEN & PAPER · ⏱ 16 min · Tier A**

6 · Sheet 1 P4: reduce the DNF, then build the MLP

① **WHAT IS BEING ASKED, IN PLAIN WORDS**

F is a yes/no function of some 0/1 inputs. Instead of a formula, it is given as a list of row numbers where $F = 1$: read the inputs, in order, as one binary number (e.g. with three inputs, 101 is 5). You must (a) write F as an OR of AND-terms (the **DNF**, one full term or "minterm" per listed row, card 5), (b) shrink

it, (c)(d) build a one-hidden-layer network for the short version, (e) compare sizes. We learn it on a smaller sibling: $G(x_1, x_2, x_3) = 1$ **on rows** $\{0, 2, 4, 6, 7\}$.

② WARM-UP BY HAND, NO FORMULAS

(sibling). Binary with 3 bits: the digits are worth 4, 2, 1. So 6 = 1·4 + 1·2 + 0·1 → **110**. Convert all five listed rows:

row	0	2	4	6	7
$x_1x_2x_3$	000	010	100	110	111

Look at them: four of the five have $x_3 = 0$, and the odd one out, 111, sits next to 110 (they differ only in x_3). That hints at two short terms. The K-map is the systematic way to see it.

③ THE ONE IDEA

Two 1-rows that **differ in exactly one input** merge into one term that leaves that input out: "x_1 on, x_2 on, x_3 off" OR "x_1 on, x_2 on, x_3 on" is just "x_1 on, x_2 on". A **K-map** is the truth table laid out so that such rows sit next to each other; then you circle blocks of 1s (sizes 1, 2, 4, 8, as big as possible, overlaps allowed).

④ BUILD IT, STEP BY STEP

(sibling).

1. Canonical DNF: one minterm per listed row, plain x_i where the bit is 1, $\bar{x}_i$ where it is 0: $G = \bar{x}_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + x_1\bar{x}_2\bar{x}_3 + x_1x_2\bar{x}_3 + x_1x_2x_3$ ("+" means OR).
   Why: each minterm is true on exactly its own row.
2. Draw the grid. With 3 inputs: 2 rows (x_1 = 0, 1) × 4 columns (x_2x_3 in the order **00, 01, 11, 10**). With 4 inputs it is 4 × 4: rows x_1x_2, columns x_3x_4, both in that order.
   Why: in this (Gray) order neighbouring cells differ in one bit, and the grid **wraps around**: the left and right edges touch, and so do the top and bottom.
3. Put a 1 in each listed cell: row 6 = 110 → row $x_1 = 1$, column $x_2x_3 = 10$; and so on.
   Why: each 1 is one minterm.

$x_1 \setminus x_2x_3$	00	01	11	10
0	1	0	0	1
1	1	0	1	1

4. Circle the biggest block: the first and last columns touch through the wrap-around, giving a block of four (rows 0, 2, 4, 6). In all four $x_3 = 0$, while x_1 and x_2 change → term $\bar{x}_3$.
   Why: inputs that change inside a block drop out; inputs that stay fixed stay in.
5. Circle the 1 left over (row 7 = 111) with its neighbour 110 (it may reuse a cell already circled): both have $x_1 = 1,\ x_2 = 1$ → term x_1x_2.
   Why: overlapping is fine and makes the block bigger, so the term shorter.
6. All five 1s are covered, so $G = \bar{x}_3 + x_1x_2$.
   Why: each block is one AND-term; G is their OR.
7. One hidden unit per term: weight $+1$ on plain literals, -1 on barred ones, 0 on absent inputs, bias = −(number of plain literals). $\bar{x}_3 \to \mathbf{w} = (0, 0, -1), b = 0$. $x_1x_2 \to \mathbf{w} = (1, 1, 0), b = -2$.
   Why: card 5's AND-term recipe: $z = 0$ exactly when the term is true, and negative otherwise.

8. Output = OR of the hidden units: weights 1, 1, bias -1.
   Why: fires if either term fires.

9. Compare widths: canonical DNF → 5 hidden units (one per minterm, e.g. 110 → $\mathbf{w} = (1, 1, -1)$, $b = -2$); reduced → 2.
   Why: each block of the K-map replaced several minterm units by one.

⑤ CHECK IT

(sibling). Row 7 = 111 (in the list): $h_1 = H(-1) = 0$, $h_2 = H(1 + 1 - 2) = H(0) = 1$, $y = H(0 + 1 - 1) = 1$ ✓. Row 4 = 100: $h_1 = H(0) = 1$, $y = 1$ ✓. Edge case, row 5 = 101 (not in the list): $h_1 = H(-1) = 0$, $h_2 = H(1 + 0 - 2) = 0$, $y = H(-1) = 0$ ✓.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Sheet 1 P4 (14 pts). $F(x_1, x_2, x_3, x_4) = 1$ on exactly the minterms $\{0, 2, 5, 7, 8, 10, 13, 15\}$, where the index is the binary number $x_1x_2x_3x_4$. (a) Canonical DNF. (b) Reduce it. (c)(d) One-hidden-layer threshold MLP for the reduced form, with every weight and bias. (e) Compare hidden widths.

Do it the same way: steps ④–⑤ with these numbers (4 bits worth 8, 4, 2, 1; a 4 × 4 K-map).

✍ Hint: S2.7 (end of this lecture)

✍ Full worked solution: S2.8 (end of this lecture)

⑦ TRY ALONE

(i) Give the hidden unit for the term $\bar{x}_1x_3$ (4 inputs) and check it on row 2 = 0010. (ii) $F = 1$ on rows $\{0, 1, 2, 3\}$ only (4 inputs). Reduce it and give the network. (iii) Same for rows $\{1, 3, 9, 11\}$.

✍ Answers: S2.9 (end of this lecture)

🧭 **Like a café order list:** if the barista accepts "tea, sugar" and "tea, no sugar", you can write just "tea". Circling a K-map block is that, done several inputs at a time.

↗ Understand more: Sheet 1 P4 worked (guide) ↗ Try P4 again without the answer (guide) ↗ K-maps from zero (guide)

🖍 PEN & PAPER · ⏱ 17 min · Tier A

7 · Sheet 1 P3: parity, shallow vs deep (the lecture's headline result)

① WHAT IS BEING ASKED, IN PLAIN WORDS

Parity $P_N = x_1 \oplus x_2 \oplus \cdots \oplus x_N$ takes N bits and outputs 1 when an **odd** number of them are 1. It is XOR of many inputs. The question compares two ways to build it: **shallow** (one hidden layer, one unit per 1-row, cards 5–6) and **deep** (a tree of the 3-perceptron XOR modules from card 3). You count units in each, for a general N and for one particular N, and say what the gap means. We learn it on the sibling $N = 4$.

Like the parity bit on an old dial-up modem line: the sender adds one bit so the count of 1s is even; if one bit gets flipped by noise, the count turns odd and the receiver knows. Parity changes whenever **any single** bit changes, which is exactly why no K-map merging is possible.

② WARM-UP BY HAND, NO FORMULAS

Take $N = 3$: $2^3 = 8$ rows. Count the 1s in each and mark odd:

$x_1x_2x_3$	000	001	010	011	100	101	110	111
number of 1s	0	1	1	2	1	2	2	3
P_3	0	**1**	**1**	0	**1**	0	0	**1**

Four 1-rows: exactly half of 8. Now look for two 1-rows that differ in one bit (so they could merge): 001 and 011 differ in one bit, but 011 is a 0-row. Every one-bit neighbour of a 1-row is a 0-row. **Nothing merges**, so the shallow net needs 4 hidden units.

③ THE ONE IDEA

Flipping any one bit flips the parity. That kills every K-map merge, so the shallow net needs one unit for every 1-row: half of all rows. But XOR can be done in pairs and then pairs of results (like a knockout bracket), and each merge costs one small module, so the deep net grows only with N.

④ BUILD IT, STEP BY STEP

(sibling, $N = 4$).

1. Rows: $2^4 = 16$.
   Why: each of 4 bits is 0 or 1, and choices multiply.
2. 1-rows: $16/2 = 8 = 2^{4-1}$.
   Why: flipping x_1 pairs each row with one of opposite parity, so odd and even rows come in equal numbers: exactly half are 1-rows.
3. No merging: a shorter term that leaves out some x_j would also be true on the row with x_j flipped, and that row has the **opposite** parity (output 0).
   Why: so every term must be a full minterm; this is the proof, not just the warm-up's observation.
4. Shallow net: **8 hidden units** + 1 OR output = 9 perceptrons.
   Why: one unit per 1-row (step 2), none merged (step 3).
5. Deep net, stage 1: 2 XOR modules on the pairs (x_1, x_2) and (x_3, x_4) → 2 values.
   Why: XOR can be regrouped and reordered freely ($\oplus$ is associative and commutative), so pairing is allowed.
6. Stage 2: 1 module XORs those 2 values → P_4.
   Why: each stage halves the number of values; stop at one.
7. Count: $2 + 1 = 3 = N - 1$ modules, in $\log_2 4 = 2$ stages. Each module is card 3's net, 2 hidden units + 1 output, so $2 \times 3 = 6$ hidden units inside the modules, $3 \times 3 = 9$ perceptrons, and $2 \times 2 = 4$ threshold layers.
   Why: each module turns two values into one, so going from N values to 1 takes N − 1 merges; each stage is 2 threshold layers deep.
8. Compare the growth: shallow needs 2^{N-1} (doubles with every extra input), deep needs $3(N-1)$ (adds 3 per input). At $N = 4$ they tie (9 vs 9); at $N = 16$ it is $2^{15} + 1 = 32769$ vs $3 \times 15 = 45$.

Why: the exam's "what does it show" is about **growth** (exponential vs linear), not one value.

⑤ CHECK IT

(sibling). Input 1011 has three 1s, so $P_4 = 1$. Run the tree: stage 1 pairs (1,0)(1,1) → 1, 0; stage 2 pair (1,0) → **1** ✓. Edge case $N = 2$: shallow $2^1 = 2$ hidden units, deep $N - 1 = 1$ module, which is the card-3 XOR net itself ✓.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Sheet 1 P3 (18 pts). $N = 2^m$. (a) How many hidden units does the one-hidden-layer DNF construction need? (b) For $N = 8$? (c) Build a deep net as a balanced binary tree of 2-input XOR modules (the P2 net). (d) For $N = 8$: (i) number of modules, (ii) hidden units inside them, (iii) number of stages, (iv) growth of the module count for general N. (e) What does this show about depth and width? (Don't claim every deep net beats every shallow one.)

Do it the same way: steps ④–⑤ with these numbers (general N for (a), then N = 8).

✍ Hint: S2.10 (end of this lecture)

✍ Full worked solution: S2.11 (end of this lecture)

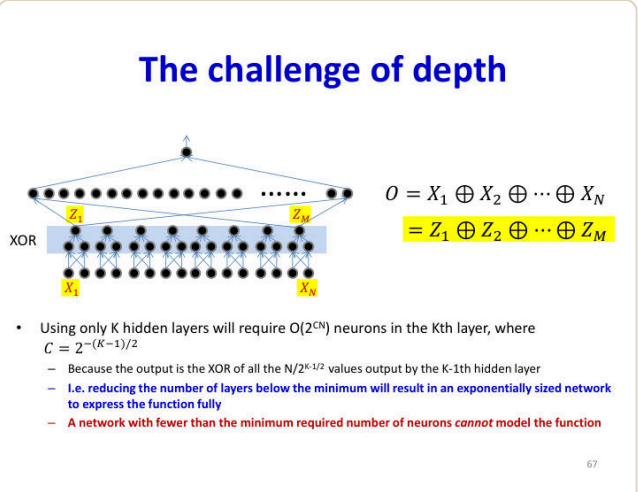
⑦ TRY ALONE

(i) $N = 32$: shallow hidden units? Deep modules, stages, total deep perceptrons? (ii) $N = 10$ (not a power of 2): shallow hidden units, deep modules, and stages (an odd value left over just waits a round). (iii) $N = 6$: same three numbers.

✍ Answers: S2.12 (end of this lecture)

🧭 **Like a World Cup knockout bracket:** 16 teams need exactly 15 matches (every match eliminates one team) played over 4 rounds. The XOR tree is the bracket: $N - 1$ modules over $\log_2 N$ stages. The shallow DNF is like listing every possible "who beat whom" outcome in advance.

Cutting depth costs width (L2 p67). With only K hidden layers instead of the full $2 \log_2 N$, the last layer needs $O(2^{CN})$ neurons with $C = 2^{-(K-1)/2}$: exponential again. A network that is too shallow (or too narrow) cannot model the function at all with fewer than that many neurons. Quick number to remember: $N = 10$ gives $2^9 + 1 = 513$ perceptrons shallow vs $3 \times 9 = 27$ deep.



L2 slide p67

↗ Understand more: Sheet 1 P3 fully worked (guide) ↗ Parity the deep way (guide) ↗ Cut the depth and the width explodes (guide)

📖 READ & UNDERSTAND · ⏱ 5 min · Tier A

8 · Decision regions: lines → polygons → any shape

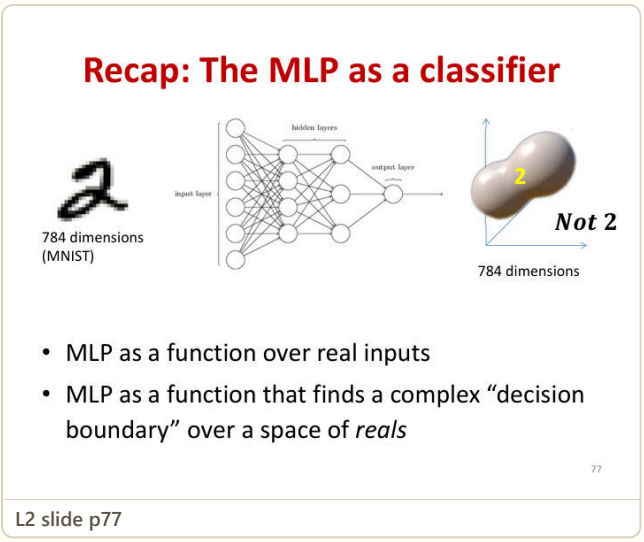
Now the inputs are real numbers, a point (x_1, x_2) in the plane. One perceptron fires on the set where $w_1x_1 + w_2x_2 + b \geq 0$: a **half-plane**, everything on one side of a straight line (L2 p77–78). E.g. $w_1 = w_2 = 1, b = -1$: the point $(1, 1)$ gives $z = 1$ (fires), $(0, 0)$ gives $z = -1$ (silent); the boundary is the line $x_1 + x_2 = 1$.

A shape is **convex** if the straight segment between any two of its points stays inside it (a disc, a triangle, a pentagon; not a star, not a ring). The recipe, in three moves:

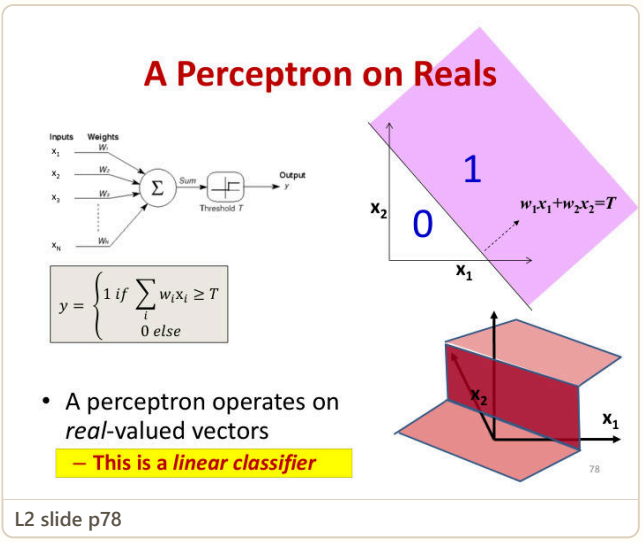
- **Convex polygon = AND of half-planes (L2 p82–88).** One hidden unit per edge, each firing on the inside of its edge. Output fires iff all fire: for a pentagon, $\sum_{i=1}^{5} y_i \geq 5$. (If you lower that to ≥ 4, points outside exactly one edge also fire, and you get a five-pointed star.)
- **Any union of polygons = OR of ANDs (L2 p89–91).** Second hidden layer = one AND per polygon; output = OR. That is **two hidden layers**, and it can draw non-convex shapes and shapes with holes.
- **Even one hidden layer suffices, approximately (L2 p97–107).** With N edges around a circle, the sum is N inside and drops to about N/2 far away. (**Skim p101–102**, the doctor said look at them: more sides shrink the area where the sum is between N/2 and N, and in the limit the sum is a near-perfect cylinder, N inside and N/2 outside. Skip the arccos formula.) Subtract N/2 and you get a "cylinder": about N/2 inside, about 0 outside. Add many small cylinders, threshold once: any shape. But it needs (in the limit) **infinitely many** neurons. So MLPs are **universal classifiers**.

🧭 **Like a food-delivery geofence on Google Maps:** the app decides "are you inside the zone?" by checking which side of each border line you are on (the half-planes), requiring all of them (the AND), and if the zone is made of several neighbourhoods, checking whether you are in any (the OR). *Where it breaks:* real apps store polygon corners, not neurons, but the logic is the same.

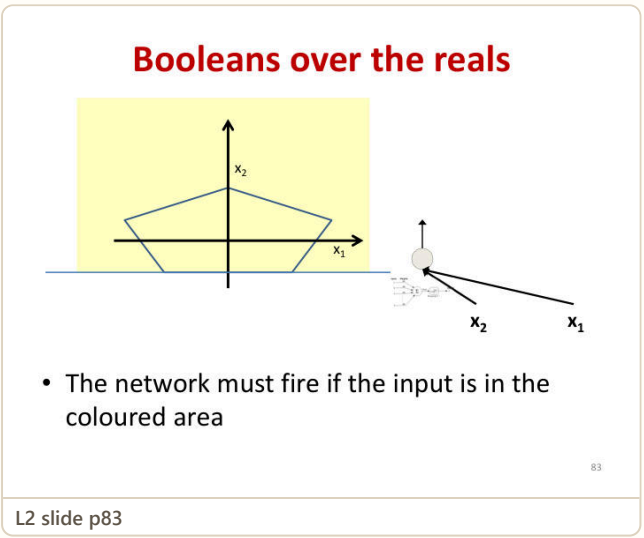
Depth pays here too (L2 p108–124). A shape that needs ~infinitely many neurons with one hidden layer took only 12 with more layers (p108). For the 16-line checkerboard: two hidden layers need $16 + 40 + 1 = 57$ neurons; the XOR-style deep net $16 + 15 \times 3 = 61$ (46 with the 2-neuron XOR). For 64 lines: $64 + 544 + 1 = 609$ vs 253 (190). The gap grows with the pattern's complexity and the input **dimension**: a shallow net's worst-case size can be exponential in the number of independent input features.



L2 slide p77



L2 slide p78



L2 slide p83

↗ Understand more: one unit per edge (guide) ↗ Non-convex shapes: OR of ANDs (guide) ↗ Circles and cylinders (guide) ↗ Watch neurons draw regions (TF Playground)

🥖 PEN & PAPER · ⏱ 17 min · Tier A

9 · Sheet 1 P5: the picture frame (a rectangle with a hole)

> ① WHAT IS BEING ASKED, IN PLAIN WORDS
>
> The inputs are now real numbers: a point (x, y) in the plane. The exam's region is a rectangle with a smaller rectangle cut out of it (a **hole**), like a picture frame. You must say why one threshold unit can't fire exactly on it, cut it into rectangles, give the units, and explain how they combine. We learn it on a sibling: the square $0 \leq x \leq 3,\ 0 \leq y \leq 3$ with the open hole $1 < x < 2,\ 1 < y < 2$ removed. ("Open" = the hole's edge lines are *not* removed, they still belong to the region.)

② WARM-UP BY HAND, NO FORMULAS

(sibling). Sketch it: a 3×3 square with a 1×1 hole in the middle. Now test four points:

point	in the big square?	in the hole?	in the frame?
$(1.5, 0.5)$	yes	no ($y \leq 1$)	**yes** (below the hole)
$(1.5, 1.5)$	yes	yes	**no**
$(0.5, 1.5)$	yes	no ($x \leq 1$)	**yes** (left of the hole)
$(4, 1)$	no ($x > 3$)	no	**no**

Notice: $(0.5, 1.5)$ and $(2.5, 1.5)$ are both in the frame, but the point halfway between them, $(1.5, 1.5)$, is in the hole.

③ THE ONE IDEA

One threshold unit fires on one side of one straight line (a **half-plane**), and a half-plane has no holes: if two points are in it, so is everything between them (it is **convex**). So one unit is hopeless. But a rectangle is **four half-planes ANDed**, and a frame is **four rectangles ORed**: lines → strips → frame.

④ BUILD IT, STEP BY STEP

(sibling).

1. Why one unit fails, as a proof: if $\mathbf{p}_1, \mathbf{p}_2$ both satisfy $\mathbf{w}^\top \mathbf{p} + b \geq 0$, their midpoint $\mathbf{m} = \frac{1}{2}(\mathbf{p}_1 + \mathbf{p}_2)$ does too, since $\mathbf{w}^\top \mathbf{m} + b = \frac{1}{2}(\mathbf{w}^\top \mathbf{p}_1 + b) + \frac{1}{2}(\mathbf{w}^\top \mathbf{p}_2 + b) \geq 0$. Take $\mathbf{p}_1 = (0.5, 1.5)$, $\mathbf{p}_2 = (2.5, 1.5)$, both in the frame; their midpoint $(1.5, 1.5)$ is in the hole.
   Why: a unit firing on the whole frame would have to fire in the hole too.
2. "$x \geq a$" is the unit $H(x - a)$, and "$x \leq a$" is $H(-x + a)$; same for y. E.g. "$y \leq 1$" is $H(-y + 1)$: at $y = 0.5$, $z = 0.5$, fires ✓; at $y = 1.5$, $z = -0.5$, silent ✓.
   Why: H fires when its input is ≥ 0, and $H(0) = 1$ keeps the edge line itself.
3. Cut the frame into four closed strips: **bottom** $[0, 3] \times [0, 1]$, **top** $[0, 3] \times [2, 3]$, **left** $[0, 1] \times [1, 2]$, **right** $[2, 3] \times [1, 2]$.
   Why: each strip is a rectangle (convex), and together they are exactly the frame (shared edges are fine, the hole is open).
4. Each strip needs 4 side units (left, right, bottom, top edge):

strip	left side	right side	bottom side	top side
bottom	$H(x)$	$H(-x + 3)$	$H(y)$	$H(-y + 1)$
top	$H(x)$	$H(-x + 3)$	$H(y - 2)$	$H(-y + 3)$
left	$H(x)$	$H(-x + 1)$	$H(y - 1)$	$H(-y + 2)$
right	$H(x - 2)$	$H(-x + 3)$	$H(y - 1)$	$H(-y + 2)$

   Why: a point is in a rectangle iff it is on the inner side of all four edges. As $(\mathbf{w}, b)$: the right strip's "$x \geq 2$" is $\mathbf{w} = (1, 0), b = -2$.
5. One AND unit per strip: weights 1, 1, 1, 1 on its four side units, bias -4.
   Why: the sum reaches 4 only if all four fire.
6. One OR output: weights 1, 1, 1, 1 on the four AND units, bias -1.
   Why: it fires if the point is in **any** strip.

7. Count: $16 + 4 + 1 = 21$ threshold units in two hidden layers + output. (Only 10 different lines appear, so 15 units if layer 1 is shared.)
   Why: 4 strips × 4 sides, 4 ANDs, 1 OR.
8. The general point: layer 1 answers "which side of this line?", layer 2 ANDs the answers into convex pieces, the output ORs the pieces. Nothing used this particular shape.
   Why: so any finite union of convex polygons, holes and all, is built the same way.

⑤ CHECK IT

(sibling). $(0.5, 1.5)$, left strip: $z = 0.5,\ 0.5,\ 0.5,\ 0.5$: all fire, AND gets $4 - 4 = 0 \to$ fires, OR fires $\to$ **1** ✓. Hole centre $(1.5, 1.5)$: bottom fails "$y \le 1$", top fails "$y \ge 2$", left fails "$x \le 1$", right fails "$x \ge 2$". No AND fires, OR gets $0 - 1 = -1 \to$ **0** ✓. Edge case $(2, 1.5)$, on the hole's open edge (so in the frame): right strip's "$x \ge 2$" is $H(0) = 1$, the rest fire too $\to$ **1** ✓.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Sheet 1 P5 (14 pts). $C = ([-2, 2] \times [-2, 1]) \setminus ((-1, 1) \times (-1, 0))$: the closed rectangle $-2 \le x \le 2,\ -2 \le y \le 1$ with the open hole $-1 < x < 1,\ -1 < y < 0$ removed. (a) Why can't one perceptron represent C? (b) Write C as a union of at most four rectangles. (c) Give the threshold units for each rectangle's four sides. (d) Explain the AND units and the OR unit. (e) Why does this show MLPs can build general regions from half-plane decisions?

Do it the same way: steps ④–⑤ with these numbers. Sketch C first; note the hole is **not** centred.

✍ Full worked solution: S2.13 (end of this lecture)

⑦ TRY ALONE

(i) Write "$x \le 2.5$" as $(\mathbf{w}, b)$ for an input (x, y). (ii) In the sibling frame, the point $(0.5, 1)$ lies in two strips. Which ones, and does the OR mind? (iii) The L-shape $[0, 2] \times [0, 1] \cup [0, 1] \times [1, 2]$: how many units, unshared and shared?

✍ Answers: S2.14 (end of this lecture)

🧭 **Like cutting a picture-frame mat with a straight ruler and a knife:** you can't cut a hole with one straight cut, but four straight strips glued together make the frame. Lines (layer 1), strips (layer 2), glue (output).

↗ Understand more: Sheet 1 P5 worked (guide) ↗ Try P5 without the answer (guide)

📖 READ & UNDERSTAND · ⏱ 5 min · Tier A

10 · Real-valued outputs: pulses, staircases, and "approximate, never exact"

So far outputs were 0/1. For **regression** (predicting a real number, like a price or a temperature) the output unit must be a **summing** (linear) unit: it outputs its weighted sum as is, with no threshold.

A pulse from 3 units (L2 p127). Two threshold units and a summing output:

$$\text{pulse}(x) = H(x - T_1) - H(x - T_2)$$

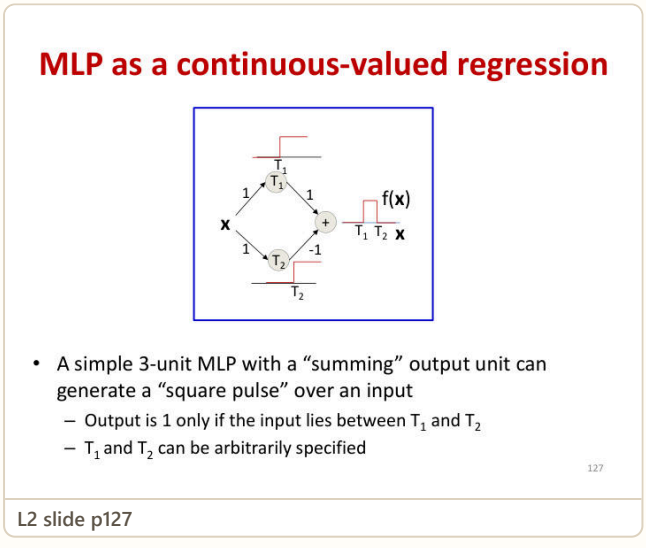
For $x < T_1$ both are 0; between T_1 and T_2 only the first is on (output 1); past T_2 both are on and cancel (output 0). E.g. $T_1 = 0.2$, $T_2 = 0.5$: at $x = 0.3$, $1 - 0 = 1$; at $x = 0.7$, $1 - 1 = 0$. Scale pulses by heights and add many narrow ones side by side, and you trace any function of one input, as closely as you like (L2 p128). In 2-D and higher the same trick uses "cylinders" from card 8 (L2 p129–130). So **a one-hidden-layer MLP is a universal approximator**.

Like a curve drawn in 8-bit Super Mario pixels: a smooth hill becomes a staircase of blocks. Smaller blocks get closer to the true curve, but at any finite size the edges are still steps. That is **approximation, not exact representation** (Poll 4, Sheet 1 P6(f), P7(f)).

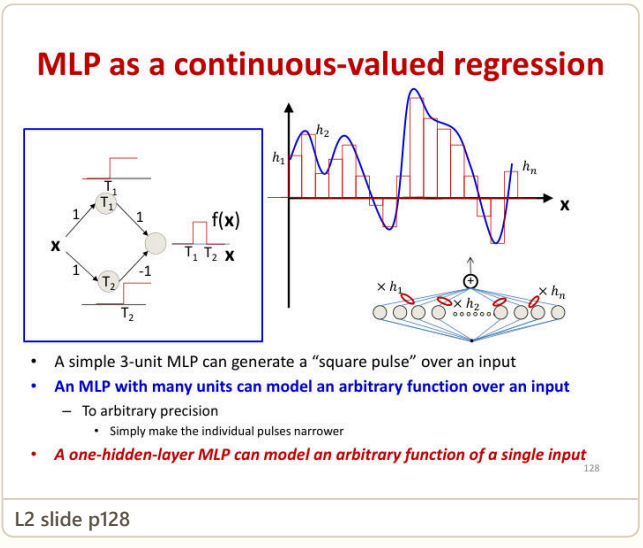
Output activation sets the range (L2 p134–136). The network can produce any map from the inputs onto the range of its output activation, and nothing outside it:

Output unit	Range of the network
Threshold	$\{0, 1\}$: a classifier only
Sigmoid $\sigma(z) = 1/(1 + e^{-z})$	$(0, 1)$: never exactly 0 or 1
tanh	$(-1, 1)$
ReLU $\max(0, z)$	$[0, \infty)$
Softplus $\log(1 + e^{z})$	$(0, \infty)$ (the slide's $[0, \infty)$ is wrong: softplus is never 0)
Summing (none)	all of $\mathbb{R}$: needed for regression

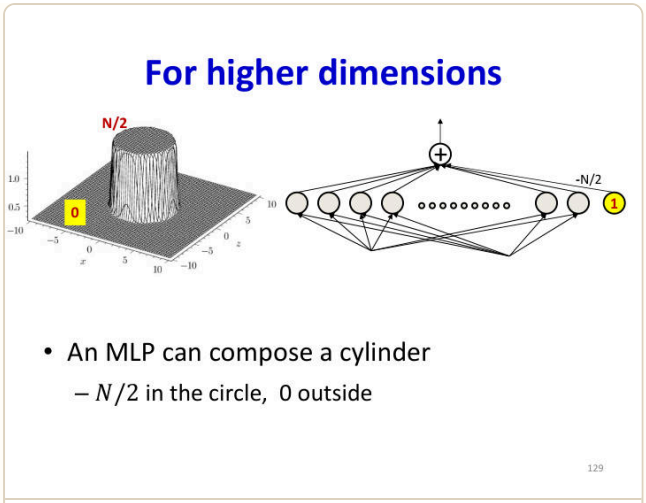
Like a thermostat vs a thermometer: a thermostat (threshold output) only says "heat on / off"; it can never tell you it is 23.4 °C. To report a real number, the output must pass the sum through unchanged.



L2 slide p127



L2 slide p128



L2 slide p129

↗ Understand more: a square pulse from three units (guide) ↗ Output activations decide the range (guide) ↗ d2l.ai: MLPs and universal approximation

PEN & PAPER · ⏱ 17 min · Tier A

11 · Sheet 1 P6: the staircase for a curve, its error bound, and K

① WHAT IS BEING ASKED, IN PLAIN WORDS

Approximate a curve $f(x)$ for x between 0 and 1 by a **staircase**. Each hidden unit $H(x - t_j)$ is a switch that turns on (outputs 1) once x reaches the point $t_j = j/K$; K is the number of steps. The output unit just adds: $g_K(x) = c_0 + c_1 H(x - t_1) + \cdots + c_K H(x - t_K)$, where c_j is how much unit j adds when it turns on. You find the c_j, prove the biggest gap between curve and staircase is at most some number over K, pick K for a gap of at most 0.01, and answer two "why" questions. ($\|f - g\|_\infty$, the "sup norm", is just that **biggest gap** $|f(x) - g(x)|$ over all x.) The exam uses x^2; we learn it on the sibling $f(x) = x^3$.

② WARM-UP BY HAND, NO FORMULAS

(sibling, $K = 4$): switches at $0.25, 0.5, 0.75, 1$. The staircase should sit at the curve's height at the start of each step:

step starts at t	0	0.25	0.5	0.75	1
height t^3	0	0.015625	0.125	0.421875	1
jump from previous	(start 0)	0.015625	0.109375	0.296875	0.578125

At $x = 0.6$: switches 1 and 2 are on ($0.25 \le 0.6, 0.5 \le 0.6$), so $g = 0.015625 + 0.109375 = 0.125$. The true value is $0.6^3 = 0.216$: gap 0.091.

③ THE ONE IDEA

Each unit adds the jump of f at its step: $c_j = f(t_j) - f(t_{j-1})$. Inside one step the curve can rise by at most (step width) × (how steep it gets), so the gap is below (steepest slope)/K.

④ BUILD IT, STEP BY STEP

(sibling, $f(x) = x^3$).

1. For x in step r, i.e. $t_r \le x < t_{r+1}$, units $1, \dots, r$ are on and the rest off, so $g_K(x) = c_0 + c_1 + \cdots + c_r$.
   Why: unit j is on iff $t_j \le x$.
2. We want that to be $f(t_r)$. For $r = 0$: $c_0 = f(0) = 0$. For $r \ge 1$, subtract the condition for $r-1$: $c_r = f(t_r) - f(t_{r-1})$.
   Why: the two sums differ only by c_r.
3. For x^3: $c_j = \dfrac{j^3 - (j-1)^3}{K^3} = \dfrac{3j^2 - 3j + 1}{K^3}$.
   Why: expand $(j-1)^3 = j^3 - 3j^2 + 3j - 1$ and subtract.
4. The network: one input x; hidden unit j has weight 1 and bias $-j/K$; the summing output has weights c_j and bias $c_0 = 0$.
   Why: $H(1 \cdot x - j/K) = H(x - t_j)$.
5. Gap inside step r, where $g_K(x) = t_r^3$: $x^3 - t_r^3 = (x - t_r)(x^2 + x t_r + t_r^2)$.
   Why: the difference-of-cubes factorisation (multiply it out to check).
6. $x - t_r < \frac{1}{K}$ (we are inside one step) and $x^2 + x t_r + t_r^2 < 1 + 1 + 1 = 3$ (both numbers are below 1), so the gap is $< \frac{3}{K}$. At $x = 1$ every unit is on, $g_K(1) = 1$, gap 0. So $\|f - g_K\|_\infty \le 3/K$.
   Why: a product of two smaller positives is smaller.
7. Gap ≤ 0.01: $\frac{3}{K} \le 0.01 \iff K \ge 300$.
   Why: multiply both sides by $100K$. (The exact worst gap is the last jump, $(3K^2 - 3K + 1)/K^3$; at $K = 300$ it is ≈ 0.00997 ✓.)
8. Why a **summing** output: the targets $0, 1/K^3, 8/K^3, \dots$ are real numbers, so the output must pass the sum through unchanged.
   Why: a threshold output only says 0 or 1, a sigmoid squashes into $(0, 1)$.
9. Exact? No: g_K is a staircase with at most $K + 1$ different values, x^3 is continuous with infinitely many.
   Why: so it is **approximation** to any ε (take $K \ge 3/\varepsilon$), never equality.

⑤ **CHECK IT**

(sibling). $K = 4$, step 3: $c_j = (3j^2 - 3j + 1)/64$ gives $1/64, 7/64, 19/64, 37/64 = 0.015625, 0.109375, 0.296875, 0.578125$, the warm-up's jumps ✓. At $x = 0.6$ the gap 0.091 is below $3/4 = 0.75$ ✓. Edge case: the worst gap for $K = 4$ is just under $1 - 0.421875 = 0.578125 = 37/64$ (right before the last switch), still below 0.75 ✓.

⑥ **NOW THE EXAM PROBLEM — YOUR TURN**

EXAM PROBLEM

Sheet 1 P6 (18 pts). $f(x) = x^2$ on $[0, 1]$, threshold hidden units, summing output, $t_j = j/K$. (a)(b) Build $g_K(x) = c_0 + \sum_{j=1}^{K} c_j H(x - t_j)$ with $g_K(x) = f(t_r)$ on each piece $[t_r, t_{r+1})$, and find $c_0, \dots, c_K$. (c) Prove $\|f - g_K\|_\infty \le 2/K$. (d) Smallest K for error at most 10^{-2}? (e) Why must the output be a summing unit? (f) Is x^2 represented exactly?

Do it the same way: steps ④–⑤ with $f(x) = x^2$ (the factorisation you need is the difference of two squares).

✍ Full worked solution: S2.15 (end of this lecture)

⑦ **TRY ALONE**

(i) Sibling x^3 with $K = 10$: what is c_3? (ii) Smallest K that guarantees a gap of at most 0.001 by the $3/K$ bound? (iii) Same construction for the straight line $f(x) = x$: what are the c_j, and what bound replaces $3/K$?

✍ Answers: S2.16 (end of this lecture)

↗ Understand more: the staircase for x^2 (guide) ↗ The error bound and choosing K (guide) ↗ Staircase lab (guide)

📖 **READ & UNDERSTAND · ⏱ 2 min · Tier C**

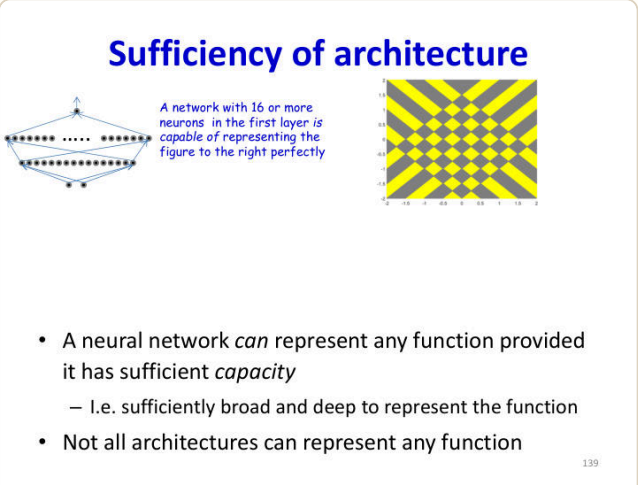
12 · Skim: width, graded activations, capacity (p137 → end)

Skim — the doctor said look at them, he didn't explain them. Read the three bold claims once and move on.

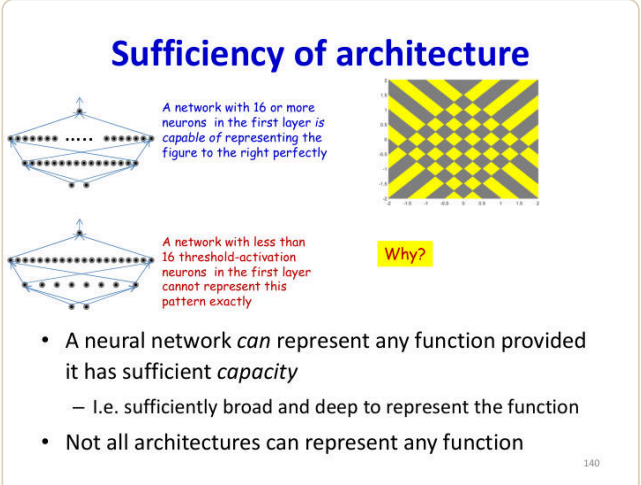
Every layer must be wide enough (p139–145). The 16-line checkerboard needs at least 16 threshold neurons in the first layer, one per line. With only 8, the network knows which cell a point is in but not where inside it, and **no extra depth** can recover that lost information.

🧭 **Like saving a photo as a tiny thumbnail:** once the detail is thrown away, no later edit brings back the true pixels. *Where it breaks:* AI upscalers invent plausible detail; they don't recover the real one.

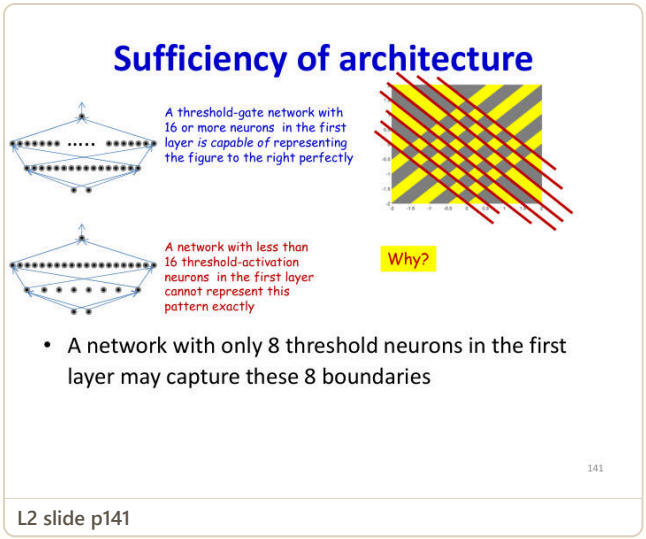
Graded activations help (p146–150). A sigmoid or ReLU unit outputs *how far* the input is from its line, not just which side, so later layers can still use that information. **Capacity (p153–154)** is how complex a function a network can represent; the **VC dimension** is one formal measure of it.



L2 slide p139



L2 slide p140



L2 slide p141

↗ Understand more: sufficiency of width (guide) ↗ Threshold vs graded units (guide)

🔁 RECALL DRILL · ⏱ 5 min · Tier A

13 · Recall drill: the five polls and Sheet 1 P7

Poll 1 (p13). Mark all true: (i) $3x + 7y$ is a linear combination of x, y; (ii) $3x + 7y + 4$ is a linear combination; (iii) $3x + 7y$ is affine; (iv) $3x + 7y + 4$ is affine.

✍ Answer: S2.17 (end of this lecture)

Poll 2 (p56). Hidden neurons for a one-hidden-layer net on 10 inputs where flipping any one bit always flips the output (the checkerboard K-map)? 20 / 256 / 512 / 1024

✍ Answer: S2.18 (end of this lecture)

Poll 3 (p80). "An XOR network needs two hidden neurons and one output neuron, because we need one hidden neuron for each boundary and an output to AND them." True or false?

✍ Answer: S2.19 (end of this lecture)

Poll 4 (p131). Any real-valued function can be modelled *exactly* by a one-hidden-layer net with infinitely many hidden neurons?

✍ Answer: S2.20 (end of this lecture)

Poll 5 (p151, skim slide). Mark all true: (1) bounded width can model any function if deep enough; (2) graded activations compensate better for insufficient width through depth than threshold/saturating ones; (3) we can always compensate for width and depth limits with more graded activations; (4) for a given accuracy, nets with more graded activations are generally smaller.

✍ Answer: S2.21 (end of this lecture)

Sheet 1 P7: true or false? (a) A one-hidden-layer MLP can represent every Boolean function, though possibly exponentially wide. (b) Because one hidden layer is universal, depth can never reduce the number of neurons. (c) If every hidden layer is too narrow, more depth necessarily fixes it. (d) More graded activations let narrower nets pass more information on. (e) A threshold output unit can directly realise any real-valued regression function. (f) Universal approximation means every real function is represented exactly by some finite one-hidden-layer MLP.

✍ Answer: S2.22 (end of this lecture)

Quick fire. (1) How many perceptrons for the XOR of 6 variables, chained? (2) Threshold for the OR of $x_1, \bar{x}_2, x_3$? (3) What does the pentagon's output compare $\sum y_i$ to? (4) Bias of a circle (cylinder) net with N edge units? (5) Depth of a net is the ____ path from input to output.

✍ Answers: S2.23 (end of this lecture)

⚠ Traps

- **Negated literal, wrong bias.** Each negated literal adds $+1$ to the bias ($\bar{x}_i = 1 - x_i$): the bias is (number of negated literals) $-$ K, not $-K$.
- $H(0) = 1$**.** The unit fires at $z = 0$. The slide (L2 p10) says it fires if the affine function is "positive", but the formula is $z \geq 0$ (non-negative). Edge-case biases (e.g. −1, +1, −2 in the XOR net) only work because of this.
- **XOR proof needs inequalities.** Sheet 1 P2(a) explicitly says a picture is not enough: write the four inequalities and add (2)+(3).
- **Depth = longest path**, not shortest; "deep" = output depth > 2.
- **Poll 3 key (L2 p80–81)** says "True" (XOR needs two hidden neurons), but p33's own 2-neuron XOR (1 hidden + skip connections) contradicts it. True only for strictly layered nets.
- **K-map edges wrap around** and the order is 00, 01, 11, 10 (Gray), not 00, 01, 10, 11. Slide error (L2 p49–50): $\bar{Y}\bar{Z} + \bar{W}X\bar{Y} + \bar{X}Y\bar{Z}$ is correct but not minimal; the four corners give $\bar{X}\bar{Z}$ (still 3 terms, 3 hidden units).
- **Parity counts:** one hidden layer needs 2^{N-1} hidden units ($2^{N-1} + 1$ with the output); deep needs $3(N - 1)$ perceptrons in $2\log_2 N$ layers. Don't mix "stages" ($\log_2 N$) with "threshold layers" ($2\log_2 N$).
- **Slide error (L2 p67):** exponent printed $N/2^{K-1/2}$; it must be $N/2^{(K-1)/2}$, matching $C = 2^{-(K-1)/2}$.
- **Slide error (L2 p68):** "30 parameters" counts only input→hidden edges; the drawn 5-6-1 net has $5 \cdot 6 + 6 = 36$ weights (+7 biases).
- **Slide error (L2 p79):** plot 3 is the XOR band (two lines), shown under "Boolean perceptrons are linear classifiers"; axis letters also swap between plots. XOR is *not* linearly separable.
- **Slide error (L2 p96):** one hexagon region is unlabelled; it should read 4.
- **Circle biases (L2 p104–107, p113):** $-N/2$ is right for one circle; two circles need $-N$, K circles $-KN/2$. p104 writes "≥ 0" and p113 "> 0": inconsistent, either is fine if you say which.
- **Slide error (L2 p135):** softplus range is $(0, \infty)$, not $[0, \infty)$. ReLU's is $[0, \infty)$.
- **Typos (L2 p109–110), names only:** "depth $\log^2(n)$" means $\log_2 n$; "Oliver Dellaleau" is Olivier Delalleau, and "exponentially fewer layers" should read fewer *units*.
- **"Approximate" ≠ "exact".** Universal approximation never means exact representation (Poll 4, P6(f), P7(f)). And universality never means "small" (P7(b)).
- **Regression needs a summing output.** A threshold or sigmoid output limits the range (P6(e), P7(e)).

Solutions and answers — Lecture 2

S2.1 · 2 · Sheet 1 P1: one perceptron for "at least K of N literals" — Hint

Count how many of the six literals are negated: each one adds $+1$ to the constant before you subtract K.

↩ back to the card

S2.2 · 2 · Sheet 1 P1: one perceptron for "at least K of N literals" — Full worked solution

(a) Step 1, plain literals. x_1, x_3, x_5 appear plain, so each literal equals its input.
Why: the input already is its 0/1 truth value.

Step 2, negated literals. $\bar{x}_2 = 1 - x_2$, $\bar{x}_4 = 1 - x_4$, $\bar{x}_6 = 1 - x_6$.
Why: $1 - x$ swaps 0 and 1 (check: $x = 0 \to 1$, $x = 1 \to 0$).

Step 3, the count. Let c = number of true literals:

$$c = x_1 + (1 - x_2) + x_3 + (1 - x_4) + x_5 + (1 - x_6)$$

Why: each true literal adds 1, each false one 0.

Step 4, collect the constants.

$$c = x_1 - x_2 + x_3 - x_4 + x_5 - x_6 + 3$$

Why: three negated literals, each contributing a "1 −", give $+3$ and a minus sign on x_2, x_4, x_6.

Step 5, subtract K = 4.

$$z = c - 4 = x_1 - x_2 + x_3 - x_4 + x_5 - x_6 - 1$$

Why: $z \geq 0$ must mean $c \geq 4$.

Step 6, read off. The perceptron computes $z = \sum_i w_i x_i + b$, so

$$\mathbf{w} = [1, -1, 1, -1, 1, -1], \qquad b = -1$$

(Threshold form: fire iff $\sum_i w_i x_i \geq 1$.)

(b) Proof. For any input $x \in \{0, 1\}^6$, steps 3–5 show the unit's total input is

$$z = x_1 - x_2 + x_3 - x_4 + x_5 - x_6 - 1 = (c - 3) - 1 = c - 4$$

This identity holds for every input, so $y = H(z) = 1 \iff z \geq 0 \iff c \geq 4$. Hence the unit outputs 1 exactly when at least four literals are true. ■

Spot checks:

input	true literals	c	z	y
101100	$x_1, \bar{x}_2, x_3, \bar{x}_6$	4	$1 - 0 + 1 - 1 + 0 - 0 - 1 = 0$	1 ✓
110110	$x_1, \bar{x}_6, x_5$	3	$1 - 1 + 0 - 1 + 1 - 0 - 1 = -1$	0 ✓

(c) General recipe. Let P = indices of plain literals, Q = indices of negated ones, $|P| + |Q| = N$. Weights and bias:

$$w_i = +1 \ (i \in P), \qquad w_i = -1 \ (i \in Q), \qquad b = |Q| - K$$

Proof, the same three steps with letters. The true-literal count is

$$c = \sum_{i \in P} x_i + \sum_{i \in Q} (1 - x_i) = \sum_{i \in P} x_i - \sum_{i \in Q} x_i + |Q|$$

so the unit computes

$$z = \sum_{i \in P} x_i - \sum_{i \in Q} x_i + |Q| - K = c - K$$

and $z \geq 0 \iff c \geq K$. ■ Special cases: $K = N$ is the AND of the literals, $K = 1$ the OR, $Q = \varnothing$ the plain "at least K" gate. Edge cases: $K \leq 0$ always fires, $K > N$ never fires.

(a) $\mathbf{w} = [1, -1, 1, -1, 1, -1]$, $b = 3 - 4 = -1$. (b) $z = c - 4$ for every input, so it fires iff $c \geq 4$ (22 of the 64 inputs: $15 + 6 + 1$). (c) $w_i = +1$ for plain, -1 for negated, $b = |Q| - K$. Any bias in $[-1, 0)$ also works for (a), e.g. -0.5.

↩ back to the card

S2.3 · 2 · Sheet 1 P1: one perceptron for "at least K of N literals" — Answers

(i) $\mathbf{w} = (-1, -1, 1)$, $b = 2 - 2 = 0$. Check $(0, 0, 0)$: two literals true, $z = 0$, fires ✓. (ii) $\mathbf{w} = (1, 1, -1, 1, -1)$, $b = 2 - 3 = -1$. Check $(1, 1, 1, 0, 1)$: true are x_1, x_2 only, count 2; $z = 1 + 1 - 1 + 0 - 1 - 1 = -1$, silent ✓. (iii) $\mathbf{w} = (-1, -1, -1, -1)$, $b = 4 - 1 = 3$: silent only on 1111 ($z = -1$). It is the 4-input **NAND**.

↩ back to the card

S2.4 · 3 · Sheet 1 P2: XOR is impossible for one perceptron, easy for three — Hint

XOR's 1-rows are 01 and 10. Which gate is 1 on "at least one on", and which on "not both on"?

↩ back to the card

S2.5 · 3 · Sheet 1 P2: XOR is impossible for one perceptron, easy for three — Full worked solution

(a) Step 1, the corner values. $z_{00} = b$, $z_{10} = w_1 + b$, $z_{01} = w_2 + b$, $z_{11} = w_1 + w_2 + b$.
Why: plug each input into $z = w_1 x_1 + w_2 x_2 + b$.

Step 2, suppose some w_1, w_2, b work and write the demand of each row ($H(z) = 1$ needs $z \geq 0$, $H(z) = 0$ needs $z < 0$):

$$(0,0) \mapsto 0: \quad b < 0 \qquad (1)$$
$$(1,0) \mapsto 1: \quad w_1 + b \geq 0 \qquad (2)$$
$$(0,1) \mapsto 1: \quad w_2 + b \geq 0 \qquad (3)$$
$$(1,1) \mapsto 0: \quad w_1 + w_2 + b < 0 \qquad (4)$$

Step 3, add (2) and (3).

$$w_1 + w_2 + 2b \geq 0 \quad \Longrightarrow \quad w_1 + w_2 + b \geq -b$$

Why: two true "≥ 0" facts add to a true one; then move one b to the right.

Step 4, use (1). $b < 0$ gives $-b > 0$, hence

$$w_1 + w_2 + b \geq -b > 0$$

which contradicts (4). So no w_1, w_2, b exist: one perceptron cannot compute XOR. ■ (Same fact via the identity: $z_{11} = z_{10} + z_{01} - z_{00}$ = non-negative + non-negative − negative > 0, so 11 would fire.)

(b) Step 5, split XOR into known gates (L2 p60). XOR = "at least one on" AND "not both on" = (OR) AND (NAND).

Why: OR alone is wrong only on 11, and NAND is 0 exactly on 11, so ANDing them removes that one row.

(c) Steps 6–8, every weight and bias.

$$h_1 = H(x_1 + x_2 - 0.5) \quad (\text{OR: } w = (1,1),\ b = -0.5)$$
$$h_2 = H(-x_1 - x_2 + 1.5) \quad (\text{NAND: } w = (-1,-1),\ b = 1.5)$$
$$y = H(h_1 + h_2 - 1.5) \quad (\text{AND: } w = (1,1),\ b = -1.5)$$

Why: OR fires once the sum reaches 1; NAND fires while $x_1 + x_2 \leq 1.5$, i.e. unless both are on; AND fires only when $h_1 + h_2 = 2$. (The slide uses the edge values $b = -1,\ +1,\ -2$, which also work because $H(0) = 1$; the half-way values leave a margin of 0.5.)

(d) Check all four inputs:

x_1x_2	z_1	h_1	z_2	h_2	z_{out}	y	**XOR**
00	−0.5	0	1.5	1	−0.5	0	0 ✓
01	0.5	1	0.5	1	0.5	1	1 ✓
10	0.5	1	0.5	1	0.5	1	1 ✓
11	1.5	1	−0.5	0	−0.5	0	0 ✓

(a) (2)+(3) give $w_1 + w_2 + b \geq -b > 0$, contradicting (4). (b)–(d) $h_1 = H(x_1 + x_2 - 0.5)$ (OR), $h_2 = H(-x_1 - x_2 + 1.5)$ (NAND), $y = H(h_1 + h_2 - 1.5)$ (AND); outputs 0, 1, 1, 0 ✓. Three perceptrons in all.

↩ back to the card

S2.6 · 3 · Sheet 1 P2: XOR is impossible for one perceptron, easy for three — Answers

(i) The output becomes an AND of h_1, h_2, but AND ("both on") and NOR ("both off") never fire together, so the output is **always 0**. (ii) $h_1 = H(0.5) = 1$, $h_2 = H(0.5) = 1$, $y = H(0.5) = 1$ ✓. (iii) E.g. $w = (-1, 1)$, $b = 0.5$: $z = 0.5,\ 1.5,\ -0.5,\ 0.5 \rightarrow$ outputs $1, 1, 0, 1$ ✓. Its only 0-row is 10, so it is "$\bar{x}_1$ OR x_2": one line cuts that corner off.

↩ back to the card

S2.7 · 6 · Sheet 1 P4: reduce the DNF, then build the MLP — Hint

Remember both wrap-arounds: the four corner cells of a 4 × 4 map are all neighbours of each other.

↩ back to the card

S2.8 · 6 · Sheet 1 P4: reduce the DNF, then build the MLP — Full worked solution

Step 1, rows to bits (digits worth 8, 4, 2, 1):

row	**0**	**2**	**5**	**7**	**8**	**10**	**13**	**15**
$x_1x_2x_3x_4$	0000	0010	0101	0111	1000	1010	1101	1111

(a) Step 2, canonical DNF (plain where the bit is 1, barred where it is 0):

$$F = \bar{x}_1\bar{x}_2\bar{x}_3\bar{x}_4 + \bar{x}_1\bar{x}_2x_3\bar{x}_4 + \bar{x}_1x_2\bar{x}_3x_4 + \bar{x}_1x_2x_3x_4$$
$$+x_1\bar{x}_2\bar{x}_3\bar{x}_4 + x_1\bar{x}_2x_3\bar{x}_4 + x_1x_2\bar{x}_3x_4 + x_1x_2x_3x_4$$

(8 terms; "+" means OR.)

(b) Step 3, the K-map (rows x_1x_2, columns x_3x_4, both Gray order 00, 01, 11, 10):

	00	**01**	**11**	**10**
00	**1**	0	0	**1**
01	0	**1**	**1**	0
11	0	**1**	**1**	0
10	**1**	0	0	**1**

Step 4, first block. The **four corners** (0000, 0010, 1000, 1010) touch through both wrap-arounds. In all four $x_2 = 0$ and $x_4 = 0$, while x_1, x_3 change → term $\bar{x}_2\bar{x}_4$.
Why: changing inputs drop out.

Step 5, second block. The **centre 2 × 2** (0101, 0111, 1101, 1111): all have $x_2 = 1,\ x_4 = 1$ → term x_2x_4.

Step 6, result. The two blocks of four cover all eight 1s, so

$$F = \bar{x}_2\bar{x}_4 + x_2x_4$$

(In words: $F = 1$ iff $x_2 = x_4$; x_1 and x_3 don't matter at all.)

(c)(d) Step 7, hidden units ($+1$ plain, -1 barred, 0 absent, bias $= -$(number of plain literals)):

$$h_1 = H(-x_2 - x_4 + 0) \quad \mathbf{w} = (0, -1, 0, -1),\ b = 0 \quad (\bar{x}_2\bar{x}_4)$$
$$h_2 = H(x_2 + x_4 - 2) \quad \mathbf{w} = (0, 1, 0, 1),\ b = -2 \quad (x_2x_4)$$

Step 8, output = OR of the hidden units:

$$y = H(h_1 + h_2 - 1) \quad \text{(weights 1, 1, bias } -1)$$

Checks: 1111 → $h_1 = H(-2) = 0, h_2 = H(0) = 1, y = H(0) = 1$ ✓ (15 is in the set). 1000 → $h_1 = H(0) = 1$, $h_2 = H(-2) = 0, y = 1$ ✓. 0110 (index 6, not in the set) → $h_1 = H(-1) = 0, h_2 = H(-1) = 0, y = H(-1) = 0$ ✓.

(e) Step 9, widths. Canonical DNF network: 8 hidden units (one per minterm, e.g. 0101 → $\mathbf{w} = (-1, 1, -1, 1), b = -2$) + 1 output. Reduced: 2 hidden + 1 output, a factor of 4 fewer hidden units.

> $F = \bar{x}_2\bar{x}_4 + x_2x_4$**. Hidden:** $(0, -1, 0, -1),\ b = 0$ **and** $(0, 1, 0, 1),\ b = -2$**; output** $H(h_1 + h_2 - 1)$**. Width 8 → 2.**

↩ back to the card

S2.9 · 6 · Sheet 1 P4: reduce the DNF, then build the MLP — Answers

(i) $\mathbf{w} = (-1, 0, 1, 0), b = -1$ (one plain literal). On 0010: $z = 0 + 0 + 1 + 0 - 1 = 0$, fires ✓. It fires on rows 2, 3, 6, 7, exactly where $x_1 = 0,\ x_3 = 1$. (ii) Rows 0–3 are 00xx: the whole top row of the K-map → $F = \bar{x}_1\bar{x}_2$, one hidden unit $\mathbf{w} = (-1, -1, 0, 0), b = 0$ (with one term, the OR on top can be dropped). (iii) 0001, 0011, 1001, 1011: always $x_2 = 0,\ x_4 = 1 \rightarrow F = \bar{x}_2x_4$, unit $\mathbf{w} = (0, -1, 0, 1), b = -1$.

↩ back to the card

S2.10 · 7 · Sheet 1 P3: parity, shallow vs deep (the lecture's headline result) — Hint

For (e), name a function where depth gives no saving at all.

↩ back to the card

S2.11 · 7 · Sheet 1 P3: parity, shallow vs deep (the lecture's headline result) — Full worked solution

(a) Step 1, rows. N bits give 2^N rows.
Why: each bit is 0 or 1.

Step 2, 1-rows. Flipping x_1 pairs every odd pattern with an even one, so exactly half the rows have output 1: the canonical DNF has 2^{N-1} minterms.

Step 3, no merging. A term missing some variable x_j would also be true on the pattern with x_j flipped, which has the opposite parity (output 0). So every term must be a full minterm, and the one-hidden-layer network needs

$$2^{N-1} \text{ hidden units} \quad (+1 \text{ OR output}), \qquad = 2^{2^m-1} \text{ for } N = 2^m$$

(b) Step 4. $N = 8$: $2^{8-1} = 2^7 = \mathbf{128}$ hidden units (129 perceptrons with the output).

(c) Steps 5–6, the tree. Each module is P2's net: $h_1 = H(a + b - 0.5)$ (OR), $h_2 = H(-a - b + 1.5)$ (NAND), out $= H(h_1 + h_2 - 1.5) = a \oplus b$. Stage 1 XORs the pairs $(x_1, x_2), (x_3, x_4), (x_5, x_6), (x_7, x_8)$; stage 2 XORs pairs of stage-1 outputs; the single last module outputs P_N. *Why it is correct:* $\oplus$ is associative and commutative, so $((x_1 \oplus x_2) \oplus (x_3 \oplus x_4)) \oplus \cdots = x_1 \oplus \cdots \oplus x_N$.

(d) Step 7, counts for $N = 8$:

- (i) modules: stage 1 has 4, stage 2 has 2, stage 3 has 1: $4 + 2 + 1 = \mathbf{7} = N - 1$ (each module turns two values into one, so 8 values → 1 takes 7 merges).
- (ii) hidden units inside them: $2 \times 7 = \mathbf{14}$ (21 perceptrons if you also count each module's output unit).
- (iii) stages: $8 \to 4 \to 2 \to 1$, so $\log_2 8 = \mathbf{3}$ (each stage is 2 threshold layers, so 6 layers in all, $2 \log_2 N$, L2 p65).
- (iv) $N - 1$ modules, $3(N - 1)$ perceptrons: **linear** growth, $\Theta(N)$.

Check on one input: 10110010 has four 1s, so $P_8 = 0$. Stage 1 pairs (1,0)(1,1)(0,0)(1,0) → 1, 0, 0, 1; stage 2 pairs (1,0)(0,1) → 1, 1; stage 3 (1,1) → 0 ✓.

(e) Step 8, the meaning. For parity, a one-hidden-layer network needs **exponentially** many units (2^{N-1}; 128 at $N = 8$), while a network of depth $2 \log_2 N$ needs only **linearly** many ($3(N - 1)$; 21 at $N = 8$). So for some functions, extra depth buys an exponential saving in width. It does **not** show deep always wins: an AND of all inputs is one unit at any depth, and Shannon's theorem (L2 p72) says almost all Boolean functions need about $2^n/n$ gates whatever the depth.

(a) $2^{N-1} = 2^{2^m-1}$. (b) 128. (c) balanced tree of P2 XOR modules. (d) 7 modules, 14 internal hidden units, 3 stages (6 threshold layers), $N - 1$ modules = linear. (e) depth can trade exponential width for linear size, for functions like parity, not for all functions.

↩ back to the card

S2.12 · 7 · Sheet 1 P3: parity, shallow vs deep (the lecture's headline result) — Answers

(i) $2^{31} = 2147483648$ hidden units; 31 modules in 5 stages, $3 \times 31 = 93$ perceptrons. (ii) $2^9 = 512$ hidden units; still $N - 1 = 9$ modules (every module removes one value); stages: $10 \to 5 \to 3 \to 2 \to 1$, i.e. $4 = \lceil \log_2 10 \rceil$. (iii) $2^5 = 32$ hidden units; 5 modules; $6 \to 3 \to 2 \to 1$, 3 stages.

↩ back to the card

S2.13 · 9 · Sheet 1 P5: the picture frame (a rectangle with a hole) — Full worked solution

(a) Step 1, convexity proof. One unit fires on a half-plane, and a half-plane is **convex**: if $\mathbf{p}_1, \mathbf{p}_2$ both satisfy $\mathbf{w}^\top \mathbf{p} + b \geq 0$, so does their midpoint $\mathbf{m}$, because

$$\mathbf{w}^\top \mathbf{m} + b = \tfrac{1}{2}(\mathbf{w}^\top \mathbf{p}_1 + b) + \tfrac{1}{2}(\mathbf{w}^\top \mathbf{p}_2 + b) \geq 0$$

Take $\mathbf{p}_1 = (-1.5, -0.5)$ and $\mathbf{p}_2 = (1.5, -0.5)$, both in C (left and right of the hole). Any unit firing on all of C fires at both, so it fires at their midpoint $(0, -0.5)$. But that point is inside the hole ($-1 < 0 < 1$, $-1 < -0.5 < 0$), not in C. Contradiction: C is not convex, so no single unit can represent it.

(b) Step 3, four closed strips (the hole is open, so its edges belong to C and the strips may include them):

- Bottom $[-2, 2] \times [-2, -1]$ · Top $[-2, 2] \times [0, 1]$ · Left $[-2, -1] \times [-1, 0]$ · Right $[1, 2] \times [-1, 0]$

Why these: below the hole ($y \leq -1$) and above it ($y \geq 0$) the frame is the full width; beside the hole ($-1 \leq y \leq 0$) it is the two side pieces.

(c) Steps 2 and 4, side units. "$x \geq a$" is $H(x - a)$; "$x \leq a$" is $H(-x + a)$; same for y. Since $H(0) = 1$, edges are included.

Rectangle	left side	right side	bottom side	top side
Bottom	$H(x + 2)$	$H(-x + 2)$	$H(y + 2)$	$H(-y - 1)$
Top	$H(x + 2)$	$H(-x + 2)$	$H(y)$	$H(-y + 1)$
Left	$H(x + 2)$	$H(-x - 1)$	$H(y + 1)$	$H(-y)$
Right	$H(x - 1)$	$H(-x + 2)$	$H(y + 1)$	$H(-y)$

Example as $(\mathbf{w}, b)$: the left strip's "$x \leq -1$" is $\mathbf{w} = (-1, 0), b = -1$.

(d) Steps 5–7, AND and OR. Each rectangle's **AND** unit: weights $1, 1, 1, 1$ on its four side units, bias -4 (fires only if all four fire, i.e. the point is inside that rectangle). The **OR** output: weights $1, 1, 1, 1$ on the four AND units, bias -1 (fires if any rectangle fires). Total $16 + 4 + 1 = 21$ threshold units in two hidden layers + output (only 10 different half-planes, so 15 units if layer 1 is shared).

Checks: $(1.5, -0.5)$: right strip's four sides give $z = 0.5, 0.5, 0.5, 0.5$, all fire, AND fires, output 1 ✓. Hole centre $(0, -0.5)$: bottom fails $y \leq -1$, top fails $y \geq 0$, left fails $x \leq -1$, right fails $x \geq 1$; no AND fires, OR has $z = -1$, output 0 ✓. $(1, -0.5)$, on the hole's open edge: right strip's $H(x - 1) = H(0) = 1$, output 1 ✓.

(e) Step 8. Layer 1 only answers "which side of this line?". Layer 2 ANDs those answers into convex pieces. The output ORs the pieces into a region that is neither convex nor hole-free. Nothing used this particular shape, so any finite union of convex polygons can be built the same way: MLPs build general regions out of half-plane decisions.

(a) one unit = half-plane = convex; $(\pm 1.5, -0.5) \in C$ but their midpoint $(0, -0.5)$ is in the hole. (b) bottom, top, left, right strips. (c) table above. (d) AND: weights 1, bias −4; OR: weights 1, bias −1; 21 units (16 + 4 + 1). (e) lines → convex pieces → union.

↩ back to the card

S2.14 · 9 · Sheet 1 P5: the picture frame (a rectangle with a hole) — Answers

(i) $\mathbf{w} = (-1, 0)$, $b = 2.5$: $z = -x + 2.5$. (ii) Bottom ($y = 1 \leq 1$) and left ($y = 1 \geq 1$): two ANDs fire, the OR gets $2 - 1 = 1 \geq 0$ and outputs 1. An OR doesn't mind overlaps. (iii) 2 rectangles: $8 + 2 + 1 = 11$ units. Distinct lines: $x \geq 0$, $x \leq 2$, $x \leq 1$, $y \geq 0$, $y \leq 1$, $y \geq 1$, $y \leq 2$ = 7, so $7 + 2 + 1 = 10$ shared.

↩ back to the card

S2.15 · 11 · Sheet 1 P6: the staircase for a curve, its error bound, and K — Full worked solution

(a)(b) Step 1, which units are on. For $x \in [t_r, t_{r+1})$, unit j is on iff $t_j \leq x$, i.e. for $j = 1, \ldots, r$. So there

$$g_K(x) = c_0 + c_1 + \cdots + c_r$$

Step 2, match the heights. We need this to equal $f(t_r)$ for every r:

$$r = 0: \quad c_0 = f(0) = 0$$

$$r \geq 1: \quad c_r = (c_0 + \cdots + c_r) - (c_0 + \cdots + c_{r-1}) = f(t_r) - f(t_{r-1})$$

Why: consecutive conditions differ only by c_r, so each unit adds the **jump** of f at its step.

Step 3, plug in x^2.

$$c_j = \frac{j^2}{K^2} - \frac{(j-1)^2}{K^2} = \frac{j^2 - (j^2 - 2j + 1)}{K^2} = \frac{2j - 1}{K^2}$$

Check: $c_1 + \cdots + c_r = (1 + 3 + \cdots + (2r-1))/K^2 = r^2/K^2 = t_r^2$ ✓ (odd numbers add up to squares). Example: $K = 4$ gives $c_j = 1/16, 3/16, 5/16, 7/16$.

Step 4, the network. One input, K hidden units with weight 1 and bias $-j/K$, summing output with weights $(2j-1)/K^2$ and bias $c_0 = 0$.

(c) Step 5, factor the gap. Take $x \in [t_r, t_{r+1})$, where $g_K(x) = t_r^2$:

$$0 \leq x^2 - t_r^2 = (x - t_r)(x + t_r)$$

Step 6, bound each factor. $x - t_r < 1/K$ (inside one piece) and $x + t_r < 2$ (both below 1), so

$$x^2 - t_r^2 < \frac{1}{K} \cdot 2 = \frac{2}{K}$$

At $x = 1$ all units are on: $g_K(1) = \sum c_j = 1 = f(1)$, error 0. So $\|f - g_K\|_\infty \leq 2/K$. ■ (The exact largest gap is the last jump, $(2K-1)/K^2$, approached on the last piece.)

(d) Step 7. $2/K \leq 0.01 \iff K \geq 200$. Check with the exact gap: $K = 199$ gives $397/39601 \approx 0.010025 > 0.01$; $K = 200$ gives $399/40000 = 0.009975$ ✓. So $K = 200$ either way.

(e) Step 8. The targets $0, 1/K^2, 4/K^2, \ldots$ are real numbers, so the output must form a weighted sum and pass it through unchanged. A threshold output could only say 0 or 1 (a classifier). A sigmoid output squashes into $(0, 1)$ and never reaches the endpoint values exactly.

(f) Step 9. No. With K units, g_K is a staircase: constant except at K jumps, so it takes at most $K + 1$ values. x^2 is continuous and takes infinitely many values on $[0, 1]$. A function with finitely many values cannot equal it. What is proved is **approximation**: for every $\varepsilon > 0$, $K \geq 2/\varepsilon$ gets the error below ε.

$c_0 = 0$, $c_j = (2j-1)/K^2$; **error** $= (x - t_r)(x + t_r) < 2/K$; $K = 200$; **output must be summing (real-valued targets); (f) No: finitely many values vs a continuous curve, only** ε-

approximation.

↩ back to the card

S2.16 · 11 · Sheet 1 P6: the staircase for a curve, its error bound, and K — Answers

(i) $c_3 = (3 \cdot 9 - 3 \cdot 3 + 1)/1000 = 19/1000 = 0.019$. (ii) $3/K \le 0.001 \iff K \ge 3000$. (iii) Every jump is $t_j - t_{j-1} = 1/K$, so $c_j = 1/K$; the gap is $x - t_r < 1/K$ (a line rises at slope 1).

↩ back to the card

S2.17 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

(i), (iii), (iv) true. (ii) false: a linear combination has no constant. (iii) is true because every linear function is affine (constant 0).

↩ back to the card

S2.18 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

512 $= 2^{10-1}$: half of the 1024 rows are 1s and none can merge.

↩ back to the card

S2.19 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

The slide's key says **True**, which holds for strictly layered nets. But p33 shows a 2-neuron XOR (1 hidden + 1 output, with skip connections), so in general it is False. In the exam: give the key's answer and add one line, "true for strictly layered nets; with skip connections 1 hidden neuron suffices (p33)".

↩ back to the card

S2.20 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

False: it can only be **approximated** (to any precision).

↩ back to the card

S2.21 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

(2) and (4). (1) is false in the lecture's logic: a too-narrow layer loses information for good. (3) "always" is too strong.

↩ back to the card

S2.22 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answer

(a) **True**: DNF, one unit per minterm; parity needs 2^{N-1}. (b) **False**: universality is about existence, not size; parity is $2^{N-1} + 1$ shallow vs $3(N-1)$ deep. (c) **False**: lost information can't be recovered ("necessarily" is the giveaway). (d) **True**: graded outputs carry "how far", not just "which side". (e) **False**: the output is stuck in $\{0, 1\}$. (f) **False**: approximation to any ε, not exact.

↩ back to the card

S2.23 · 13 · Recall drill: the five polls and Sheet 1 P7 — Answers

(1) $3 \times 5 = 15$ (5 XORs, 3 perceptrons each; $3(N-1)$). (2) $L - N + 1 = 2 - 3 + 1 = 0$: fire iff $x_1 - x_2 + x_3 \geq 0$. (3) 5 (all five edges); with 4 you get a star. (4) $-N/2$ for one circle. (5) Longest.

↩ back to the card

LECTURE 3

Learning the network: ERM, the perceptron algorithm, ADALINE, MADALINE, smooth activations

Tier A · must ⏱ 137 min Slides: lec3.learning.pdf (138 p) + Assignment 3 CMU video (Lecture 3)

Guide tutorial 7 Guide tutorial 8

Lectures 1–2 showed that a network *can* represent almost anything if the weights are right. Lecture 3 asks the real question: **how do we find those weights from examples?** It tries the old answers (the perceptron rule, ADALINE, MADALINE), shows exactly why they break on multi-layer networks, and ends with the modern recipe: smooth activations + a smooth error averaged over the training data (ERM), which is what gradient descent and backprop (Lectures 4–5) then minimise.

What the exam can ask from this lecture

- **Run the perceptron algorithm by hand** (Assignment 3 P1): every update, the final boundary, R, the margin, the $(R/\gamma)^2$ bound.
- **One perceptron update vs one ADALINE update** on the same input (Assignment 3 P2), and show ADALINE is gradient descent.
- **MADALINE**: which hidden unit to flip and why (Assignment 3 P3); why it is greedy, not gradient descent.
- **ERM by hand**: minimise $\frac{1}{N}\sum\frac{1}{2}(wx_i - d_i)^2$, get $w^* = \sum x_i d_i / \sum x_i^2$ (Assignment 3 P4); empirical vs true risk.
- Why threshold networks can't be trained by small changes (zero derivative), why a sigmoid fixes it, $\sigma' = \sigma(1-\sigma)$, logistic regression = $P(y=1 \mid x)$.
- The three class polls (Polls 1–3) — CMU reuses them in quizzes.

⌛ **Only 20 minutes?** Card 4 (perceptron by hand), then card 7 (perceptron vs ADALINE): in each, read ① and ③, skim the twin build ④, then do the exam problem ⑥ yourself and check it against the full worked solution — skip ② and ⑦. Then read the Traps box. Those two hand-runs are the most likely numericals.

📖 READ & UNDERSTAND · ⏱ 9 min · Tier A

1 · Learning = choosing W so the network matches examples (ERM)

A network is a function with knobs. Write it $Y = f(X; W)$: X is the input, Y the output, and W is **every weight and every bias** in the network. The architecture (how many layers and units) is fixed beforehand; **learning means setting** W (L3 p11).

There is some true function $g(X)$ we want to copy (e.g. "is this a cat?"). We never see g fully. We only get **training samples** $(X_1, d_1), \ldots, (X_N, d_N)$, where d_i is the desired output (label), and labels can be noisy: $d_i = g(X_i) + \text{noise}$ (L3 p25).

To measure "how wrong" we use a **divergence** $\text{div}(f, d)$: a number that is 0 when output equals target, > 0 otherwise, and — the key new requirement — **differentiable** in f (has a slope you can compute). Example: $\frac{1}{2}(f-d)^2$. The 0/1 "right or wrong" count fails the last test (L3 p128–129).

What we truly want small is the **risk** (true risk): the average divergence over *all* possible inputs, weighted by how likely each input is ($P(X)$ is that probability):

$$R(W) = \mathbb{E}\big[\,\text{div}(f(X;W), g(X))\big]$$

We can't compute it (it needs g everywhere). So we use the **empirical risk** — the average over the training set — and minimise that. This is **Empirical Risk Minimisation (ERM)** (L3 p132–137):

$$\text{Loss}(W) = \frac{1}{N}\sum_{i=1}^{N} \text{div}\left(f(X_i; W),\, d_i\right), \qquad \hat{W} = \underset{W}{\text{argmin}}\, \text{Loss}(W)$$

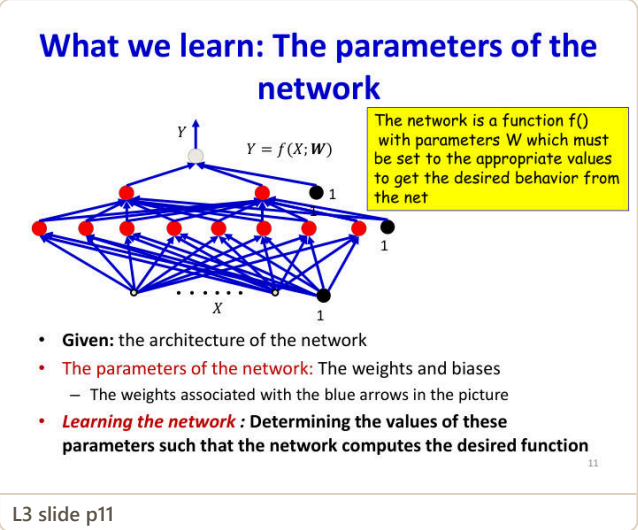
E.g. two samples with targets $d = 1$ and $d = 0$, current outputs 0.8 and 0.3, divergence $\frac{1}{2}(f-d)^2$: $\text{Loss} = \frac{1}{2}\left(\frac{1}{2}(0.2)^2 + \frac{1}{2}(0.3)^2\right) = \frac{1}{2}(0.02 + 0.045) = 0.0325$. Change W, the outputs change, the number changes.

"argmin" means "the W that makes it smallest" (the location, not the smallest value). The slides call it a "Loss" even though it is an error; for a fixed training set it is a function of W only.

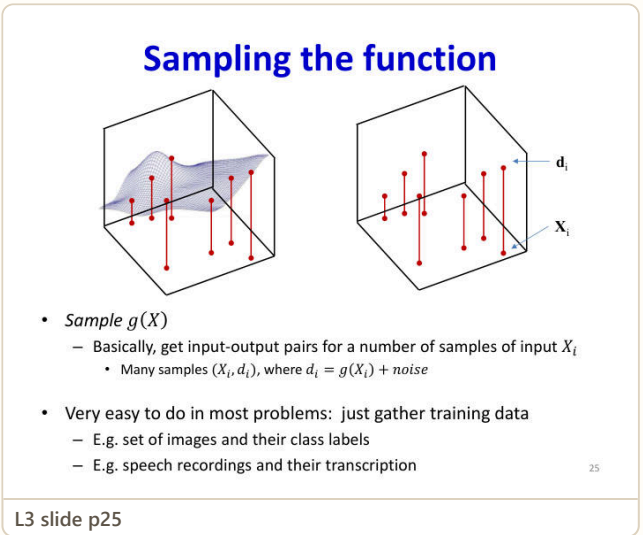
Like tasting soup with a spoon. You want to know if the whole pot (true risk) is salty enough, but you can only taste a few spoonfuls (training samples). Adjusting the salt until the spoonfuls taste right is ERM. More spoonfuls, drawn from all over the pot, give a better judgement. *Where it breaks:* if a spoonful has a lump of salt in it (a noisy or wrong label), you might fix the pot for that one spoonful and ruin it elsewhere — exactly the "empirical ≠ true risk" gap of card 2.

Like ChatGPT's training. The model has billions of knobs (W); it is shown a huge set of text examples and the knobs are tuned to make its next-token predictions match the real next tokens *on those examples*. That is ERM at giant scale — nobody can evaluate it on "all possible text".

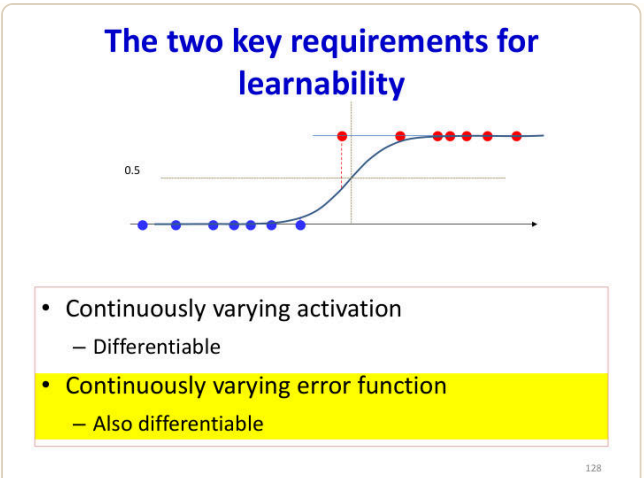
Poll 1 in one line: "Universal approximator" does **not** mean any network of any size works — it needs enough capacity (units). And the weights can **not** be learned perfectly from just a few samples.



L3 slide p11



L3 slide p25



L3 slide p128

↗ Understand more: guide 7.3 — training data and the empirical error ↗ guide 8.15 — divergence, risk, ERM ↗ Wikipedia: Empirical risk minimization

PEN & PAPER · ⏱ 22 min · Tier A

2 · Assignment 3 P4 — ERM by hand for $f(x;w)=wx$

① WHAT YOU'RE BEING ASKED, IN PLAIN WORDS

The "network" here has **one knob**, a single number w. Given an input number x it outputs $f(x;w)=wx$: a straight line through the origin whose slope is w. We get four training examples (x_i, d_i): x_i is the input, d_i the **desired output** (the label). For each example the **miss** is $wx_i - d_i$ and the penalty is $\frac{1}{2}(\text{miss})^2$ (half the squared miss; squaring makes it positive, the ½ only tidies the derivative). The **empirical risk** $L(w)$ is the average penalty over the four examples. Task: find the w that makes $L(w)$ smallest (call it w^*), then explain what this can and cannot promise about new data, and what one bad label does.

② WARM-UP BY HAND, NO FORMULAS

— on a smaller twin of the exam data: **three** examples $(1, 1.1), (2, 1.9), (3, 3.2)$. Just try a few slopes and score them:

x	d	$w=0.9$**: output, miss, ½miss²**	$w=1$**: output, miss, ½miss²**	$w=1.1$**: output, miss, ½miss²**
1	1.1	0.9, −0.2, 0.020	1, −0.1, 0.005	1.1, 0, 0
2	1.9	1.8, −0.1, 0.005	2, 0.1, 0.005	2.2, 0.3, 0.045
3	3.2	2.7, −0.5, 0.125	3, −0.2, 0.020	3.3, 0.1, 0.005
average (÷3)		$L(0.9)=0.05$	$L(1)=0.01$	$L(1.1)\approx 0.0167$

L goes 0.05 → 0.01 → 0.0167 as w goes 0.9 → 1 → 1.1: **down, then up — a bowl**, with its bottom a bit to the right of $w=1$.

③ THE ONE IDEA

$L(w)$ is a bowl-shaped curve (a parabola) in w. At the bottom of a bowl the **slope is zero**. So: write the slope $L'(w)$ (the **derivative**, "how fast L changes when w is nudged"), set it to

0, solve for w.

④ BUILD IT, STEP BY STEP

(on the twin data, $N = 3$).

1. Empirical risk: $L(w) = \frac{1}{3}\left[\frac{1}{2}(w-1.1)^2 + \frac{1}{2}(2w-1.9)^2 + \frac{1}{2}(3w-3.2)^2\right]$.
   Why: one "½ miss²" per example (the table, with w left as a letter), then the average — that is all "empirical risk" means.
2. One example's slope in w: $(wx_i - d_i)\cdot x_i$.
   Why: chain rule — for $\frac{1}{2}u^2$ the slope is u times the slope of u; here $u = wx_i - d_i$, and nudging w by 1 changes u by x_i. (E.g. example 3 at $w=1$: $(3-3.2)\times 3 = -0.6$.)
3. Average: $L'(w) = \frac{1}{N}\sum_i (wx_i - d_i)\,x_i$.
   Why: the slope of an average is the average of the slopes.
4. Split the sum: $L'(w) = \frac{1}{N}\left(w\sum_i x_i^2 - \sum_i x_i d_i\right)$.
   Why: $(wx_i - d_i)x_i = wx_i^2 - x_i d_i$, and w is the same in every term so it comes out.
5. Set it to 0: $w^* = \dfrac{\sum_i x_i d_i}{\sum_i x_i^2}$.
   Why: bottom of the bowl ⇔ slope 0.
6. It is a bottom, not a top: $L''(w) = \frac{1}{N}\sum_i x_i^2 > 0$.
   Why: a positive second derivative means the curve bends upward.
7. Numbers: $\sum x_i d_i = 1(1.1) + 2(1.9) + 3(3.2) = 1.1 + 3.8 + 9.6 = 14.5$ and $\sum x_i^2 = 1+4+9 = 14$, so $w^* = 14.5/14 \approx 1.0357$.
   Why: compute the two sums separately, then divide — fewer slips.
8. Empirical vs true risk: L averages only these 3 noisy examples; the true risk averages over *every* input the world can produce, against the true g.
   Why: a different 3 examples would give a different w^*, so w^* fits *this sample*, noise included.
9. One corrupted label: change d_k to $d_k + \Delta$. Only the top sum contains d_k, through the single term $x_k d_k$, so $w^*_{\text{new}} = \dfrac{\sum x_i d_i + x_k\Delta}{\sum x_i^2} = w^* + \dfrac{x_k\Delta}{\sum x_i^2}$.
   Why: this gives the shift as a formula — and shows it grows with x_k (leverage).
10. Twin numbers: corrupt d_3 from 3.2 to 10.2 ($k=3$, $x_3=3$, $\Delta = 7$): shift $= 3\times 7/14 = 1.5$, new $w^* \approx 2.5357$.
    Why: g did not change, yet ERM followed the one bad point, because squared loss makes a miss of ≈7 count ≈49 times as much as a miss of 1.

⑤ CHECK IT

$L(1.0357) \approx 0.0070$, below $L(1) = 0.01$ ✓. Sign check: $L'(1) = \frac{1}{3}(14 - 14.5) \approx -0.167 < 0$, the bowl still slopes down at $w=1$, so the bottom is a little to the right ✓ — just as the warm-up table suggested. Leverage check: the same $\Delta = 7$ on d_1 ($x_1 = 1$) shifts w^* by only $7/14 = 0.5$, a third as much ✓.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

A scalar regression network is $f(x; w) = wx$ and the training set is $(1, 1.2)$, $(2, 1.9)$, $(3, 3.2)$, $(4, 3.9)$. Use squared divergence $\text{div}(f(x;w), d) = \frac{1}{2}(f(x;w) - d)^2$.

a. Write the empirical risk $L(w) = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{2}(wx_i - d_i)^2$.

b. Differentiate $L(w)$ and solve exactly for the empirical-risk minimiser w^*.

c. Compute the numerical value of w^*.

d. Explain carefully why minimising empirical risk does not necessarily minimise the unknown true risk $\mathbb{E}[\text{div}(f(X;w), g(X))]$.

e. Suppose one training target is replaced by a highly corrupted value. Explain how this can move the empirical minimiser even though the underlying target function $g(x)$ has not changed. (Worked with numbers: d_4 corrupted from 3.9 to 13.9.)

Do it the same way: steps ④–⑤ with these numbers (four examples, so $N = 4$).

✍ Hint: S3.1 (end of this lecture)

✍ Full worked solution: S3.2 (end of this lecture)

⑦ TRY ALONE

(1) Data $(1, 2)$, $(2, 4)$: what is w^*? (2) Data $(1, 1)$, $(2, 3)$: w^*? (3) In the exam data, corrupt d_1 by $+10$ (1.2 → 11.2) instead of d_4: new w^*?

✍ Answers: S3.3 (end of this lecture)

🧭 **Like one fake 1-star review on a product with four reviews.** The average rating (the empirical estimate) crashes, but the product (the truth, g) didn't change. And a review from a "verified heavy buyer" (large x_k) moves the score more than one from a casual buyer.

↗ Understand more: guide 7.4 — Assignment 3 P4 worked ↗ guide 7.5 — P4 practice

📖 READ & UNDERSTAND · ⏱ 10 min · Tier A

3 · The perceptron algorithm: learn only from mistakes

A **perceptron** is one unit: it computes a weighted sum $z = \mathbf{w}^\top \mathbf{x} + b$ (the dot product $\mathbf{w}^\top \mathbf{x} = w_1 x_1 + w_2 x_2 + \ldots$) and outputs the class by its sign: $+1$ if $z > 0$, -1 otherwise. Its **decision boundary** is the line (or plane) $\mathbf{w}^\top \mathbf{x} + b = 0$.

Bias trick. Append a constant 1 to every input and the bias to the weights: $\tilde{\mathbf{x}} = [x_1, x_2, 1]$, $\tilde{\mathbf{w}} = [w_1, w_2, b]$. Then $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$ with no separate b, and the boundary passes through the origin of this bigger space (L3 p38). Careful: a unit that "fires when $\mathbf{w}^\top \mathbf{x} \geq T$" has $b = -T$.

Geometry. $\mathbf{w}$ is **perpendicular to the boundary** and points into the $+1$ side. A point is correctly classified when its angle with $\mathbf{w}$ is under 90° (positive class) or over 90° (negative class) (L3 p40–42).

The algorithm (Rosenblatt, L3 p43–45), labels $y_i \in \{+1, -1\}$, start from $W = 0$:

- Cycle through the training points. For each, compute $O(X_i) = \text{sign}(W^\top X_i)$.
- If it's wrong, $W \leftarrow W + y_i X_i$: add a misclassified positive, subtract a misclassified negative. If it's right, do nothing.
- Stop after a full pass with no mistakes.

Assignment 3 writes the same thing with a learning rate η and labels d_i: if $d_i(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i) \le 0$ then $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, d_i\, \tilde{\mathbf{x}}_i$. The quantity $d_i(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i)$ is the **signed score**: positive = right side. **A score of exactly 0 counts as a mistake.**

Why an update helps that point. Substitute the update into the signed score and use $d^2 = 1$:

$$d\,(\tilde{\mathbf{w}} + \eta d\tilde{\mathbf{x}})^\top \tilde{\mathbf{x}} = d\,\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} + \eta\,\|\tilde{\mathbf{x}}\|^2$$

So each update raises that point's signed score by $\eta\|\tilde{\mathbf{x}}\|^2$ ($\|\cdot\|$ is the length). *E.g.* $\tilde{\mathbf{x}} = [3, 1, 1]$, $\eta = 1$: $\|\tilde{\mathbf{x}}\|^2 = 9 + 1 + 1 = 11$, so a signed score of -2 becomes 9. It can still break *other* points.

Convergence (L3 p57). If the classes are **linearly separable** (some line splits them perfectly) and W starts at 0, the algorithm stops after at most

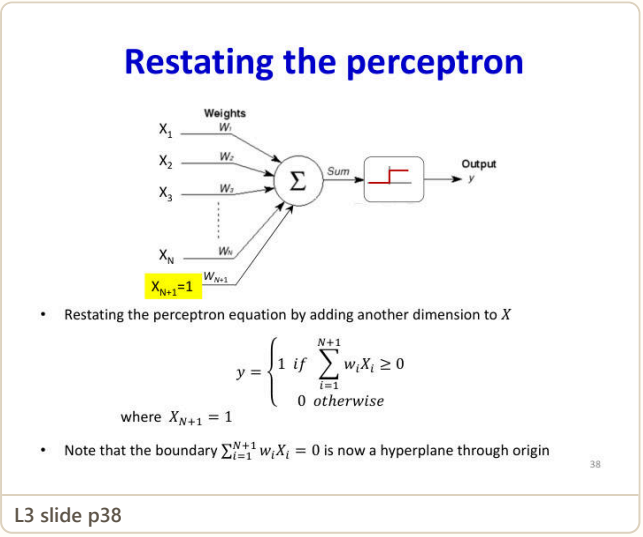
$$\left(\frac{R}{\gamma}\right)^2 \text{ mistakes}$$

R = length of the longest training vector; γ = the **margin**, the best-case closest distance of any point to a separating boundary (same as an SVM's margin). *E.g.* $R = 2$, $\gamma = 0.5$: at most $(2/0.5)^2 = 16$ mistakes. Intuition from the slide: it takes many increments of size γ to undo an error made by a step of size R. Proof idea: after k updates, progress along the true direction $\ge k\gamma$, but length $\le \sqrt{k}R$; so $k\gamma \le \sqrt{k}R$, i.e. $k \le (R/\gamma)^2$.

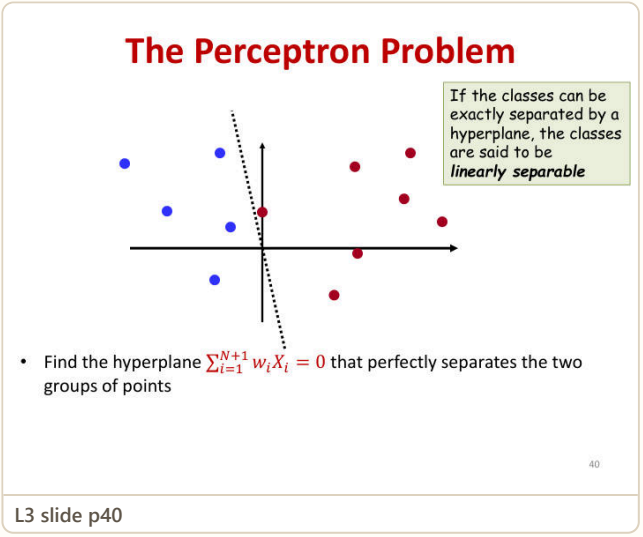
Not separable? Some point is always on the wrong side, so the algorithm **never converges** (L3 p59). **Reflection trick** (p60–61): replace each point by $X' = yX$ (negate the negatives); then we just need one W with $W^\top X'_i > 0$ for all i.

Like your email spam filter. It only changes when you hit "Report spam" or "Not spam" — i.e. only on its mistakes — and each correction nudges the weights of the words in *that* email towards the right answer. Emails it already got right teach it nothing. *Where it breaks:* a real filter outputs a probability and learns from every email; the perceptron is the pure "mistakes only" version.

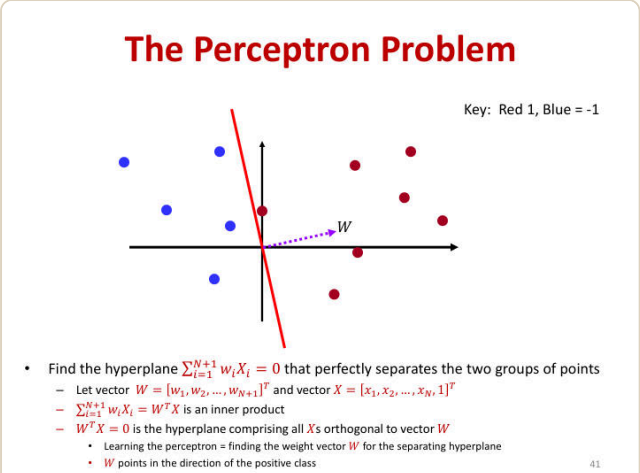
$(R/\gamma)^2$ is like parallel parking. A big car (large R, big steering corrections) in a tight gap (small γ) needs many back-and-forth moves; a small car in a wide gap goes in at once. If no gap exists (not separable), you shuffle forever.



L3 slide p38



L3 slide p40



L3 slide p41

↗ Understand more: guide 7.8 — add the mistakes ↗ guide 7.10 — the $(R/\gamma)^2$ bound 🎥 CMU L3 video — follow slides p43–63 ↗ Wikipedia: Perceptron

🖍 PEN & PAPER · ⏱ 24 min · Tier A

4 · Assignment 3 P1 — run the perceptron by hand

① WHAT YOU'RE BEING ASKED, IN PLAIN WORDS

Four dots on a page: two are labelled $+1$, two are labelled -1. A perceptron draws a straight line $w_1 x_1 + w_2 x_2 + b = 0$ and says $+1$ on one side, -1 on the other. Its **score** for a point is $w_1 x_1 + w_2 x_2 + b$: positive $\rightarrow$ it says $+1$. We must **learn** w_1, w_2, b from zero by the perceptron rule, write every change, then measure two lengths (R and the margin γ) that the convergence theorem uses.

The notation, one symbol at a time. d is the true label ($+1$ or -1). The "tilde" vectors just glue the bias in: $\tilde{\mathbf{x}} = [x_1, x_2, 1]$ (the point with a 1 stuck on the end) and $\tilde{\mathbf{w}} = [w_1, w_2, b]$. Then the score is the **dot product** $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$ — multiply matching entries and add: $w_1 x_1 + w_2 x_2 + b \cdot 1$. The **signed score** $d \cdot$ score is positive exactly when the point is on its correct side. η (eta) is the step size, here 1.

The rule. Visit the points in order. If signed score ≤ 0 (wrong side, *or exactly on the line*), do $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, d\, \tilde{\mathbf{x}}$: add the point if it is a $+1$, subtract it if it is a -1. Otherwise change nothing. Stop after a whole pass with no change.

② WARM-UP BY HAND: ONE VISIT

— on a twin data set with three points: $A = (0, 1) \rightarrow +1$, $B = (1, 0) \rightarrow +1$, $C = (0, -1) \rightarrow -1$. Start from $\tilde{\mathbf{w}} = [0, 0, 0]$, $\eta = 1$, and visit A, so $\tilde{\mathbf{x}} = [0, 1, 1]$, $d = +1$.

step	arithmetic
score	$0 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 = 0$
signed score	$(+1) \cdot 0 = 0 \leq 0 \rightarrow$ **mistake** (on the line counts)
update	$[0, 0, 0] + (+1)[0, 1, 1] = [0, 1, 1]$
new signed score	$0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 = 2 > 0 \rightarrow$ now correct

③ THE ONE IDEA

An update raises the offending point's signed score by exactly $\|\tilde{\mathbf{x}}\|^2$ (its squared length): above, 0 became $0 + (0^2 + 1^2 + 1^2) = 2$. So each mistake pushes the line towards fixing that

point. If *some* line separates the classes, the theorem promises this stops after at most $(R/\gamma)^2$ mistakes.

④ BUILD IT, STEP BY STEP

(the whole run on the twin, then the lengths the exam always asks).

1. Augment every point: $A \to [0,1,1]$, $B \to [1,0,1]$, $C \to [0,-1,1]$.
   Why: the appended 1 lets the bias b be learned like any other weight.
2. One table row per visit: score $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$, signed score d·score, mistake if ≤ 0, new $\tilde{\mathbf{w}}$.
   Why: the update uses the weights **as they are at that moment**, so write them down after every visit.

pass	point	score with current $\tilde{\mathbf{w}}$	d·score	action → $\tilde{\mathbf{w}}$
1	A $[0,1,1]$, $+1$	0 (with $[0,0,0]$)	0	**mistake**: $+[0,1,1] \to [0,1,1]$
1	B $[1,0,1]$, $+1$	$0+0+1=1$	1	correct
1	C $[0,-1,1]$, -1	$0-1+1=0$	0	**mistake**: $-[0,-1,1] \to [0,2,0]$
2	A	$0+2+0=2$	2	correct
2	B	$0+0+0=0$	0	**mistake**: $+[1,0,1] \to [1,2,1]$
2	C	$0-2+1=-1$	1	correct
3	A, B, C	$3,\ 2,\ -1$	$3,\ 2,\ 1$	all correct → **stop**

3. Keep going until a **whole pass** has no mistake.
   Why: a later update (pass 2, B) can break a point that was fine before, so a single clean visit proves nothing. Here: 3 passes, 3 updates.
4. Boundary: $\tilde{\mathbf{w}} = [1,2,1]$ means $w_1 = 1$, $w_2 = 2$, $b = 1$, so $x_1 + 2x_2 + 1 = 0$.
   Why: the boundary is where the score is exactly 0.
5. Verify: the pass-3 scores $3, 2, -1$ have signs $+, +, -$ = the labels.
   Why: the exam wants this written, not assumed.
6. Theorem conditions: the data are linearly separable (we just found a separator) and we started from $\mathbf{0}$.
   Why: those are exactly the two hypotheses of the convergence theorem.
7. $\|\tilde{\mathbf{x}}\| = \sqrt{x_1^2 + x_2^2 + 1}$; here $\|[0,1,1]\| = \|[1,0,1]\| = \|[0,-1,1]\| = \sqrt{2}$, so $R = \sqrt{2} \approx 1.4142$.
   Why: length = square root of the sum of squares (Pythagoras), **the appended 1 counts**, and R is the largest length — it caps how far one update can move $\tilde{\mathbf{w}}$.
8. Margin of a given $\tilde{\mathbf{w}}$ = smallest signed score $\div \|\tilde{\mathbf{w}}\|$. For $[1,2,1]$: $1/\sqrt{1+4+1} = 1/\sqrt{6} \approx 0.4082$.
   Why: dividing by the length turns a score into a true distance (augmented space, the slides' convention); the closest point is the one in danger.
9. Bound with that margin: $(R/\gamma)^2 = 2 \div \frac{1}{6} = 12$. With the better direction $[1,1,0]$: signed scores $1, 1, 1$, length $\sqrt{2}$, $\gamma = 1/\sqrt{2} \approx 0.7071$, bound $= 2 \div \frac{1}{2} = 4$.
   Why: the theorem's γ is the margin of the **best** separator, so the best direction gives the true (tightest) bound.
10. Smaller margin → more updates: each update moves $\tilde{\mathbf{w}}$ by up to R but gains only about γ towards the ideal direction, and fixing one point near the line can push a near-boundary point of the other class across.
    Why: this is the "$1/\gamma^2$" in the bound, in words.

⑤ CHECK IT

Every update raised its point's signed score by $\|\tilde{\mathbf{x}}\|^2 = 2$: A 0 → 2, C 0 → 2 (with $[0, 2, 0]$: $-(0 - 2 + 0) = 2$), B 0 → 2 ✓. Actual updates 3 ≤ bound 4 ✓. Edge case: a **very wrong** point with signed score -10 and $\|\tilde{\mathbf{x}}\|^2 = 6$ needs two visits ($-10 \to -4 \to 2$); one update does not always finish the job.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

Training set $(\mathbf{x}_i, d_i)$: $(2, 1) \to +1$, $(1, 2) \to +1$, $(-1, -2) \to -1$, $(-2, -1) \to -1$; classifier $y = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. Use the perceptron update $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, d_i\, \tilde{\mathbf{x}}_i$ only when a sample is misclassified, with $\tilde{\mathbf{x}} = [x_1, x_2, 1]^\top$, $\tilde{\mathbf{w}} = [w_1, w_2, b]^\top$, $\eta = 1$, $\tilde{\mathbf{w}}^{(0)} = \mathbf{0}$.

a. Process the four samples in the order shown. Record every update during the first pass.

b. Continue until an entire pass produces no update.

c. Give the final decision boundary.

d. Verify that all four points are classified correctly.

e. Explain why the perceptron convergence theorem applies here.

f. Let $R = \max_i \|\tilde{\mathbf{x}}_i\|_2$. Compute R, then explain qualitatively why a smaller margin may lead to more perceptron updates even when the data remain linearly separable.

g. (Extra, often asked.) Compute the margin of your final $\tilde{\mathbf{w}}$ and the bound $(R/\gamma)^2$; then with the best direction $[1, 1, 0]$.

Do it the same way: steps ④–⑤ with these numbers.

✍ Hint: S3.4 (end of this lecture)

✍ Full worked solution: S3.5 (end of this lecture)

⑦ TRY ALONE

(1) $\tilde{\mathbf{w}} = [0, 0, 0]$, visit $\tilde{\mathbf{x}} = [1, -1, 1]$ with $d = -1$: mistake? new $\tilde{\mathbf{w}}$? (2) $\tilde{\mathbf{w}} = [1, 1, 0]$, visit the point $(-1, 3)$ with $d = -1$: signed score, new $\tilde{\mathbf{w}}$, new signed score? (3) For a point whose signed score is -10 and $\|\tilde{\mathbf{x}}\|^2 = 6$, how many consecutive updates on it until it is correct?

✍ Answers: S3.6 (end of this lecture)

↗ Understand more: guide 7.11 — P1 by hand ↗ guide 7.12 — run it yourself ↗ guide 7.14 — perceptron lab (watch every update)

📖 READ & UNDERSTAND · ⏱ 6 min · Tier A

5 · The wall: why the perceptron rule can't train a multi-layer network

Problem 1 — hidden units have no targets. The slides' "double pentagon" (L3 p64–76): two pentagon-shaped regions, built by 10 first-layer units (one per edge line) plus units that combine them. The training data only say the final answer (inside a pentagon or not). To train one edge unit with the

perceptron rule, you'd need *its own* label for every point — "which side of *my* line is this point on?" — and those labels are **not given**.

With the original labels, one edge line can't separate the data (it isn't linearly separable for that single line), so the perceptron rule never converges. **Poll 2 key: No**, it can't learn the remaining neuron; **both** problems are real: the data aren't separable, and you'd have to relabel them.

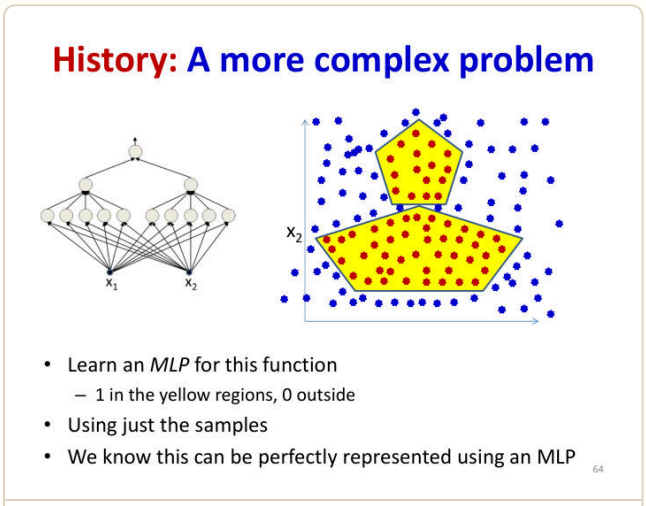
Searching for labels is hopeless: each unit can label each of N points 0 or $1 \to 2^N$ choices per unit, and 10 units jointly $\to (2^N)^{10} = 2^{10N}$. Training a threshold network this way is a **combinatorial (exponential) search** — the slides call it NP-complex (p92).

Problem 2 — the error is flat. Take 5 points $x = 1, \ldots, 5$ with targets $0, 0, 1, 1, 1$ and a threshold unit "$y = 1$ if $x \geq T$". At $T = 4.5$ outputs are $0, 0, 0, 0, 1$: two mistakes, error $2/5 = 0.4$. Move T to 4.4 or 4.6 — still 0.4. The error is a **staircase**: constant between samples, jumping at them. So its derivative (slope) is **0 almost everywhere** and undefined at the jumps (L3 p94–96).

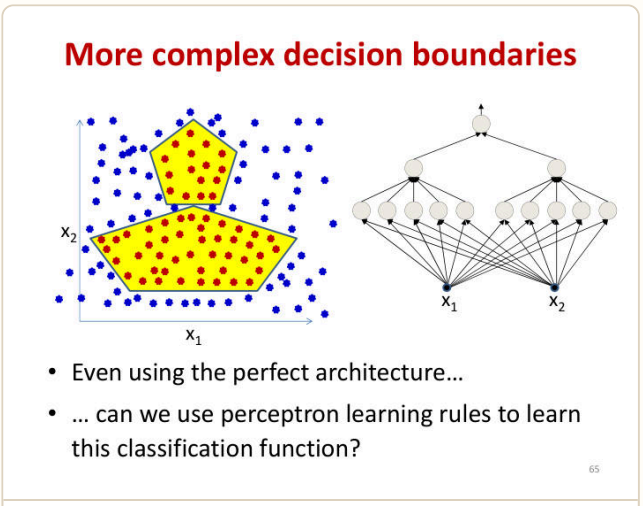
Zero slope here means "flat", **not** "at the minimum" (the best error is 0). Small weight changes give no signal about which direction helps. Realising this stalled neural-network research for over a decade (p93). Plus real data are rarely separable (p97).

Like a marble on a staircase. A marble in a bowl rolls to the bottom because the floor slopes. Put it on a flat stair tread and it just sits there — it can't "feel" which way the ground floor is, even though it's far from the bottom. Gradient methods are the marble; a threshold error is the staircase.

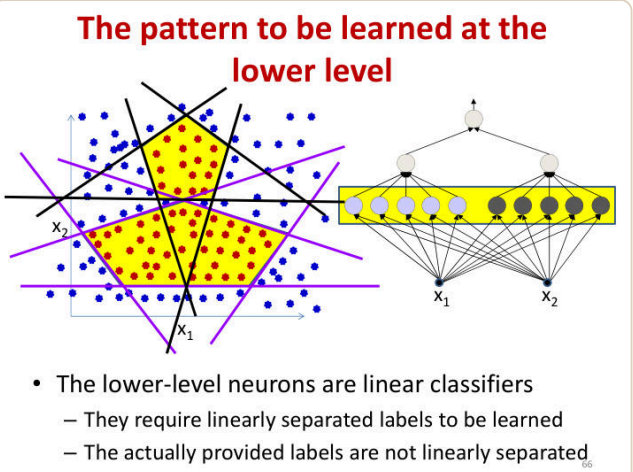
Hidden labels are like a combination lock with $10N$ binary dials and no click to tell you when one dial is right — you'd have to try 2^{10N} combinations.



L3 slide p64



L3 slide p65



L3 slide p66

↗ Understand more: guide 8.2 — Poll 2 and relabelling ↗ guide 8.10 — flat error, zero derivatives

📖 READ & UNDERSTAND · ⏱ 8 min · Tier A

6 · ADALINE and MADALINE: train on the raw sum z, flip the least-sure unit

The slides mark these as "skipped in class — check the quiz" (L3 p77): **hidden slides are quiz material**, and Assignment 3 P2–P3 test them.

ADALINE (ADAptive LINear Element, Widrow & Hoff 1960). As a machine it is an ordinary threshold unit: $z = \sum_i w_i x_i$ (bias included as $x = 1$), output $y = 1$ if $z \geq 0$ else 0. The new part is **training**: pretend the output is the raw sum z and minimise the squared gap to the binary target d (L3 p80–82):

$$\text{Err} = \tfrac{1}{2}(d - z)^2, \qquad \frac{\partial \text{Err}}{\partial w_i} = -(d - z)\, x_i$$

Gradient descent means "step each weight against its slope, scaled by a learning rate η". That gives the **delta rule** (also called LMS, least mean squares, or the Widrow–Hoff rule):

$$\delta = d - z, \qquad w_i \leftarrow w_i + \eta\, \delta\, x_i$$

E.g. $d = 1$, $z = 0.4$, input $x_i = 2$, $\eta = 0.1$: $\delta = 0.6$, so w_i grows by $0.1 \times 0.6 \times 2 = 0.12$.

Because z changes smoothly with the weights, this error has a real slope — unlike the staircase of card 5.

One update shape, three rules (L3 p83–84) — they differ only in where the error δ is measured:

rule	error δ	update
Perceptron	$d - y$ (after the threshold; only 0 or ±1)	$w_i \mathrel{+}= \eta\, \delta\, x_i$
ADALINE / LMS	$d - z$ (before the threshold; any real number)	$w_i \mathrel{+}= \eta\, \delta\, x_i$
Generalized delta rule	$d - y$, $y = f(z)$, f smooth	$w_i \mathrel{+}= \eta\, \delta\, f'(z)\, x_i$

Consequence: the perceptron stops once the class is right ($\delta = 0$); ADALINE keeps updating until z itself reaches d — even on correctly classified points. The generalized rule with $f(z) = z$ is ADALINE; it is the seed of backprop (p84).

MADALINE (Many ADALINEs, L3 p85–91) trains a whole threshold network greedily:

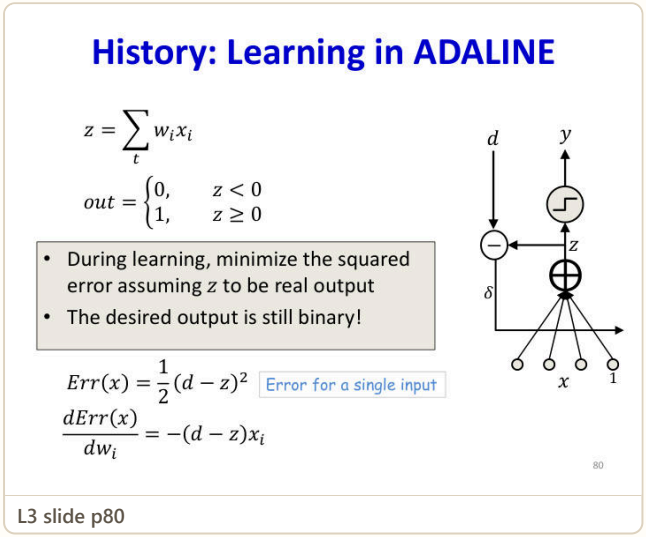
1. Classify an input. If the output is wrong:
2. find the hidden unit whose z is **closest to 0** (smallest $|z|$);

3. tentatively flip its output and recompute the network output;
4. if the error drops, make the flipped value that unit's target and train it with the ADALINE rule.

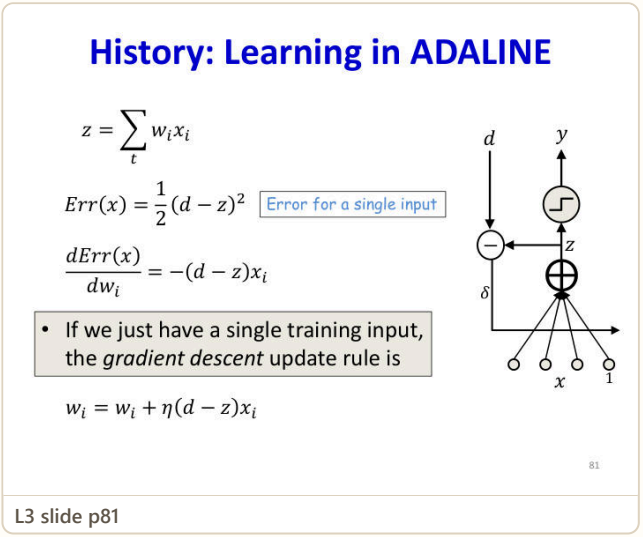
Why smallest $|z|$? It needs the smallest weight change to flip — the **minimum disturbance** principle: it disturbs the network's answers on other inputs least. Verdict (p91): fine for small nets, but too expensive and too greedy for large ones.

🧭 **Perceptron vs ADALINE is pass/fail vs a numeric grade.** A pass/fail teacher gives no feedback to a student who passed with 51%. A teacher who marks out of 100 still says "you got 80, aim for 100" — that's ADALINE pushing $z = 0.8$ towards $d = 1$ even though the class is already right.

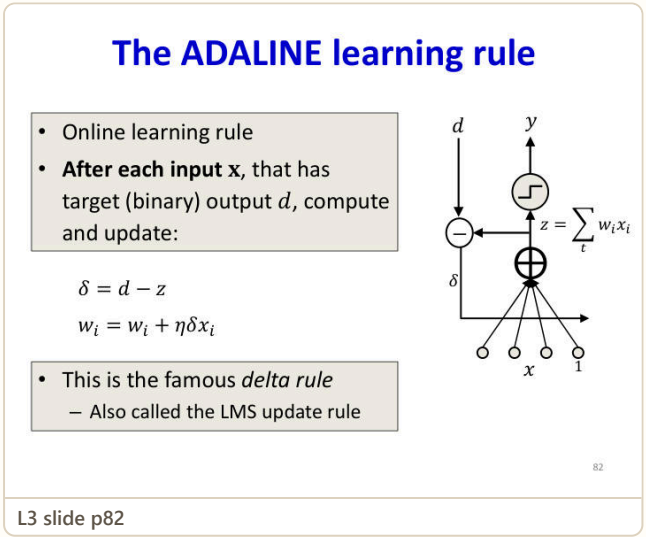
🧭 **MADALINE is like a group-project vote that came out wrong.** You ask the member who was most on the fence (51/49) to switch, because convincing them costs least. If the switch fixes the result, they're coached to hold that view. *Where it breaks:* fixing today's decision may spoil others, and sometimes two members must switch together — a single-switch rule never finds that.



L3 slide p80



L3 slide p81



L3 slide p82

↗ Understand more: guide 8.3 — ADALINE ↗ guide 8.4 — the delta rule family ↗ guide 8.7 — MADALINE ↗ Wikipedia: ADALINE

🖍 **PEN & PAPER · ⏱ 20 min · Tier A**

7 · Assignment 3 P2 — one perceptron step vs one ADALINE step

① WHAT YOU'RE BEING ASKED, IN PLAIN WORDS

One unit, one training example. The unit computes a **raw sum** $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$ (multiply each weight by its input and add; the last input is a constant 1 that carries the bias) and then a **yes/no output** $y = 1$ if $z \geq 0$, else 0. The target (desired output) is d. Two learning rules both have the shape

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, \delta\, \tilde{\mathbf{x}}$$

(η = step size, δ = "the error") and differ **only in which error** they plug in: the perceptron uses $\delta_P = d - y$ (measured after the yes/no, so only 0 or ±1); ADALINE uses $\delta_A = d - z$ (measured on the raw sum, any real number). Task: do one step of each, then show ADALINE's step is **gradient descent** — "move every weight a little against the slope of the error".

② WARM-UP BY HAND

— a smaller twin with two weights: $\tilde{\mathbf{x}} = [1, 1]$ (one input $x_1 = 1$ plus the constant 1), $\tilde{\mathbf{w}} = [0.5, 0]$, $d = 1$, $\eta = 0.1$.

quantity	arithmetic	value
raw sum z	$0.5 \cdot 1 + 0 \cdot 1$	0.5
yes/no output y	$0.5 \geq 0$	1 — the class is already right
does the perceptron see an error?	$d - y = 1 - 1$	no (0)
does ADALINE see an error?	$d - z = 1 - 0.5$	yes (0.5)

③ THE ONE IDEA

The perceptron's error dies as soon as the *class* is right; ADALINE's error dies only when the *raw sum* equals the target. And ADALINE's δ_A is exactly minus the slope of $E = \frac{1}{2}(d - z)^2$, so its step *is* a gradient-descent step.

④ BUILD IT, STEP BY STEP

(on the twin).

1. $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = 0.5 \cdot 1 + 0 \cdot 1 = 0.5$.
   Why: multiply matching entries and add, the bias entry included.
2. $y = 1$ because $z = 0.5 \geq 0$.
   Why: the threshold rule; note "$\geq$", so $z = 0$ would also give 1.
3. $\delta_P = d - y = 1 - 1 = 0$.
   Why: the perceptron measures the error **after** the threshold.
4. $\delta_A = d - z = 1 - 0.5 = 0.5$.
   Why: ADALINE measures it **before** the threshold, on the raw sum.
5. Perceptron update: $\eta\, \delta_P\, \tilde{\mathbf{x}} = 0.1 \cdot 0 \cdot [1, 1] = [0, 0]$, so $\tilde{\mathbf{w}}$ stays $[0.5, 0]$. ADALINE update: $0.1 \cdot 0.5 \cdot [1, 1] = [0.05, 0.05]$, so $\tilde{\mathbf{w}} = [0.55, 0.05]$.
   Why: same update shape $\tilde{\mathbf{w}} + \eta\delta\tilde{\mathbf{x}}$, only δ differs.
6. New $z = 0.55 + 0.05 = 0.6$; shortcut: z rises by $\eta\, \delta_A\, \|\tilde{\mathbf{x}}\|^2 = 0.1 \cdot 0.5 \cdot 2 = 0.1$.
   Why: $(\tilde{\mathbf{w}} + \eta\delta\tilde{\mathbf{x}})^\top \tilde{\mathbf{x}} = z + \eta\delta\, \tilde{\mathbf{x}}^\top \tilde{\mathbf{x}}$ — a free arithmetic check.

7. Why they differ here: the class is already right, so the perceptron is satisfied, but $z = 0.5$ is not yet the target 1, so ADALINE keeps pushing. *Why it matters:* they minimise different things — mistakes vs the squared gap $(d - z)^2$.
8. Gradient-descent proof, twin version: $z = w_1 x_1 + w_2 x_2$, so $\partial z/\partial w_1 = x_1$ and $\partial z/\partial w_2 = x_2$.
   Why: only the term $w_i x_i$ contains w_i. (∂ = "slope with respect to one variable, others held fixed".)
9. $E = \frac{1}{2}u^2$ with $u = d - z$: $\partial E/\partial w_i = u \cdot \partial u/\partial w_i = (d - z) \cdot (-x_i) = -(d - z)\, x_i$.
   Why: chain rule along $w_i \to z \to u \to E$; d is fixed, so u falls by x_i when w_i grows by 1.
10. Gradient descent: $w_i \leftarrow w_i - \eta\, \partial E/\partial w_i = w_i + \eta\, (d - z)\, x_i$ — the ADALINE rule with $\delta_A = d - z$. Numbers: $\nabla E = -0.5 \cdot [1, 1] = [-0.5, -0.5]$ and $[0.5, 0] - 0.1 \cdot [-0.5, -0.5] = [0.55, 0.05]$, the same as step 5.
    Why: "minus the slope" walks downhill, and it lands exactly on ADALINE's answer.

⑤ CHECK IT

Slope by nudging: $E(0.5, 0) = \frac{1}{2}(0.5)^2 = 0.125$; nudge w_1 to $0.51 \to z = 0.51$, $E = \frac{1}{2}(0.49)^2 = 0.12005$; rise ÷ run $= -0.00495/0.01 = -0.495 \approx -0.5$ = the first entry of ∇E ✓. Edge case: $z = 3$, $d = 1 \to \delta_A = -2$: ADALINE pulls a *correct* point's sum **down**.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

For one augmented input $\tilde{\mathbf{x}} = [2, -1, 1]^\top$, $d = 1$, $\tilde{\mathbf{w}} = [0.3, -0.4, -0.2]^\top$, let $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$ and $y = 1$ if $z \geq 0$, $y = 0$ if $z < 0$.

a. Compute z and y.

b. Compute the perceptron error signal $\delta_P = d - y$.

c. Compute the ADALINE error signal $\delta_A = d - z$.

d. With learning rate $\eta = 0.1$, compute one perceptron update and one ADALINE update.

e. Explain why the two methods can update differently even when the thresholded output is already correct.

f. Show that the ADALINE update is a gradient-descent step for $E(\mathbf{w}) = \frac{1}{2}(d - z)^2$.

Do it the same way: steps ④–⑤ with these numbers (three weights now, the last one is the bias).

✍ Hint: S3.7 (end of this lecture)

✍ Full worked solution: S3.8 (end of this lecture)

⑦ TRY ALONE

(1) $\tilde{\mathbf{x}} = [1, 2, 1]$, $\tilde{\mathbf{w}} = [0, 0, 0]$, $d = 1$, $\eta = 0.5$: both updates, and the new z after ADALINE. (2) $\tilde{\mathbf{x}} = [1, -1, 1]$, $\tilde{\mathbf{w}} = [-1, 0, 0]$, $d = 1$, $\eta = 0.1$: both updates. (3) In (2), what is ∇E?

✍ Answers: S3.9 (end of this lecture)

↗ Understand more: guide 8.5 — P2 worked ↗ guide 8.6 — P2 practice ↗ Wikipedia: Delta rule

✏ PEN & PAPER · ⏱ 14 min · Tier A

8 · Assignment 3 P3 — the MADALINE flip

① WHAT YOU'RE BEING ASKED, IN PLAIN WORDS

MADALINE ("many ADALINEs") is a network with a **hidden layer** of threshold units feeding an output unit. Each hidden unit j has a raw sum z_j and outputs 1 if $z_j \geq 0$, else 0. When the network gets an example wrong, nobody tells us what each hidden unit *should* have said. MADALINE's recipe (L3 p87–90): pick the hidden unit whose z_j is **closest to 0**, flip its output (1↔0), recompute the network's answer; if the error went down, treat the flipped value as that unit's target d_j and train only that unit with the ADALINE rule $\mathbf{w}_j \leftarrow \mathbf{w}_j + \eta\,(d_j - z_j)\,\mathbf{x}$. Task: run this reasoning and explain why it is "greedy".

② WARM-UP BY HAND

A tiny made-up network: 3 hidden units, the output says 1 if **at least 2** hidden units say 1 (a majority vote). Target 1. Hidden sums $z = (-0.5,\ 2.0,\ -1.3)$.

unit	z_j	output	$\lvert z_j \rvert$ **(how sure)**
1	−0.5	0	0.5 ← smallest
2	2.0	1	2.0
3	−1.3	0	1.3

Outputs $(0, 1, 0)$ → only one vote → network says 0, target 1: **wrong**. Flip unit 1: $(1, 1, 0)$ → two votes → network says 1: **fixed**, so keep it. Unit 1's target becomes $d_1 = 1$ and its ADALINE error is $\delta = d_1 - z_1 = 1 - (-0.5) = 1.5$.

③ THE ONE IDEA: MINIMUM DISTURBANCE

A z near 0 means the unit is barely deciding; a **small weight change** flips it, so the network's answers on *other* inputs are disturbed least. Rank by $\lvert z \rvert$ (distance from 0), not by z.

④ BUILD IT, STEP BY STEP

1. Write $\lvert z_j \rvert$ for every hidden unit.
   Why: distance from the threshold 0 = how much work a flip costs.
2. Pick the smallest.
   Why: minimum disturbance.
3. Current output: 1 if $z_j \geq 0$, else 0; the tentative output is the other one.
   Why: flipping is the only change tried.
4. Recompute the network output; keep the flip only if the error drops.
   Why: a trial, not a calculation of slopes.
5. If kept: d_j = flipped value, $\delta = d_j - z_j$, update $\mathbf{w}_j \leftarrow \mathbf{w}_j + \eta\,\delta\,\mathbf{x}$.
   Why: now unit j has a target, so plain ADALINE (card 7) can train it on its raw sum.

⑤ CHECK IT ON THE WARM-UP

$\delta = +1.5 > 0$, so the update **raises** z_1 — towards ≥ 0, i.e. towards output 1 = the flipped value ✓.
Edge case: hidden sums $(0.3, -0.8, -2.0)$, same vote, target 1. Unit 1 ($\lvert z \rvert = 0.3$) is tried first: $(1, 0, 0) \rightarrow (0, 0, 0)$, still wrong (one fewer vote) → the error does not drop → **reject**, unit 1 is not

trained. (The original Widrow algorithm then tries the next-closest unit — background, not on the slides.)

⑥ **NOW THE EXAM PROBLEM — YOUR TURN**

EXAM PROBLEM

A MADALINE network has three hidden ADALINE units with pre-activation values $z_1 = -2.4$, $z_2 = 0.15$, $z_3 = 1.1$. The hidden outputs are thresholded at zero. Suppose the network currently misclassifies a training example. The MADALINE rule searches for the hidden unit whose z_j is closest to zero, tentatively flips that unit's binary output, and keeps the flip only if network error is reduced.

a. Which hidden unit is examined first? Why?

b. What binary output would that unit have before and after the tentative flip?

c. Suppose the tentative flip corrects the network output. If the desired hidden-unit target is therefore the flipped value, write the ADALINE update for that hidden unit in terms of its input vector $\mathbf{x}$, current weight vector $\mathbf{w}_j$, and learning rate η.

d. Explain why this procedure is greedy rather than true gradient descent on the complete multilayer network.

e. Give one reason why such a greedy procedure can fail to find a globally good network even if each accepted local change reduces the current error.

Do it the same way: steps ④–⑤ with these numbers.

✍ Hint: S3.10 (end of this lecture)

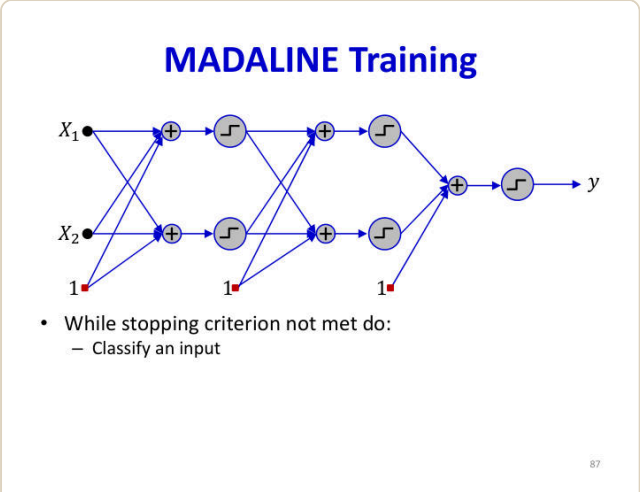
✍ Full worked solution: S3.11 (end of this lecture)

⑦ **TRY ALONE**

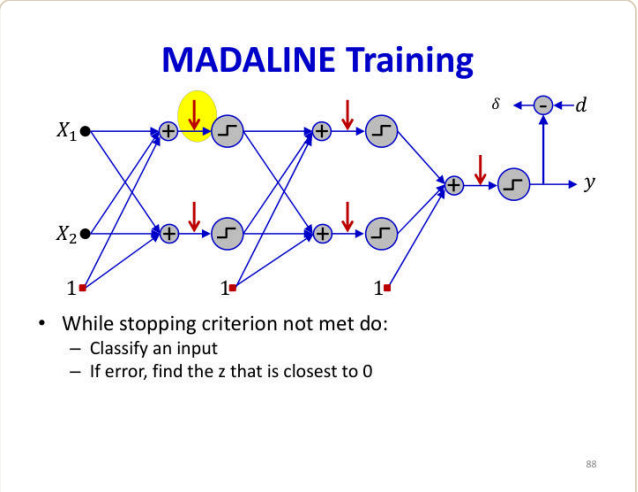
(1) Hidden sums $(0.7, -0.05, -3)$: which unit first, and its output before/after? (2) Hidden sums $(-1.2, 0.4, -0.6)$ and the flip is accepted: which unit, and its δ with a 0/1 target? (3) True or false: "MADALINE tries the unit with the most negative z first, because it is the most wrong."

✍ Answers: S3.12 (end of this lecture)

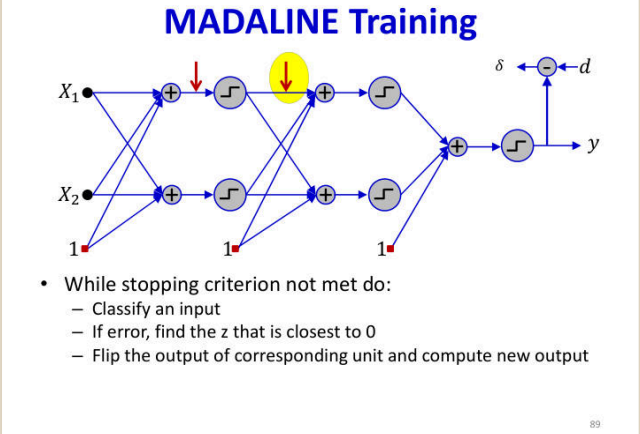
🧭 **Like tuning an old radio with several dials** by only ever turning the one that's closest to a click and keeping it if the station sounds better right now. You'll often improve things — but you can end up stuck where only turning two dials at once would bring in the station.



L3 slide p87



L3 slide p88



L3 slide p89

↗ Understand more: guide 8.8 — P3 worked ↗ guide 8.9 — the flip carried one step further (why ±1 targets)

📖 READ & UNDERSTAND · ⏱ 7 min · Tier A

9 · The fix: a smooth activation (sigmoid) and logistic regression

Replace the step with a smooth S-curve, the **sigmoid** (L3 p98–106):

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

It goes smoothly from 0 to 1, equals 0.5 at $z = 0$, and its slope σ' is **always positive** (at most $\frac{1}{4}$, at $z = 0$). *E.g.* $\sigma(2) \approx 0.8808$, so $\sigma'(2) \approx 0.8808 \times 0.1192 \approx 0.105$. Now nudging the boundary changes the *distance* between outputs and targets even when no point changes class, so the error has a slope that says which way to move.

Poll 3 key. Threshold activation: (1) moving the threshold doesn't change the error unless it passes a misclassified sample — **true**; (2) we can't tell if a move is in the right direction — **true**; (3) the derivative of the classification error tells us which way to move — **false** (it's 0). Sigmoid: all three statements — error changes only when the 0.5 crossover passes a sample; shifting changes the total distance to the targets; that distance's derivative tells us which way to shift — **true**.

Logistic regression (L3 p106–123) = a perceptron with a sigmoid. When classes overlap, the fraction of class-1 points near each x rises smoothly like an S, so the sigmoid output is read as a probability:

$$P(Y=1 \mid X) = \frac{1}{1+\exp\!\big(-\sum_i w_i x_i - b\big)}$$

Example: $\mathbf{w} = (1,1)$, $b = -1$. At $(1,1)$: $z = 1$, $P = \sigma(1) \approx 0.731$. At $(0,0)$: $z = -1$, $P \approx 0.269$. At $(0.5, 0.5)$: $z = 0$, $P = 0.5$. The decision "class 1 if $P > 0.5$" is the same as $z > 0$, so the boundary is the **straight line** $x_1 + x_2 = 1$ — not curved.

Differentiable units → differentiable network (p125–127). For one sigmoid unit, by the chain rule:

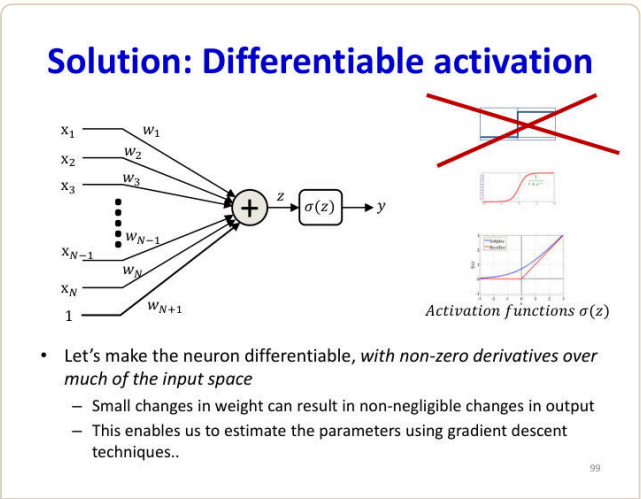
$$\frac{\partial y}{\partial w_i} = \sigma'(z)\, x_i, \qquad \frac{\partial y}{\partial x_i} = \sigma'(z)\, w_i$$

E.g. at $z = 0$ with $x_i = 3$: $\partial y / \partial w_i = 0.25 \times 3 = 0.75$.

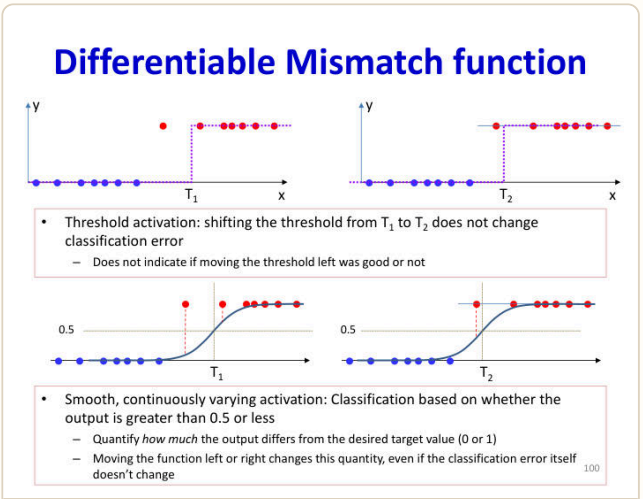
Chain units and each link multiplies in one finite, non-zero slope, so the network output has a usable derivative for every weight. A threshold anywhere on the path puts in a factor 0 and kills it. With a differentiable divergence on top (card 1), the training loss is smooth in W — ready for gradient descent (Lecture 4) and backprop (Lecture 5).

Light switch vs dimmer. A threshold unit is a light switch: turning the knob a little does nothing until it suddenly clicks. A sigmoid is a dimmer: every small turn makes the room a little brighter or darker, so you always know which way to turn.

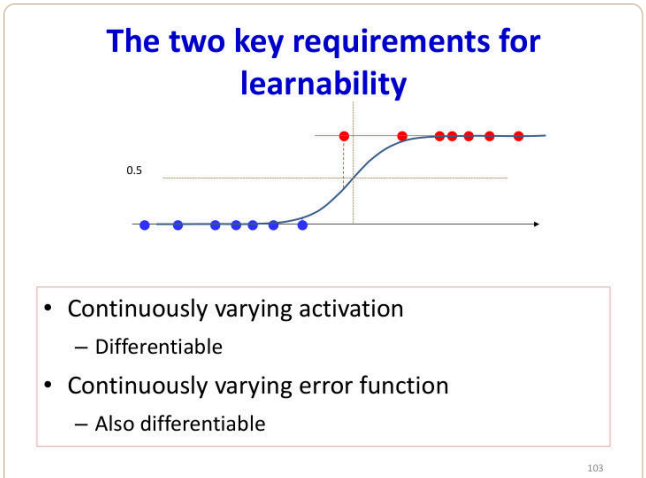
Like a spam filter saying "93% spam" instead of just "spam". The sigmoid's output is that confidence. Claude/ChatGPT do the many-class version (softmax over the vocabulary); the sigmoid is the two-class case.



L3 slide p99



L3 slide p100



L3 slide p103

↗ Understand more: guide 8.11 — smooth activation (Poll 3) ↗ guide 8.12 — logistic regression ↗ guide 8.13 — differentiable network ↗ Wikipedia: Logistic regression

PEN & PAPER · ⏱ 12 min · Tier B

10 · Derive $\sigma' = \sigma(1-\sigma)$ and take one generalized-delta step

① WHAT YOU'RE BEING ASKED, IN PLAIN WORDS

The **sigmoid** $\sigma(z) = \dfrac{1}{1+e^{-z}}$ squashes any number z into the range 0 to 1 ($e \approx 2.718$; e^{-z} is huge for very negative z and tiny for very positive z). Show that its slope $\sigma'(z)$ can be written using σ itself: $\sigma(1-\sigma)$. Then use that slope to get the learning rule for a sigmoid unit (output $y = \sigma(z)$, error $E = \frac{1}{2}(d-y)^2$), and take one step with numbers.

② WARM-UP BY HAND

— on a twin function, the **steeper sigmoid** $s(z) = \dfrac{1}{1+e^{-2z}}$ (same S-shape, squeezed sideways by 2). Estimate its slope by nudging z a little either side (rise ÷ run), and compare with $2s(1-s)$:

z	$s(z)$	**nudge estimate** $\frac{s(z+0.01)-s(z-0.01)}{0.02}$	$2s(1-s)$
0	0.5	0.5000	$2 \times 0.5 \times 0.5 = 0.5$
1	0.8808	0.2100	$2 \times 0.8808 \times 0.1192 \approx 0.2100$

They agree. The derivation below explains why — and the same four moves work for σ itself.

③ THE ONE IDEA

Write the S-curve as $(1+e^{\cdots})^{-1}$, differentiate with the chain rule, then notice that the leftover fraction $\frac{e^{\cdots}}{1+e^{\cdots}}$ is just "1 minus the function".

④ BUILD IT, STEP BY STEP

(on the twin s).

1. $s = u^{-1}$ with $u = 1+e^{-2z}$.
   Why: a power is easy to differentiate.
2. $\frac{d}{du}u^{-1} = -u^{-2}$ and $\frac{du}{dz} = -2e^{-2z}$.

Why: power rule; and the slope of e^{-2z} is $-2e^{-2z}$ (chain rule on the exponent $-2z$, whose slope is -2).

3. $s' = -u^{-2} \cdot (-2e^{-2z}) = \dfrac{2e^{-2z}}{(1+e^{-2z})^2}$.
   Why: chain rule; the two minus signs cancel.
4. Split it: $2 \cdot \dfrac{1}{1+e^{-2z}} \cdot \dfrac{e^{-2z}}{1+e^{-2z}} = 2\,s\,(1-s)$.
   Why: $\frac{e^{-2z}}{1+e^{-2z}} = \frac{(1+e^{-2z})-1}{1+e^{-2z}} = 1 - s$.
5. Learning rule for a unit with output $y = s(z)$, $z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$, error $E = \frac{1}{2}(d-y)^2$: the chain is $w_i \to z \to y \to E$, so $\dfrac{\partial E}{\partial w_i} = \underbrace{-(d-y)}_{\partial E/\partial y} \cdot \underbrace{s'(z)}_{dy/dz} \cdot \underbrace{x_i}_{\partial z/\partial w_i}$.
   Why: chain rule — multiply the three slopes.
6. $\Delta w_i = -\eta\, \partial E/\partial w_i = \eta\,(d-y)\,s'(z)\,x_i$.
   Why: step against the slope (gradient descent).
7. Numbers: $\tilde{\mathbf{x}} = [1, 1]$, $\tilde{\mathbf{w}} = [0.5, 0]$, $d = 1$, $\eta = 0.1 \to z = 0.5$, $y = s(0.5) \approx 0.7311$, $s'(0.5) = 2(0.7311)(0.2689) \approx 0.3932$, $d - y \approx 0.2689$, so $\Delta\tilde{\mathbf{w}} = 0.1 \times 0.2689 \times 0.3932 \times [1, 1] \approx [0.0106,\ 0.0106]$. *Why this order:* z first, then y, then the slope at that z, then multiply.

⑤ CHECK IT

At $z = 0$ the formula gives 0.5, the warm-up's estimate ✓ — the steepest point of the curve. Edge case: $z = 5 \to s \approx 0.99995$, $s' \approx 0.00009$: the curve is flat out there ("saturated"), so Δw_i is almost zero — a saturated unit barely learns.

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

a. Show that $\sigma(z) = 1/(1+e^{-z})$ has $\sigma'(z) = \sigma(z)(1-\sigma(z))$.

b. Derive the generalized delta rule $\Delta w_i = \eta\,(d-y)\,\sigma'(z)\,x_i$ by gradient descent on $E = \frac{1}{2}(d-y)^2$, $y = \sigma(z)$.

c. Apply it to the Assignment 3 P2 unit: $\tilde{\mathbf{x}} = [2, -1, 1]$, $\tilde{\mathbf{w}} = [0.3, -0.4, -0.2]$, $d = 1$, $\eta = 0.1$.

Do it the same way: steps ④–⑤ with σ in place of s (the exponent is $-z$, so no factor 2) and the numbers in (c).

✍ Hint: S3.13 (end of this lecture)

✍ Full worked solution: S3.14 (end of this lecture)

⑦ TRY ALONE

(1) A sigmoid unit outputs $\sigma = 0.9$. What is σ' there? (2) $\sigma(-0.8) \approx 0.31$: what is $\sigma'(-0.8)$, and how does it compare with $\sigma'(0.8)$? (3) One generalized-delta step: $\tilde{\mathbf{x}} = [1, 1]$, $\tilde{\mathbf{w}} = [0, 0]$, $d = 1$, $\eta = 1$.

✍ Answers: S3.15 (end of this lecture)

↗ Understand more: guide 8.14 — slopes through a chain ↗ Wikipedia: Sigmoid function

🔁 **RECALL DRILL · ⏱ 5 min · Tier A**

11 · Recall drill: the three polls + quick checks

Poll 1 (p30). "Since neural networks are universal approximators, any network of any architecture can approximate any function to arbitrary precision." True or false? And which of these are true: the architecture needs enough capacity / the network is a parametric function of its weights and biases / the parameters must be learned / the parameters can be learned perfectly from a few samples?

✍ Answer: S3.16 (end of this lecture)

Poll 2 (p68). Double pentagon, all neurons learned except one. Can the perceptron algorithm learn it from the given data? What goes wrong?

✍ Answer: S3.17 (end of this lecture)

Poll 3 (p101). Threshold: (1) error changes only when the threshold passes a misclassified sample; (2) we can't know if a move is in the right direction; (3) the derivative of the classification error tells us which way to move. Sigmoid: (1) error changes only when the 0.5 crossover passes a sample; (2) shifting changes the total distance to the targets; (3) its derivative tells us which way to shift.

✍ Answer: S3.18 (end of this lecture)

Q1. A unit fires when $\mathbf{w}^\top \mathbf{x} \geq T$ with $\mathbf{w} = (2, 1)$, $T = 4$. Write $\tilde{\mathbf{w}}$ and z at $\mathbf{x} = (3, -1)$.

✍ Answer: S3.19 (end of this lecture)

Q2. ADALINE: $\tilde{\mathbf{x}} = [2, 1]$, $\tilde{\mathbf{w}} = [1, 0]$, $d = 1$, $\eta = 0.1$. New $\tilde{\mathbf{w}}$?

✍ Answer: S3.20 (end of this lecture)

Q3. Boundary $2x_1 - x_2 + 3 = 0$, class $+1$ where it's > 0. Which vector is perpendicular to the boundary and points into the $+1$ side?

✍ Answer: S3.21 (end of this lecture)

Q4. MADALINE hidden units have $z = -0.9$ and $z = -0.3$. Which is tried first?

✍ Answer: S3.22 (end of this lecture)

Q5. Logistic regression, $\mathbf{w} = (2, -1)$, $b = 0$: $P(Y = 1 \mid X)$ at $X = (1, 2)$?

✍ Answer: S3.23 (end of this lecture)

Q6. What does the perceptron algorithm do on data that are not linearly separable? And what are the two conditions of the $(R/\gamma)^2$ guarantee?

✍ Answer: S3.24 (end of this lecture)

↗ guide 7.13 — reflection, the "trivial" solution, non-separable data

⚠ Traps

- **Score 0 is a mistake.** In P1 the very first sample scores 0 with $\tilde{\mathbf{w}} = \mathbf{0}$ and must be updated ($d \cdot \text{score} \leq 0$).
- **Include the appended 1 in R:** the point $(3, 1)$ becomes $[3, 1, 1]$, so $\|[3, 1, 1]\| = \sqrt{11}$, not $\sqrt{10}$.
- **γ is the best-case margin** (of the best separator), not the margin of the W the algorithm returns; the final W's margin only gives a looser bound. Margin = smallest signed score ÷ $\|\tilde{\mathbf{w}}\|$ — don't forget to divide.
- **ADALINE's δ uses z, not y.** Feed y into it and you're back to the perceptron rule. ADALINE updates even correctly classified points.
- **MADALINE ranks by $|z|$**, not by z: the most negative unit is the hardest to flip, not the first to try.

- **Zero derivative ≠ minimum.** The threshold error is a staircase: flat everywhere, yet far from the best value.
- **Logistic regression's boundary is a straight line** ($\mathbf{w}^\top \mathbf{x} + b = 0$), even though the sigmoid is curved.
- $\partial y / \partial w_i = \sigma'(z)\, x_i$ but $\partial y / \partial x_i = \sigma'(z)\, w_i$ — weight-derivative is scaled by the input, and vice versa.
- **Slide error, L3 p58:** the side labels read "+1 (blue)" and "−1 (Red)", but the key on these slides is Red = +1, Blue = −1. Trust the key.
- **Slide error, L3 p62:** the "trivial solution" $W = \frac{1}{N}\sum_i y_i X_i$ is **not** guaranteed to separate separable data. Counterexample: reflected points $(10, 0)$ and $(-1, 1)$ are separable ($W = (0.1, 1)$ scores 1 and 0.9), but their mean $(4.5, 0.5)$ gives $(-1, 1) \cdot (4.5, 0.5) = -4 < 0$.
- **Slide error, L3 p69:** "data shown on slide 68" should be slide 67 (68 is the poll itself).
- **Slide error, L3 p79:** "Hopf and Widrow" → **Widrow and Hoff** (1960; hence "Widrow–Hoff rule"). And $z = \sum_t w_i x_i$ on p79–82 should read $z = \sum_i w_i x_i$.

Solutions and answers — Lecture 3

S3.1 · 2 · Assignment 3 P4 — ERM by hand for f(x;w)=wxf(x;w)=wxf(x;w)=wx — Hint

Make a four-row table with columns x_i, d_i, $x_i d_i$, x_i^2 and add each column. For (e), only one term of one sum changes.

↩ back to the card

S3.2 · 2 · Assignment 3 P4 — ERM by hand for f(x;w)=wxf(x;w)=wxf(x;w)=wx — Full worked solution

(a) Empirical risk. One squared divergence per training pair, averaged over the $N = 4$ pairs:

$$L(w) = \frac{1}{N}\sum_{i=1}^{N} \tfrac{1}{2}\,(wx_i - d_i)^2$$

$$L(w) = \tfrac{1}{4}\Big[\tfrac{1}{2}(w-1.2)^2 + \tfrac{1}{2}(2w-1.9)^2 + \tfrac{1}{2}(3w-3.2)^2 + \tfrac{1}{2}(4w-3.9)^2\Big]$$

Why: "empirical" = averaged over the samples we actually have.

(b) Differentiate and solve. Each term $\frac{1}{2}u^2$ with $u = wx_i - d_i$ has slope $u \cdot \frac{du}{dw} = (wx_i - d_i)\,x_i$ (chain rule):

$$L'(w) = \frac{1}{N}\sum_{i}(wx_i - d_i)\,x_i$$

Why: the derivative of an average is the average of the derivatives.

$$L'(w) = \frac{1}{N}\Big(w\sum_i x_i^2 - \sum_i x_i d_i\Big)$$

Why: expand $(wx_i - d_i)x_i = wx_i^2 - x_i d_i$; w is common to every term.

$$L'(w) = 0 \iff w^* = \frac{\sum_i x_i d_i}{\sum_i x_i^2}$$

Why: the minimum of a smooth curve has zero slope.

$$L''(w) = \frac{1}{N}\sum_i x_i^2 = \frac{30}{4} = 7.5 > 0$$

Why: positive curvature, so this stationary point is the bottom of a bowl (the unique minimum), not a top.

(c) Numbers.

i	x_i	d_i	$x_i d_i$	x_i^2
1	1	1.2	1.2	1
2	2	1.9	3.8	4
3	3	3.2	9.6	9
4	4	3.9	15.6	16
sum			30.2	30

$$w^* = \frac{30.2}{30} \approx 1.0067$$

Check: $L'(w^*) = \frac{1}{4}(1.0067 \times 30 - 30.2) = 0$ ✓, and $L(w^*) \approx 0.01233$, the smallest value the empirical risk can take.

(d) Why empirical ≠ true risk. $L(w)$ averages the divergence over only 4 samples, not over the whole input distribution $P(X)$. The labels are noisy ($d_i = g(x_i) + \text{noise}$), so w^* fits this particular sample, noise included; a different 4 samples would give a different w^*. The true risk $\mathbb{E}[\text{div}(f(X;w), g(X))]$ needs g and $P(X)$, which we never have, so it is never computed — minimising L is only a proxy. The two agree better as N grows.

(e) One corrupted target. Replace d_k by $d_k + \Delta$. Only the numerator contains targets, and only its k-th term contains d_k:

$$w^*_{\text{new}} = \frac{\sum_i x_i d_i + x_k \Delta}{\sum_i x_i^2} = w^* + \frac{x_k \Delta}{\sum_i x_i^2}$$

With d_4: $3.9 \to 13.9$, i.e. $k = 4$, $x_4 = 4$, $\Delta = 10$:

$$\Delta w^* = \frac{4 \times 10}{30} \approx 1.3333, \qquad w^*_{\text{new}} = \frac{30.2 + 40}{30} = 2.34$$

More than double. ERM never sees g; it only sees the samples, and the corrupted pair $(4, 13.9)$ is one of them with full weight. Squared loss charges $\frac{1}{2}(\text{residual})^2$, so one residual of ≈10 dominates the sum and the line bends towards that one bad point. The true risk (and g) has not changed at all — only the empirical risk moved. Leverage: on d_1 ($x_1 = 1$) the same corruption shifts w^* by only $10/30 \approx 0.3333$, a quarter as much, because points far from the origin pull harder on a line through the origin.

(a) $L(w) = \frac{1}{4}\sum_{i=1}^{4} \frac{1}{2}(wx_i - d_i)^2$**; (b)** $w^* = \dfrac{\sum x_i d_i}{\sum x_i^2}$**,** $L'' = 7.5 > 0$**; (c)** $w^* = 30.2/30 \approx 1.0067$**; (e) corrupted** d_4**:** $\Delta w^* = 4/3 \approx 1.333$**, new** $w^* = 2.34$**.**

↩ back to the card

S3.3 · 2 · Assignment 3 P4 — ERM by hand for f(x;w)=wxf(x;w)=wxf(x;w)=wx — Answers

(1) $\frac{1\cdot 2 + 2\cdot 4}{1+4} = \frac{10}{5} = 2$ — the points lie exactly on $y = 2x$. (2) $\frac{1+6}{1+4} = 1.4$. (3) $\frac{30.2 + 1\times 10}{30} = 1.34$, a shift of only $1/3$ (vs $4/3$ for d_4): the shift is $x_k\Delta / \sum x_i^2$, and $x_1 = 1$ is small.

↩ back to the card

S3.4 · 4 · Assignment 3 P1 — run the perceptron by hand — Hint

One table row per visit: score, signed score, mistake?, new $\tilde{\mathbf{w}}$. Remember a score of exactly 0 is a mistake — that decides the very first visit.

↩ back to the card

S3.5 · 4 · Assignment 3 P1 — run the perceptron by hand — Full worked solution

Set-up. Augment: $\tilde{\mathbf{x}}_1 = [2,1,1]$, $\tilde{\mathbf{x}}_2 = [1,2,1]$, $\tilde{\mathbf{x}}_3 = [-1,-2,1]$, $\tilde{\mathbf{x}}_4 = [-2,-1,1]$. Mistake test: $d_i(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i) \le 0$ (a score of 0 counts, since $\text{sign}(0)$ is no class). Update on a mistake: $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + d_i \tilde{\mathbf{x}}_i$ ($\eta = 1$).

(a) Pass 1.

sample	$\tilde{\mathbf{x}}$, d	score $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$	$d\cdot$score	action
1	$[2,1,1]$, $+1$	0 (with $[0,0,0]$)	$0 \le 0$	**mistake**: $\tilde{\mathbf{w}} = [0,0,0] + [2,1,1] = [2,1,1]$
2	$[1,2,1]$, $+1$	$2+2+1 = 5$	$5 > 0$	correct
3	$[-1,-2,1]$, -1	$-2-2+1 = -3$	$3 > 0$	correct

4	$[-2,-1,1]$, -1	$-4-1+1=-4$	$4>0$	correct

Why sample 1 is a mistake: with $\tilde{\mathbf{w}}=\mathbf{0}$ every score is 0, so the first visit always updates. After it, its signed score is $0+\|\tilde{\mathbf{x}}_1\|^2=6$. One update in pass 1.

(b) Pass 2 with $\tilde{\mathbf{w}}=[2,1,1]$:

$$[2,1,1]\cdot[2,1,1]=6,\quad [2,1,1]\cdot[1,2,1]=5,\quad [2,1,1]\cdot[-1,-2,1]=-3,\quad [2,1,1]\cdot[-2,-1,1]=-4$$

Signed scores $6,5,3,4$, all >0 → no update in this pass → **stop**. *Why stop now:* the rule ends only after a whole pass without an update. Total: **2 passes, 1 update.**

(c) Boundary. $\tilde{\mathbf{w}}=[2,1,1]$ means $\mathbf{w}=(2,1)$, $b=1$; the boundary is where the score is 0:

$$2x_1+x_2+1=0 \quad (\text{i.e. } x_2=-2x_1-1), \qquad +1 \text{ side: } 2x_1+x_2+1>0$$

(d) Verify.

point	score $2x_1+x_2+1$	sign	label	✓
$(2,1)$	$4+1+1=6$	+	$+1$	✓
$(1,2)$	$2+2+1=5$	+	$+1$	✓
$(-1,-2)$	$-2-2+1=-3$	−	-1	✓
$(-2,-1)$	$-4-1+1=-4$	−	-1	✓

(e) Why the theorem applies. Its two hypotheses hold: the data are linearly separable ($[2,1,1]$ separates them, as (d) verified) and the algorithm started from $\tilde{\mathbf{w}}=\mathbf{0}$. Hence it must stop after at most $(R/\gamma)^2$ updates, where $\gamma>0$ is the margin of the best separator.

(f) R. Every $\tilde{\mathbf{x}}_i$ has the same length (include the appended 1!):

$$\|[2,1,1]\|=\|[1,2,1]\|=\|[-1,-2,1]\|=\|[-2,-1,1]\|=\sqrt{4+1+1}$$
$$R=\sqrt{6}\approx 2.4495$$

Smaller margin → more updates. Each mistake moves $\tilde{\mathbf{w}}$ by a vector of length up to R but gains only about γ along the ideal separating direction. With a small γ the gains are tiny, and a step that fixes one point can push a near-boundary point of the other class across, so $\tilde{\mathbf{w}}$ zig-zags. The bound $(R/\gamma)^2$ grows like $1/\gamma^2$.

(g) Margins and bounds. Margin of a $\tilde{\mathbf{w}}$ = smallest signed score divided by $\|\tilde{\mathbf{w}}\|$. For $[2,1,1]$: signed scores $6,5,3,4$, smallest 3; $\|\tilde{\mathbf{w}}\|=\sqrt{6}$:

$$\gamma=\frac{3}{\sqrt{6}}\approx 1.2247, \qquad \left(\frac{R}{\gamma}\right)^2=\frac{6}{9/6}=4$$

For the best direction $[1,1,0]$: signed scores $3,3,3,3$ (e.g. $-(-1-2+0)=3$), length $\sqrt{2}$:

$$\gamma=\frac{3}{\sqrt{2}}\approx 2.1213, \qquad \left(\frac{R}{\gamma}\right)^2=\frac{6}{4.5}\approx 1.3333$$

The theorem's γ is the best-case one, so the bound is ≤ 1.33, i.e. at most 1 update — and we made exactly 1. ✓

(a)–(b) 1 update (sample 1, pass 1), stop after pass 2 → $\tilde{\mathbf{w}}=[2,1,1]$; (c) boundary $2x_1+x_2+1=0$; (d) scores $6,5,-3,-4$ all correct; (f) $R=\sqrt{6}\approx 2.449$; (g) margin of $[2,1,1]$: $3/\sqrt{6}\approx 1.225$, bound 4; best direction $[1,1,0]$: $\gamma=3/\sqrt{2}\approx 2.121$, bound $4/3$.

↩ back to the card

S3.6 · 4 · Assignment 3 P1 — run the perceptron by hand — Answers

(1) Score 0 → signed score 0 → mistake; $\tilde{\mathbf{w}} = [0,0,0] - [1,-1,1] = [-1,1,-1]$. (2) $\tilde{\mathbf{x}} = [-1,3,1]$, score $-1+3+0 = 2$, signed -2 → mistake; $\tilde{\mathbf{w}} = [1,1,0] - [-1,3,1] = [2,-2,-1]$; new signed score $-(-2-6-1) = 9 = -2+11$ ($\|\tilde{\mathbf{x}}\|^2 = 1+9+1 = 11$) ✓. (3) Two: $-10 \to -4 \to 2$.

↩ back to the card

S3.7 · 7 · Assignment 3 P2 — one perceptron step vs one ADALINE step — Hint

Watch the sign in $(-0.4)(-1)$. For (f), write the proof for a general weight w_i, then plug in the numbers to show both routes give the same $\tilde{\mathbf{w}}$.

↩ back to the card

S3.8 · 7 · Assignment 3 P2 — one perceptron step vs one ADALINE step — Full worked solution

(a) Multiply matching entries and add (the last entry is the bias times 1):

$$z = 0.3(2) + (-0.4)(-1) + (-0.2)(1) = 0.6 + 0.4 - 0.2 = 0.8$$

$0.8 \geq 0$, so $y = 1$ — already the right class ($d = 1$).

(b) The perceptron measures the error after the threshold:

$$\delta_P = d - y = 1 - 1 = 0$$

(c) ADALINE measures it on the raw sum:

$$\delta_A = d - z = 1 - 0.8 = 0.2$$

(d) Both use $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\,\delta\,\tilde{\mathbf{x}}$. Perceptron: $\eta\,\delta_P\,\tilde{\mathbf{x}} = 0.1 \times 0 \times [2,-1,1] = \mathbf{0}$, so $\tilde{\mathbf{w}}$ stays $[0.3, -0.4, -0.2]$. ADALINE:

$$\eta\,\delta_A\,\tilde{\mathbf{x}} = 0.1 \times 0.2 \times [2,-1,1] = [0.04, -0.02, 0.02]$$

$$\tilde{\mathbf{w}}_{\text{new}} = [0.34,\ -0.42,\ -0.18]$$

$$z_{\text{new}} = 0.34(2) + (-0.42)(-1) + (-0.18)(1) = 0.68 + 0.42 - 0.18 = 0.92$$

Shortcut check: z rises by $\eta\,\delta\,\|\tilde{\mathbf{x}}\|^2 = 0.1 \times 0.2 \times 6 = 0.12$. ✓ It moved towards the target 1.

(e) The perceptron's error uses the thresholded y: once the class is right, $\delta_P = 0$ and it stops, however close z is to the boundary. ADALINE's uses the raw z: $\delta_A = 0$ only when z equals d exactly, so it keeps adjusting correct samples (here pushing z from 0.8 up to 1, further from the boundary). It can even *lower* a correct score ($z = 3$, $d = 1$ gives $\delta_A = -2$). They minimise different things: the perceptron counts mistakes; ADALINE minimises the squared distance of z from d.

(f) Only the term $w_i x_i$ of z contains w_i, so $\partial z / \partial w_i = x_i$. Chain rule:

$$\frac{\partial E}{\partial w_i} = (d - z) \cdot \frac{\partial (d - z)}{\partial w_i} = -(d - z)\,x_i$$

$$w_i - \eta \frac{\partial E}{\partial w_i} = w_i + \eta\,(d - z)\,x_i = w_i + \eta\,\delta_A\,x_i$$

That is exactly the ADALINE rule (for every i, bias included). Numbers:

$$\nabla E = -(d - z)\,\tilde{\mathbf{x}} = -0.2 \times [2,-1,1] = [-0.4,\ 0.2,\ -0.2]$$

$$\tilde{\mathbf{w}} - 0.1\,\nabla E = [0.34,\ -0.42,\ -0.18]\ \checkmark$$

(a) $z = 0.8$, $y = 1$; (b) $\delta_P = 0$; (c) $\delta_A = 0.2$; (d) perceptron: no change; ADALINE $\tilde{\mathbf{w}} = [0.34, -0.42, -0.18]$, new $z = 0.92$; (e) perceptron stops once the class is right, ADALINE until

$z = d$; **(f)** $\partial E/\partial w_i = -(d - z)x_i$, $\nabla E = [-0.4, 0.2, -0.2]$, $\tilde{\mathbf{w}} - \eta\nabla E$ **= the ADALINE update.**

↩ back to the card

S3.9 · 7 · Assignment 3 P2 — one perceptron step vs one ADALINE step — Answers

(1) $z = 0$, $y = 1$ ($0 \geq 0$); $\delta_P = 0 \to$ no change; $\delta_A = 1 \to \tilde{\mathbf{w}} = 0.5 \cdot 1 \cdot [1, 2, 1] = [0.5, 1, 0.5]$, new $z = 0.5 + 2 + 0.5 = 3$ — it **overshoots** the target 1 because $\eta\|\tilde{\mathbf{x}}\|^2 = 0.5 \times 6 = 3 > 1$. (2) $z = -1$, $y = 0$ (wrong class). Perceptron: $\delta_P = 1$, $\tilde{\mathbf{w}} = [-1, 0, 0] + 0.1[1, -1, 1] = [-0.9, -0.1, 0.1]$. ADALINE: $\delta_A = 2$, $\tilde{\mathbf{w}} = [-1, 0, 0] + 0.2[1, -1, 1] = [-0.8, -0.2, 0.2]$. (3) $\nabla E = -(d - z)\tilde{\mathbf{x}} = -2[1, -1, 1] = [-2, 2, -2]$.

↩ back to the card

S3.10 · 8 · Assignment 3 P3 — the MADALINE flip — Hint

Rank by $|z_j|$, not by z_j. For (d), compare with gradient descent: how many weights move at once, how the change is chosen, how many examples are looked at. For (e), think of the ⑤ edge case and of a fix that needs two units to change together.

↩ back to the card

S3.11 · 8 · Assignment 3 P3 — the MADALINE flip — Full worked solution

(a) Compare **absolute values** (distance from the threshold 0):

unit	z_j	output (1 if $z_j \geq 0$)	$\lvert z_j \rvert$
1	−2.4	0	2.4
2	0.15	1	0.15 ← smallest
3	1.1	1	1.1

Unit 2 is examined first.

Why: it is the least confident unit, so the smallest weight change flips it (minimum disturbance) and the network's answers on other inputs are disturbed least. (Unit 1 is the most negative, but that makes it the *hardest* to flip.)

(b) $z_2 = 0.15 \geq 0$, so output **1 before, 0 after** the tentative flip (in ±1 coding: $+1 \to -1$).

Why: a flip replaces the current binary output by the other value.

(c) The flip is kept, so the flipped value becomes unit 2's target d_2, and it is trained with the ADALINE rule on its own linear sum $z_2 = \mathbf{w}_2^\top \mathbf{x}$ (not on its thresholded output, which has no usable slope):

$$\mathbf{w}_2 \leftarrow \mathbf{w}_2 + \eta \left(d_2 - \mathbf{w}_2^\top \mathbf{x}\right) \mathbf{x}$$

$$d_2 = 0: \quad \delta = 0 - 0.15 = -0.15$$

Negative, so the update $\Delta\mathbf{w}_2 = -0.15\,\eta\,\mathbf{x}$ lowers z_2 (by $0.15\,\eta\|\mathbf{x}\|^2$) towards the threshold and past it — towards output 0, the flipped value. *Why check the sign:* the step must move z_2 towards its new target. (With ±1 coding, $d_2 = -1$ and $\delta = -1.15$, a much stronger push past the threshold.)

(d) Greedy, not gradient descent. Gradient descent moves *all* weights together, each in proportion to its effect on the total error, using derivatives of a smooth error. MADALINE instead:

- changes **one unit** at a time;
- chooses it by a **heuristic** ($|z|$ nearest 0), not by how much it would reduce the error;
- decides by a **discrete trial** (flip and look) on the **current example only**;

- keeps the change without revisiting it.

A threshold network has zero derivative almost everywhere anyway, so there is no gradient to follow — only local trial-and-error.

(e) The error is reduced on the *current* example only — a flip that fixes it can break others, so the network can cycle. And since only single-unit changes that help right now are accepted, it can get stuck where no single flip helps but a coordinated change of several units would (a local optimum). The least confident unit is also not necessarily the one that should change.

(a) Unit 2 (smallest $|z| = 0.15$); (b) output $1 \to 0$; (c) $\mathbf{w}_2 \leftarrow \mathbf{w}_2 + \eta(d_2 - \mathbf{w}_2^\top \mathbf{x})\mathbf{x}$ with $d_2 = 0$, $\delta = -0.15$; (d) greedy: one unit, heuristic choice, one example, discrete trial, no gradient; (e) a fix for this example can break others, and no single flip may help where a joint change would (local optimum).

↩ back to the card

S3.12 · 8 · Assignment 3 P3 — the MADALINE flip — Answers

(1) Unit 2 ($|z| = 0.05$); output 0 before, 1 after. (2) Unit 2 ($|z| = 0.4$); output $1 \to 0$, so $d_2 = 0$ and $\delta = 0 - 0.4 = -0.4$ (pushes z_2 down). (3) **False** — the most negative z is the *hardest* to flip; MADALINE picks the smallest $|z|$.

↩ back to the card

S3.13 · 10 · Derive σ′=σ(1−σ)\sigma' = \sigma(1-\sigma)σ′=σ(1−σ) and take one generalized-delta step — Hint

For (a) the only change from step ② of ④ is that the slope of e^{-z} is $-e^{-z}$. For (c) you already know this unit's z from card 7.

↩ back to the card

S3.14 · 10 · Derive σ′=σ(1−σ)\sigma' = \sigma(1-\sigma)σ′=σ(1−σ) and take one generalized-delta step — Full worked solution

(a) Write σ as a power of $u = 1 + e^{-z}$:

$$\sigma = u^{-1}, \qquad \frac{d\sigma}{du} = -u^{-2}, \qquad \frac{du}{dz} = -e^{-z}$$

Why: power rule, and the slope of e^{-z} is $-e^{-z}$ (the exponent $-z$ has slope -1). Chain rule, the two minus signs cancel:

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Split the fraction into two factors:

$$\sigma'(z) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

The first factor is σ; the second is $1 - \sigma$ (add and subtract 1 on top):

$$\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \sigma(z)$$

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

(b) $E = \frac{1}{2}(d-y)^2$ depends on w_i through the chain $w_i \to z \to y \to E$; multiply the three slopes:

$$\frac{\partial E}{\partial y} = -(d-y), \qquad \frac{dy}{dz} = \sigma'(z), \qquad \frac{\partial z}{\partial w_i} = x_i$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y} \cdot \frac{dy}{dz} \cdot \frac{\partial z}{\partial w_i} = -(d-y) \cdot \sigma'(z) \cdot x_i$$

Gradient descent steps against the slope:

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i} = w_i + \eta\,(d-y)\,\sigma'(z)\,x_i$$

That is the generalized delta rule; with $f(z) = z$ ($f' = 1$) it reduces to ADALINE.

(c) Raw sum first (as in card 7):

$$z = 0.3(2) + (-0.4)(-1) + (-0.2)(1) = 0.8$$

$$y = \sigma(0.8) \approx 0.6900, \qquad \sigma'(0.8) = 0.6900 \times 0.3100 \approx 0.2139, \qquad d - y \approx 0.3100$$

$$\Delta\tilde{\mathbf{w}} = 0.1 \times 0.3100 \times 0.2139 \times [2, -1, 1] \approx [0.0133,\ -0.0066,\ 0.0066]$$

$$\tilde{\mathbf{w}}_{\text{new}} \approx [0.3133,\ -0.4066,\ -0.1934]$$

Compare ADALINE's 0.04 on w_1: the sigmoid's error is bigger (0.31 vs 0.2) but the step is smaller, because the factor $\sigma' \approx 0.21$ shrinks it. Where the sigmoid is flat, σ' is tiny and learning slows.

> $\sigma' = \sigma(1-\sigma)$; $\Delta w_i = \eta(d-y)\sigma'(z)x_i$; **numbers:** $y \approx 0.6900$, $\sigma' \approx 0.2139$, $\Delta\tilde{\mathbf{w}} \approx [0.0133, -0.0066, 0.0066]$.

↩ back to the card

S3.15 · 10 · Derive σ′=σ(1−σ)\sigma' = \sigma(1-\sigma)σ′=σ(1−σ) and take one generalized-delta step — Answers

(1) $0.9 \times 0.1 = 0.09$. (2) $0.31 \times 0.69 \approx 0.2139$ — the same as at $+0.8$ (the slope is symmetric about 0). (3) $z = 0$, $y = 0.5$, $\sigma' = 0.25$, $\Delta\tilde{\mathbf{w}} = 1 \times 0.5 \times 0.25 \times [1, 1] = [0.125, 0.125]$.

↩ back to the card

S3.16 · 11 · Recall drill: the three polls + quick checks — Answer

False — it needs sufficient capacity. The first three statements are true; "perfectly from just a few samples" is **false**.

↩ back to the card

S3.17 · 11 · Recall drill: the three polls + quick checks — Answer

No. Both listed problems are real: the data are not linearly separable for that one line, and you would need to relabel the data (labels for that unit are not given).

↩ back to the card

S3.18 · 11 · Recall drill: the three polls + quick checks — Answer

Threshold: **1 and 2 true, 3 false** (derivative is 0). Sigmoid: **all three true**.

↩ back to the card

S3.19 · 11 · Recall drill: the three polls + quick checks — Answer

$b = -T = -4$, $\tilde{\mathbf{w}} = [2, 1, -4]$; $z = 6 - 1 - 4 = 1 \geq 0$: fires.

↩ back to the card

S3.20 · 11 · Recall drill: the three polls + quick checks — Answer

$z = 2$ ($y = 1$, already right) but $\delta = 1 - 2 = -1$: $\tilde{\mathbf{w}} = [1, 0] + 0.1(-1)[2, 1] = [0.8, -0.1]$. ADALINE pulls z *down* to 1.5.

↩ back to the card

S3.21 · 11 · Recall drill: the three polls + quick checks — Answer

$(2, -1)$ — the coefficient vector $\mathbf{w}$.

↩ back to the card

S3.22 · 11 · Recall drill: the three polls + quick checks — Answer

The one with $z = -0.3$ (smallest $|z|$).

↩ back to the card

S3.23 · 11 · Recall drill: the three polls + quick checks — Answer

$z = 2 - 2 = 0$, so $P = \sigma(0) = 0.5$: on the boundary.

↩ back to the card

S3.24 · 11 · Recall drill: the three polls + quick checks — Answer

It never converges (keeps updating forever). Guarantee needs linearly separable data and W initialised to 0.

↩ back to the card

LECTURE 4

Derivatives, gradients, the Hessian and gradient descent

Tier A · must ⏱ 68 min Slides: lec4.learning.pdf, p1–47 only (p48 → end is not on the exam) CMU video

Guide tutorial 9

Lecture 3 said: training = pick the weights that make the average error (the **Loss**) as small as possible. That is "minimise a function", and this part of Lecture 4 is the maths of minimising: derivatives, the gradient, the Hessian, solving for a minimum by hand, and, when no formula exists, **gradient descent** (keep stepping downhill). Lecture 5 (backprop) is just the way to get the gradient for a network.

What the exam can ask from this lecture

- Derivative vs gradient: shapes (row 1×N vs column N×1), and which direction increases / decreases f fastest (Polls 1 and 3, p46).
- Find and classify a minimum: f′ = 0 then f″; ∇f = 0 then Hessian eigenvalues (the 3-variable example of p35–37 is a classic numerical).
- Run gradient descent by hand for a few steps, in one variable and in several; say what a too-big step does.
- The road from the "trivial" sign algorithm (p41–42) to the update $x^{k+1} = x^k - \eta^k f'(x^k)$ (p43), and the ascent/descent rules of p47.

⏳ **Only 20 minutes?** Read card 5 (6 min), then card 6 steps 1–5 on the practice valley (about 7 min), then card 6's exam problem on your own, checked against its worked solution (about 7 min). If 18 more minutes appear, card 4: it is the classic numerical.

📖 READ & UNDERSTAND · ⏱ 7 min · Tier A

1 · Training is "minimise a function"; the derivative is the local exchange rate

Recall the setup (L4 p2–3). You have T training pairs (X_i, d_i): an input X_i and the answer you want d_i (the *desired output*). The network $f(X; W)$ gives an output; W is the list of all its weights and biases. The **divergence** div is a number saying how far the output is from d_i. The Loss is its average:

$$\text{Loss}(W) = \frac{1}{T}\sum_{i=1}^{T} \text{div}\big(f(X_i; W),\, d_i\big)$$

$$\hat{W} = \arg\min_{W} \text{Loss}(W)$$

E.g. with T = 2 examples whose divergences are 0.4 and 0.2, Loss = (0.4 + 0.2)/2 = 0.3.

"argmin" means "the W that makes it smallest". So training is just **function minimisation**, and the rest of the lecture is how to minimise a function.

Derivative, one variable (p5–7). Zoom in far enough on any smooth curve and it looks like a straight line. So a tiny nudge Δx moves the output by $\Delta y = \alpha\, \Delta x$. That multiplier α is the **derivative**, written $f'(x)$ or dy/dx. Positive: f is rising there. Negative: falling. Zero: locally flat. Big $|f'(x)|$: steep. *E.g.* $f(x) = x^2$ at $x = 3$: $f'(3) = 6$, so a nudge of 0.01 should move f by about 0.06 (truly $3.01^2 - 9 = 0.0601$).

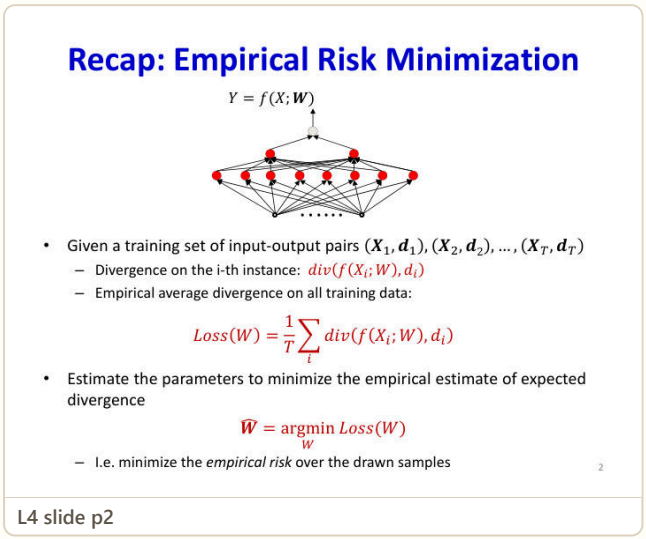
Many inputs (p8–10). A *vector* is just a list of numbers, here $\mathbf{x} = [x_1, \ldots, x_D]^\top$ written as a column (the $^\top$, "transpose", turns a row into a column and back). Now a nudge to each input adds up:

$$\Delta y = \frac{\partial y}{\partial x_1}\Delta x_1 + \frac{\partial y}{\partial x_2}\Delta x_2 + \cdots + \frac{\partial y}{\partial x_D}\Delta x_D = \nabla_{\mathbf{x}} y\ \Delta\mathbf{x}$$

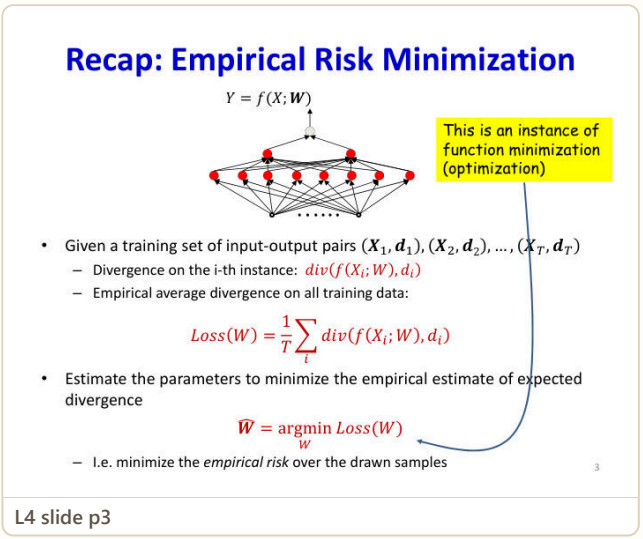
Each $\partial y/\partial x_i$ (a *partial derivative*) is the rate when only x_i moves. Since $\Delta\mathbf{x}$ is a D×1 column and Δy is one number, the multiplier $\nabla_{\mathbf{x}} y$ must be a **1×D row vector: that is the derivative**. The **gradient** is its transpose, a D×1 column the same shape as $\mathbf{x}$. This course keeps the two names apart, and Poll 1 tests it.

E.g. $y = x_1^2 + 3x_2$ at $(1, 5)$: the derivative is the row $[2x_1,\ 3] = [2,\ 3]$. Nudge both inputs by 0.1: $\Delta y \approx 2(0.1) + 3(0.1) = 0.5$ (truly $1.1^2 + 15.3 - 16 = 0.51$).

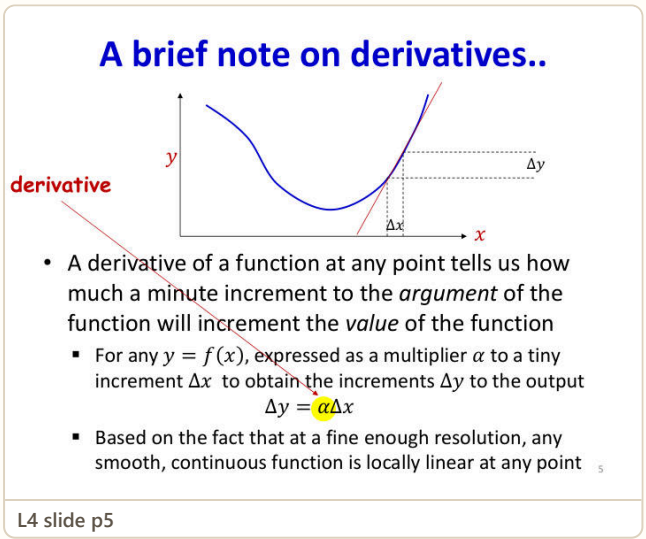
Like an exchange rate. Change a small amount of dollars and the pounds you get = rate × dollars. The derivative is that rate at the current point. With several currencies in your wallet (several inputs), the total change is the sum of rate × change for each one: one rate per input, sitting in a row. *Where it breaks:* a real exchange rate is the same for 10 or 1000 dollars; a derivative only holds for *tiny* nudges, because the curve bends.



L4 slide p2



L4 slide p3



L4 slide p5

↗ Understand more: the derivative as a multiplier (guide 9.2) ↗ row vs column (guide 9.3)

📖 **READ & UNDERSTAND · ⏱ 6 min · Tier A**

2 · The gradient points uphill, steepest; minus the gradient is steepest downhill

Why does the gradient have a direction worth caring about (p11–16)? The change in f for a small step $d\mathbf{x}$ is $df = \nabla_{\mathbf{x}} f \; d\mathbf{x}$, which is an *inner product* (multiply matching entries, add them up). For two vectors $\mathbf{u}, \mathbf{v}$:

$$\mathbf{u}^\top \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta$$

Here $\|\cdot\|$ is a vector's length and θ is the angle between them. *E.g.* $\mathbf{u} = [3, 4]$, $\mathbf{v} = [4, 3]$: $\mathbf{u}^\top \mathbf{v} = 12 + 12 = 24$, both lengths are 5, so $\cos\theta = 24/25 = 0.96$ (they point almost the same way). For a step of fixed length, $df = \|\nabla f\| \, \|d\mathbf{x}\| \cos\theta$ depends only on the angle:

- θ = 0° (step along the gradient): cos = +1, **f increases fastest**.
- θ = 180° (step exactly opposite): cos = −1, **f decreases fastest**. This is the direction gradient descent uses (Poll 4).
- θ = 90° (perpendicular): cos = 0, f does not change. You are walking along a **level curve** (a line where f is constant), so the gradient is always **perpendicular to the level curves** (p18).
- Any θ below 90° still increases f, so there are *many* uphill directions; the gradient is just the steepest one (Poll 1).
- At the top of a hill or the bottom of a valley the gradient is 0 (p17).

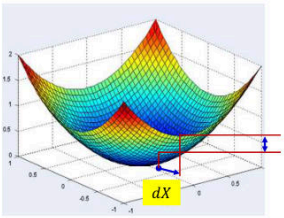
The Hessian (p19) is the table of all second derivatives: entry (i, j) is $\dfrac{\partial^2 f}{\partial x_i \, \partial x_j}$ (differentiate by x_i, then by x_j). It is a D×D symmetric matrix and it describes **curvature**: bowl, dome or saddle. For two variables:

$$\nabla^2 f = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \dfrac{\partial^2 f}{\partial x_1 \partial x_2} \\ \dfrac{\partial^2 f}{\partial x_2 \partial x_1} & \dfrac{\partial^2 f}{\partial x_2^2} \end{bmatrix}$$

E.g. $f = x_1^2 x_2$: first derivatives $2x_1x_2$ and x_1^2; differentiate again to get $\mathbf{H} = \begin{bmatrix} 2x_2 & 2x_1 \\ 2x_1 & 0 \end{bmatrix}$, which at $(1, 3)$ is $\begin{bmatrix} 6 & 2 \\ 2 & 0 \end{bmatrix}$ (symmetric, as promised).

🧭 **Like a hiking map with contour lines, in fog.** You can only feel the slope under your feet. The gradient is the arrow pointing straight up the steepest part of the slope, always at right angles to the contour line you are standing on. Walk along the contour and you stay at the same height. *Where it breaks:* on a real map "steepest" depends on your scale; the gradient is only the steepest direction for a *very small* step from where you stand.

Gradient of a scalar function of a vector



- The *derivative* $\nabla_X f(X)$ of a scalar function $f(X)$ of a multi-variate input X is a multiplicative factor that gives us the change in $f(X)$ for tiny variations in X

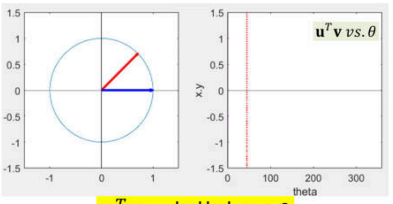
$$df(X) = \nabla_X f(X) dX$$

  - $\nabla_X f(X) = \left[\frac{\partial f(X)}{\partial x_1} \quad \frac{\partial f(X)}{\partial x_2} \quad \cdots \quad \frac{\partial f(X)}{\partial x_n}\right]$

This is a vector inner product. To understand its behavior lets consider a well-known property of inner products

L4 slide p11

A well-known vector property



$$\mathbf{u}^T\mathbf{v} = |\mathbf{u}||\mathbf{v}| \cos\theta$$

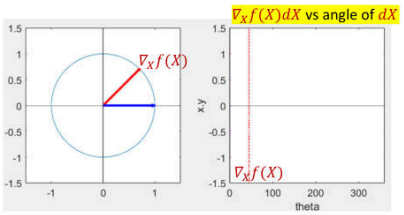
- The inner product between two vectors of fixed lengths is maximum when the two vectors are aligned
  - i.e. when $\theta = 0$

12

L4 slide p12

Properties of Gradient

Blue arrow is dX



- $df(X) = \nabla_X f(X) dX$
- For an increment dX of any given length $df(X)$ is max if dX is aligned with $\nabla_X f(X)^T$
  - The function $f(X)$ increases most rapidly if the input increment dX is exactly in the direction of $\nabla_X f(X)^T$
- The gradient is the direction of fastest increase in $f(X)$

13

L4 slide p13

↗ Understand more: the gradient points uphill (guide 9.4) ↗ level curves and the Hessian (guide 9.5) ↗ Hessian (Wikipedia)

📖 **READ & UNDERSTAND** · ⏱ **6 min** · **Tier A**

3 · Where is the minimum? First derivative zero, then check the curvature

One variable (p22–31). At every max, min or flat bend the slope is zero, so start by solving $f'(x) = 0$. These solutions are the *critical points*. Then look at $f''(x)$, the derivative of the derivative (how fast the slope itself changes):

- $f'' > 0$: the curve is a cup → **minimum**.
- $f'' < 0$: the curve is a cap → **maximum**.
- $f'' = 0$: **the test is silent**. It could be an inflection point (x^3 at 0) or still a minimum (x^4 at 0). The slide's honest version (p31): $f'' \geq 0$ at minima, ≤ 0 at maxima.

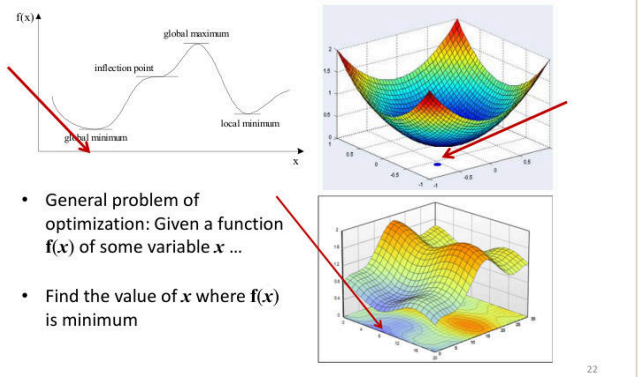
E.g. $f(x) = x^2 - 4x$: $f'(x) = 2x - 4 = 0$ gives $x = 2$; $f'' = 2 > 0$, so a minimum, $f(2) = -4$.

Many variables (p32–34). Same two steps. (1) Solve $\nabla_{\mathbf{x}} f = \mathbf{0}$ (every partial derivative zero). (2) Compute the Hessian there and find its **eigenvalues**: the numbers λ with $\mathbf{H}\mathbf{v} = \lambda\mathbf{v}$ for some direction $\mathbf{v}$. For a symmetric Hessian they are the curvatures along its main directions, found by solving $\det(\mathbf{H} - \lambda\mathbf{I}) = 0$.

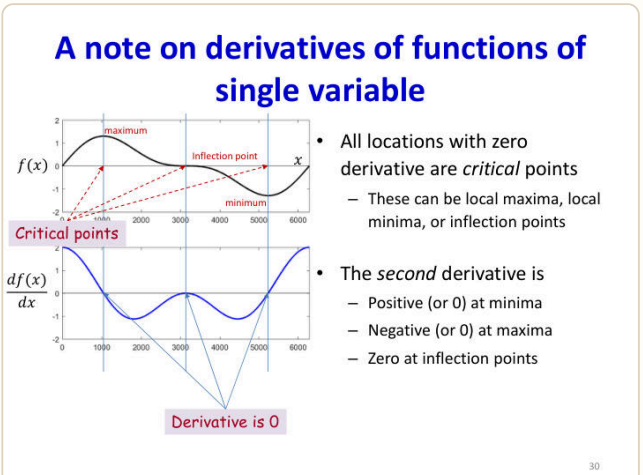
- All $\lambda > 0$ (*positive definite*): curves up in every direction → **minimum**.
- All $\lambda < 0$ (*negative definite*): **maximum**.
- Some positive, some negative: a **saddle** (up one way, down another), neither.

E.g. $\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$ (diagonal, so its eigenvalues are just 2 and −3): mixed signs, a saddle.

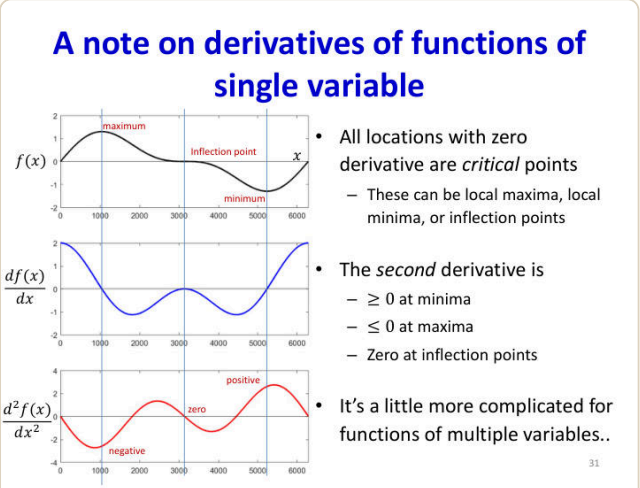
🧭 **Like a marble.** Put it at the bottom of a bowl (all curvatures positive): nudge it any way and it rolls back, a minimum. On top of an upturned bowl it rolls off in every direction, a maximum. On a Pringles crisp it sits still in the middle but rolls off sideways: a saddle. In all three the ground is flat right under the marble (gradient 0), which is why step (2) is needed.



L4 slide p22



L4 slide p30



L4 slide p31

↗ Understand more: f′ = 0 then f″ (guide 9.6) ↗ Hessian eigenvalues (guide 9.7)

🖍 **PEN & PAPER · ⏱ 18 min · Tier A**

4 · Worked example (p35–37): minimise a function of three variables by hand

Step 1 · What you're being asked, in plain words

f is a machine: feed it three numbers x_1, x_2, x_3 and it returns one number. **"Minimise" means: find the three inputs that make the output as small as possible**, and show that nothing nearby is

lower.

The tools, each in one line. A *partial derivative* $\partial f/\partial x_1$ is the slope of f when only x_1 moves (the other two are frozen and act like constants). The *gradient* ∇f is the list of all three slopes. The *Hessian* $\mathbf{H}$ is the 3×3 table of "slopes of the slopes" (curvature), and its *eigenvalues* are the curvatures along its main directions.

Step 2 · Warm-up by hand, no formulas

Practise on a smaller cousin with two inputs: $g(x_1, x_2) = x_1^2 - x_1x_2 + x_2^2 - 3x_2$. Just try some inputs:

input (x_1, x_2)	**g, term by term**	**g**
(0, 0)	0 − 0 + 0 − 0	0
(1, 1)	1 − 1 + 1 − 3	−2
(2, 2)	4 − 4 + 4 − 6	−2

Now feel the slope at (0, 0) by nudging one input at a time by 0.1. Only $x_1 = 0.1$: g = 0.01, a rise of 0.01 for a 0.1 step, so slope ≈ 0.1, nearly flat. Only $x_2 = 0.1$: g = 0.01 − 0.3 = −0.29, so slope ≈ −2.9: **the ground falls steeply as x_2 grows**. So the origin is not the bottom, and the bottom lies somewhere with positive x_2, as (1, 1) and (2, 2) suggest.

Step 3 · The one idea

At the bottom of a smooth valley the ground is flat in every direction, so every partial derivative is 0. That gives one equation per variable. But flat ground also happens on a hilltop or a saddle, so the Hessian's eigenvalues then decide: all positive means the ground curves up every way, a minimum.

Step 4 · Build the recipe on the warm-up function

Same $g(x_1, x_2) = x_1^2 - x_1x_2 + x_2^2 - 3x_2$. Five moves: slopes, set them to 0, solve, Hessian, eigenvalues.

1. $\dfrac{\partial g}{\partial x_1} = 2x_1 - x_2$.
   Why: $x_1^2 \to 2x_1$; in $-x_1x_2$ the x_2 is a frozen constant, so it gives $-x_2$; the terms without x_1 give 0.
2. $\dfrac{\partial g}{\partial x_2} = -x_1 + 2x_2 - 3$.
   Why: the same, with x_1 frozen this time.
3. Set both to 0. The first gives $x_2 = 2x_1$. Put that into the second: $-x_1 + 4x_1 - 3 = 0$, so $x_1 = 1$ and $x_2 = 2$.
   Why: flat in both directions at once.
4. Hessian: differentiate each slope again. $2x_1 - x_2$ gives 2 (by x_1) and −1 (by x_2); $-x_1 + 2x_2 - 3$ gives −1 and 2. So $\mathbf{H} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$.
   Why: curvature = how the slopes themselves change.
5. Eigenvalues: subtract λ on the diagonal and set the determinant to 0: $(2 - \lambda)^2 - (-1)(-1) = 0$, so $2 - \lambda = \pm 1$ and $\lambda = 1$ or 3. Both positive → **minimum**, with $g(1, 2) = 1 - 2 + 4 - 6 = -3$.

Step 5 · Check it

Slopes at (1, 2): $2(1) - 2 = 0$ ✓ and $-1 + 4 - 3 = 0$ ✓. Eigenvalues: sum 1 + 3 = 4 = diagonal sum 2 + 2 ✓; product 3 = det = 2·2 − (−1)(−1) ✓. Neighbours are higher: g(2, 2) = −2, g(1, 1) = −2, g(0, 2) = −2, all above −3 ✓.

Edge case: $x_1^2 - x_2^2$ has slopes $(2x_1, -2x_2) = (0, 0)$ at the origin, but its Hessian has eigenvalues 2 and −2: mixed signs, a **saddle**. Flat ground alone proves nothing; step (5) is required.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

Minimise

$$f(x_1, x_2, x_3) = x_1^2 + x_1(1 - x_2) + x_2^2 - x_2x_3 + x_3^2 + x_3$$

(a) Find the gradient. (b) Solve $\nabla f = \mathbf{0}$. (c) Find the Hessian and its eigenvalues. (d) Is the point a minimum? What is f there?

Do it the same way: steps 4–5 with these numbers, now with three variables (three slopes, a 3×3 Hessian).

✍ Hint: S4.1 (end of this lecture)

✍ Full worked solution: S4.2 (end of this lecture)

Step 7 · Try alone

1. $f(x) = x^2 - 6x + 1$. Where is the minimum, and what is f there?
2. $f(x_1, x_2) = x_1^2 + 3x_2^2 - 2x_1 + 6x_2$. Find the flat point and classify it.
3. $f(x_1, x_2) = x_1^3 - 3x_1 + x_2^2 - 2x_2$ has *two* flat points. Classify each (here the Hessian depends on where you stand).

✍ Answers: S4.3 (end of this lecture)

↗ Understand more: the same example step by step (guide 9.8) ↗ practice: a Hessian that moves (guide 9.9)

📖 READ & UNDERSTAND · ⏱ 6 min · Tier A

5 · Gradient descent: no formula for the minimum? Walk downhill in small steps

For a real network you cannot solve $\nabla\text{Loss} = 0$ on paper (p38). So you guess a starting point x^0 and improve it again and again: an **iterative** method (p39). Two questions: which direction, and how big a step.

Direction (p40). If $f'(x) > 0$, moving right goes up, so move left; if $f'(x) < 0$, move right. The slides build the rule in three versions:

- **p41, "trivial" algorithm:** while $f'(x^k) \neq 0$: if the sign of $f'(x^k)$ is positive, $x^{k+1} = x^k - \text{step}$, else $x^{k+1} = x^k + \text{step}$.
- **p42, same thing in one line:** $x^{k+1} = x^k - \text{sign}\big(f'(x^k)\big) \cdot \text{step}$. sign(·) is +1 for a positive number and −1 for a negative one.
- **p43, real gradient descent:** replace the sign by the slope itself, scaled by η:

$$x^{k+1} = x^k - \eta^k \, f'(x^k)$$

η ("eta") is the **step size** (learning rate), and k counts the steps. *E.g.* $f(x) = x^2$ at $x^k = 1$ with η = 0.1: $f'(1) = 2$, so $x^{k+1} = 1 - 0.1 \cdot 2 = 0.8$. Why is p43 better than p41? A fixed step never shrinks, so near the minimum it keeps jumping over it. Using f′ makes the step **shrink automatically** as the slope flattens near the bottom (card 7 shows both by hand).

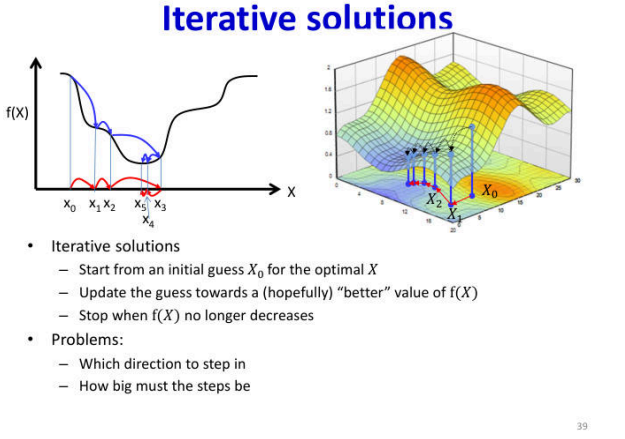
Many variables (p46–47). The gradient (transposed, so it is a column like $\mathbf{x}$) is the steepest way up, so:

$$\text{minimum (descent):} \quad \mathbf{x}^{k+1} = \mathbf{x}^k - \eta^k \, \nabla_{\mathbf{x}} f(\mathbf{x}^k)^\top$$
$$\text{maximum (ascent):} \quad \mathbf{x}^{k+1} = \mathbf{x}^k + \eta^k \, \nabla_{\mathbf{x}} f(\mathbf{x}^k)^\top$$

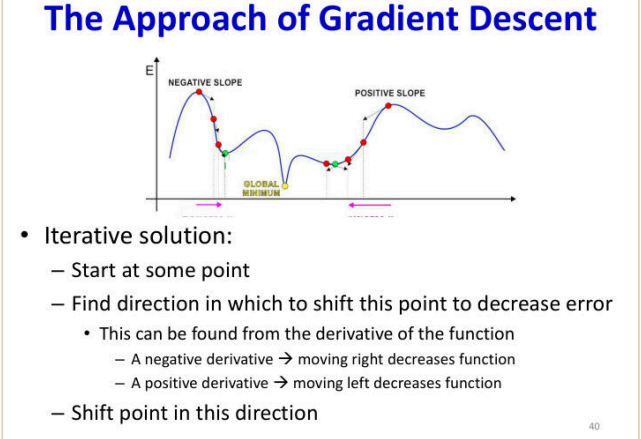
E.g. $f = x_1^2 + x_2^2$ at $\mathbf{x}^k = [1, -2]^\top$, η = 0.1: the gradient is $[2, -4]^\top$, so descent gives $[0.8, -1.6]^\top$ (closer to the bottom at 0) and ascent gives $[1.2, -2.4]^\top$.

"Many solutions to choosing step size" (p47) is the subject of Lecture 6. Training a network is this same update with $\mathbf{x}$ replaced by all the weights W and f by Loss(W) (background, not on the exam).

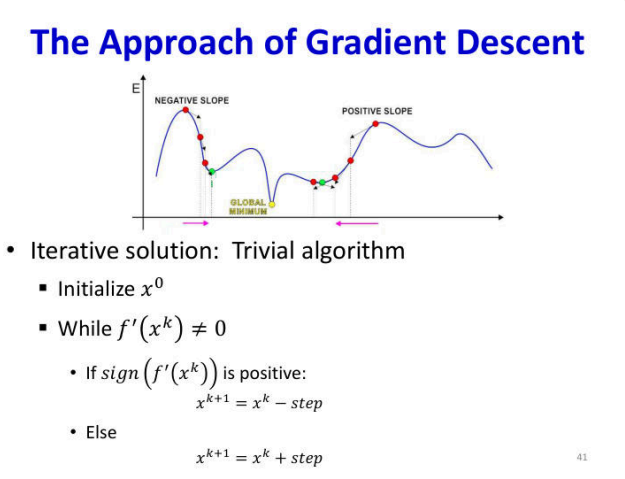
Like the "hot and cold" game, or adjusting an old shower. Each turn you feel which way is warmer and move that way, by an amount that shrinks as you get close. Turn the shower tap too hard (η too big) and you swing from freezing to scalding and back, never settling. *Where it breaks:* in the game someone tells you the answer's direction directly; gradient descent only knows the local slope, so in a hilly landscape it can settle in a nearby dip that is not the lowest one.



L4 slide p39



L4 slide p40



L4 slide p41

↗ Understand more: from sign steps to η·f′ (guide 9.10) ↗ many variables and stopping (guide 9.11) ↗ 3Blue1Brown: gradient descent ↗ d2l: gradient descent

✏️ PEN & PAPER · ⏱ 12 min · Tier A

6 · Run gradient descent by hand, and watch a big step go wrong

Step 1 · What you're being asked, in plain words

You get a one-input function shaped like a valley, and you must find its lowest point *without* solving for it: **gradient descent**. Stand at some x, measure the slope $f'(x)$, and step *against* it by $\eta\times$ slope. η ("eta", the step size or learning rate) is a number you choose. $x^0, x^1, x^2, \ldots$ are your positions after 0, 1, 2… steps. You also check what happens when η is too big.

Step 2 · Warm-up by hand, no formulas

Practise on $f(x) = (x-1)^2$, a valley whose lowest point is at $x = 1$ (there f = 0; anywhere else the square is positive). Pretend you don't know that.

Stand at $x = 3$: $f(3) = 4$. Nudge right to 3.1: $f(3.1) = 2.1^2 = 4.41$. Going right raised f by 0.41 for a 0.1 step, so the slope is about $0.41/0.1 \approx 4$: **downhill is to the left**.

Stand at $x = 2$: $f(2) = 1$, $f(2.1) = 1.21$. Slope about 2. Notice the slope is steeper at 3 (2 away from the bottom) than at 2 (1 away): **slope ∝ distance from the bottom**.

Step 3 · The one idea

Step = −η × slope. The slope is big far from the bottom and 0 at the bottom, so the steps shrink by themselves. For a parabola the slope is exactly proportional to the distance from the bottom, so every step multiplies the remaining distance by the same number.

Step 4 · Build it, step by step

1. $f(x) = x^2 - 2x + 1$, so $f'(x) = 2x - 2 = 2(x-1)$.
   Why: expand the square, differentiate term by term. At $x = 3$ this gives 4, as the warm-up felt.
2. Update: $x^{k+1} = x^k - \eta \cdot 2(x^k - 1)$.
   Why: new position = old position − η × slope.
3. Three steps from $x^0 = 3$ with $\eta = 0.25$: $x^1 = 3 - 0.25 \cdot 2(3-1) = 3 - 1 = 2$; $x^2 = 2 - 0.25 \cdot 2(1) = 1.5$; $x^3 = 1.5 - 0.25 \cdot 2(0.5) = 1.25$.
   Why: plug the current x into the slope, scale by η, subtract; repeat with the new x.
4. Bigger steps, two each from $x^0 = 3$. η = 0.5: $3 - 0.5 \cdot 4 = 1$, then stays at 1 (slope 0). η = 1: $3 - 4 = -1$, then $-1 - 1 \cdot 2(-2) = 3$: back where it started. η = 1.2: $3 - 4.8 = -1.8$, then $-1.8 - 1.2 \cdot 2(-2.8) = 4.92$: farther away each time.
   Why: same rule; only η changed.
5. Write $e^k = x^k - 1$, the *gap* (signed distance to the answer). Subtract 1 from both sides of the update: $e^{k+1} = e^k - 2\eta e^k = (1 - 2\eta)\, e^k$.
   Why: the gap is multiplied by the same factor $(1 - 2\eta)$ every step, which explains all of step 4.

6. The gap shrinks only if $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$. Factor 0 (η = 0.5) lands in one step; factor −1 (η = 1) bounces forever; $|\text{factor}| > 1$ (η > 1) diverges.
   Why: repeated multiplication by a number shrinks only if the number's size is below 1.

Step 5 · Check it

η = 0.25: gaps 2 → 1 → 0.5 → 0.25, ratio 0.5 = $1 - 2(0.25)$ ✓. η = 1.2: gaps 2 → −2.8 → 3.92, ratio −1.4 = $1 - 2(1.2)$ ✓. **Edge case:** start at $x^0 = 1$. The slope is $2(1 - 1) = 0$, so the step is 0 and you stay put: the bottom is a resting point.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

$f(x) = (x - 3)^2$, start at $x^0 = 0$.

(a) Write $f'(x)$ and the update rule. (b) Do 4 steps with η = 0.25. (c) Do 2 steps with η = 0.5, with η = 1 and with η = 1.2. (d) Show that the distance to the answer, $x^k - 3$, is multiplied by the same factor every step, and say which η values work.

Do it the same way: steps 4–5 with these numbers.

✍ Full worked solution: S4.4 (end of this lecture)

Step 7 · Try alone

1. $f(x) = x^2$, $x^0 = 4$, $\eta = 0.25$. Three steps.
2. $f(x) = (x + 1)^2$, $x^0 = 1$, $\eta = 0.75$. Two steps. What does the gap do?
3. $f(x) = 2x^2$. Find the gap factor per step, the η that lands in one step, and the η values that diverge.

✍ Answers: S4.5 (end of this lecture)

↗ Understand more: twice the step size (guide 9.12) ↗ d2l: gradient descent (the learning-rate pictures)

🖍 PEN & PAPER · ⏱ 10 min · Tier A

7 · Two more by hand: the sign algorithm stalls, and descent in three variables

Step 1 · What you're being asked, in plain words

Two short jobs. **A:** the p42 *sign algorithm* only looks at whether the slope is positive or negative, then moves a *fixed* distance ("step") the downhill way. sign(·) is +1 for a positive number, −1 for a negative one. You compare it with real gradient descent (p43), which moves η × slope. **B:** gradient descent when f has three inputs: the slope is now a list of three numbers (the gradient), and each coordinate steps against its own slope.

Step 2 · Warm-up by hand, no formulas

Sign algorithm on $f(x) = x^2$ (bottom at 0), start 2.5, fixed step 1. At 2.5 the curve rises to the right, so step left: 1.5. Still rising to the right: 0.5. Still: −0.5. Now it rises to the *left*, so step right: 0.5. Then −0.5, 0.5, −0.5… It never gets closer than 0.5.

Two inputs, $f = x_1^2 + x_2^2$ (bottom at (0, 0)), at (1, 2). Slope in x_1 alone: $2x_1 = 2$. Slope in x_2 alone: $2x_2 = 4$. With η = 0.25, move each coordinate against its own slope: $1 - 0.25(2) = 0.5$ and $2 - 0.25(4) = 1$. New point (0.5, 1); f fell from 5 to 1.25.

Step 3 · The one idea

A fixed step can't shrink, so near the bottom it keeps jumping over it; a step of η × slope shrinks as the ground flattens. With several inputs, do the one-variable update on every coordinate at the same time, using slopes all measured at the current point.

Step 4 · Build it, step by step

1. Sign rule: $x^{k+1} = x^k - \operatorname{sign}\big(f'(x^k)\big) \cdot \text{step}$.
   Why: positive slope → minus one step (left); negative → plus one step (right).
2. Gradient rule: $x^{k+1} = x^k - \eta\, f'(x^k)$.
   Why: same direction, but the length now follows the slope.
3. Vector rule: $\mathbf{x}^{k+1} = \mathbf{x}^k - \eta\, \nabla f(\mathbf{x}^k)^\top$.
   Why: it is the gradient rule written once for all coordinates.
4. Recipe for one vector step: (i) plug the current point into each partial derivative; (ii) multiply the list by η; (iii) subtract coordinate by coordinate; (iv) compute f to see it fell.

Step 5 · Check it

Warm-up B: $f(0.5, 1) = 0.25 + 1 = 1.25 < 5$ ✓. Warm-up A with the gradient rule instead, η = 0.25: $2.5 - 0.25(2 \cdot 2.5) = 1.25$, then 0.625… it keeps halving, never bouncing ✓. **Edge case:** at the bottom every slope is 0, so the gradient rule doesn't move; the sign rule would also stop there, but only if it happens to land exactly on it.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

A (p41–43). $f(x) = (x - 3)^2$, start at $x^0 = 0.5$. Run the p42 sign algorithm with step = 1 for five steps. Does it ever stop? Then do one step of p43 with η = 0.25 from the same start.

B (p47). Take card 4's function, whose gradient you found there:

$$\nabla f^\top = \begin{bmatrix} 2x_1 + 1 - x_2 \\ -x_1 + 2x_2 - x_3 \\ -x_2 + 2x_3 + 1 \end{bmatrix}$$

Start at $\mathbf{x}^0 = [0, 0, 0]^\top$ and do two steps of gradient descent with η = 0.25. Is f going down towards the minimum value −1?

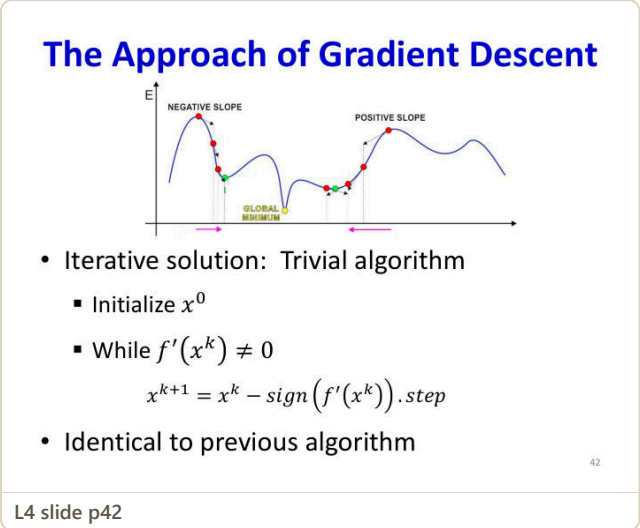
Do it the same way: steps 4–5 with these numbers (A like warm-up A, B with the step 4 recipe (i)–(iv)).

✍ Full worked solution: S4.6 (end of this lecture)

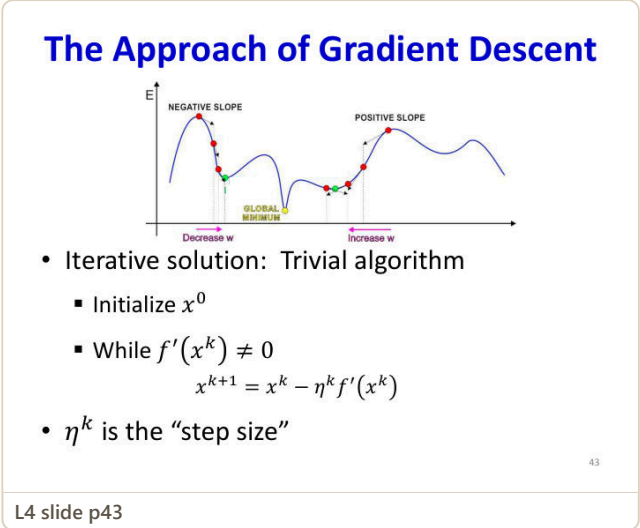
Step 7 · Try alone

1. Sign algorithm on $f(x) = (x+2)^2$, start 0, step 0.5. Run it until it stops (or doesn't).
2. One gradient step on the same f from 0 with η = 0.1.
3. $f = x_1^2 + x_2^2$ from (2, −2) with η = 0.5; and card 4's function from (−1, −1, −1) with any η. Where do you end up after one step?

✍ Answers: S4.7 (end of this lecture)



L4 slide p42



L4 slide p43

↗ Understand more: from sign steps to η·f′ (guide 9.10) ↗ gradient descent in many variables (guide 9.11)

🔁 **RECALL DRILL · ⏱ 3 min · Tier A**

8 · Recall drill: the class polls and quick checks

1. **Poll 1 / Poll 3** (p20–21, p44–45). Select all true: (A) at any X there may be many directions in which f increases; (B) the gradient's direction is where f increases fastest; (C) the gradient is the derivative of f w.r.t. X.

   ✍ Answer: S4.8 (end of this lecture)

2. **Poll 1 Q2.** y = f(x), x an N×1 column. Shape of dy/dx?

   ✍ Answer: S4.9 (end of this lecture)

3. **Poll 2** (p24–25). Which criterion holds at a minimum of f(x)? (1) f′ = 0 is the only condition; (2) f′ = 0 and f″ negative; (3) f′ = 0 and f″ positive.

   ✍ Answer: S4.10 (end of this lecture)

4. Which direction decreases f fastest from a point? Which leaves f unchanged?

   ✍ Answer: S4.11 (end of this lecture)

5. Hessian eigenvalues 3, −1 at a point where ∇f = 0. What is the point?

   ✍ Answer: S4.12 (end of this lecture)

6. $x^k = 2$, $f'(x^k) = -4$, η = 0.1. What is x^{k+1}? And for a *maximum* (ascent)?

   ✍ Answer: S4.13 (end of this lecture)

7. Why does the p41 sign algorithm usually never stop on its own?

✍️ Answer: S4.14 (end of this lecture)

⚠️ Traps

- **Derivative ≠ gradient here.** Derivative = 1×N row; gradient = its transpose, N×1 column. Polls 1 and 3 test this.
- **Poll 2 key** "$f' = 0$ and $f'' > 0$" is sufficient, not necessary: x^4 has a minimum with $f'' = 0$. The slide itself (p31) says $f'' \geq 0$ at minima. Pick the key on the quiz; know the counterexample.
- **$f'' = 0$ does not mean inflection.** x^3 (inflection) and x^4 (minimum) both have $f''(0) = 0$.
- **Slide p43 figure:** the "Increase w / Decrease w" labels are swapped. Rule: positive slope → decrease x; negative slope → increase x.
- **Ascent vs descent sign:** minimum ⇒ minus the gradient; maximum ⇒ plus (p47). The gradient is transposed so it has the same column shape as x.

Solutions and answers — Lecture 4

S4.1 · 4 · Worked example (p35–37): minimise a function of three variables by hand — Hint

Expand the bracket before differentiating. For the eigenvalues, expand the 3×3 determinant along the first row and look for a common factor.

↩ back to the card

S4.2 · 4 · Worked example (p35–37): minimise a function of three variables by hand — Full worked solution

Expand first. $x_1(1 - x_2) = x_1 - x_1x_2$, so

$$f = x_1^2 + x_1 - x_1x_2 + x_2^2 - x_2x_3 + x_3^2 + x_3$$

Why: each term is now a plain power or a plain product, so its partial derivatives can be read off one at a time.

(a) Gradient: one slope per variable (freeze the other two each time).

$$\frac{\partial f}{\partial x_1} = 2x_1 + 1 - x_2$$

Why: $x_1^2 \to 2x_1$, $x_1 \to 1$, $-x_1x_2 \to -x_2$ (x_2 frozen); terms without x_1 give 0.

$$\frac{\partial f}{\partial x_2} = -x_1 + 2x_2 - x_3$$

Why: $-x_1x_2 \to -x_1$, $x_2^2 \to 2x_2$, $-x_2x_3 \to -x_3$.

$$\frac{\partial f}{\partial x_3} = -x_2 + 2x_3 + 1$$

Why: $-x_2x_3 \to -x_2$, $x_3^2 \to 2x_3$, $x_3 \to 1$.

$$\nabla f^\top = \begin{bmatrix} 2x_1 + 1 - x_2 \\ -x_1 + 2x_2 - x_3 \\ -x_2 + 2x_3 + 1 \end{bmatrix}$$

(b) Flat in every direction: set all three to zero.

$$(1)\quad 2x_1 + 1 - x_2 = 0 \;\Rightarrow\; x_2 = 2x_1 + 1$$

$$(3)\quad -x_2 + 2x_3 + 1 = 0 \;\Rightarrow\; x_2 = 2x_3 + 1$$

Why: equations (1) and (3) each give x_2 directly, so solve them for x_2 first.

Equate the two expressions for x_2: $2x_1 + 1 = 2x_3 + 1$, so $x_3 = x_1$. Substitute $x_2 = 2x_1 + 1$ and $x_3 = x_1$ into (2):

$$-x_1 + 2(2x_1 + 1) - x_1 = 0$$

$$-x_1 + 4x_1 + 2 - x_1 = 0$$

$$2x_1 + 2 = 0 \;\Rightarrow\; x_1 = -1$$

$$x_3 = x_1 = -1, \qquad x_2 = 2(-1) + 1 = -1$$

Check in all three: (1) $-2 + 1 + 1 = 0$ ✓; (2) $1 - 2 + 1 = 0$ ✓; (3) $1 - 2 + 1 = 0$ ✓.

(c) Hessian: differentiate each slope again. Row i holds the derivatives of $\partial f / \partial x_i$ by x_1, x_2, x_3:

slope	by x_1	by x_2	by x_3
$2x_1 + 1 - x_2$	2	−1	0

$-x_1 + 2x_2 - x_3$	-1	2	-1
$-x_2 + 2x_3 + 1$	0	-1	2

$$\mathbf{H} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

Why: curvature = how the slopes change. H is symmetric (as it must be) and constant, because f is quadratic.

Eigenvalues. Subtract λ on the diagonal and set the determinant to zero, expanding along the first row (its last entry is 0, so only two terms survive):

$$\begin{aligned} \det(\mathbf{H} - \lambda\mathbf{I}) &= (2-\lambda)\big[(2-\lambda)^2 - (-1)(-1)\big] - (-1)\big[(-1)(2-\lambda) - (-1)\cdot 0\big] \\ &= (2-\lambda)\big[(2-\lambda)^2 - 1\big] - (2-\lambda) \\ &= (2-\lambda)\big[(2-\lambda)^2 - 2\big] = 0 \end{aligned}$$

Why: $(2-\lambda)$ is a common factor, so the cubic splits into "first factor = 0" or "bracket = 0".

$$2 - \lambda = 0 \;\Rightarrow\; \lambda = 2$$

$$(2-\lambda)^2 = 2 \;\Rightarrow\; \lambda = 2 \mp \sqrt{2} = 0.586 \text{ or } 3.414$$

Check: sum $2 + 0.586 + 3.414 = 6$ = diagonal sum $2 + 2 + 2$ ✓; product $2 \cdot (4 - 2) = 4 = \det \mathbf{H}$ ✓.

(d) Classify and evaluate. All three eigenvalues are positive, so H is positive definite: the ground curves up in every direction, a **minimum** (and, since H is the same everywhere, the only one, so the global minimum).

$$f(-1,-1,-1) = 1 + (-1)(1-(-1)) + 1 - (-1)(-1) + 1 + (-1) = 1 - 2 + 1 - 1 + 1 - 1 = -1$$

Check: move one coordinate from −1 to 0, e.g. $f(0,-1,-1) = 0$, $f(-1,0,-1) = 0$, $f(-1,-1,0) = 0$: all higher than −1 ✓.

(a) $\nabla f^\top = [2x_1 + 1 - x_2,\ -x_1 + 2x_2 - x_3,\ -x_2 + 2x_3 + 1]^\top$. (b) $\mathbf{x} = [-1,-1,-1]^\top$. (c) H as above, eigenvalues $2 - \sqrt{2} = 0.586$, 2, $2 + \sqrt{2} = 3.414$. (d) All > 0 ⇒ positive definite ⇒ minimum, $f = -1$.

↩ back to the card

S4.3 · 4 · Worked example (p35–37): minimise a function of three variables by hand — Answers

1. $f'(x) = 2x - 6 = 0$ gives $x = 3$; $f'' = 2 > 0$, a minimum; $f(3) = 9 - 18 + 1 = -8$.
2. Slopes $(2x_1 - 2,\ 6x_2 + 6) = (0,0)$ give $(1,-1)$. $\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$, eigenvalues 2 and 6, both positive: minimum, $f = 1 + 3 - 2 - 6 = -4$.
3. Slopes $(3x_1^2 - 3,\ 2x_2 - 2)$ vanish at (1, 1) and (−1, 1). $\mathbf{H} = \begin{bmatrix} 6x_1 & 0 \\ 0 & 2 \end{bmatrix}$. At (1, 1): eigenvalues 6, 2 → local minimum, f = −3. At (−1, 1): −6, 2 → saddle, f = 1. Not a global minimum: f(−3, 1) = −19.

↩ back to the card

S4.4 · 6 · Run gradient descent by hand, and watch a big step go wrong — Full worked solution

(a) Slope and update rule.

$$f(x) = x^2 - 6x + 9 \;\Rightarrow\; f'(x) = 2x - 6 = 2(x-3)$$

Why: expand the square and differentiate term by term. Check the sign: at $x = 0$, $f'(0) = -6 < 0$, so downhill is to the right, towards 3.

$$x^{k+1} = x^k - \eta\, f'(x^k) = x^k - \eta \cdot 2(x^k - 3)$$

Why: new position = old position − η × slope.

(b) Four steps with η = 0.25. Each line: plug the current x into the slope, scale by η, subtract.

$$x^1 = 0 - 0.25 \cdot 2(0 - 3) = 0 + 1.5 = 1.5$$

$$x^2 = 1.5 - 0.25 \cdot 2(1.5 - 3) = 1.5 + 0.75 = 2.25$$

$$x^3 = 2.25 - 0.25 \cdot 2(2.25 - 3) = 2.25 + 0.375 = 2.625$$

$$x^4 = 2.625 - 0.25 \cdot 2(2.625 - 3) = 2.625 + 0.1875 = 2.8125$$

Why it slows down: the steps 1.5, 0.75, 0.375, 0.1875 halve each time, because the slope halves as the gap halves.

(c) Bigger steps, two each from $x^0 = 0$.

η	x^1	x^2	what happens
0.5	$0 - 0.5 \cdot (-6) = 3$	$3 - 0.5 \cdot 0 = 3$	lands on the minimum in one step, then stays (slope 0)
1	$0 - 1 \cdot (-6) = 6$	$6 - 1 \cdot 6 = 0$	bounces 0 → 6 → 0 → 6 forever
1.2	$0 - 1.2 \cdot (-6) = 7.2$	$7.2 - 1.2 \cdot 8.4 = -2.88$	**diverges**: next 11.232, −8.525…

Why: same rule, only η changes; at 7.2 the slope is $2(7.2 - 3) = 8.4$.

(d) The gap factor. Let $e^k = x^k - 3$. Subtract 3 from both sides of the update:

$$x^{k+1} - 3 = (x^k - 3) - 2\eta(x^k - 3)$$

$$e^{k+1} = (1 - 2\eta)\, e^k$$

Why: the slope $2(x^k - 3)$ is exactly $2e^k$, so the gap is multiplied by the same number every step, and $e^k = (1 - 2\eta)^k e^0$.

η	**factor** $1 - 2\eta$	**gaps from** $e^0 = -3$	**verdict**
0.25	0.5	−3, −1.5, −0.75, …	converges
0.5	0	−3, 0	one step
1	−1	−3, 3, −3, …	oscillates forever
1.2	−1.4	−3, 4.2, −5.88, …	diverges

The gap shrinks to 0 exactly when $|1 - 2\eta| < 1$, i.e. $-1 < 1 - 2\eta < 1$, i.e. $0 < \eta < 1$.

(a) $f'(x) = 2(x - 3)$, $x^{k+1} = x^k - 2\eta(x^k - 3)$. (b) 1.5, 2.25, 2.625, 2.8125. (c) η = 0.5: 3, 3; η = 1: 6, 0; η = 1.2: 7.2, −2.88. (d) gap × $(1 - 2\eta)$ each step: converges for 0 < η < 1, η = 0.5 in one step, η = 1 oscillates, η > 1 diverges.

This is a preview of Lecture 6: for a parabola with second derivative a (here a = 2), the best step is η = 1/a and anything from 2/a up fails to converge.

↩ back to the card

S4.5 · 6 · Run gradient descent by hand, and watch a big step go wrong — Answers

1. $f' = 2x$, factor $1 - 2(0.25) = 0.5$: 2, 1, 0.5.

2. $f' = 2(x+1)$, factor $1 - 2(0.75) = -0.5$. Gap $2 \to -1 \to 0.5$, so x = −2, then −0.5: it overshoots to the other side each step but still closes in.
3. $f' = 4x$, so $x^{k+1} = (1-4\eta)x^k$. One step: η = 0.25 (factor 0). Bounces forever at η = 0.5 (factor −1); diverges for η > 0.5. Matches 1/a and 2/a with a = 4.

↩ back to the card

S4.6 · 7 · Two more by hand: the sign algorithm stalls, and descent in three variables — Full worked solution

A, sign algorithm (p42). Rule: $x^{k+1} = x^k - \text{sign}\big(f'(x^k)\big) \cdot 1$, with $f'(x) = 2(x-3)$.

Why: only the sign of the slope is used: negative left of 3 (step +1), positive right of 3 (step −1).

k	x^k	$f'(x^k) = 2(x^k - 3)$	sign	x^{k+1}
0	0.5	−5	−	1.5
1	1.5	−3	−	2.5
2	2.5	−1	−	3.5
3	3.5	+1	+	2.5
4	2.5	−1	−	3.5

Does it stop? No. It **bounces 2.5 ↔ 3.5 forever, 0.5 away from the answer**.

Why: the step is always 1, so it can never land on 3 from 0.5 (3 − 0.5 = 2.5 is not a whole number of steps), and the stopping test $f'(x) = 0$ is never met at the points it visits.

A, one p43 step, η = 0.25.

$$x^1 = 0.5 - 0.25 \cdot 2(0.5 - 3) = 0.5 - 0.25 \cdot (-5) = 0.5 + 1.25 = 1.75$$

Why it is better: the gap to 3 went from 2.5 to 1.25 (halved: factor $1 - 2\eta = 0.5$, card 6), and the next steps shrink with the slope instead of overshooting.

B, step 1 at $\mathbf{x}^0 = [0, 0, 0]^\top$. (i) Plug in:

$$\nabla f(\mathbf{x}^0)^\top = [2 \cdot 0 + 1 - 0,\ \ -0 + 0 - 0,\ \ -0 + 0 + 1] = [1,\ 0,\ 1]$$

(ii)–(iii) Scale by η and subtract, coordinate by coordinate:

$$\mathbf{x}^1 = [0, 0, 0] - 0.25\,[1, 0, 1] = [-0.25,\ 0,\ -0.25]$$

(iv) $f(\mathbf{x}^1) = 0.0625 - 0.25 + 0 - 0 + 0.0625 - 0.25 = -0.375$, below $f(\mathbf{x}^0) = 0$ ✓.

B, step 2 at $\mathbf{x}^1$. *Why recompute:* the slopes change when the point moves.

$$\frac{\partial f}{\partial x_1} = 2(-0.25) + 1 - 0 = 0.5$$

$$\frac{\partial f}{\partial x_2} = -(-0.25) + 2(0) - (-0.25) = 0.5$$

$$\frac{\partial f}{\partial x_3} = -0 + 2(-0.25) + 1 = 0.5$$

$$\mathbf{x}^2 = [-0.25,\ 0,\ -0.25] - 0.25\,[0.5,\ 0.5,\ 0.5] = [-0.375,\ -0.125,\ -0.375]$$

(iv) $f(\mathbf{x}^2) = 0.1406 - 0.375 \cdot 1.125 + 0.0156 - 0.0469 + 0.1406 - 0.375 = -0.547$.

k	$\mathbf{x}^k$	$f(\mathbf{x}^k)$
0	[0, 0, 0]	0

1	[−0.25, 0, −0.25]	−0.375
2	[−0.375, −0.125, −0.375]	−0.547

Going down? Yes: 0 → −0.375 → −0.547, heading for −1 at $[-1, -1, -1]$, and each coordinate is moving towards −1.

A: sign steps 1.5, 2.5, 3.5, 2.5, 3.5… (never settles); the η·f′ step gives 1.75. B: x^1 = [−0.25, 0, −0.25] (f = −0.375), x^2 = [−0.375, −0.125, −0.375] (f = −0.547).

Preview of Lecture 6: with η = 0.7 the same run gives f = −0.42, −0.35, +0.09, going *up*. For a quadratic, η must stay below 2 ÷ (largest Hessian eigenvalue) = 2 ÷ 3.414 = 0.586.

↩ back to the card

S4.7 · 7 · Two more by hand: the sign algorithm stalls, and descent in three variables — Answers

1. Slope $2(x + 2)$ is positive, so it moves left: −0.5, −1, −1.5, −2. At −2 the slope is exactly 0 and it stops. Lucky: 2 is a whole number of 0.5-steps. From 0.5 with step 1 it would never land.
2. $f'(0) = 4$, so $x^1 = 0 - 0.1 \cdot 4 = -0.4$.
3. Gradient (4, −4), so $(2, -2) - 0.5(4, -4) = (0, 0)$: the minimum in one step. Card 4's gradient at (−1, −1, −1) is (0, 0, 0), so you don't move: you are already at the minimum.

↩ back to the card

S4.8 · 8 · Recall drill: the class polls and quick checks — Answer

A and B. C is false in this course: the derivative is a 1×N row, the gradient is its transpose (a column).

↩ back to the card

S4.9 · 8 · Recall drill: the class polls and quick checks — Answer

1 × N row vector (so that Δy = (dy/dx) Δx is a single number).

↩ back to the card

S4.10 · 8 · Recall drill: the class polls and quick checks — Answer

(3), the intended answer. Strictly it is sufficient, not necessary: x^4 has a minimum at 0 with f″ = 0. (2) describes a maximum.

↩ back to the card

S4.11 · 8 · Recall drill: the class polls and quick checks — Answer

Exactly opposite to the gradient (p46). Perpendicular to the gradient = along the level curve: no change.

↩ back to the card

S4.12 · 8 · Recall drill: the class polls and quick checks — Answer

A saddle point (mixed signs): neither minimum nor maximum.

↩ back to the card

S4.13 · 8 · Recall drill: the class polls and quick checks — Answer

Descent: $2 - 0.1 \cdot (-4) = 2.4$. Ascent: $2 + 0.1 \cdot (-4) = 1.6$.

↩ back to the card

S4.14 · 8 · Recall drill: the class polls and quick checks — Answer

Its step is fixed, so near the minimum it jumps over it back and forth; its stopping test $f' = 0$ is almost never hit exactly. $\eta \cdot f'$ shrinks the step as the slope flattens.

↩ back to the card

LECTURE 5 · STANFORD CS231N L4

Neural networks and backpropagation on computational graphs

Tier A · must ⏱ 115 min Slides: deep- stanford - lecture_4.pdf, slides 32–39 and 49–92 (PDF p32–39, p48–91)

📗 CS231n backprop notes 📘 Guide tutorial 11 🎥 3Blue1Brown: backprop

The CMU Lecture 5 was replaced by this Stanford one. It does the same job in a friendlier way: to train a network you need the slope of the loss with respect to every weight, and **backpropagation** gets all of them by drawing the formula as a graph of tiny operations and passing numbers backwards through it. Lecture 4 gave you derivatives and gradient descent; this lecture is the machine that produces the gradients gradient descent eats.

What the exam can ask from this lecture

- Fill a computational graph by hand: forward values first, then gradients backwards ($f = (x + y)z$ with $x = -2$, $y = 5$, $z = -4$; the sigmoid neuron with $w = [2, -3, -3]$, $x = [-1, -2]$).
- Name and use the three gradients at a node: **local**, **upstream**, **downstream** = upstream × local.
- Local gradients of the basic gates: +, ×, ×(−1), +1, e^x, $1/x$.
- Derive the sigmoid's derivative $\sigma'(x) = (1 - \sigma(x))\sigma(x)$ and use it as one "sigmoid gate".
- The add gate is a gradient distributor (and, from the examples, the multiply gate swaps its inputs).
- Short theory: more neurons = more capacity; regularise with λ, not by shrinking the net; the brain analogy and its limits.

⌛ **Only 20 minutes?** Do card 4 steps 1–6 (learn on the sibling graph, then fill the slides' graph yourself; skip "Try alone"), then card 6 step 4 items 7–8 and the exam problem part (b). If 30 more minutes appear, card 5 steps 4–6.

📖 READ & UNDERSTAND · ⏱ 5 min · Tier B

1 · How many layers and neurons? More neurons = more capacity, but regularise with λ

A **neuron** here is one small unit that takes a weighted sum of its inputs and squashes it. A **hidden layer** is a row of neurons between the input and the output. Slide 32 trains the same 2-D classifier (red vs green dots) with 3, 6 and 20 hidden neurons. With 3 the boundary is a gentle curve; with 20 it twists around almost every dot. **More neurons = more capacity**: the net can draw more complicated boundaries.

More capacity is not automatically better: a boundary that snakes around every training dot is **overfitting** (memorising the training set, doing worse on new points). Slide 33's advice: **do not shrink the network to fight overfitting; keep it big and use stronger regularisation instead.** The loss becomes

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i\big(f(x_i, W),\, y_i\big) + \lambda R(W)$$

Symbols: N training examples; $f(x_i, W)$ the network's output for example x_i with weights W; y_i its correct label; L_i the error on that one example; $R(W)$ the **regulariser**, a penalty that grows when the

weights are large (typically the sum of squared weights); λ ("lambda") a knob for how much that penalty counts. Slide 33 shows λ = 0.001, 0.01, 0.1: **bigger λ → smoother, simpler boundary**.

E.g. with $N = 2$ examples whose errors are $L_1 = 0.3$ and $L_2 = 0.5$, weights $(1, -2)$ so $R(W) = 1^2 + (-2)^2 = 5$, and $\lambda = 0.1$: $L = \frac{0.3+0.5}{2} + 0.1 \cdot 5 = 0.4 + 0.5 = 0.9$. Here the penalty is as large as the error itself, so training will push the weights down.

🧭 **Like a big engine with a speed limiter.** A small car (few neurons) simply cannot go fast. A big engine (many neurons) can, and λ is the limiter you set: you keep the power for when it's needed but stop it from doing anything wild. *Where it breaks:* λ doesn't cap anything hard; it just makes large weights cost more, so the net uses them only when they really reduce the error.

Try it: the TensorFlow Playground (linked on the slide) lets you change the neuron count and the regularisation rate and watch the boundary change live.

↗ Understand more: TensorFlow Playground ↗ d2l: weight decay (the λR(W) term)

📖 READ & UNDERSTAND · ⏱ 5 min · Tier B

2 · The biological neuron, the artificial one, and the activation function

Slides 34–37 put a real brain cell next to the maths. A real neuron has **dendrites** (branches that receive signals from other cells), a **cell body** that combines them, and an **axon** (a long cable) that carries its own signal out to the next cells through **synapses** (the contact points).

The artificial copy (slide 36): input x_0 arrives "along an axon from another neuron", the synapse multiplies it by a weight w_0, the cell body adds everything up plus a bias b, and an **activation function** f decides the output:

$$\text{output} = f\Big(\sum_i w_i x_i + b\Big)$$

E.g. weights $(2, -1)$, inputs $(1, 3)$, bias $b = 0.5$: the sum is $2 \cdot 1 + (-1) \cdot 3 + 0.5 = -0.5$, and with the sigmoid below as f the output is $\sigma(-0.5) = 0.38$.

The **activation function** is the non-linear squashing step. Without it, stacking layers would just give one big linear function. The slide's example is the **sigmoid** (slide 37):

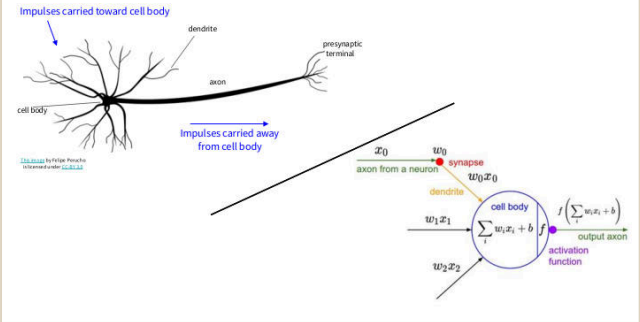
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It maps any number into (0, 1): very negative → near 0, very positive → near 1, and $\sigma(0) = 0.5$; e.g. $\sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1.135} = 0.88$. (Background, not on these slides: other common choices are tanh and ReLU = max(0, x); the next cards only need the sigmoid.)

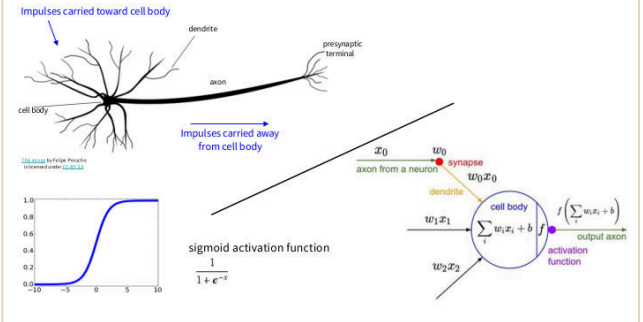
Layers (slide 39). Real neurons connect in tangled patterns; artificial ones are **organised into regular layers for computational efficiency**: every neuron in one layer feeds every neuron in the next, so a whole layer is one matrix multiply on a GPU. Slide 40 adds that **randomly wired** networks can work too (Xie et al., 2019), so the neat layering is a convenience, not a law.

🧭 **Like a voting committee.** Each input is a member's opinion, the weight is how much you trust that member, the sum is the tally, and the sigmoid turns the tally into a confidence between 0 and 1. *Where it breaks:* the slides themselves warn (slide 41, background) that real neurons are far more complex

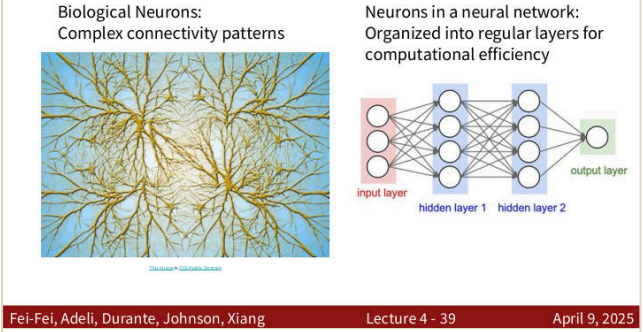
(dendrites compute non-linearly, a synapse is not one number), so "neural network" is a loose inspiration, not a brain model.



Stanford L4, PDF p36



Stanford L4, PDF p37



Stanford L4, PDF p38

↗ Understand more: CS231n notes, neural networks part 1 ↗ 3Blue1Brown ch1: what is a neural network

📖 **READ & UNDERSTAND · ⏱ 10 min · Tier A**

3 · Computational graphs, the chain rule, and the three gradients at every node

The problem (background, slides 43–48): to train, gradient descent needs $\partial L/\partial w$ for every weight w, i.e. "if I nudge this weight a tiny bit, how much does the loss L change?" Deriving one big formula on paper is tedious and must be redone whenever the model changes. The fix is to **break the formula into tiny operations** and let each one handle only its own small derivative.

A **computational graph** draws a formula as boxes (nodes = one simple operation: +, ×, exp, 1/x…) joined by wires (the values flowing between them). **Backpropagation** (slide 49) runs it twice: a **forward pass** left → right computing every wire's value, then a **backward pass** right → left computing every wire's gradient.

The chain rule is the one fact that makes this work. If f depends on q, and q depends on y, then a nudge to y moves q, which moves f, and the rates multiply:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y}$$

E.g. $q = 3y$ and $f = 2q$: nudging y by 0.01 moves q by 0.03, which moves f by 0.06, so $\partial f/\partial y = 2 \cdot 3 = 6$.

The node view (slides 66–71). Zoom in on one node f with inputs x, y and output z:

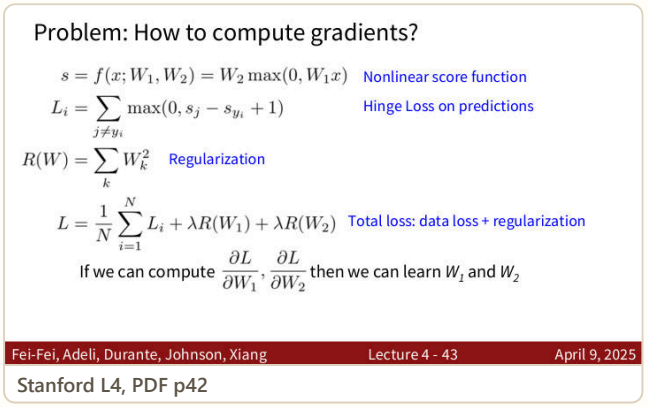
- **Local gradient**: $\partial z/\partial x$ and $\partial z/\partial y$. The node knows these on its own, from its own formula (for $z = xy$: $\partial z/\partial x = y$).
- **Upstream gradient**: $\partial L/\partial z$, arriving from the right (from everything after this node). The node does not need to know what is out there.
- **Downstream gradient**: what the node sends left to each input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x} \qquad \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y}$$

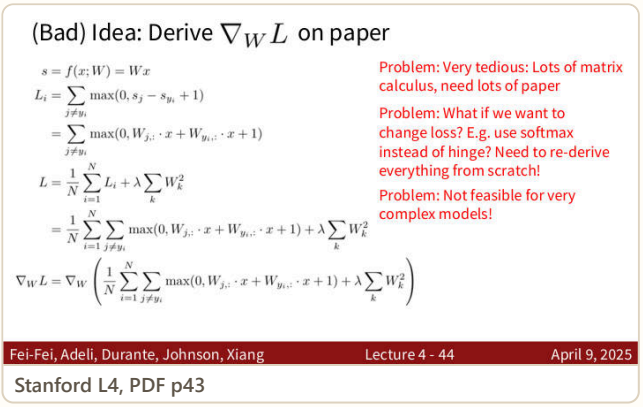
E.g. a node $z = xy$ with $x = 4$, $y = 3$, receiving upstream gradient $\partial L/\partial z = 2$: $\partial L/\partial x = 2 \cdot 3 = 6$ and $\partial L/\partial y = 2 \cdot 4 = 8$.

In words: **downstream = upstream × local**. That downstream gradient becomes the upstream gradient of the node before it, and so on back to the inputs. The very last output starts with gradient $\partial f/\partial f = 1$ (a nudge to f changes f by exactly that nudge).

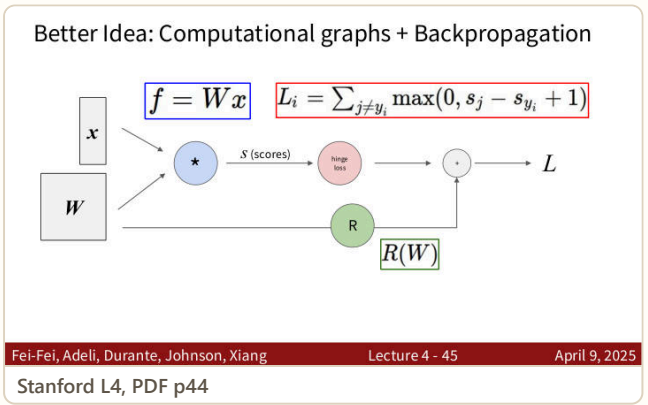
Like gears in a bicycle drivetrain. Each gear only knows its own ratio (the local gradient). To know how much the back wheel turns per pedal turn, you multiply the ratios along the chain (the chain rule). Backprop walks the chain from the wheel back to the pedals, carrying the running product (the upstream gradient). *Where it breaks:* gear ratios are fixed; a node's local gradient depends on the values that flowed through it in the forward pass, which is why the forward pass must be done (and stored) first.



Stanford L4, PDF p42



Stanford L4, PDF p43



Stanford L4, PDF p44

↗ Understand more: CS231n notes, "Backpropagation, intuitions" ↗ the chain rule (guide 11.3) ↗ 3Blue1Brown ch4: backprop calculus

PEN & PAPER · ⏱ 22 min · Tier A

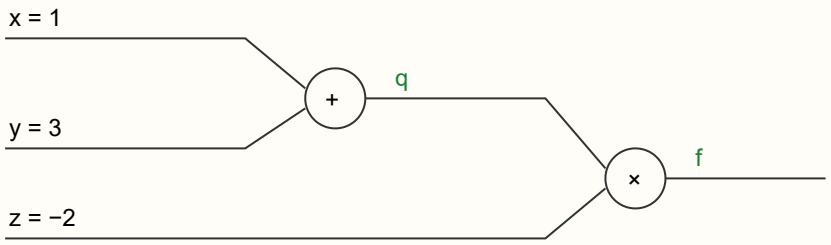
4 · Backprop by hand: learn it on f = (x + y)·z at x = 1, y = 3, z = −2, then do the slides' graph yourself (slides 50–65)

Step 1 · What you're being asked, in plain words

A **computational graph** is a formula drawn as a small circuit: inputs on the left, one simple operation per circle (+ or ×), and a wire carrying each in-between value to the right. "Do backprop on it" means two passes: first write every wire's **value** (forward pass, left to right), then every wire's **gradient** (backward pass, right to left).

The **gradient on a wire**, written $\partial f/\partial x$ ("the partial derivative of f with respect to x"), answers one question: **if this wire is nudged up by a tiny amount, by how many times that amount does the output f move?** A gradient of −2 means "nudge x up by 0.01 and f goes *down* by 0.02". "Partial" only means every other input is held still while this one is nudged.

Step 2 · Warm-up by hand, no formulas: f = (x + y)·z at x = 1, y = 3, z = −2



Forward: $x + y = 1 + 3 = 4$, then $f = 4 \cdot (-2) = -8$.

Backward, by brute force: nudge each input by 0.01 (holding the other two still), recompute f, and see how much f moved.

nudge	new f	change in f	change ÷ 0.01
x: 1 → 1.01	$(1.01 + 3) \cdot (-2) = -8.02$	−0.02	**−2**
y: 3 → 3.01	$(1 + 3.01) \cdot (-2) = -8.02$	−0.02	**−2**
z: −2 → −1.99	$4 \cdot (-1.99) = -7.96$	+0.04	**4**

So the answers are $\partial f/\partial x = -2$, $\partial f/\partial y = -2$, $\partial f/\partial z = 4$. Now nudge each **gate on its own** to see where those numbers come from:

gate	nudge	its output moves by	slope (local gradient)
$+ : q = x + y$	x: 1 → 1.01	q: 4 → 4.01, i.e. +0.01	**1** (same for y)
$\times : f = q \cdot z$	q: 4 → 4.01	f: −8 → −8.02, i.e. −0.02	**−2** = z's value
$\times : f = q \cdot z$	z: −2 → −1.99	f: −8 → −7.96, i.e. +0.04	**4** = q's value

Two rules fall out. **Add gate: slope 1 for each input. Multiply gate: each input's slope is the *other* input's value** (the slope of $q \cdot z$ as q moves is z).

Step 3 · The one idea

Through a chain of gates, slopes multiply. If a nudge to x moves q by 1× the nudge, and a nudge to q moves f by −2× the nudge, then a nudge to x moves f by $1 \cdot (-2) = -2\times$ the nudge. That is the chain rule, and it is all of backprop: at every gate, **gradient sent left (downstream) = gradient arriving from the right (upstream) × the gate's own slope (local)**.

Step 4 · Build it, step by step (same graph, x = 1, y = 3, z = −2)

1. **Name the middle wire:** $q = x + y$, so $f = q \cdot z$.
   Why: every gate needs a named output so it can get a row in the tables; now the graph is one + gate feeding one × gate.
2. **Forward, left to right:** $q = 1 + 3 = 4$; $f = q \cdot z = 4 \cdot (-2) = -8$.
   Why: the × gate's local slopes are the values q and z, so they must be known before going backward.
3. **Local slopes of each gate, at these values:** + gate: $\partial q/\partial x = 1$, $\partial q/\partial y = 1$. × gate: $\partial f/\partial q = z = -2$, $\partial f/\partial z = q = 4$.
   Why: the warm-up's two rules (add → 1, multiply → the other input).
4. **Start at the output:** $\partial f/\partial f = 1$.
   Why: nudging f by 0.01 moves f by 0.01, so the slope is 1. This 1 is the first upstream gradient.
5. **× gate** (upstream 1): $\partial f/\partial z = 1 \cdot q = 1 \cdot 4 = 4$ and $\partial f/\partial q = 1 \cdot z = 1 \cdot (-2) = -2$.
   Why: downstream = upstream × local, and the local is the other input.
6. **+ gate** (its upstream is now $\partial f/\partial q = -2$, its local slope is 1):
   Why: the + gate copies whatever arrives to both inputs.

$$\frac{\partial f}{\partial x} = \underbrace{\frac{\partial f}{\partial q}}_{\text{upstream}} \cdot \underbrace{\frac{\partial q}{\partial x}}_{\text{local}} = (-2) \cdot 1 = -2 \qquad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = (-2) \cdot 1 = -2$$

7. **Write every wire's pair (value, gradient) on the graph**, in the order you found them.
   Why: that filled graph is exactly what the exam marks.

wire	value (forward)	gradient ∂f/∂wire (backward)
f	−8	1
z	−2	4
q	4	−2
y	3	−2
x	1	−2

Step 5 · Check it

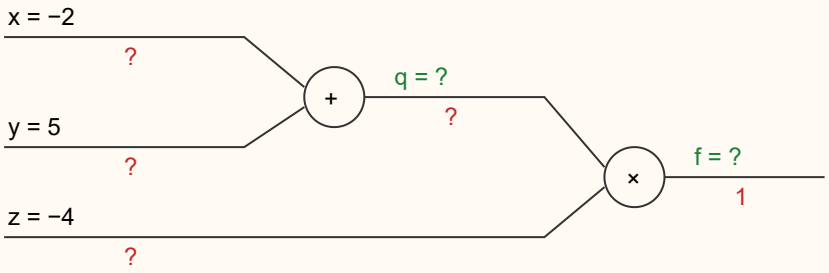
On the warm-up: the chain-rule answers −2, −2, 4 are exactly the brute-force nudge table's last column. ✓

Edge case, an input equal to 0: f = x·y at x = 0, y = 7. The rule says $\partial f/\partial y = x = 0$. Nudge y to 7.01: $f = 0 \cdot 7.01 = 0$, no change at all. ✓ A zero on one side of a × gate blocks the gradient to the other side.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

For $f(x, y, z) = (x + y)\, z$ with $x = -2$, $y = 5$, $z = -4$, let $q = x + y$. Draw the computational graph, write each wire's forward value and its gradient $\partial f/\partial(\text{wire})$.



(a) Find q and f. (b) Give the local gradients of the + node and the × node. (c) Find $\partial f/\partial z$, $\partial f/\partial q$, $\partial f/\partial y$, $\partial f/\partial x$, naming the upstream and local gradient in each.

Do it the same way: steps 4–5 with these numbers. Fill the five-row table (f, z, q, y, x) before you look.

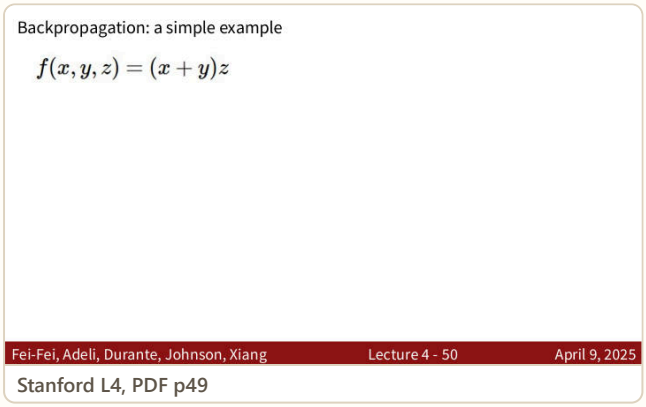
✍ Hint: S5.1 (end of this lecture)

✍ Full worked solution: S5.2 (end of this lecture)

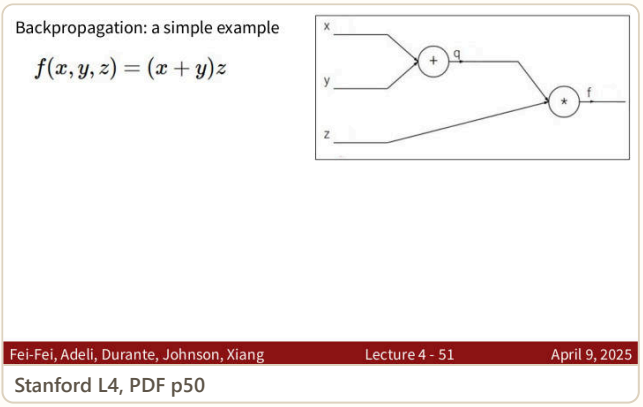
Step 7 · Try alone

1. $f = x \cdot y$ at $x = 4, y = -5$: the value of f and both gradients.
2. $f = (x + y) \cdot z$ at $x = 3, y = -1, z = -2$: forward values and all three input gradients.
3. $f = (x \cdot y + z) \cdot w$ at $x = 1, y = 2, z = 3, w = -1$ (three gates: ×, +, ×; name the wires $m = xy$, $r = m + z$): all four input gradients.

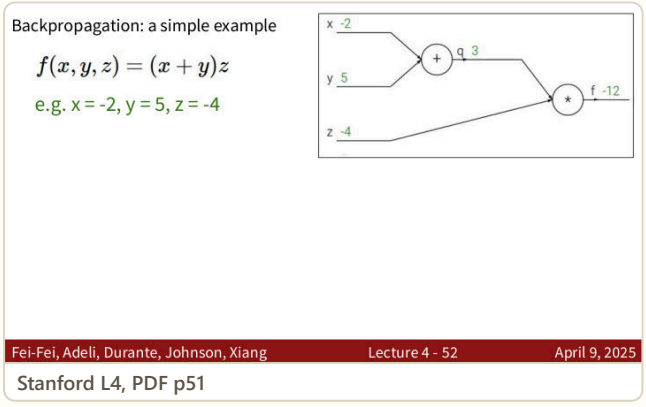
✍ Answers: S5.3 (end of this lecture)



Stanford L4, PDF p49



Stanford L4, PDF p50



Stanford L4, PDF p51

↗ Understand more: CS231n notes, the same f = (x + y)z example ↗ forward pass: compute and store every value (guide 11.5)

PEN & PAPER · ⏱ 32 min · Tier A

5 · The bigger example: a sigmoid neuron, gate by gate (slides 72–88)

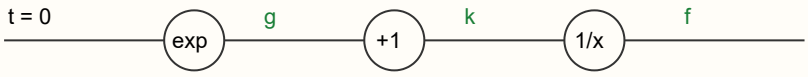
Step 1 · What you're being asked, in plain words

This is **one neuron**. It has two inputs x_0, x_1, a **weight** for each (w_0, w_1: how much that input counts) and a **bias** w_2 (a number added no matter what the inputs are). It computes the weighted sum $w_0x_0 + w_1x_1 + w_2$ and squashes it with the sigmoid:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Here $e \approx 2.718$, and e^t ("exp of t") is one gate. The slides cut this formula into **eight tiny gates**: two ×, two +, a "×(−1)" that flips the sign, an exp, a "+1", and a "1/x". The task: write every wire's forward value, then every wire's gradient, ending with the gradients on the weights w_0, w_1, w_2 (those are what training will change).

Step 2 · Warm-up by hand: just the last three gates, t → exp → +1 → 1/x, at t = 0



Forward: $g = e^0 = 1$, then $k = 1 + 1 = 2$, then $f = 1/2 = 0.5$.

Brute force: nudge t to 0.01. Then $g = e^{0.01} = 1.01005$, $k = 2.01005$, $f = 0.49750$. f moved by -0.0025, so the slope is $-0.0025/0.01 = -0.25$.

Gate by gate, nudging each gate's input by 0.01 on its own:

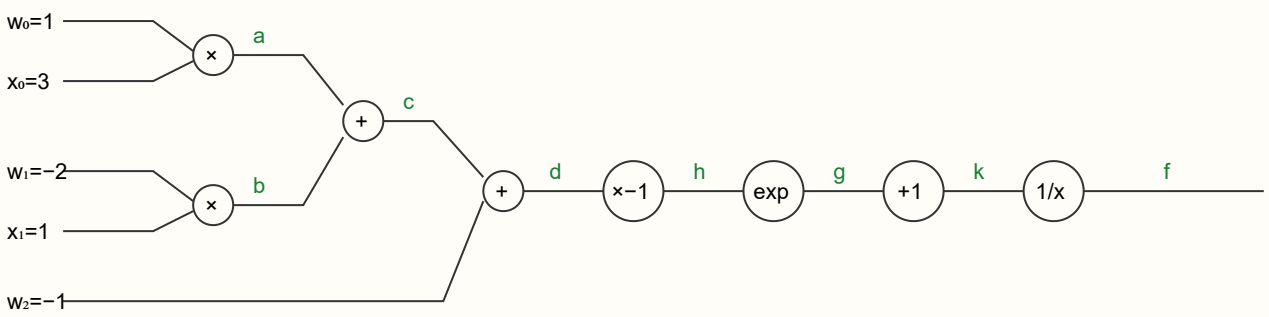
gate	input → input + 0.01	output moves by	slope (local gradient)
1/x	2 → 2.01	0.5 → 0.49751, i.e. −0.0025	−0.25
+1	1 → 1.01	2 → 2.01, i.e. +0.01	1
exp	0 → 0.01	1 → 1.01005, i.e. ≈ +0.01	≈ 1

Multiply the three slopes: $(-0.25) \cdot 1 \cdot 1 = -0.25$. **Exactly the brute-force answer.** That is backprop working on a chain of three gates, and it is the right-hand end of the graph built in step 4.

Step 3 · The one idea

Never differentiate the whole formula. Every gate is one of six kinds, and each kind has a one-line local slope. Evaluate that slope at the value that went *into* the gate on the forward pass, then multiply right to left: downstream = upstream × local.

Step 4 · Build it, step by step, on a sibling neuron: $w_0 = 1$, $x_0 = 3$, $w_1 = -2$, $x_1 = 1$, $w_2 = -1$



1. **Cut the formula into named wires, from the inside out:** $a = w_0 x_0$, $b = w_1 x_1$, $c = a + b$, $d = c + w_2$, $h = -d$, $g = e^h$, $k = 1 + g$, $f = 1/k$.
   Why: each name is one gate's output, so each wire gets one row in the tables. (The wire after ×(−1) is called h so it is not confused with the number e.)
2. **Write the cheat-sheet of local slopes** (slide 75 gives the first four).
   Why: the backward pass only ever needs these six lines.

gate	local slope	tiny example
×(−1)	-1, always	—
exp	e^x at its input x	input 0 → slope 1
+1 (or + any constant)	1	—
1/x	$-1/x^2$ at its input x	input 2 → slope −0.25
+ (two wires)	1 to each input: copies the upstream	upstream 0.5 → 0.5 and 0.5
× (two wires)	each input gets the **other** input	inputs 3, −2: slopes −2 and 3

3. **Forward table**, left to right.
   Why: the exp and 1/x slopes need the values that went in, and the × slopes need the inputs.

wire	computation	value
a	$w_0 x_0 = 1 \cdot 3$	3
b	$w_1 x_1 = (-2) \cdot 1$	−2
c	$a + b = 3 + (-2)$	1
d	$c + w_2 = 1 + (-1)$	0
h	$-d = -0$	0
g	$e^h = e^0$	1
k	$1 + g = 1 + 1$	2
f	$1/k = 1/2$	0.5

4. **Backward table**, right to left, each row upstream × local.
   Why: step 3's rule, applied once per gate; each row's result is the next row's upstream.

gate	upstream × local	gradient	why this local
output	start	∂f/∂f = 1	nudge f, f moves the same
1/x	$1 \cdot \left(-\frac{1}{2^2}\right)$	∂f/∂k = −0.25	$-1/x^2$ at its input k = 2
+1	$(-0.25) \cdot 1$	∂f/∂g = −0.25	adding a constant: slope 1
exp	$(-0.25) \cdot e^0$	∂f/∂h = −0.25	e^x at its input h = 0

×(−1)	$(-0.25)\cdot(-1)$	∂f/∂d = 0.25	always −1
+ (c, w2)	$0.25\cdot 1$ to **both**	∂f/∂c = 0.25, ∂f/∂w2 = 0.25	+ copies
+ (a, b)	$0.25\cdot 1$ to both	∂f/∂a = 0.25, ∂f/∂b = 0.25	+ copies
× (w0, x0)	w0: $0.25\cdot x_0 = 0.25\cdot 3$; x0: $0.25\cdot w_0 = 0.25\cdot 1$	∂f/∂w0 = 0.75, ∂f/∂x0 = 0.25	× swaps
× (w1, x1)	w1: $0.25\cdot x_1 = 0.25\cdot 1$; x1: $0.25\cdot w_1 = 0.25\cdot(-2)$	∂f/∂w1 = 0.25, ∂f/∂x1 = −0.5	× swaps

5. **Read off the weights' gradients** at the far left: $\partial f/\partial w_0 = 0.75$, $\partial f/\partial w_1 = 0.25$, $\partial f/\partial w_2 = 0.25$.
   Why: these are the numbers gradient descent uses to update the weights.
6. **When the numbers are not clean** (they won't be in the exam), round every value to 2 decimals as you go, like the slides do, and keep using the rounded value. E.g. a 1/x gate whose input is 1.61 has local slope $-1/1.61^2 = -0.39$.
   Why: it keeps the arithmetic short and matches the slides' answers.

Step 5 · Check it

On the warm-up: the right-hand end of this graph is the warm-up chain with t = h = 0, and the table gives ∂f/∂h = −0.25, the warm-up's brute-force slope. ✓

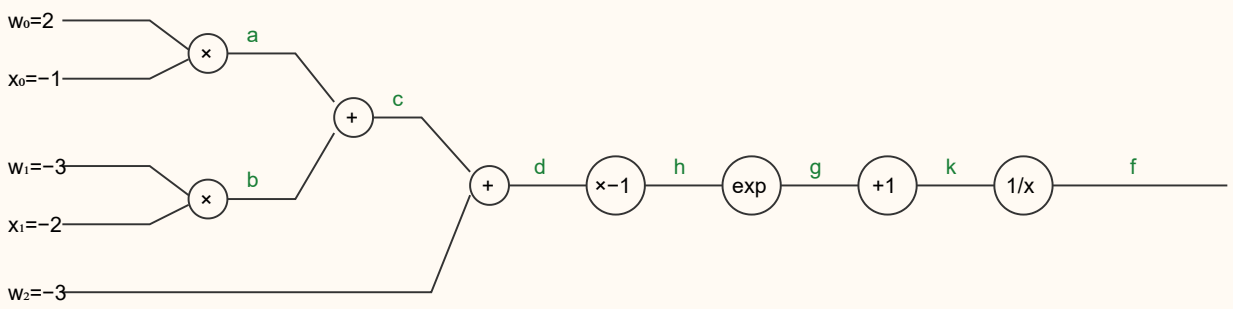
By nudging a weight: w_0 from 1 to 1.01 makes $a = 3.03$ and $d = 0.03$, so $f = 1/(1+e^{-0.03}) = 0.507499$. f moved by 0.0075, and $0.0075/0.01 = 0.75$, the table's ∂f/∂w0. ✓

Edge case, input vs output: the 1/x gate here had input 2 and output 0.5. Using the input, $-1/2^2 = -0.25$ ✓. Plugging the output by mistake gives $-1/0.5^2 = -4$, sixteen times too big. Always plug the gate's **input** into e^x and $-1/x^2$.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

For $f(w,x) = \dfrac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$ with $w_0 = 2$, $x_0 = -1$, $w_1 = -3$, $x_1 = -2$, $w_2 = -3$, using the gate graph below:



(a) Forward pass: compute every wire, to 2 decimals. (b) Backward pass: starting from $\partial f/\partial f = 1$, find the gradient on k, g, h, d, c, w_2, a, b and on w_0, x_0, w_1, x_1, writing each as upstream × local. (The slides leave w_1 and x_1 blank.)

Do it the same way: steps 4–5 with these numbers. Two tables (forward, then backward), one row per gate.

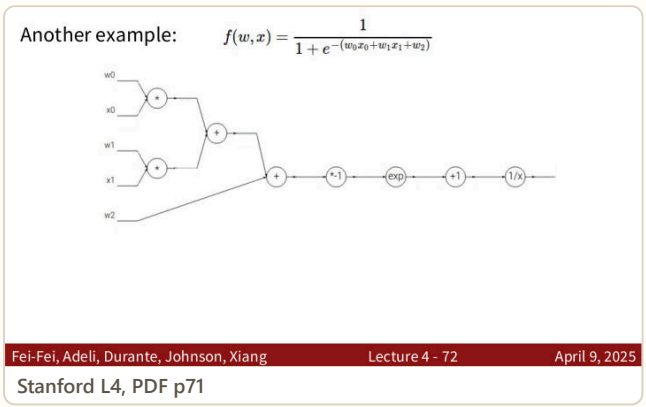
✍ Hint: S5.4 (end of this lecture)

✍ Full worked solution: S5.5 (end of this lecture)

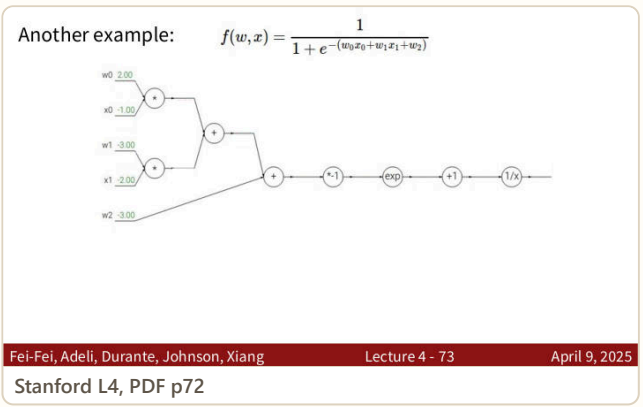
Step 7 · Try alone

1. A 1/x gate receives input 4 and upstream gradient 1. What does it send left?
2. In the exam problem, suppose x_0 were 0 instead of −1. Without redoing the whole graph, what is $\partial f/\partial w_0$, and why?
3. Redo the exam problem with the bias changed to $w_2 = -2$ (everything else the same). Forward values, then all five leaf gradients, carrying 3 decimals.

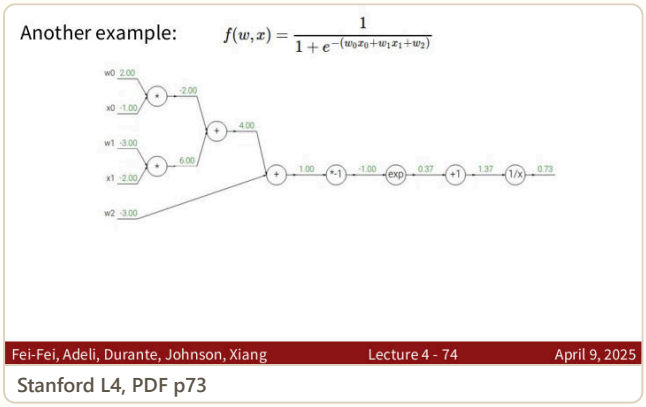
✍ Answers: S5.6 (end of this lecture)



Stanford L4, PDF p71



Stanford L4, PDF p72



Stanford L4, PDF p73

↗ Understand more: CS231n notes, "Modularity: sigmoid example" ↗ a whole backward pass by hand (guide 11.10)

🖉 PEN & PAPER · ⏱ 16 min · Tier A

6 · The sigmoid gate: local gradient σ(1 − σ), derived and used (slides 89–91)

Step 1 · What you're being asked, in plain words

The **sigmoid** $\sigma(x) = \dfrac{1}{1+e^{-x}}$ squashes any number into (0, 1). In card 5 it took four gates (×−1, exp, +1, 1/x). Slide 89's point: a graph is not unique, so those four gates may be drawn as **one "sigmoid" box**, provided its local gradient is known. **Choose the split where each local gradient is easy to write.**

The **derivative** $d\sigma/dx$ is the slope of the sigmoid curve at x: how much σ moves per tiny nudge of x. It is exactly the sigmoid box's local gradient. "Show that $d\sigma/dx = (1-\sigma)\sigma$" means: start from the formula for σ and reach that expression by legal algebra steps.

To learn the method without giving the exam proof away, this card works on a sibling: the **steep sigmoid** $\sigma_2(x) = \dfrac{1}{1+e^{-2x}}$ (the same S-curve, squeezed to rise twice as fast). The claim to reach is $\sigma_2'(x) = 2\,(1-\sigma_2(x))\,\sigma_2(x)$.

Step 2 · Warm-up by hand, no formulas: nudge the steep sigmoid

x	$\sigma_2(x)$	$\sigma_2(x+0.01)$	change ÷ 0.01	$2(1-\sigma_2)\cdot\sigma_2$
0.5	0.73106	0.73497	0.391	$2 \cdot 0.269 \cdot 0.731 = 0.393$
−1	0.11920	0.12132	0.212	$2 \cdot 0.881 \cdot 0.119 = 0.210$

The brute-force slope and $2(1-\sigma_2)\sigma_2$ agree (the small gap is only because 0.01 is not infinitely small). So the claim is believable before any algebra: **the slope can be read off the output alone**.

Step 3 · The one idea

Differentiate once, then **split the fraction into two factors**: one is the function itself, and the other is "1 – the function" in disguise (add and subtract 1 on top to see it).

Step 4 · Build it, step by step (on σ_2)

Two tools first. **(i) Chain rule for a power:** $\dfrac{d}{dx}u^{-1} = -u^{-2}\cdot u'$, where u' is the slope of the inside. E.g. $u = 2x$: $\dfrac{d}{dx}\dfrac{1}{2x} = -\dfrac{1}{(2x)^2}\cdot 2$, which at $x=1$ is -0.5. **(ii)** $\dfrac{d}{dx}e^{-2x} = -2e^{-2x}$ (exp, times the slope of the inside $-2x$, which is −2). E.g. at $x=0$ the slope is −2.

1. Write $\sigma_2 = u^{-1}$ with $u = 1+e^{-2x}$.
   Why: a power is easier to differentiate than a fraction.
2. Slope of the inside: $u' = 0 + (-2e^{-2x}) = -2e^{-2x}$.
   Why: the constant 1 has slope 0; tool (ii) for the rest.
3. Tool (i): $\sigma_2' = -(1+e^{-2x})^{-2}\cdot(-2e^{-2x}) = \dfrac{2e^{-2x}}{(1+e^{-2x})^2}$.
   Why: the two minus signs cancel.
4. Split: $\dfrac{2e^{-2x}}{(1+e^{-2x})^2} = 2\cdot\dfrac{e^{-2x}}{1+e^{-2x}}\cdot\dfrac{1}{1+e^{-2x}}$.
   Why: $\dfrac{a}{b^2} = \dfrac{a}{b}\cdot\dfrac{1}{b}$; the last factor is σ_2.
5. Add and subtract 1 on top of the middle factor: $\dfrac{1+e^{-2x}-1}{1+e^{-2x}} = 1 - \dfrac{1}{1+e^{-2x}} = 1-\sigma_2$.
   Why: that rewrites it using the function itself.
6. Together: $\sigma_2'(x) = 2\,(1-\sigma_2(x))\,\sigma_2(x)$.
   Why: multiply the three factors back.
7. **Use it as one gate.** A steep-sigmoid box gets input 1, outputs $\sigma_2(1) = 0.88$, and receives upstream gradient 0.5. Local $= 2(1-0.88)(0.88) = 0.211$; downstream $= 0.5\cdot 0.211 = 0.106$.
   Why: downstream = upstream × local, and the local needs only the stored output.

8. **Its slope at two points:** at $x = 0$, $\sigma_2 = 0.5$, so $2 \cdot 0.5 \cdot 0.5 = 0.5$ (the steepest point). At $x = 1.5$, $\sigma_2 = 0.953$, so $2 \cdot 0.047 \cdot 0.953 = 0.090$.
   Why: far from 0 the curve flattens, so the slope shrinks.

Step 5 · Check it

On the warm-up: at $x = 0.5$, step 3's form gives $\frac{2e^{-1}}{(1+e^{-1})^2} = \frac{0.7358}{1.8711} = 0.393$, and $2(1 - 0.731)(0.731) = 0.393$ ✓, both matching the nudge 0.391.

Edge case, far out: at $x = 5$, $\sigma_2 \approx 0.99995$, so $2(1-\sigma_2)\sigma_2 \approx 0.00009$: almost flat, just as the curve looks far to the right.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

(a) Show that $\frac{d\sigma}{dx} = (1 - \sigma(x))\,\sigma(x)$ for $\sigma(x) = \frac{1}{1+e^{-x}}$.
(b) In card 5's neuron, treat ×−1, exp, +1, 1/x as one sigmoid box. Its input is 1.00, its output 0.73, the upstream gradient 1.00. Do the backward step through the box in one line.
(c) What is the sigmoid's local gradient at x = 0, and at x = 4?

Do it the same way: steps 4–5 with these numbers. For (a), the six algebra steps with $-x$ where σ_2 had $-2x$.

✍ Hint: S5.7 (end of this lecture)

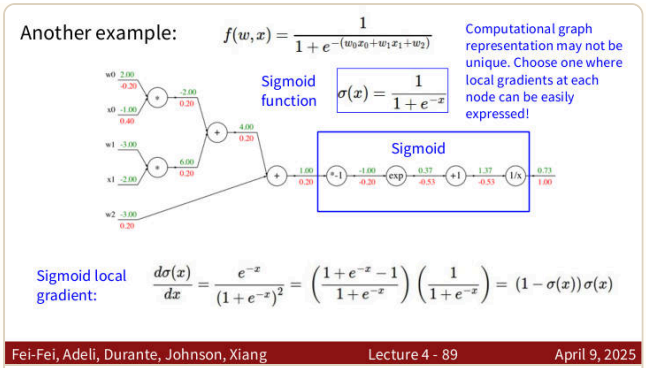
✍ Full worked solution: S5.8 (end of this lecture)

Step 7 · Try alone

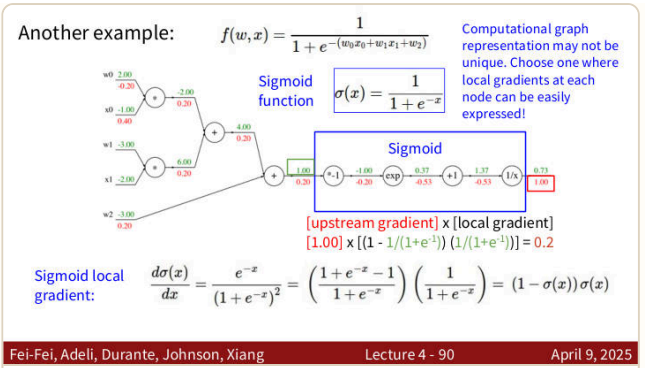
1. $\sigma(-1) = 0.27$. What is $\sigma'(-1)$? Compare with $\sigma'(1)$.
2. A sigmoid box outputs 0.9 and receives upstream gradient 2. What does it send left?
3. Why can $(1-\sigma)\sigma$ never exceed 0.25? (Try σ = 0.4, 0.3, 0.5.)

✍ Answers: S5.9 (end of this lecture)

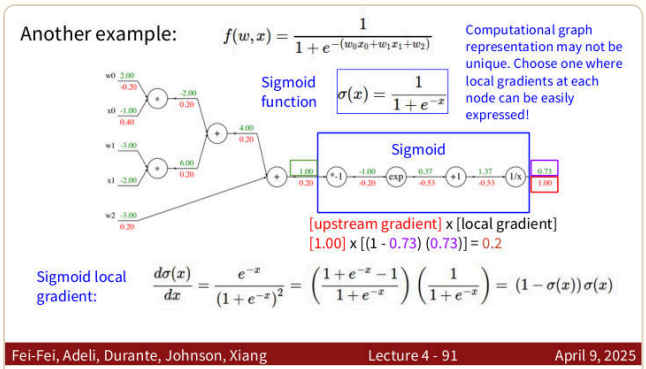
🧭 **Like packing a recipe into one button.** "Toast" on a microwave is really a sequence of power levels and timings; only what the button does as a whole matters. Grouping gates into a sigmoid box is the same: fewer steps, same result. *Where it breaks:* a group is allowed only when its local gradient is known in closed form, as derived above.



Stanford L4, PDF p88



Stanford L4, PDF p89



Stanford L4, PDF p90

↗ Understand more: CS231n notes, sigmoid gate ↗ d2l: activation functions and the sigmoid's derivative

📖 READ & UNDERSTAND · ⏱ 4 min · Tier A

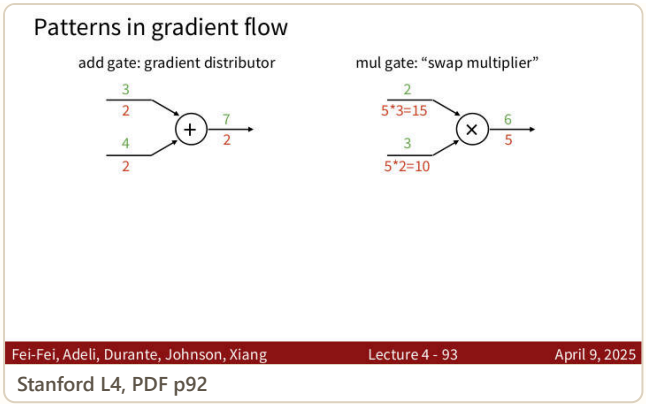
7 · Patterns in gradient flow: the add gate distributes, the multiply gate swaps

Add gate = gradient distributor (slide 92). Forward: 3 + 4 = 7. Backward with upstream gradient 2: the local gradient of $x + y$ is 1 for each input, so **both inputs get 2**, an unchanged copy of the upstream. You saw it in card 5's build: the 0.25 arriving at each + gate went to both inputs unchanged.

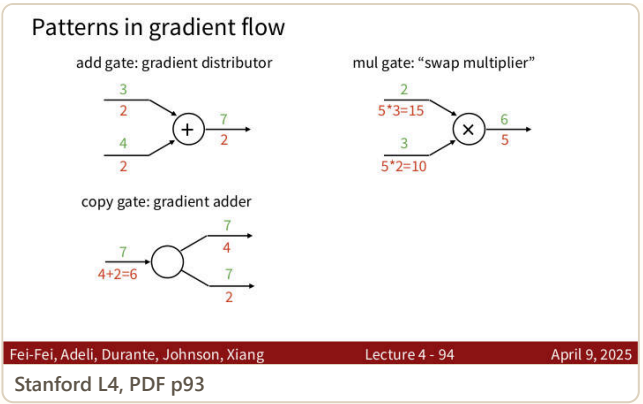
Multiply gate = "swap multiplier" (seen in cards 4 and 5). For xy, the local gradient on x is y and on y is x, so each input receives **upstream × the other input's value**. Card 4's build: $\partial f/\partial z = 1 \cdot q = 4$ and $\partial f/\partial q = 1 \cdot z = -2$. Card 5's build: w_0 got $0.25 \cdot x_0 = 0.75$, x_0 got $0.25 \cdot w_0 = 0.25$.

(Background, slides 93–95 are past the scope line: a **max gate** routes the whole upstream gradient to whichever input was larger and gives 0 to the other; a **copy gate**, where one value feeds two places, adds the gradients coming back from both.)

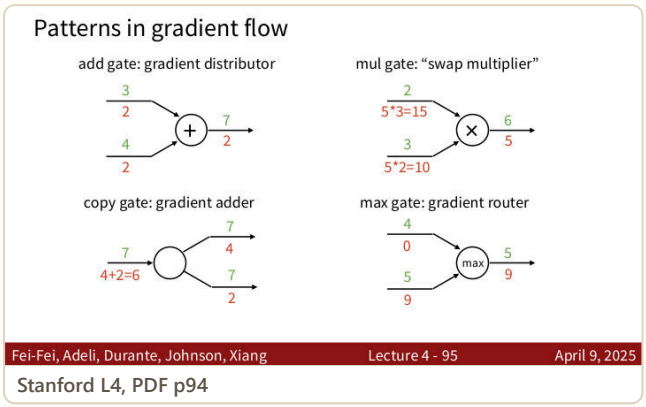
🧭 **Like splitting a restaurant bill.** At an add gate, "how much does the total change if this item costs one pound more?" is the same answer for every item, so everyone gets the same share of the blame. At a multiply gate (price × quantity), how much the bill moves when the price rises depends on the *quantity*, and vice versa: the swap. *Where it breaks:* a real bill is split by amounts; the add gate ignores the inputs' sizes entirely (3 and 4 both get 2).



Stanford L4, PDF p92



Stanford L4, PDF p93



Stanford L4, PDF p94

↗ Understand more: CS231n notes, "Patterns in backward flow" ↗ multiplicative units (guide 12.4)

🖍 **PEN & PAPER · ⏱ 16 min · Tier A**

8 · Practice on a new graph: f = w · σ(x·y + z)

Step 1 · What you're being asked, in plain words

Not from the slides; built so the numbers come out clean. Four gates, left to right: multiply x by y (call the result m), add z ($r = m + z$), squash with a sigmoid box ($s = \sigma(r)$), multiply by w ($f = w \cdot s$). The job is the same as cards 4 and 5: every wire's value going forward, then every wire's gradient coming back. The method is built below on $x = 3$, $y = 1$, $z = -3$, $w = 2$; the exam version uses other numbers.

Step 2 · Warm-up by hand: only the last two gates, r → σ → ×2, at r = 0

Forward: $s = \sigma(0) = 0.5$, then $f = 2 \cdot 0.5 = 1$.

Brute force: nudge r to 0.01. Then $s = \sigma(0.01) = 0.50250$ and $f = 2 \cdot 0.50250 = 1.00500$. f moved by 0.005, so $\partial f/\partial r = 0.005/0.01 = 0.5$.

Gate by gate: the × gate gives s the other input, 2; the sigmoid box's slope is $s(1 - s) = 0.5 \cdot 0.5 = 0.25$. Product: $2 \cdot 0.25 = 0.5$. Same answer. ✓

Step 3 · The one idea

A mixed graph is just the three rules stacked: × swaps (each input gets the other input), + copies (same number to both inputs), σ multiplies by $s(1 - s)$. The gradient on any input is the product of the local slopes along its path from f.

Step 4 · Build it, step by step ($x = 3$, $y = 1$, $z = -3$, $w = 2$)

1. **Name the wires** $m = xy$, $r = m + z$, $s = \sigma(r)$, $f = ws$.
   Why: one row per gate.
2. **Forward**, left to right: $m = 3 \cdot 1 = 3$; $r = 3 + (-3) = 0$; $s = \sigma(0) = 0.5$; $f = 2 \cdot 0.5 = 1$.
   Why: the swaps need x, y, w, s, and the sigmoid slope needs s.
3. **Local slopes, in letters:** last ×: $\partial f/\partial w = s$, $\partial f/\partial s = w$. Sigmoid: $\partial s/\partial r = s(1 - s)$. +: $\partial r/\partial m = 1$, $\partial r/\partial z = 1$. First ×: $\partial m/\partial x = y$, $\partial m/\partial y = x$.
   Why: the cheat-sheet from card 5 plus card 6's sigmoid result.
4. **Last × gate** (upstream $\partial f/\partial f = 1$): $\partial f/\partial w = 1 \cdot s = 0.5$; $\partial f/\partial s = 1 \cdot w = 2$.
   Why: × swaps.
5. **Sigmoid box** (upstream 2): local $= s(1 - s) = 0.5 \cdot 0.5 = 0.25$, so $\partial f/\partial r = 2 \cdot 0.25 = 0.5$.
   Why: upstream × local, local read off the stored output s.
6. **+ gate** (upstream 0.5): $\partial f/\partial z = 0.5 \cdot 1 = 0.5$ and $\partial f/\partial m = 0.5 \cdot 1 = 0.5$.
   Why: + copies.
7. **First × gate** (upstream 0.5): $\partial f/\partial x = 0.5 \cdot y = 0.5 \cdot 1 = 0.5$; $\partial f/\partial y = 0.5 \cdot x = 0.5 \cdot 3 = 1.5$.
   Why: × swaps.

The same thing in one line for x, as a product along its path from f:

$$\frac{\partial f}{\partial x} = \underbrace{w}_{\text{last }\times} \cdot \underbrace{s(1-s)}_{\sigma} \cdot \underbrace{1}_{+} \cdot \underbrace{y}_{\text{first }\times} = 2 \cdot 0.25 \cdot 1 \cdot 1 = 0.5$$

Step 5 · Check it

On the warm-up: the first two factors are $w \cdot s(1 - s) = 2 \cdot 0.25 = 0.5$, which is the brute-force $\partial f/\partial r = 0.5$. ✓

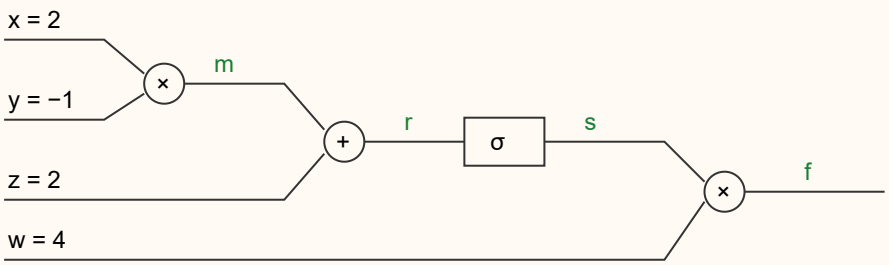
By nudging: y from 1 to 1.01 makes $m = 3.03$, $r = 0.03$, $s = \sigma(0.03) = 0.50750$, $f = 1.01500$: a change of 0.015, and $0.015/0.01 = 1.5$ = ∂f/∂y. ✓

Edge case, w = 0: then $f = 0 \cdot s = 0$ whatever x, y, z are, so nudging them changes nothing: the path product has a leading factor $w = 0$, so $\partial f/\partial x = \partial f/\partial y = \partial f/\partial z = 0$ ✓. Only $\partial f/\partial w = s$ survives.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

Inputs $x = 2$, $y = -1$, $z = 2$, $w = 4$, and $f = w \cdot \sigma(x \cdot y + z)$ drawn as below.



(a) Forward: m, r, s, f. (b) Backward: $\partial f/\partial w$, $\partial f/\partial s$, $\partial f/\partial r$, $\partial f/\partial z$, $\partial f/\partial m$, $\partial f/\partial x$, $\partial f/\partial y$, using the sigmoid box's local gradient $s(1 - s)$.

Do it the same way: steps 4–5 with these numbers.

✍️ Full worked solution: S5.10 (end of this lecture)

Step 7 · Try alone

1. Same graph with $x = 1$, $y = 1$, $z = -1$, $w = 3$: forward values and all four input gradients.
2. Same graph with $x = 1$, $y = 1$, $z = 0$, $w = 1$ (now $r \neq 0$; use $\sigma(1) = 0.73$).
3. In the exam problem, which input moves f the most per unit nudge, and in which direction?

✍️ Answers: S5.11 (end of this lecture)

↗ Practise more: backprop lab, watch every forward and backward value (guide 11.12) ↗ 3Blue1Brown ch3: what backprop is really doing

🔁 RECALL DRILL · ⏱ 5 min · Tier A

9 · Quick recall

1. Name the three gradients at a node and how they combine.

   ✍️ Answer: S5.12 (end of this lecture)

2. Why does the backward pass start with the number 1?

   ✍️ Answer: S5.13 (end of this lecture)

3. Add gate, inputs 3 and 4, upstream gradient 2. Gradients on the inputs?

   ✍️ Answer: S5.14 (end of this lecture)

4. Multiply gate, inputs x = 2 and y = 3, upstream gradient 5. Gradients on x and y?

   ✍️ Answer: S5.15 (end of this lecture)

5. A 1/x gate receives input 2 and upstream gradient 3. Downstream gradient?

   ✍️ Answer: S5.16 (end of this lecture)

6. A sigmoid gate outputs 0.73 and receives upstream gradient 1. Downstream?

   ✍️ Answer: S5.17 (end of this lecture)

7. Your net overfits. Shrink it, or raise λ?

   ✍️ Answer: S5.18 (end of this lecture)

8. Is the computational graph of a formula unique?

   ✍️ Answer: S5.19 (end of this lecture)

⚠️ Traps

- **Forgetting the upstream.** In f = (x + y)z at x = 1, y = 3, z = −2, ∂q/∂y = 1 is only the local gradient; ∂f/∂y = (−2)(1) = −2.
- **Which value the local gradient uses.** 1/x and exp: evaluate at the gate's *input* (1.37, −1). Sigmoid box: σ(1 − σ) uses its *output* (0.73). Multiply gate: each input gets the *other* input's value (∂f/∂wo uses xo = −1).
- **The add gate copies, it does not halve.** Upstream 0.20 → 0.20 to *both* inputs (slide 85, "both inputs!").

- **Rounding.** The slides round every step to 2 decimals (−0.53, −0.20, 0.20). Exact values: −0.534, −0.197, 0.197, and leaves −0.197, 0.393, −0.393, −0.590, 0.197. Either is fine if you say you rounded.
- **Slide numbering:** the PDF has no slide 38, so from slide 39 on the printed slide number is one ahead of the PDF page (slide 92, the add gate, is PDF page 91).
- **Don't regularise by shrinking the network** (slide 33): keep capacity, raise λ.

Solutions and answers — Lecture 5 · Stanford CS231n L4

S5.1 · 4 · Backprop by hand: learn it on f = (x + y)·z at x = 1, y = 3, z = −2, then do the slides' graph yourself (slides 50–65) — Hint

Forward first: the + gate, then the × gate. Backward: the × gate gives each input 1 × the *other* input; the + gate gives both its inputs whatever arrived at q. Check one answer by nudging x by 0.01.

↩ back to the card

S5.2 · 4 · Backprop by hand: learn it on f = (x + y)·z at x = 1, y = 3, z = −2, then do the slides' graph yourself (slides 50–65) — Full worked solution

(a) Forward pass, left to right.

- $q = x + y = -2 + 5 = 3.$
  Why: the + gate is the first gate; its inputs are both known.
- $f = q \cdot z = 3 \cdot (-4) = -12.$
  Why: the × gate needs q, which now exists.

(b) Local gradients (slides 53–54), evaluated at the forward values.

- Add node $q = x + y$: $\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1.$
  Why: nudging either input moves the sum by exactly the nudge.
- Multiply node $f = qz$: $\frac{\partial f}{\partial q} = z = -4, \frac{\partial f}{\partial z} = q = 3.$
  Why: each input's slope is the other input's value.

(c) Backward pass, right to left; every line is downstream = upstream × local.

- $\frac{\partial f}{\partial f} = 1.$
  Why: the output's gradient with respect to itself is 1; it is the first upstream (slide 57).
- $\frac{\partial f}{\partial z} = \underbrace{1}_{\text{upstream}} \cdot \underbrace{q}_{\text{local}} = 1 \cdot 3 = 3.$
  Why: × gate, z gets the other input q (slide 59).
- $\frac{\partial f}{\partial q} = \underbrace{1}_{\text{upstream}} \cdot \underbrace{z}_{\text{local}} = 1 \cdot (-4) = -4.$
  Why: × gate, q gets the other input z (slide 61).
- $\frac{\partial f}{\partial y} = \underbrace{\frac{\partial f}{\partial q}}_{\text{upstream}} \cdot \underbrace{\frac{\partial q}{\partial y}}_{\text{local}} = (-4) \cdot 1 = -4.$
  Why: the + gate copies the −4 that arrived at q (slide 63).
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = (-4) \cdot 1 = -4.$
  Why: same copy to the other input (slide 65).

The filled graph, in the order found:

wire	value (forward)	gradient ∂f/∂wire (backward)
x	−2	−4

y	5	−4
z	−4	3
q	3	−4
f	−12	1

Check by nudging: x from −2 to −1.99 gives $q = 3.01$, $f = 3.01 \cdot (-4) = -12.04$, a change of $-0.04 = (-4)(0.01)$. ✓ And z from −4 to −3.99 gives $f = 3 \cdot (-3.99) = -11.97$, a change of $+0.03 = 3 \cdot 0.01$. ✓

Forward: $q = 3$, $f = -12$. **Local:** $\partial q/\partial x = \partial q/\partial y = 1$, $\partial f/\partial q = z = -4$, $\partial f/\partial z = q = 3$. **Backward:** $\partial f/\partial f = 1$, $\partial f/\partial z = 3$, $\partial f/\partial q = -4$, $\partial f/\partial y = -4$, $\partial f/\partial x = -4$.

↩ back to the card

S5.3 · 4 · Backprop by hand: learn it on f = (x + y)·z at x = 1, y = 3, z = −2, then do the slides' graph yourself (slides 50–65) — Answers

1. $f = -20$. $\partial f/\partial x = y = -5$, $\partial f/\partial y = x = 4$.

2. $q = 2$, $f = -4$. $\partial f/\partial z = q = 2$; $\partial f/\partial q = z = -2$, copied by the + gate: $\partial f/\partial x = \partial f/\partial y = -2$.

3. $m = 2$, $r = 5$, $f = -5$. Last ×: $\partial f/\partial w = r = 5$, $\partial f/\partial r = w = -1$. + copies: $\partial f/\partial z = -1$, $\partial f/\partial m = -1$. First × swaps: $\partial f/\partial x = (-1) \cdot y = -2$, $\partial f/\partial y = (-1) \cdot x = -1$.

↩ back to the card

S5.4 · 5 · The bigger example: a sigmoid neuron, gate by gate (slides 72–88) — Hint

Here $d \neq 0$, so exp and 1/x give decimals: $e^{-1} = 0.37$ (to 2 decimals). Round each new value to 2 decimals before using it in the next row. Check: the gradient arriving at d should equal the sigmoid slope $\sigma(d)(1 - \sigma(d))$ (card 6).

↩ back to the card

S5.5 · 5 · The bigger example: a sigmoid neuron, gate by gate (slides 72–88) — Full worked solution

(a) Forward pass (slide 74), left to right, rounding to 2 decimals:

wire	computation	value	why
a	$w_0 x_0 = 2 \cdot (-1)$	−2.00	first × gate
b	$w_1 x_1 = (-3) \cdot (-2)$	6.00	second × gate; minus × minus = plus
c	$a + b = -2 + 6$	4.00	first + gate
d	$c + w_2 = 4 + (-3)$	1.00	add the bias
h	$-d = -1 \cdot 1$	−1.00	×(−1) gate
g	$e^h = e^{-1}$	0.37	exp gate (0.3679)
k	$1 + g = 1 + 0.37$	1.37	+1 gate
f	$1/k = 1/1.37$	0.73	1/x gate (0.7299)

(b) Backward pass, right to left; every row is upstream × local, and its result is the next row's upstream:

gate (slide)	upstream × local	gradient	why this local
output (76)	start	∂f/∂f = 1.00	nudge f, f moves the same
1/x (77)	$(1.00)\cdot\left(-\frac{1}{1.37^2}\right) = 1.00\cdot(-0.53)$	∂f/∂k = −0.53	$-1/x^2$ at its input k = 1.37
+1 (79)	$(-0.53)\cdot(1)$	∂f/∂g = −0.53	adding a constant: slope 1
exp (81)	$(-0.53)\cdot e^{-1} = (-0.53)(0.37)$	∂f/∂h = −0.20	e^x at its input h = −1
×−1 (83)	$(-0.20)\cdot(-1)$	∂f/∂d = 0.20	always −1
+ with w2 (85)	$(0.20)\cdot(1)$ to **both** inputs	∂f/∂c = 0.20, ∂f/∂w2 = 0.20	+ copies
+ of a, b (86)	$(0.20)\cdot(1)$ to both	∂f/∂a = 0.20, ∂f/∂b = 0.20	+ copies
× of w0, x0 (87)	w0: $0.2\cdot x_0 = 0.2\cdot(-1)$; x0: $0.2\cdot w_0 = 0.2\cdot 2$	∂f/∂w0 = −0.20, ∂f/∂x0 = 0.40	× swaps
× of w1, x1	w1: $0.2\cdot x_1 = 0.2\cdot(-2)$; x1: $0.2\cdot w_1 = 0.2\cdot(-3)$	∂f/∂w1 = −0.40, ∂f/∂x1 = −0.60	× swaps

The filled graph (value / gradient), left to right: w_0 2 / −0.20; x_0 −1 / 0.40; w_1 −3 / −0.40; x_1 −2 / −0.60; w_2 −3 / 0.20; a −2 / 0.20; b 6 / 0.20; c 4 / 0.20; d 1 / 0.20; h −1 / −0.20; g 0.37 / −0.53; k 1.37 / −0.53; f 0.73 / 1.00.

Check: the four gates ×(−1), exp, +1, 1/x together are the sigmoid, so ∂f/∂d must be $\sigma(1)(1-\sigma(1)) = 0.73\cdot 0.27 = 0.197 \approx 0.20$ ✓ (slide 91). Unrounded (carrying 0.1966 instead of 0.20) the leaf gradients are −0.197, 0.393, −0.393, −0.590, 0.197; the slides round at every step.

Forward: a = −2, b = 6, c = 4, d = 1, h = −1, g = 0.37, k = 1.37, f = 0.73. Backward: ∂f/∂k = ∂f/∂g = −0.53, ∂f/∂h = −0.20, ∂f/∂d = ∂f/∂c = ∂f/∂a = ∂f/∂b = 0.20. Leaves: $\partial f/\partial w_0 = -0.20$, $\partial f/\partial x_0 = 0.40$, $\partial f/\partial w_1 = -0.40$, $\partial f/\partial x_1 = -0.60$, $\partial f/\partial w_2 = 0.20$**.**

↩ back to the card

S5.6 · 5 · The bigger example: a sigmoid neuron, gate by gate (slides 72–88) — Answers

1. Local slope $-1/4^2 = -0.0625$, so it sends $1\cdot(-0.0625) = -0.0625$.

2. 0. The × gate gives w_0 the upstream × the other input, and the other input is $x_0 = 0$. An input that is zero gives its weight no gradient: that weight gets no update from this example.

3. Forward: a = −2, b = 6, c = 4, d = 2, h = −2, g = e^{-2} = 0.135, k = 1.135, f = 0.881. Backward: ∂f/∂k = $-1/1.135^2$ = −0.776; ∂f/∂g = −0.776; ∂f/∂h = $-0.776\cdot 0.135$ = −0.105; ∂f/∂d = 0.105; ∂f/∂c = ∂f/∂w2 = 0.105; ∂f/∂a = ∂f/∂b = 0.105. Leaves: ∂f/∂w0 = $0.105\cdot(-1)$ = −0.105, ∂f/∂x0 = $0.105\cdot 2$ = 0.210, ∂f/∂w1 = $0.105\cdot(-2)$ = −0.210, ∂f/∂x1 = $0.105\cdot(-3)$ = −0.315, ∂f/∂w2 = 0.105.

↩ back to the card

S5.7 · 6 · The sigmoid gate: local gradient σ(1 − σ), derived and used (slides 89–91) — Hint

Tool (ii) becomes $\frac{d}{dx}e^{-x} = -e^{-x}$ (the inside $-x$ has slope −1), so there is no factor 2 at the end. For (c), first find σ(0) and σ(4), then use the result of (a).

↩ back to the card

S5.8 · 6 · The sigmoid gate: local gradient σ(1 − σ), derived and used (slides 89–91) — Full worked solution

(a) Every line: one step of maths, then why.

1. $\sigma(x) = (1 + e^{-x})^{-1}$.
   Why: write the fraction as a power, u^{-1} with $u = 1 + e^{-x}$.
2. $u' = \frac{d}{dx}(1 + e^{-x}) = 0 + (-e^{-x}) = -e^{-x}$.
   Why: the constant has slope 0; the slope of e^{-x} is e^{-x} times the inside's slope −1.
3. $\frac{d\sigma}{dx} = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2}$.
   Why: power rule with the chain rule, $\frac{d}{dx}u^{-1} = -u^{-2}u'$; the two minus signs cancel.
4. $\frac{e^{-x}}{(1 + e^{-x})^2} = \frac{e^{-x}}{1 + e^{-x}} \cdot \frac{1}{1 + e^{-x}}$.
   Why: split $\frac{a}{b^2} = \frac{a}{b} \cdot \frac{1}{b}$; the second factor is σ(x).
5. $\frac{e^{-x}}{1 + e^{-x}} = \frac{1 + e^{-x} - 1}{1 + e^{-x}} = 1 - \frac{1}{1 + e^{-x}} = 1 - \sigma(x)$.
   Why: add and subtract 1 in the numerator, then split the fraction.
6. $\frac{d\sigma}{dx} = (1 - \sigma(x))\,\sigma(x)$.
   Why: put the two factors back together. ■

Written as the slide writes it (slide 91):

$$\frac{d\sigma}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}}\right)\left(\frac{1}{1 + e^{-x}}\right) = (1 - \sigma(x))\,\sigma(x)$$

So only the forward output σ is needed, and it is already stored from the forward pass.

(b) Local gradient $= (1 - 0.73)(0.73) = 0.27 \cdot 0.73 = 0.197 \approx 0.20$.
Why: (a) with the stored output 0.73. Downstream = upstream × local = 1.00 · 0.20 = 0.20.
Why: the chain rule at one node. This is the same ∂f/∂d = 0.20 that the four small gates gave in card 5 (slide 91: "[1.00] x [(1 − 0.73)(0.73)] = 0.2").

(c) At x = 0: $\sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$, so $(1 - 0.5)(0.5) = \mathbf{0.25}$, the largest the sigmoid's slope ever gets. At x = 4: $\sigma(4) = \frac{1}{1 + e^{-4}} = \frac{1}{1.0183} = 0.982$, so $(1 - 0.982)(0.982) = 0.018 \cdot 0.982 \approx \mathbf{0.018}$.
Why: far from 0 the curve is nearly flat. (Background: a saturated sigmoid passes back almost no gradient, which is why deep sigmoid nets learn slowly.)

(a) $\sigma'(x) = (1 - \sigma(x))\sigma(x)$**. (b) Sigmoid box in card 5:** $1.00 \times (1 - 0.73)(0.73) = 0.20$**. (c) σ′(0) = 0.25, σ′(4) ≈ 0.018.**

↩ back to the card

S5.9 · 6 · The sigmoid gate: local gradient σ(1 − σ), derived and used (slides 89–91) — Answers

1. $(1 - 0.27)(0.27) = 0.73 \cdot 0.27 \approx 0.197$, the same as σ′(1): the curve's slope is symmetric around 0.

2. $2 \cdot (1 - 0.9)(0.9) = 2 \cdot 0.09 = 0.18$.

3. With $p = \sigma$: 0.4 · 0.6 = 0.24, 0.3 · 0.7 = 0.21, 0.5 · 0.5 = 0.25. The product $p(1-p)$ peaks when the two factors are equal, at p = 0.5, i.e. at x = 0.

↩ back to the card

S5.10 · 8 · Practice on a new graph: f = w · σ(x·y + z) — Full worked solution

(a) Forward, left to right.

- $m = x \cdot y = 2 \cdot (-1) = -2$.
  Why: first × gate.
- $r = m + z = -2 + 2 = 0$.
  Why: + gate.
- $s = \sigma(0) = \dfrac{1}{1+e^0} = \dfrac{1}{2} = 0.5$.
  Why: sigmoid box; $e^0 = 1$.
- $f = w \cdot s = 4 \cdot 0.5 = 2$.
  Why: last × gate.

(b) Backward, start at $\partial f/\partial f = 1$; each line is upstream × local.

- Last × gate (swap), upstream 1: $\partial f/\partial w = 1 \cdot s = 0.5$; $\partial f/\partial s = 1 \cdot w = 4$.
  Why: each input gets the other input.
- Sigmoid box, upstream 4: local $= s(1-s) = 0.5 \cdot 0.5 = 0.25$, so $\partial f/\partial r = 4 \cdot 0.25 = 1$.
  Why: card 6's local gradient, from the stored output s.
- + gate (copies), upstream 1: $\partial f/\partial z = 1 \cdot 1 = 1$ and $\partial f/\partial m = 1 \cdot 1 = 1$.
  Why: local slope 1 to each input.
- First × gate (swap), upstream 1: $\partial f/\partial x = 1 \cdot y = 1 \cdot (-1) = -1$; $\partial f/\partial y = 1 \cdot x = 1 \cdot 2 = 2$.
  Why: each input gets the other input.

wire	value (forward)	gradient ∂f/∂wire (backward)
x	2	−1
y	−1	2
z	2	1
w	4	0.5
m	−2	1
r	0	1
s	0.5	4
f	2	1

Check by nudging y from −1 to −0.99: $m = -1.98$, $r = 0.02$, $s = \sigma(0.02) \approx 0.505$, $f \approx 2.02$, a change of about $+0.02 = 2 \cdot 0.01$. ✓

Forward: $m = -2$, $r = 0$, $s = 0.5$, $f = 2$. **Backward:** $\partial f/\partial w = 0.5$, $\partial f/\partial s = 4$, $\partial f/\partial r = 1$, $\partial f/\partial z = 1$, $\partial f/\partial m = 1$, $\partial f/\partial x = -1$, $\partial f/\partial y = 2$.

↩ back to the card

S5.11 · 8 · Practice on a new graph: f = w · σ(x·y + z) — Answers

1. $m = 1, r = 0, s = 0.5, f = 1.5$. $\partial f/\partial w = 0.5$, $\partial f/\partial s = 3$, $\partial f/\partial r = 3 \cdot 0.25 = 0.75$, $\partial f/\partial z = 0.75$, $\partial f/\partial x = 0.75 \cdot y = 0.75$, $\partial f/\partial y = 0.75 \cdot x = 0.75$.

2. $m = 1, r = 1, s = 0.73, f = 0.73$. $\partial f/\partial w = 0.73$, $\partial f/\partial s = 1$, $\partial f/\partial r = (1 - 0.73)(0.73) \approx 0.20$ (exactly 0.197), so $\partial f/\partial z = \partial f/\partial x = \partial f/\partial y \approx 0.20$.

3. y: $\partial f/\partial y = 2$ is the largest in size, so raising y by 0.01 raises f by about 0.02. (x has −1: raising x lowers f.)

↩ back to the card

S5.12 · 9 · Quick recall — Answer

Local ($\partial z/\partial x$, from the node's own formula), upstream ($\partial L/\partial z$, arriving from the right), downstream ($\partial L/\partial x$ = upstream × local, sent to the left).

↩ back to the card

S5.13 · 9 · Quick recall — Answer

It is $\partial f/\partial f$: nudging the output by a tiny amount changes it by exactly that amount.

↩ back to the card

S5.14 · 9 · Quick recall — Answer

2 and 2. The add gate copies the upstream to every input (distributor).

↩ back to the card

S5.15 · 9 · Quick recall — Answer

x gets 5 · 3 = 15, y gets 5 · 2 = 10 (swap: each gets upstream × the other input).

↩ back to the card

S5.16 · 9 · Quick recall — Answer

Local gradient $-1/x^2 = -1/4$, so $3 \cdot (-0.25) = -0.75$. Use the *input* 2, not the output 0.5.

↩ back to the card

S5.17 · 9 · Quick recall — Answer

$1 \cdot (1 - 0.73)(0.73) \approx 0.20$. The sigmoid's local gradient uses its *output*.

↩ back to the card

S5.18 · 9 · Quick recall — Answer

Raise λ (stronger regularisation). Slide 33: don't use network size as the regulariser.

↩ back to the card

S5.19 · 9 · Quick recall — Answer

No. Pick the split where every local gradient is easy, e.g. one sigmoid box instead of ×−1, exp, +1, 1/x.

↩ back to the card

LECTURE 6

Optimization part 1: does gradient descent get there, and how fast?

Tier A · must ⏱ 92 min Slides: lec6_convergence.pdf (155 p) · in scope: p36–45 and p81 → end 🎥 CMU video

📘 d2l: gradient descent

Lectures 3–5 gave you the machine: a loss, its gradient (from backprop) and the update $w \leftarrow w - \eta \cdot$ gradient. This lecture asks the uncomfortable questions: does that update land on the answer we wanted, does it land at all, and how fast? Almost everything reduces to one picture, **a marble in a bowl**, and one number, the step size η compared with the bowl's curvature.

What the exam can ask from this lecture

- The three in-scope class polls (Poll 2–4), almost word for word — card 8.
- Definitions: saddle point (Hessian eigenvalues of mixed sign), convex, "converge", convergence rate R and "linear" convergence.
- The modified update $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{A}^{-1} \nabla E^T$, Newton's method ($\eta = 1$, $\mathbf{H}^{-1}$), one Newton step by hand, and why Newton is not used for big nets.
- One step of RProp, QuickProp, momentum or Nesterov by hand; learning-rate decay schedules.
- True/false reasoning: "a too-large η is always bad", "RProp reverses on a sign flip", "momentum is faster on a constant slope".

⏳ **Only 20 minutes?** Do card 8 (the polls, 5 min), then card 6 rungs ①–⑤ for RProp only, skipping QuickProp (9 min), then card 3 (Newton, 7 min), then read the Traps box.

📖 READ & UNDERSTAND · ⏱ 8 min · Tier A

1 · The landscape: saddles, convex bowls, and what "converge" means

Loss surface = the loss plotted as a height over every possible weight setting. Training is walking downhill on it, using the gradient (the vector of slopes, one per weight) from backprop. Two words you need:

- The **Hessian H** is the table (matrix) of all second derivatives $\partial^2 E / \partial w_i \partial w_j$. It says how the surface *curves*. Its **eigenvalues** are the curvatures along the surface's main directions: positive = curves up (a valley that way), negative = curves down (a ridge).
- A **saddle point** (p37): slope is zero, but the surface goes up in some directions and down in others, i.e. the Hessian has some positive and some negative eigenvalues. Gradient descent can crawl to a near stop there because the gradient is tiny.

Local minima vs saddles (p36–39). The popular hypothesis: in **large** networks saddle points are far more common than local minima (their number grows exponentially with network size), and most local minima are about equally good, close to the global one. Not true for small networks: there you can hit bad local minima. p38's paper list is "watch this space" (skim). Bottom line (p39): gradient descent is not guaranteed to find the "true" solution, even when one exists.

🧭 **Like a horse saddle, or a mountain pass on Google Maps terrain view.** At the pass you are at the lowest point of the ridge you crossed, but the highest point of the road down. Standing exactly

there, the ground feels flat.

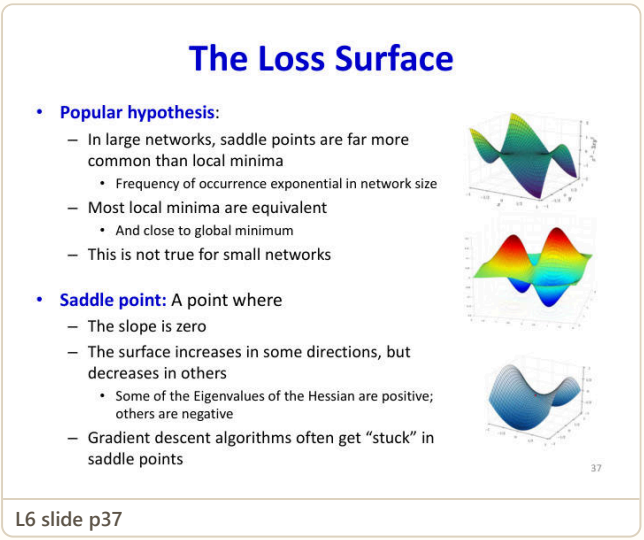
Convex (p42): a surface that curves upward everywhere, like a bowl. Test: the straight line between any two points on the surface never goes below it. A convex function has **one** valley, so any minimum is the global one. Neural-network losses are **not** convex; we study convex bowls anyway because they are the only case we can analyse ("streetlight effect": looking for your keys where the light is).

Converge (p43): the updates arrive at a fixed point (gradient 0, further updates change nothing). The alternatives: *jitter* around the minimum, or *diverge* (fly off). **Convergence rate** (p44):

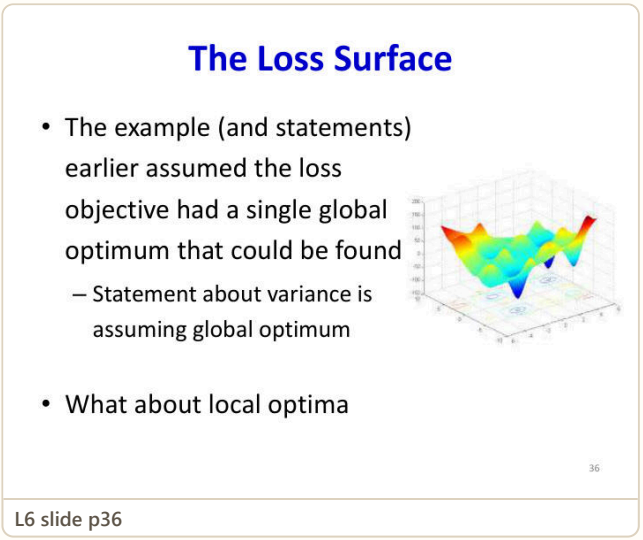
$$R = \frac{f(x^{(k+1)}) - f(x^*)}{f(x^{(k)}) - f(x^*)}$$

where x^* is the optimum (e.g. if the gap above the optimum shrinks from 8 to 2 in one step, $R = 2/8 = 0.25$). If R stays below a constant < 1, the gap shrinks like R^k: exponentially fast, which the slides confusingly call **"linear" convergence** (it looks like a straight line on a log plot). In formula form (p44): $f(x^{(k)}) - f(x^*) \leq R^k \left(f(x^{(0)}) - f(x^*)\right)$; e.g. with $R = 0.5$ and a starting gap of 8, after 3 steps the gap is at most $0.5^3 \cdot 8 = 1$.

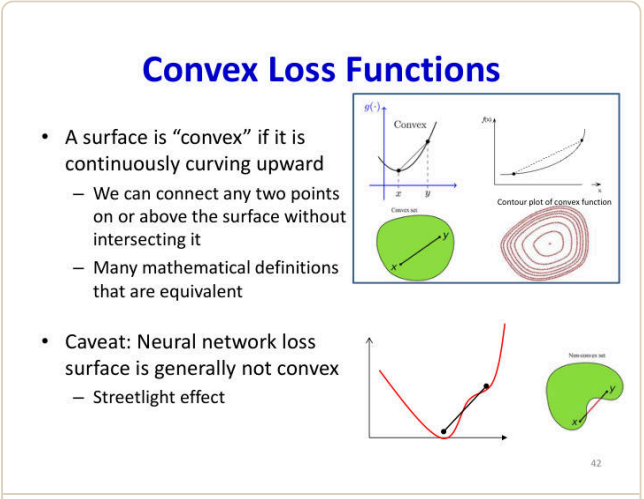
Like rewinding a cassette with a pencil when each turn winds half of what is left. Halving the remaining gap every step is "linear convergence": 10 steps leave about 1/1000. *Where it breaks:* real losses are not perfect bowls, so the ratio R changes from place to place.



L6 slide p37



L6 slide p36



L6 slide p42

↗ Understand more: d2l, local minima and saddle points ↗ d2l: convexity ↗ guide: Hessian eigenvalues (Tutorial 9)

PEN & PAPER · ⏱ 15 min · Tier A

2 · Pen: the quadratic bowl set-up and measuring the convergence rate

① WHAT IS BEING ASKED, IN PLAIN WORDS

There is one weight w (a single number) and a loss $E(w)$ that we want as small as possible. The loss is a **parabola**, a U-shaped curve, i.e. a bowl. Its general form (p45) is $E(w) = \frac{1}{2}aw^2 + bw + c$ with $a > 0$; a is the **curvature** (the second derivative $E'' = a$: big a = narrow, steep bowl).

Gradient descent repeats one move: read the **slope** $E'(w)$ at the current point, and step against it by "learning rate × slope": $w_{\text{new}} = w - \eta\, E'(w)$. The learning rate η is a small fixed number we choose. The bottom of the bowl is w^*, and the **gap** is how much higher the current loss is than the bottom's loss, $E(w) - E(w^*)$. The **convergence rate** R = new gap ÷ old gap: the fraction of the gap that survives one step.

② WARM-UP BY HAND, NO FORMULAS

A small bowl: $E(w) = w^2 - 2w$. Its slope at w is $2w - 2$. Start at $w = 0$ with $\eta = 0.25$. The bottom is at $w = 1$, where the loss is $1 - 2 = -1$ (step 3 below shows how to find it), so the gap is $E(w) - (-1)$.

step	w	slope $2w-2$	move $0.25\times$slope	new w	gap before → after	after ÷ before
1	0	−2	−0.5	0.5	1 → 0.25	0.25
2	0.5	−1	−0.25	0.75	0.25 → 0.0625	0.25
3	0.75	−0.5	−0.125	0.875	0.0625 → 0.015625	0.25

Look at the last column: **every step keeps exactly a quarter of the gap**. The distance to the bottom halves each time (1, 0.5, 0.25, 0.125), so the gap, which goes like distance squared, quarters.

③ THE ONE IDEA

On a parabola the slope is proportional to the distance from the bottom. So every step removes the **same fraction** of the remaining distance, the distance is multiplied by a fixed number $(1 - \eta a)$, and the gap by its square. A constant ratio R is exactly what "linear convergence" means.

④ BUILD IT, STEP BY STEP

(on the warm-up bowl $E = w^2 - 2w$, start $w^{(0)} = 0$, $\eta = 0.25$; the general rule and the warm-up numbers side by side).

1. Read off a, b, c by matching $E = \frac{1}{2}aw^2 + bw + c$: $w^2 = \frac{1}{2} \cdot 2 \cdot w^2$, so $a = 2, b = -2, c = 0$.
   Why: every later line uses a and b; a is **twice** the w^2 coefficient because of the $\frac{1}{2}$.
2. Slope: $E'(w) = aw + b = 2w - 2$.
   Why: differentiate $\frac{1}{2}aw^2 + bw + c$ term by term.
3. Bottom: $E'(w^*) = 0 \Rightarrow w^* = -b/a = 1$, and $E^* = E(1) = 1 - 2 = -1$.
   Why: the bottom of a bowl is where the slope is zero; plug it back in for the lowest loss.

4. Gradient steps, $w^{(k+1)} = w^{(k)} - \eta\, E'(w^{(k)})$: $w^{(1)} = 0 - 0.25(-2) = 0.5$; $w^{(2)} = 0.5 - 0.25(2 \cdot 0.5 - 2) = 0.5 + 0.25 = 0.75$.
   Why: "minus η × slope" moves against the slope; a negative slope makes w grow.
5. Gaps $E(w) - E^*$: $E(0) = 0 \to$ gap 1; $E(0.5) = 0.25 - 1 = -0.75 \to$ gap 0.25; $E(0.75) = 0.5625 - 1.5 = -0.9375 \to$ gap 0.0625.
   Why: R is measured on the gap above the bottom, never on the raw loss (losses can be negative).
6. Measured rate: $R_1 = 0.25/1 = 0.25$, $R_2 = 0.0625/0.25 = 0.25$.
   Why: R = new gap ÷ old gap, one ratio per step.
7. Predict it. The slope is "curvature × distance": $E'(w) = a(w - w^*)$ (since $a(w + b/a) = aw + b$). Subtract w^* from the update: $w_{\text{new}} - w^* = (w - w^*) - \eta a(w - w^*) = (1 - \eta a)(w - w^*)$. Here $1 - 0.25 \cdot 2 = 0.5$: the distance halves (1, 0.5, 0.25 ✓).
   Why: this is the line that proves the ratio never changes.
8. Gap = $\frac{1}{2}a(w - w^*)^2$ (expand: $\frac{1}{2}a(w + b/a)^2 = \frac{1}{2}aw^2 + bw + \frac{b^2}{2a}$, and $E^* = c - \frac{b^2}{2a}$), so $R = (1 - \eta a)^2 = 0.5^2 = 0.25$, the same every step: **linear convergence**.
   Why: the gap is distance squared, so it gets the square of the factor.
9. Bound after k steps: $E(w^{(k)}) - E^* \le R^k\big(E(w^{(0)}) - E^*\big) = 1 \cdot 0.25^k$ (equality on an exact bowl).
   Why: each step multiplies the gap by R once.

(background, not on the exam) Line 7 also gives the famous rules: $\eta = 1/a$ makes the factor 0 (one jump to the bottom, so $\eta_{opt} = 1/a = 1/E''$), and $\eta > 2/a = 2\eta_{opt}$ makes $|1 - \eta a| > 1$, so it diverges. You need this for Newton (card 3) and Poll 2 (card 8).

⑤ CHECK IT

The predicted $R = 0.25$ (line 8) equals the measured one (line 6) ✓. Bound at $k = 2$: $1 \cdot 0.25^2 = 0.0625$, the gap in line 5 ✓. Edge cases on the same bowl from $w = 0$: $\eta = 0.5 = 1/a$ gives $0 \to 1$ in one jump (factor 0); $\eta = 1 = 2/a$ gives $0 \to 2 \to 0 \to 2$ (bounces forever, factor -1); $\eta = 1.2$ gives $0 \to 2.4 \to -0.96 \to 3.744$, distances to 1 of $-1,\ 1.4,\ -1.96,\ 2.744$ (factor -1.4: diverges).

⑥ NOW THE EXAM PROBLEM — YOUR TURN

EXAM PROBLEM

$E(w) = 2w^2 - 8w$ (so $a = 4$, $b = -8$, $c = 0$), start at $w^{(0)} = 0$, $\eta = 0.1$. (a) Find w^* and $E(w^*)$. (b) Do 2 gradient steps. (c) Compute the convergence rate R of the loss gap for each step, and bound the gap after k steps.

Do it the same way: steps ④–⑤ with these numbers.

✍ Hint: S6.1 (end of this lecture)

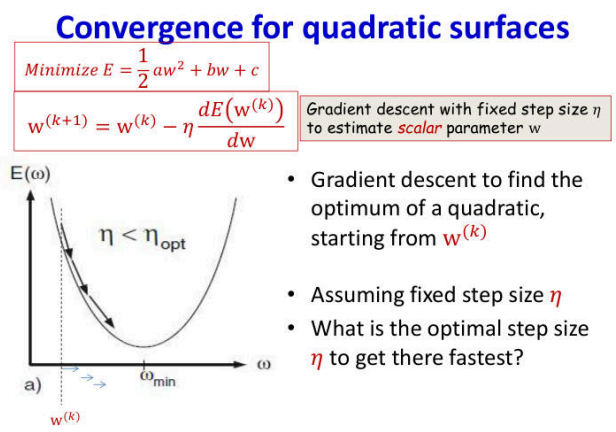
✍ Full worked solution: S6.2 (end of this lecture)

⑦ TRY ALONE

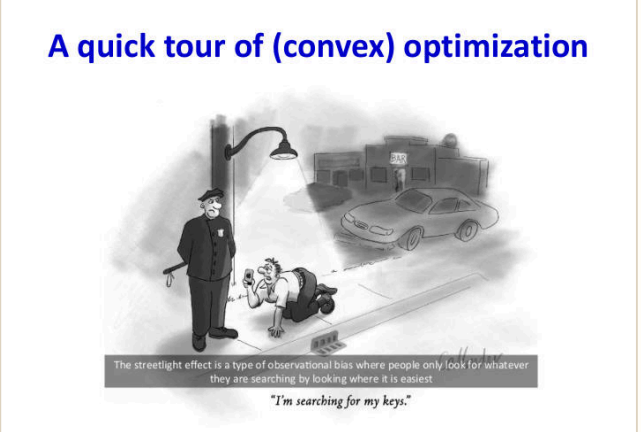
(i) $E = w^2$ from $w = 1$ with $\eta = 0.1$: what is R? (ii) $E = 3w^2$: R for $\eta = 0.1$, the one-jump η, and the η above which it diverges. (iii) $E = w^2 - 4w$ from $w = 0$ with $\eta = 1.5$: do 2 steps and the gaps. What happens?

✍ Answers: S6.3 (end of this lecture)

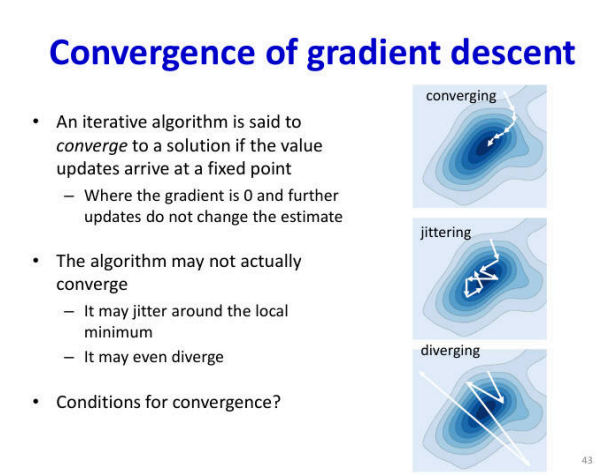
Like a marble rolling in a salad bowl, or the "hot and cold" game. Small nudges creep to the bottom, each one closing the same fraction of what is left. *Where it breaks:* a real marble carries speed between moves; plain gradient descent forgets everything each step (momentum, card 7, adds the memory back).



L6 slide p45



L6 slide p41



L6 slide p43

↗ Understand more: d2l, one-dimensional gradient descent and learning rate ↗ CMU L6 video, slides p40–45

📖 READ & UNDERSTAND · ⏱ 7 min · Tier A

3 · The modified update with A^{-1} → Newton's method, and why we can't use it

Many weights. With a vector of weights $\mathbf{w}$ the bowl is $E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{b}^T\mathbf{w} + c$, where $\mathbf{A}$ is a symmetric matrix (a square table of numbers) that plays the role of a; it is exactly the Hessian of E (card 1).

(background, not on the exam) Different directions of this bowl have different curvatures, so they need **different step sizes** ($\eta_{opt} = 1/\text{curvature}$ in each), and one shared η is too slow in flat directions or diverges in steep ones. The fix is to stretch the axes, $\hat{\mathbf{w}} = \mathbf{A}^{1/2}\mathbf{w}$, so the bowl becomes round: $E = \frac{1}{2}\hat{\mathbf{w}}^T\hat{\mathbf{w}} + \hat{\mathbf{b}}^T\hat{\mathbf{w}} + c$, where every direction has the same curvature.

The modified update rule (p81–82). In the round bowl the learning rate is "independent of direction", so plain gradient descent works. Translating back with $\nabla_{\hat{\mathbf{w}}}E^T = \mathbf{A}^{-1/2}\nabla_{\mathbf{w}}E^T$ gives:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta\, \mathbf{A}^{-1}\, \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)})^T$$

Every gradient is multiplied by $\mathbf{A}^{-1}$ (the inverse matrix, the multi-weight version of "divide by a") before stepping: this is the **scaling** that gives each direction its own step size. E.g. $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$ (steep in w_1, flat in w_2) and gradient $(8, 2)$: $\mathbf{A}^{-1}\nabla E^T = (8/4,\ 2/1) = (2, 2)$, so the steep direction's big slope is divided by 4 and both weights move by the same amount.

Non-axis-aligned quadratics (p83–84). If $\mathbf{A}$ is not diagonal, E has cross terms $a_{ij} w_i w_j$ and the elliptical contours are tilted. Their main axes are the **eigenvectors** of $\mathbf{A}$ (the directions $\mathbf{A}$ only stretches, never turns), and the optimal step along each axis is inversely proportional to its **eigenvalue** (that stretch factor). It is just a rotation of the axis-aligned case, so the same $\mathbf{A}^{-1}$ rule fixes it.

Any smooth loss (p85–86). A second-order Taylor expansion around the current point, $E(\mathbf{w}) \approx E(\mathbf{w}^{(k)}) + \nabla E\,(\mathbf{w} - \mathbf{w}^{(k)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(k)})^T \mathbf{H} (\mathbf{w} - \mathbf{w}^{(k)})$, has the same quadratic form, with the Hessian $\mathbf{H}$ in place of $\mathbf{A}$:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta\, \mathbf{H}^{-1}\, \nabla_{\mathbf{w}} E^T$$

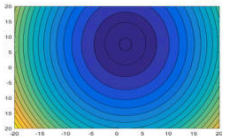
With $\eta = 1$ this is **Newton's method**: fit a quadratic bowl at the current point and jump straight to its bottom (p87–97). E.g. with one weight, slope $E' = 6$ and curvature $E'' = 3$ give the step $w \leftarrow w - 6/3 = w - 2$. For a true quadratic the optimal η is exactly 1, and it must not exceed 2. It is the multi-weight version of $\eta_{opt} = 1/E''$ (card 2's background line).

Issues (decisive list for the exam):

1. **Size** (p98): a net with 100,000 parameters has a Hessian with $100{,}000^2 = 10^{10}$ entries, and inverting it is even worse.
2. **Non-convexity** (p99–100): where the surface curves down in some direction, $\mathbf{H}$ is not positive definite and Newton happily moves *towards a maximum or saddle* (it jumps to wherever the slope is zero). Needs extra checks.
3. **Approximations** (p101): BFGS and L-BFGS (low-memory), Levenberg–Marquardt (adds a constant to the diagonal, "diagonal loading", to force positive definiteness), other quasi-Newton methods. Not popular for large networks anymore.

🧭 **Like Google Maps choosing a route from the real road shape instead of "always head straight at the destination".** Gradient descent walks straight downhill (perpendicular to the contour, which in a long valley points at the wall, not the exit). Newton uses the curvature to aim at the bottom directly. *Where it breaks:* Google Maps' map is cheap; for a neural net, the Hessian "map" is billions of numbers and changes at every step.

Returning to our problem



$$E = \frac{1}{2}\hat{\mathbf{w}}^T\hat{\mathbf{w}} + \hat{\mathbf{b}}^T\hat{\mathbf{w}} + c$$

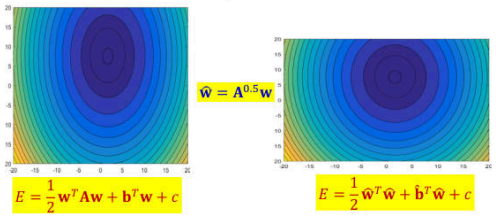
- Gradient descent rule:
  - $\hat{\mathbf{w}}^{(k+1)} = \hat{\mathbf{w}}^{(k)} - \eta\nabla_{\hat{\mathbf{w}}}E\left(\hat{\mathbf{w}}^{(k)}\right)^T$
  - Learning rate is now independent of direction
- Using $\hat{\mathbf{w}} = \mathbf{A}^{0.5}\mathbf{w}$, and $\nabla_{\hat{\mathbf{w}}}E(\hat{\mathbf{w}})^T = \mathbf{A}^{-0.5}\nabla_{\mathbf{w}}E(\mathbf{w})^T$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta\mathbf{A}^{-1}\nabla_{\mathbf{w}}E\left(\mathbf{w}^{(k)}\right)^T$$

81

L6 slide p81

Modified update rule



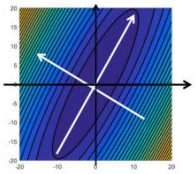
- $\hat{\mathbf{w}}^{(k+1)} = \hat{\mathbf{w}}^{(k)} - \eta\nabla_{\hat{\mathbf{w}}}E\left(\hat{\mathbf{w}}^{(k)}\right)^T$
- Leads to the modified gradient descent rule

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta\mathbf{A}^{-1}\nabla_{\mathbf{w}}E\left(\mathbf{w}^{(k)}\right)^T$$

82

L6 slide p82

For non-axis-aligned quadratics..



$$E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{w}^T\mathbf{b} + c$$

$$E = \frac{1}{2}\sum_i a_{ii}w_i^2 + \sum_{i\neq j} a_{ij}w_iw_j + \sum_i b_iw_i + c$$

- If $\mathbf{A}$ is not diagonal, the contours are not axis-aligned
  - Because of the cross-terms $a_{ij}w_iw_j$
  - The major axes of the ellipsoids are the *Eigenvectors* of $\mathbf{A}$, and their diameters are proportional to the Eigen values of $\mathbf{A}$
- But this does not affect the discussion
  - This is merely a rotation of the space from the axis-aligned case
  - The component-wise optimal learning rates along the major and minor axes of the equal-contour ellipsoids will be different, causing problems
    - The optimal rates along the axes are Inversely proportional to the *eigenvalues* of $\mathbf{A}$

83

L6 slide p83

↗ Understand more: Newton's method in optimisation (Wikipedia) ↗ d2l: Newton's method and preconditioning (section "Adaptive Methods")

PEN & PAPER · ⏱ 17 min · Tier A

4 · Pen: Newton in one step, Newton's speed, and Newton going wrong

① WHAT IS BEING ASKED, IN PLAIN WORDS

Newton's method does one thing per step: at the current point, pretend the loss is a parabola (a bowl) with the **same slope** and the **same curvature** as the real loss there, then jump straight to that bowl's bottom. With one weight the step is $w_{\text{new}} = w - E'(w)/E''(w)$: slope E' (which way is downhill and how steep) divided by curvature E'' (how fast the slope changes). With many weights the slope becomes the **gradient** vector $\mathbf{g}$ (one slope per weight), the curvature becomes the **Hessian** matrix $\mathbf{H}$ (card 1), and "divide by" becomes "multiply by the inverse": $\mathbf{w}_{\text{new}} = \mathbf{w} - \mathbf{H}^{-1}\mathbf{g}$. The problem asks for one 2-weight step, three 1-weight steps, and one step where it misbehaves.

② WARM-UP BY HAND, NO FORMULAS

Loss $E(w) = w^2 - 6w$, standing at $w = 0$. List the slope at a few points (slope $= 2w - 6$):

w	0	1	2	3
slope	−6	−4	−2	0

The slope is -6 now and rises by 2 for every unit you move right. To bring it to 0 you need $6/2 = 3$ units. So the bottom is at $w = 3$, found from information at $w = 0$ alone.

③ THE ONE IDEA

Distance to the bottom = slope ÷ curvature. The slope tells you how far you are from flat, the curvature tells you how fast you approach flat, so their ratio is how far to go. Newton is exactly that sentence, with matrices when there are many weights.

④ BUILD IT, STEP BY STEP

(four small sibling problems, one for each kind of exam part).

Part A, the rule (warm-up bowl $E = w^2 - 6w$ at $w = 0$):

1. 1-D rule: $w_{\text{new}} = w - \dfrac{E'(w)}{E''(w)}$. Here $E' = 2w - 6 = -6$, $E'' = 2$, so $w_{\text{new}} = 0 - (-6)/2 = 3$.
   Why: it is the sentence of ③; the minus sign sends you downhill.
2. On any parabola $E = \frac{1}{2}aw^2 + bw + c$: $E' = aw + b$, $E'' = a$, so $w - \frac{aw+b}{a} = -\frac{b}{a} = w^*$ from **any** start.
   Why: the pretend bowl is the real bowl, so one jump is exact.

Part B, two weights ($\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $\mathbf{b} = (-3, -3)$, start $(0, 0)$):

3. Gradient $\mathbf{g} = \mathbf{A}\mathbf{w} + \mathbf{b} = (0, 0) + (-3, -3) = (-3, -3)$; Hessian $\mathbf{H} = \mathbf{A}$.
   Why: the same as $aw + b$ and a, one row per weight.
4. Inverse of a 2×2 matrix: $\begin{pmatrix} p & q \\ r & s \end{pmatrix}^{-1} = \frac{1}{ps-qr}\begin{pmatrix} s & -q \\ -r & p \end{pmatrix}$ (swap the diagonal, negate the other two, divide by the determinant $ps - qr$). Here $ps - qr = 2 \cdot 2 - 1 \cdot 1 = 3$, so $\mathbf{A}^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$.
   Why: $\mathbf{A}^{-1}$ undoes $\mathbf{A}$, the 2-D "divide by a".
5. Multiply, row times column: $\mathbf{A}^{-1}\mathbf{g} = \frac{1}{3}\big(2(-3) - 1(-3),\ -1(-3) + 2(-3)\big) = \frac{1}{3}(-3, -3) = (-1, -1)$.
   Why: this is "slope ÷ curvature" for both weights at once.
6. Newton step: $\mathbf{w}_{\text{new}} = \mathbf{w} - \mathbf{A}^{-1}\mathbf{g} = (0, 0) - (-1, -1) = (1, 1)$.
   Why: rule 1 with matrices; it is card 3's update with $\eta = 1$.

Part C, a loss that is not a parabola ($E = e^w - w$, true minimum at $w = 0$, start $w = 1$):

7. $E' = e^w - 1$, $E'' = e^w$, so the step is $w - \dfrac{e^w - 1}{e^w} = w - 1 + e^{-w}$.
   Why: rule 1; simplify once, then reuse.
8. Step 1: $1 - 1 + e^{-1} = 0.3679$. Step 2: $0.3679 - 1 + e^{-0.3679} = 0.0601$. Step 3: $0.0601 - 1 + e^{-0.0601} = 0.0018$. Errors (distance to 0): $1 \to 0.368 \to 0.060 \to 0.0018$.
   Why: the pretend bowl is only approximately right, so it takes several jumps, but the number of correct digits roughly **doubles** each step.

Part D, curving down ($E = \frac{1}{4}w^4 - 2w^2$, minima at ± 2, a maximum at 0, start $w = 0.5$):

9. $E' = w^3 - 4w = 0.125 - 2 = -1.875$; $E'' = 3w^2 - 4 = 0.75 - 4 = -3.25$. **Look at the sign of E'' first**: negative.
   Why: $E'' < 0$ means the pretend "bowl" is upside down (a hill), and its "bottom" is really its **top**.

10. Step: $0.5 - \dfrac{-1.875}{-3.25} = 0.5 - 0.5769 = -0.0769$, right next to the maximum at 0. The loss rises from $E(0.5) = -0.4844$ to $E(-0.0769) = -0.0118$.
    Why: the formula only looks for slope = 0; it cannot tell a minimum from a maximum. Plain GD would go to $0.5 + 1.875\eta$, towards the minimum at 2.

⑤ **CHECK IT**

Part A: $E'(3) = 6 - 6 = 0$ ✓ (matches the warm-up table). Part B: $\mathbf{A}(1,1)^T + \mathbf{b} = (2+1-3,\ 1+2-3) = (0,0)$ ✓, the minimum in one step; and $\mathbf{A}\mathbf{A}^{-1}$ has diagonal $\frac{1}{3}(2 \cdot 2 - 1 \cdot 1) = 1$ ✓. Part C: for this loss the next error is about half the square of the last one: $0.368^2/2 = 0.068 \approx 0.060$, $0.060^2/2 = 0.0018$ ✓ (that is "digits double"). Part D: the loss went **up** ✓, so Newton made things worse. Simplest edge case: $E = -w^2$ at $w = 1$: $1 - (-2)/(-2) = 0$, the top of the hill.

⑥ **NOW THE EXAM PROBLEM — YOUR TURN**

EXAM PROBLEM

(a) $E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{b}^T\mathbf{w}$ with $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} -5 \\ -5 \end{pmatrix}$, start $(0,0)$. Do one Newton step ($\eta = 1$); the gradient is $\nabla E^T = \mathbf{A}\mathbf{w} + \mathbf{b}$.
(b) 1-D Newton on $E(w) = e^w - 2w$ from $w = 0$; update $w \leftarrow w - E'(w)/E''(w)$. Do 3 steps; the true minimum is $\ln 2 = 0.6931$.
(c) $E(w) = \frac{1}{4}w^4 - \frac{1}{2}w^2$ (minima at ± 1, a maximum at 0). Do one Newton step from $w = 0.3$.

Do it the same way: steps ④–⑤ with these numbers ((a) = Part B, (b) = Part C, (c) = Part D).

✍ Hint: S6.4 (end of this lecture)

✍ Full worked solution: S6.5 (end of this lecture)

⑦ **TRY ALONE**

(i) One Newton step on $E = 5w^2 - 10w + 7$ from $w = 100$. (ii) One Newton step for $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}$, $\mathbf{b} = (-2,-2)$ from $(0,0)$. (iii) $E = \cos w$ at $w = 0.5$: before computing, predict from the sign of E'' whether Newton heads for a minimum or a maximum; then do the step.

✍ Answers: S6.6 (end of this lecture)

↗ Understand more: Hessian matrix (Wikipedia) ↗ guide: level curves and the Hessian (Tutorial 9)

📖 **READ & UNDERSTAND · ⏱ 3 min · Tier A**

5 · A too-big step is not always bad: learning-rate decay

Everything so far assumed one convex bowl, where $\eta > 2\eta_{opt}$ means divergence. Real losses have many bumps. A step bigger than $2\eta_{opt}$ can **jump out of a poor local minimum** (p103). But if it stays big forever you never settle anywhere. Compromise: **start large, shrink over time** (p104–105).

schedule	formula	$\eta_0 = 1$: $k = 0, 1, 2, 3$
"linear" (really $1/k$)	$\eta_k = \eta_0/(k+1)$	1, 0.5, 0.333, 0.25

quadratic	$\eta_k = \eta_0/(k+1)^2$	1, 0.25, 0.111, 0.0625
exponential ($\beta = 0.5$)	$\eta_k = \eta_0 e^{-\beta k}$	1, 0.607, 0.368, 0.223
step decay (common for nets)	train at fixed η until the (held-out) loss stagnates, then $\eta \leftarrow \alpha\eta$, $\alpha \approx 0.1$; repeat	e.g. 0.1 → 0.01 → 0.001

Lesson of p110: **many convergence problems come from forcing the same learning rate on every parameter**. That motivates the next two cards.

🧭 **Like tuning an old radio dial.** You spin it fast to get near the station (and past bad static), then turn it more and more gently to lock in. Spin fast forever and you never land on the station. *Where it breaks:* the radio has one dial; a network has millions, each wanting a different speed.

↗ Understand more: d2l, learning-rate scheduling

🖍 PEN & PAPER · ⏱ 20 min · Tier A

6 · Pen: RProp and QuickProp — each weight gets its own step

① WHAT IS BEING ASKED, IN PLAIN WORDS

Both methods treat **every weight separately** (no shared η, no coupling between weights) and use the derivative from backprop only as a hint (p111–112). Write $D = dE/dw$ for the derivative (the slope) of the loss with respect to one weight: $D < 0$ means "the loss goes down if w grows", $D > 0$ means "the loss goes down if w shrinks".

- **RProp** ("resilient propagation", p113–123) keeps a **step size** Δ per weight and looks only at the **sign** of D, never its size.
- **QuickProp** (p126–131) does a cheap 1-D Newton step (card 4) per weight, estimating the curvature from the last two slopes.

The problems ask you to run RProp by hand (a table of tries, keeps and undos) and to do one QuickProp step.

② WARM-UP BY HAND, NO FORMULAS

RProp. Loss $(w-1)^2$, bottom at 1, slope $2(w-1)$. Start at $w = 0$ (slope −2: go right) with step 0.4. Rule: same sign → step ×1.2; sign flipped → go back, step ×0.5.

try w	slope	sign vs last kept	what happens	next step
0.4	−1.2	same (−)	keep	0.48
0.88	−0.24	same (−)	keep	0.576
1.456	+0.912	flipped	jumped past 1: back to 0.88	0.288
1.168	+0.336	flipped	back to 0.88 again	0.144
1.024	+0.048	flipped	back to 0.88 again	0.072
0.952	−0.096	same (−)	keep	0.0864

The step grows while things go well and shrinks around the bottom, and the steepness of the slope never mattered. *QuickProp*. Loss $(w-5)^2$: slope at $w=0$ is −10, at $w=1$ it is −8. The slope rose by 2 over a move of 1, so it rises 2 per unit; to climb from −8 to 0 takes $8/2=4$ more units: $w=5$, the bottom.

③ THE ONE IDEA

RProp: the **sign** of the slope tells you which side of the bottom you are on; the same side again = speed up, the other side = you jumped over it, so go back and take half. *QuickProp:* two slopes give the **curvature for free** (change in slope ÷ distance moved), and then it is card 4's "slope ÷ curvature".

④ BUILD IT, STEP BY STEP

RProp, the slide's algorithm (p122), growth $\alpha=1.2$, shrink $\beta=0.5$ (this β is NOT momentum's β), run on the warm-up: $E=(w-1)^2$, $D=2(w-1)$, $w_0=0$, $\Delta_0=0.4$.

1. Start: $D_{prev}=D(w_0)$ and $\Delta w=\text{sign}(D_{prev})\cdot\Delta_0$. Here $D_{prev}=2(0-1)=-2$, $\Delta w=-0.4$.
   Why: Δw carries the step size and the direction; the update subtracts it, so it moves against the slope.
2. Try: $w\leftarrow w-\Delta w$, then compute the new D. Here $w=0-(-0.4)=0.4$, $D=2(0.4-1)=-1.2$.
   Why: backprop gives D at the new point.
3. Same sign as D_{prev}: keep the point, $\Delta w\leftarrow\alpha\Delta w$, $D_{prev}\leftarrow D$. Here -1.2 and -2 agree: $\Delta w=1.2(-0.4)=-0.48$, $D_{prev}=-1.2$. Next try $0.4+0.48=0.88$, $D=-0.24$, same again: $\Delta w=-0.576$, $D_{prev}=-0.24$.
   Why: still on the same side of the bottom, so speed up.
4. Sign flipped: $w\leftarrow w+\Delta w$ (undo the step), $\Delta w\leftarrow\beta\Delta w$, and D_{prev} is **not** updated. Here try $0.88+0.576=1.456$, $D=+0.912$: flipped against -0.24, so $w=1.456+(-0.576)=0.88$, $\Delta w=0.5(-0.576)=-0.288$.
   Why: you overshot; back up, then try again the **same way** with half the step. (Optional caps Δ_{max}, Δ_{min}.)
5. Keep going with the same two rules. Try $0.88+0.288=1.168$, $D=+0.336$, flipped against the **kept** -0.24: undo to 0.88, $\Delta w=-0.144$. Try 1.024, $D=+0.048$: flipped, undo, $\Delta w=-0.072$. Try 0.952, $D=-0.096$: same, keep, $\Delta w=-0.0864$.
   Why: it halves until a try lands on the near side of the bottom, then grows again.

QuickProp (p126–131): Newton's update $\mathbf{w}\leftarrow\mathbf{w}-\mathbf{H}^{-1}\nabla E^T$ with two changes. Siblings: $E=(w-5)^2$ (a parabola) and $E=e^w-3w$ (not a parabola, true minimum $\ln 3=1.0986$), both with previous point $w^{(k-1)}=0$ and current point $w^{(k)}=1$.

6. Each weight on its own: use only its own second derivative $E''(w_i)$, not the full Hessian.
   Why: no $N\times N$ matrix to store or invert.
7. Estimate it from the last two slopes (a finite difference, assuming the loss is locally a parabola):

$$E''\approx\frac{E'(w^{(k)})-E'(w^{(k-1)})}{\Delta w^{(k-1)}},\qquad \Delta w^{(k-1)}=w^{(k)}-w^{(k-1)}$$

   For $(w-5)^2$: $E'=2(w-5)$, $E'(0)=-10$, $E'(1)=-8$, so $E''\approx(-8-(-10))/(1-0)=2$.
   Why: change in slope ÷ distance moved is what curvature means.
8. Newton step with the estimate: $w^{(k+1)}=w^{(k)}-\dfrac{E'(w^{(k)})}{E''_{\text{approx}}}=1-\dfrac{-8}{2}=5$.

Why: card 4, rule 1.

9. Same for $e^w - 3w$: $E' = e^w - 3$, $E'(0) = -2$, $E'(1) = e - 3 = -0.2817$; $E'' \approx (-0.2817 + 2)/1 = 1.7183$; $w = 1 - \dfrac{-0.2817}{1.7183} = 1 + 0.1640 = 1.1640$.

   Why: the same three lines; the answer is close to 1.0986 but not exact, because the loss is not a parabola. Fast on many problems; unstable on non-convex losses.

⑤ CHECK IT

RProp: the five tries match the warm-up table row by row ✓; the steps run $0.4 \to 0.48 \to 0.576$ ($\times 1.2$) then $0.288 \to 0.144 \to 0.072$ ($\times 0.5$) ✓. Edge case: rows 4–5 still compare with -0.24 (the last *kept* slope), because a flip does not update D_{prev}. QuickProp: at $w = 5$, $E' = 2(5 - 5) = 0$ ✓, the exact bottom (the parabola assumption is true). At $w = 1.1640$, $E' = e^{1.164} - 3 = 0.20$, small but not 0 ✓, so one more QuickProp step would refine it.

⑥ NOW THE EXAM PROBLEMS — YOUR TURN

EXAM PROBLEM

A (RProp). $E(w) = (w - 3)^2$, so $D = 2(w - 3)$. Start $w = 0$, initial step $\Delta_0 = 0.5$, $\alpha = 1.2$, $\beta = 0.5$. Run RProp until it has undone a step twice, and one step more.

B (QuickProp). Previous point $w^{(k-1)} = 0$, current $w^{(k)} = 1$. One QuickProp step for (i) $E = (w - 3)^2$, (ii) $E = e^w - 2w$ (true minimum 0.6931).

Do it the same way: steps ④–⑤ with these numbers (A = lines 1–5, B = lines 6–9).

✍ Hint: S6.7 (end of this lecture)

✍ Full worked solution (A, RProp): S6.8 (end of this lecture)

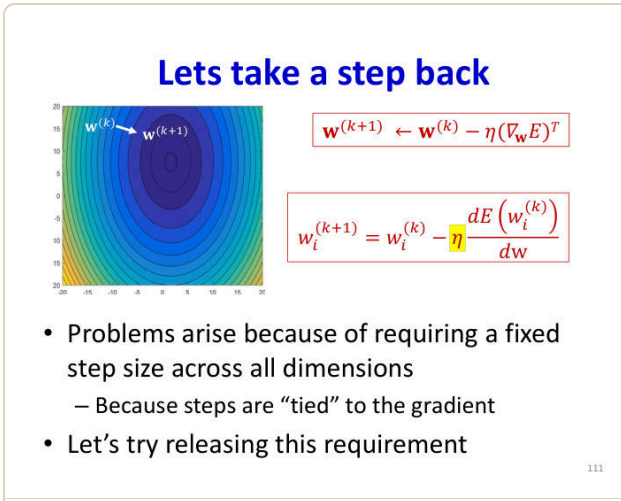
✍ Full worked solution (B, QuickProp): S6.9 (end of this lecture)

⑦ TRY ALONE

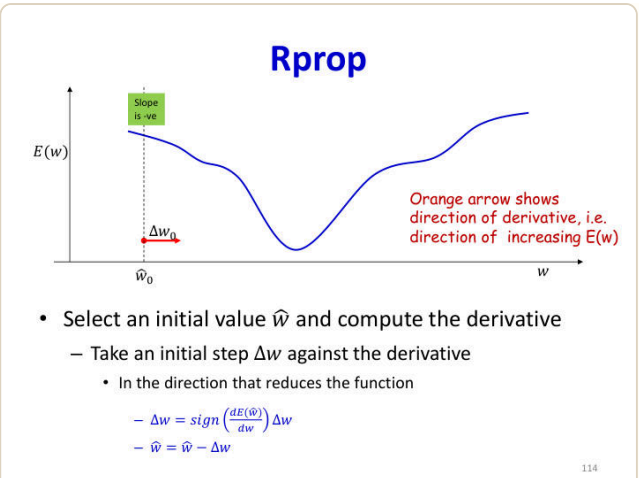
(i) The slope was $+3$; after a step it is $+1$. What does RProp do with the step size and direction? (ii) RProp on $E = (w + 2)^2$ from $w = 0$ with $\Delta_0 = 1$: list the first four tried w values and say which ones are undone. (iii) QuickProp on $E = w^2 - 2w$ with previous point 3 and current point 2 (a step to the *left*).

✍ Answers: S6.10 (end of this lecture)

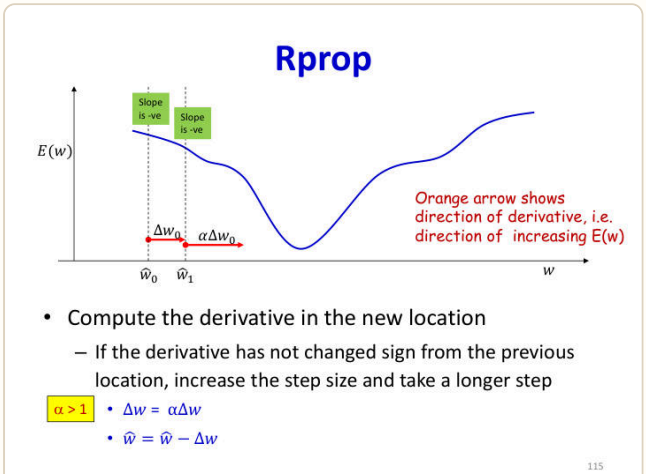
🧭 **RProp is like adjusting the shower temperature.** Still too cold? Turn the knob a bit more each time. Suddenly too hot? Go back to where you were and turn half as much. You only feel "colder/hotter", never by how much. *Where it breaks:* RProp does this separately for millions of knobs, and with mini-batches the sign itself gets noisy.



L6 slide p111



L6 slide p114



L6 slide p115

↗ Understand more: Rprop (Wikipedia) ↗ CMU L6 video, slides p111–131

🖍 PEN & PAPER · ⏱ 17 min · Tier A

7 · Pen: momentum and Nesterov — remember where you were going

① WHAT IS BEING ASKED, IN PLAIN WORDS

The problem these methods fix (p133–135): with one shared η, some directions converge smoothly while others **zig-zag** from one valley wall to the other. Instead of stepping by the current gradient alone, keep a **running average of past steps** and move by that.

Notation: ΔW is the step actually taken (think "velocity"); β (about 0.9) is the fraction of the previous step you keep; $-\eta\,\nabla Loss$ is the usual gradient-descent "push". **Momentum** adds the push measured where you stand. **Nesterov** adds the push measured at the point you would reach by just coasting with the kept part of the old step. The problem asks for three steps of each on a simple bowl, compared with plain gradient descent.

② WARM-UP BY HAND, NO FORMULAS

Rule: new step = 0.8 × previous step + this push. Two cases, pushes of size 0.1:

step	pushes always the same way	new step	pushes alternating (zig-zag)	new step

1	+0.1	0.1	+0.1	0.1
2	+0.1	0.08 + 0.1 = 0.18	−0.1	0.08 − 0.1 = −0.02
3	+0.1	0.144 + 0.1 = 0.244	+0.1	−0.016 + 0.1 = 0.084
4	+0.1	0.1952 + 0.1 = 0.2952	−0.1	0.0672 − 0.1 = −0.0328

Steady pushes **pile up**: the step grows towards 0.5, five times the push. Alternating pushes **partly cancel**: the steps keep flipping sign and, if you carry on, settle at about ±0.056, about half the push. So the steady direction speeds up while the zig-zag direction calms down.

③ THE ONE IDEA

The step is a **memory of all past pushes**, each one faded by another factor of β: $\Delta W^{(k)} = p_k + \beta p_{k-1} + \beta^2 p_{k-2} + \dots$ Pushes that agree add up; pushes that disagree cancel. Nesterov keeps the same memory but measures the new push where the memory is about to carry you, so it notices an approaching bottom one step earlier.

④ BUILD IT, STEP BY STEP

(sibling: $Loss = w^2$, gradient $2w$, but $w^{(0)} = 2$, $\eta = 0.1$, $\beta = 0.8$, $\Delta W^{(0)} = 0$).

1. Plain GD for comparison: $w \leftarrow w - 0.1 \cdot 2w = 0.8w$: $1.6,\ 1.28,\ 1.024$.
   Why: the baseline every method is judged against.
2. Momentum step (p136–142): $\Delta W^{(k)} = \beta\, \Delta W^{(k-1)} - \eta\, \nabla_W Loss\big(W^{(k-1)}\big)^T$.
   Why: kept part of the old step + the push at where you are.
3. Move: $W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$ (a **plus**: the minus is already inside ΔW).
   Why: the step already points downhill.
4. Run momentum: $k=1$: $\Delta W = 0.8 \cdot 0 - 0.1 \cdot 2(2) = -0.4$, $w = 1.6$. $k=2$: $\Delta W = 0.8(-0.4) - 0.1 \cdot 2(1.6) = -0.32 - 0.32 = -0.64$, $w = 0.96$. $k=3$: $\Delta W = 0.8(-0.64) - 0.1 \cdot 2(0.96) = -0.512 - 0.192 = -0.704$, $w = 0.256$. $k=4$: $\Delta W = -0.5632 - 0.0512 = -0.6144$, $w = -0.3584$.
   Why: the steps grow (0.4, 0.64, 0.704) because the pushes agree, so it gets near 0 much faster than GD, then **rolls past** the bottom.
5. Nesterov (p143–150): look-ahead point $\tilde{W} = W^{(k-1)} + \beta\, \Delta W^{(k-1)}$, then $\Delta W^{(k)} = \beta\, \Delta W^{(k-1)} - \eta\, \nabla_W Loss\big(\tilde{W}\big)^T$ and $W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$.
   Why: $\tilde{W}$ is where coasting alone would take you; the slope there brakes before overshooting (p149).
6. Run Nesterov: $k=1$: $\tilde{w} = 2$, $\Delta W = -0.4$, $w = 1.6$. $k=2$: $\tilde{w} = 1.6 + 0.8(-0.4) = 1.28$, $\Delta W = -0.32 - 0.1 \cdot 2(1.28) = -0.32 - 0.256 = -0.576$, $w = 1.024$. $k=3$: $\tilde{w} = 1.024 + 0.8(-0.576) = 0.5632$, $\Delta W = -0.4608 - 0.1126 = -0.5734$, $w = 0.4506$. $k=4$: $\tilde{w} = -0.0082$ (already past 0, gradient there is negative), $\Delta W = -0.4587 + 0.0016 = -0.4571$, $w = -0.0066$.
   Why: it overshoots by only 0.0066 against momentum's 0.358.
7. Constant gradient g: the steps are $-\eta g\,(1 + \beta + \beta^2 + \dots) \to -\dfrac{\eta g}{1-\beta}$. With $g = 1$, $\eta = 0.1$, $\beta = 0.8$: $-0.1,\ -0.18,\ -0.244,\ \dots \to -0.5$.
   Why: the geometric series $1 + \beta + \beta^2 + \dots = 1/(1-\beta)$; 5× the push here, but in the **same direction**: only the step size changes.

⑤ **CHECK IT**

Warm-up, steady column: line 7 says the step after k pushes is $0.1(1 + 0.8 + \cdots + 0.8^{k-1}) = 0.5(1 - 0.8^k)$; for $k = 4$: $0.5(1 - 0.4096) = 0.2952$ ✓, heading to $0.1/(1 - 0.8) = 0.5$. Edge case $\beta = 0$: $\Delta W = -\eta\nabla$, which is plain gradient descent ✓. Another edge: with $\Delta W^{(0)} = 0$ the look-ahead point is the current point, so the **first** Nesterov step equals the first momentum step (1.6 in both runs ✓).

⑥ **NOW THE EXAM PROBLEM — YOUR TURN**

EXAM PROBLEM

$Loss = w^2$ (gradient $2w$, minimum 0), $w^{(0)} = 1$, $\eta = 0.1$, $\beta = 0.9$, $\Delta W^{(0)} = 0$. (a) 3 steps of plain GD. (b) 3 steps of momentum (also the 4th w). (c) 3 steps of Nesterov (also the 4th w). (d) On a constant slope (gradient always 1) what do momentum's steps become?

Do it the same way: steps ④–⑤ with these numbers.

✍ Hint: S6.11 (end of this lecture)

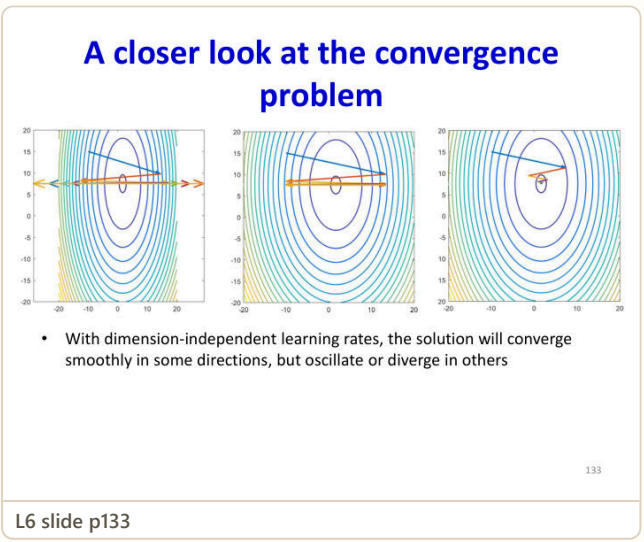
✍ Full worked solution: S6.12 (end of this lecture)

⑦ **TRY ALONE**

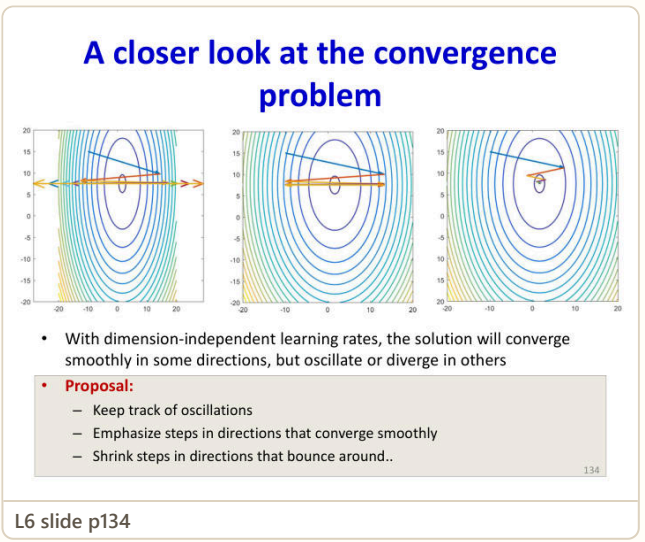
(i) Momentum with $\beta = 0.5$, $\eta = 0.1$ and a constant gradient of 2: the first three steps and the limit. (ii) $Loss = 2w^2$ (gradient $4w$), $w^{(0)} = 1$, $\eta = 0.1$, $\beta = 0.5$, $\Delta W^{(0)} = 0$: two steps of momentum and two of Nesterov. (iii) Why are the first momentum and first Nesterov steps always equal when $\Delta W^{(0)} = 0$?

✍ Answers: S6.13 (end of this lecture)

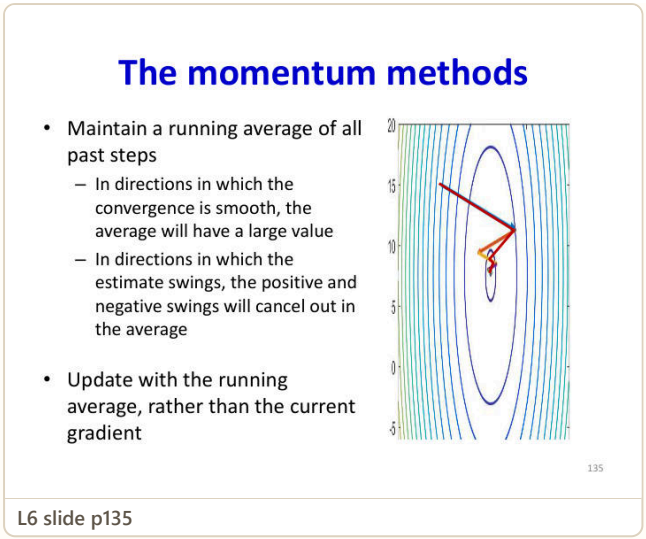
🧭 **Like a bicycle rolling down a bumpy valley.** Its speed carries it through small bumps and side-to-side wobbles cancel, but it can roll past the bottom. Nesterov is the rider who looks a few metres ahead and starts braking before the dip ends. *Where it breaks:* a real bike loses energy to friction; here $\beta < 1$ plays that role.



L6 slide p133



L6 slide p134



L6 slide p135

↗ Understand more: Distill, "Why Momentum Really Works" ↗ d2l: momentum

🔁 RECALL DRILL · ⏱ 5 min · Tier A

8 · Drill: the three in-scope class polls, then six quick ones

Poll 2 (p107). Mark all true: (A) Step sizes greater than twice the inverse of the second derivative can cause gradient descent to diverge. (B) This is always a bad thing. (C) Gradient descent will not converge without decaying learning rates.

✍️ Answer: S6.14 (end of this lecture)

Poll 3 (p124). The derivative w.r.t. w is positive; after a step it becomes negative. Mark all true: (A) RProp will revert to the earlier estimate and take a smaller step. (B) RProp will change direction and begin taking steps in the opposite direction.

✍️ Answer: S6.15 (end of this lecture)

Poll 4 (p152). On a flat surface of constant slope, momentum methods converge faster than vanilla gradient descent. True or false?

✍️ Answer: S6.16 (end of this lecture)

Quick ones

1. How do you recognise a saddle point from the Hessian? Are saddles or local minima more common in large networks?
2. Loss gaps go 10, 4, 1.6, 0.64. Convergence rate R? What kind of convergence?
3. Write the modified (scaled) update rule for $E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{b}^T\mathbf{w} + c$. What do you get with $\mathbf{H}$ and $\eta = 1$?
4. Three reasons Newton's method isn't used for big networks.
5. Step decay with $\eta_0 = 0.1$, $\alpha = 0.1$: the first three learning rates?
6. Which method uses only the *sign* of the derivative? Which estimates E'' from two consecutive slopes? Which takes the gradient at a look-ahead point?

✍️ Answers: S6.17 (end of this lecture)

⚠️ Traps

- **Two different βs:** RProp's $\beta = 0.5$ shrinks the step after a sign flip; momentum's $\beta = 0.9$ is how much of the old step you keep.
- **RProp after a sign flip does not reverse direction**: it undoes the step, halves it, and steps the same way again (Poll 3).
- **Momentum signs:** $\Delta W = \beta \Delta W - \eta \nabla$, then $W = W + \Delta W$ (plus). Don't subtract twice.
- **"Linear convergence"** means the error shrinks by a constant factor per step (exponentially fast), not by a constant amount.
- **Slide error p84:** $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \mathbf{A}^{-1}\mathbf{b}$ should be $\mathbf{w}^{(k)} - \eta \mathbf{A}^{-1} \nabla E^T = \mathbf{w}^{(k)} - \eta \mathbf{A}^{-1}(\mathbf{A}\mathbf{w}^{(k)} + \mathbf{b})$; with $\eta = 1$ that lands on $\mathbf{w}^* = -\mathbf{A}^{-1}\mathbf{b}$.
- **Slide error p83:** the ellipse "diameters are proportional to the eigenvalues". The contour $\frac{1}{2}\lambda x^2 = c$ gives half-width $\sqrt{2c/\lambda}$: axis lengths are proportional to $1/\sqrt{\lambda}$ (steep direction = narrow).
- **Naming:** p105's "linear decay" $\eta_0/(k+1)$ is really $1/k$ decay. "Nestorov" = Nesterov. p44 calls $x^{(k+1)}$ "the k-th iteration" (it is the $(k+1)$-th).

Solutions and answers — Lecture 6

S6.1 · 2 · Pen: the quadratic bowl set-up and measuring the convergence rate — Hint

The losses come out negative. R uses the gap $E(w) - E^*$, so find E^* first.

↩ back to the card

S6.2 · 2 · Pen: the quadratic bowl set-up and measuring the convergence rate — Full worked solution

Step 1, read off the bowl. $2w^2 = \frac{1}{2} \cdot 4 \cdot w^2$, so $a = 4, b = -8, c = 0$.
Why: a is twice the w^2 coefficient.

Step 2, slope. $E'(w) = aw + b = 4w - 8$.
Why: differentiate term by term.

Step 3, part (a). $E'(w^*) = 0 \Rightarrow 4w^* - 8 = 0 \Rightarrow w^* = -b/a = 2$, and $E^* = E(2) = 2 \cdot 4 - 8 \cdot 2 = 8 - 16 = -8$.
Why: the minimum of a bowl is where the slope is zero.

Step 4, part (b). $w^{(1)} = 0 - 0.1\,(4 \cdot 0 - 8) = 0 + 0.8 = 0.8$.
Why: the slope -8 is negative, so the step moves right.
$w^{(2)} = 0.8 - 0.1\,(4 \cdot 0.8 - 8) = 0.8 - 0.1(-4.8) = 0.8 + 0.48 = 1.28$.
Why: same rule at the new point; the slope is smaller, so the step is smaller.

Step 5, gaps. $E(0) = 0$; $E(0.8) = 2(0.64) - 6.4 = 1.28 - 6.4 = -5.12$; $E(1.28) = 2(1.6384) - 10.24 = 3.2768 - 10.24 = -6.9632$. Gaps to $E^* = -8$: $8,\ 2.88,\ 1.0368$.
Why: R is defined on the gap above the minimum.

Step 6, part (c), measured rate.

$$R_1 = \frac{2.88}{8} = 0.36, \qquad R_2 = \frac{1.0368}{2.88} = 0.36$$

Step 7, predicted rate. Factor $1 - \eta a = 1 - 0.1 \cdot 4 = 0.6$: distances to $w^* = 2$ go $2 \to 1.2 \to 0.72$ ($1.2/2 = 0.72/1.2 = 0.6$ ✓). So $R = (1 - \eta a)^2 = 0.6^2 = 0.36$ at **every** step: **linear convergence** (exponentially fast).
Why: the gap is $\frac{1}{2}a(w - w^*)^2$, distance squared. Since $\eta = 0.1 < 1/a = 0.25$, the factor is positive and w never overshoots 2.

Step 8, bound. $E(w^{(k)}) - E^* \le R^k\big(E(w^{(0)}) - E^*\big) = 8 \cdot 0.36^k$ (equality here, because E is an exact bowl). Check $k = 2$: $8 \cdot 0.1296 = 1.0368$ ✓. About 16 steps bring the gap below 10^{-6}.

(a) $w^* = 2$, $E^* = -8$. (b) $w^{(1)} = 0.8$, $w^{(2)} = 1.28$. (c) Gaps $8 \to 2.88 \to 1.0368$, $R_1 = R_2 = 0.36 = (1 - \eta a)^2$ (linear convergence); gap after k steps $\le 8 \cdot 0.36^k$.

↩ back to the card

S6.3 · 2 · Pen: the quadratic bowl set-up and measuring the convergence rate — Answers

(i) $a = 2$: factor 0.8, $R = 0.64$ (gaps $1, 0.64, 0.4096\ldots$). (ii) $a = 6$: $R = (1 - 0.6)^2 = 0.16$; one jump at $\eta = 1/6$; diverges for $\eta > 1/3$. (iii) $a = 2$, $w^* = 2$, $E^* = -4$: w: $0 \to 6 \to -6$; gaps $4 \to 16 \to 64$, so $R = 4 > 1$: it **diverges**, because $\eta = 1.5 > 2/a = 1$ (factor $1 - 3 = -2$: each step flips side and doubles the distance).

↩ back to the card

S6.4 · 4 · Pen: Newton in one step, Newton's speed, and Newton going wrong — Hint

(a) The determinant is 5. (b) The step simplifies to $w - 1 + 2e^{-w}$. (c) Check the sign of E'' before stepping.

↩ back to the card

S6.5 · 4 · Pen: Newton in one step, Newton's speed, and Newton going wrong — Full worked solution

(a) Step 1, gradient. $\nabla E^T = \mathbf{A}\mathbf{w} + \mathbf{b} = \mathbf{A}\mathbf{0} + \mathbf{b} = (-5, -5)$.

Why: at the origin only $\mathbf{b}$ survives.

Step 2, Hessian. $\mathbf{H} = \mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$.

Why: the second derivative of $\frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w}$ is $\mathbf{A}$.

Step 3, inverse. Determinant $3 \cdot 2 - 1 \cdot 1 = 5$; swap the diagonal, negate the off-diagonal: $\mathbf{A}^{-1} = \frac{1}{5}\begin{pmatrix} 2 & -1 \\ -1 & 3 \end{pmatrix}$.

Why: the 2×2 inverse formula.

Step 4, multiply.

$$\mathbf{A}^{-1}\nabla E^T = \tfrac{1}{5}\big(2(-5) - 1(-5),\ -1(-5) + 3(-5)\big) = \tfrac{1}{5}(-5, -10) = (-1, -2)$$

Step 5, Newton step ($\eta = 1$). $\mathbf{w}^{(1)} = (0, 0) - (-1, -2) = (1, 2)$.

Step 6, check. $\mathbf{A}(1,2)^T + \mathbf{b} = (3 + 2 - 5,\ 1 + 4 - 5) = (0, 0)$: gradient zero, so $(1, 2)$ is the **minimum, reached in one step** (exact because E is quadratic and $\mathbf{A}$ is positive definite). For comparison, plain GD with $\eta = 0.4$ goes $(2, 2) \to (0.8, 1.6) \to (1.2, 2.0)$, still wobbling.

(b) Step 1, derivatives. $E' = e^w - 2$, $E'' = e^w$, so $w_{\text{new}} = w - \dfrac{e^w - 2}{e^w} = w - 1 + 2e^{-w}$.

Why: rule 1, simplified once.

Step 2, iterate.

$w^{(1)} = 0 - \dfrac{1 - 2}{1} = 1.$

$w^{(2)} = 1 - \dfrac{e - 2}{e} = 1 - \dfrac{0.7183}{2.7183} = 1 - 0.2642 = 0.7358.$

$w^{(3)} = 0.7358 - \dfrac{e^{0.7358} - 2}{e^{0.7358}} = 0.7358 - \dfrac{0.0871}{2.0871} = 0.7358 - 0.0417 = 0.6940.$

Step 3, errors. Distance to $\ln 2 = 0.6931$: $0.307 \to 0.043 \to 0.0009$ (step 4: 4×10^{-7}). The number of correct digits roughly **doubles** each step (quadratic convergence); it needed several steps only because E is not a parabola.

(c) Step 1, derivatives at 0.3. $E' = w^3 - w = 0.027 - 0.3 = -0.273$; $E'' = 3w^2 - 1 = 0.27 - 1 = -0.73$.

Why: negative E'' = curving *down*: warning sign.

Step 2, Newton step. $0.3 - \dfrac{-0.273}{-0.73} = 0.3 - 0.374 = -0.074$.

Step 3, what happened. $E(0.3) = 0.0020 - 0.0450 = -0.0430$ and $E(-0.074) = -0.0027$: the loss went **up**. Newton moved towards the **maximum** at 0 because it only looks for zero slope. Plain GD would have moved to $0.3 + 0.273\eta$, towards the minimum at 1. (This is why Newton needs a positive-definite $\mathbf{H}$, or a fix like Levenberg–Marquardt.)

(a) $\mathbf{w}^{(1)} = (1, 2)$, the exact minimum in one step. (b) $1,\ 0.7358,\ 0.6940 \to 0.6931$ (digits double). (c) $w = -0.074$: Newton heads for the maximum because $E'' < 0$; the loss rises.

↩ back to the card

S6.6 · 4 · Pen: Newton in one step, Newton's speed, and Newton going wrong — Answers

(i) $E' = 10w - 10 = 990$, $E'' = 10$: $100 - 99 = 1$, the exact minimum (a parabola, so any start works). (ii) $\mathbf{g} = (-2, -2)$, $\det = 8 - 4 = 4$, $\mathbf{A}^{-1} = \frac{1}{4}\begin{pmatrix} 2 & -2 \\ -2 & 4 \end{pmatrix}$, $\mathbf{A}^{-1}\mathbf{g} = \frac{1}{4}(0, -4) = (0, -1)$, new point $(0, 1)$; check $\mathbf{A}(0,1)^T + \mathbf{b} = (2-2,\ 2-2) = (0,0)$ ✓. (iii) $E'' = -\cos 0.5 = -0.8776 < 0$: a hill, so it heads for a **maximum**. $E' = -\sin 0.5 = -0.4794$; step $0.5 - (-0.4794)/(-0.8776) = 0.5 - 0.5463 = -0.046$, next to the maximum of cos at 0.

↩ back to the card

S6.7 · 6 · Pen: RProp and QuickProp — each weight gets its own step — Hint

A: compare each new D with the last *kept* D, not the last tried one. B (ii): $E'(1) = e - 2$.

↩ back to the card

S6.8 · 6 · Pen: RProp and QuickProp — each weight gets its own step — Full worked solution (A, RProp)

Start. $D_{prev} = 2(0 - 3) = -6$ (negative: downhill is to the right). $\Delta w = \text{sign}(-6) \times 0.5 = -0.5$, so $w - \Delta w$ moves right.

Why: line 1.

iter	try $w = w - \Delta w$	$D = 2(w - 3)$	sign vs D_{prev}	action	new Δw
1	$0 + 0.5 = 0.5$	−5	same (vs −6)	keep, $D_{prev} = -5$	$1.2(-0.5) = -0.6$
2	$0.5 + 0.6 = 1.1$	−3.8	same	keep, $D_{prev} = -3.8$	−0.72
3	$1.1 + 0.72 = 1.82$	−2.36	same	keep, $D_{prev} = -2.36$	−0.864
4	$1.82 + 0.864 = 2.684$	−0.632	same	keep, $D_{prev} = -0.632$	−1.0368
5	$2.684 + 1.0368 = 3.7208$	+1.4416	**flipped**	undo → $3.7208 - 1.0368 = 2.684$	$0.5(-1.0368) = -0.5184$
6	$2.684 + 0.5184 = 3.2024$	+0.4048	**flipped** (vs −0.632)	undo → 2.684	−0.2592
7	$2.684 + 0.2592 = 2.9432$	−0.1136	same (vs −0.632)	keep, $D_{prev} = -0.1136$	$1.2(-0.2592) = -0.3110$

Why each row. Rows 1–4: same sign as the last kept slope, so the step grows ×1.2 ($0.5 \to 0.6 \to 0.72 \to 0.864 \to 1.0368$). Row 5: the try jumped past 3, so it is undone and the step halves. Row 6: still compared with -0.632 (a flip never updates D_{prev}), flipped again, undo and halve. Row 7 (the "one step more"): it lands short of 3, same side, so it is kept and the step grows again. The step size never looked at how big D was.

RProp tries: 0.5, 1.1, 1.82, 2.684 (kept), 3.7208 (undone → 2.684, step 0.5184), 3.2024 (undone → 2.684, step 0.2592), 2.9432 (kept, step 0.3110) … closing in on 3.

↩ back to the card

S6.9 · 6 · Pen: RProp and QuickProp — each weight gets its own step — Full worked solution (B, QuickProp)

(i) Step 1, slopes. $E' = 2(w-3)$: $E'(0) = -6$, $E'(1) = -4$.

Step 2, curvature estimate. $E'' \approx \dfrac{-4-(-6)}{1-0} = 2$.

Why: change in slope ÷ distance moved.

Step 3, Newton step. $w = 1 - \dfrac{-4}{2} = 1+2 = 3$. *Check:* $E'(3) = 0$: **exact**, because the parabola assumption is true.

(ii) Step 1, slopes. $E' = e^w - 2$: $E'(0) = 1-2 = -1$, $E'(1) = e-2 = 0.7183$.

Why: the sign changed, so the minimum lies between 0 and 1.

Step 2, curvature estimate. $E'' \approx \dfrac{0.7183-(-1)}{1-0} = 1.7183$.

Step 3, Newton step. $w = 1 - \dfrac{0.7183}{1.7183} = 1 - 0.418 = 0.582$. *Check:* $E'(0.582) = e^{0.582} - 2 = -0.21 \neq 0$, so close to 0.6931 but not exact (the loss is not a parabola); the next QuickProp step would refine it.

QuickProp: (i) $w = 3$ (exact minimum); (ii) $w = 0.582$ (true minimum 0.6931).

↩ back to the card

S6.10 · 6 · Pen: RProp and QuickProp — each weight gets its own step — Answers

(i) Same sign: step ×1.2, keep going the same way (here: decreasing w). (ii) $D_{prev} = +4$, $\Delta w = +1$. Tries: -1 ($D = 2$, same, keep, step 1.2); -2.2 ($D = -0.4$, flipped: undo to -1, step 0.6); -1.6 ($D = 0.8$, same as the kept $+2$: keep, step 0.72); -2.32 ($D = -0.64$, flipped: undo to -1.6, step 0.36). (iii) $E' = 2w - 2$: $E'(3) = 4$, $E'(2) = 2$; $\Delta w = 2-3 = -1$; $E'' \approx (2-4)/(-1) = 2$; $w = 2 - 2/2 = 1$, the exact minimum (watch the signs: the negative step gives a positive curvature).

↩ back to the card

S6.11 · 7 · Pen: momentum and Nesterov — remember where you were going — Hint

Write a table with columns k, look-ahead $\tilde{w}$, gradient, ΔW, w. Keep 4 decimals.

↩ back to the card

S6.12 · 7 · Pen: momentum and Nesterov — remember where you were going — Full worked solution

(a) Plain GD. $w \leftarrow w - 0.1 \cdot 2w = 0.8w$: $1 \to$ **0.8, 0.64, 0.512**.

Why: each step keeps 80% of the distance to 0.

(b) Momentum, $\Delta W = 0.9\,\Delta W_{\text{old}} - 0.1 \cdot 2w$, then $w \leftarrow w + \Delta W$.

Why: lines 2–3; the gradient is taken where you stand.

$k=1$: $\Delta W = 0.9 \cdot 0 - 0.1 \cdot 2(1) = -0.2$, $w = 0.8$.

$k=2$: $\Delta W = 0.9(-0.2) - 0.1 \cdot 2(0.8) = -0.18 - 0.16 = -0.34$, $w = 0.46$.

$k=3$: $\Delta W = 0.9(-0.34) - 0.1 \cdot 2(0.46) = -0.306 - 0.092 = -0.398$, $w = 0.062$.

$k=4$: $\Delta W = 0.9(-0.398) - 0.1 \cdot 2(0.062) = -0.3582 - 0.0124 = -0.3706$, $w = 0.062 - 0.3706 = -0.3086$: it **rolled past** the bottom, because the stored velocity outweighs the small push near 0.

(c) Nesterov (look-ahead point $\tilde{w} = w + \beta\Delta W$, gradient taken there).

Why: line 5.
$k=1$: $\tilde{w}=1$, $\Delta W=-0.2$, $w=0.8$.
$k=2$: $\tilde{w}=0.8+0.9(-0.2)=0.62$, $\Delta W=-0.18-0.1\cdot 2(0.62)=-0.18-0.124=-0.304$, $w=0.496$.
$k=3$: $\tilde{w}=0.496+0.9(-0.304)=0.2224$, $\Delta W=-0.2736-0.1\cdot 2(0.2224)=-0.2736-0.0445=-0.3181$, $w=0.496-0.3181=0.1779$.
$k=4$: $\tilde{w}=0.1779+0.9(-0.3181)=-0.1084$; the gradient there is negative, so the push is **positive** and it brakes: $\Delta W=-0.2863+0.0217=-0.2646$, $w=-0.0867$. Overshoot 3.5× smaller than momentum's -0.3086.

Comparison. After 3 steps: GD 0.512, momentum 0.062, Nesterov 0.178. Momentum gets closest first but overshoots most; Nesterov is between them and settles faster.

(d) Constant slope. $\Delta W^{(k)}=0.9\,\Delta W^{(k-1)}-0.1\cdot 1$: $-0.1,\ -0.19,\ -0.271,\ -0.3439,\ldots$, i.e. $-0.1(1-0.9^k)\to -\eta/(1-\beta)=-0.1/0.1=-1$.
Why: the geometric series of line 7. Momentum just makes the step up to 10× longer, the same as using a bigger η. Nothing about the *direction* changes, which is why Poll 4 says it does not "converge faster" on a constant slope.

GD: 0.8, 0.64, 0.512. Momentum: 0.8, 0.46, 0.062 (then −0.309). Nesterov: 0.8, 0.496, 0.178 (then −0.087). Constant slope: momentum step $\to \eta/(1-\beta)=1$.

↩ back to the card

S6.13 · 7 · Pen: momentum and Nesterov — remember where you were going — Answers

(i) $-0.2,\ -0.3,\ -0.35$, limit $-0.2/(1-0.5)=-0.4$. (ii) Momentum: $\Delta W=-0.4$, $w=0.6$; $\Delta W=0.5(-0.4)-0.1\cdot 4(0.6)=-0.2-0.24=-0.44$, $w=0.16$. Nesterov: $w=0.6$; $\tilde{w}=0.6+0.5(-0.4)=0.4$, $\Delta W=-0.2-0.1\cdot 4(0.4)=-0.36$, $w=0.24$. (iii) The look-ahead point is $w+\beta\cdot 0=w$, so both measure the gradient at the same place.

↩ back to the card

S6.14 · 8 · Drill: the three in-scope class polls, then six quick ones — Answer

Only (A). (A) $\eta>2/E''=2\eta_{opt}$ diverges on a bowl (card 2's background line). (B) false: on bumpy losses a big step can escape poor local minima (p103, card 5). (C) false: on a convex bowl a fixed small η converges (card 2's warm-up: $R=0.25$ every step).

↩ back to the card

S6.15 · 8 · Drill: the three in-scope class polls, then six quick ones — Answer

Only (A). The sign flip means "overshot": go back, halve the step, step again the same way (card 6 warm-up, rows 3–5).

↩ back to the card

S6.16 · 8 · Drill: the three in-scope class polls, then six quick ones — Answer

False (slide key: "momentum only changes step size"). The averaged step points the same way as the gradient and just grows to $\eta/(1-\beta)$, like a larger η. Momentum's real benefit is cancelling zig-zags, and a constant slope has none.

↩ back to the card

S6.17 · 8 · Drill: the three in-scope class polls, then six quick ones — Answers

1. Gradient zero, eigenvalues of mixed sign (some $+$, some $-$). Saddles (hypothesis, p37).
2. $R = 0.4$ every step (constant) $\rightarrow$ "linear" convergence, i.e. exponentially fast.
3. $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{A}^{-1} \nabla E^T$; with the Hessian and $\eta = 1$ it is Newton's method.
4. Hessian has N^2 entries (10^{10} for 10^5 weights); inverting it is even costlier; in non-convex regions it isn't positive definite, so Newton can head to a maximum/saddle.
5. 0.1, 0.01, 0.001 (each time the held-out loss stagnates).
6. RProp; QuickProp; Nesterov.

↩ back to the card

LECTURE 8

Choosing the divergence, and batch normalisation

Tier A · must ⏱ 61 min Slides: lec8.optimizersandregularizers.pdf, p1–46 only (p47 → end is not on the exam)

🎥 CMU video (Lecture 8, first half) 📘 d2l: batch norm

Lectures 3–6 taught you how to make the training loss small by gradient descent. The exam part of Lecture 8 is about two things that make that descent slow or unstable: **the wrong divergence** (L2 on a classifier goes flat exactly when the network is badly wrong), and **each mini-batch living in a different place** (covariate shift), fixed by **batch normalisation**. It opens with a 5-slide recap of the faster training methods (Lecture 7 itself is not on the exam).

What the exam can ask from this lecture

- The recap (p2–6) at the level of one sentence each: batch vs SGD vs mini-batch; shrinking learning rates; what momentum, RMSprop and Adam change.
- L2 vs KL: which for regression, which for classification, which is convex in the pre-sigmoid value z; why $\nabla_z = y - d$ for both (Poll 1).
- Covariate shift and what BN normalises over (Poll 2).
- Batch norm: where it sits, the training forward pass by hand (mean, variance, normalise, γ, β), and $\partial L/\partial\gamma$, $\partial L/\partial\beta$ (p44–46).

⏳ **Only 20 minutes?** Do card 5 steps 2–5 on the two-number batch (about 7 min), then its exam problem on your own, checked against its worked solution (about 8 min), then the three polls in card 7 (5 min). If 7 more minutes appear, card 2 (L2 vs KL).

📖 READ & UNDERSTAND · ⏱ 6 min · Tier B

1 · The recap slides (p2–6): faster steps and smoother steps

The set-up (p2–4). The loss is the average divergence over the training inputs, $L(W) = \frac{1}{N}\sum_X \operatorname{div}\big(f(X;W), d(X)\big)$, and gradient descent repeats $W_k = W_{k-1} - \eta\,\nabla_W L^\top$, with the gradient computed by backprop. Because L is an average, its gradient is **the average of the per-example gradients**.

Incremental methods (p5). *Batch* descent averages over all N examples before each step: exact but slow. **SGD** (stochastic gradient descent) steps after each single random example; **mini-batch** descent steps after a small random subset (say 64). They update far more often, but each step is noisier ("greater variance"), so they need a **shrinking learning rate** η_k that is "not absolute summable, but square summable":

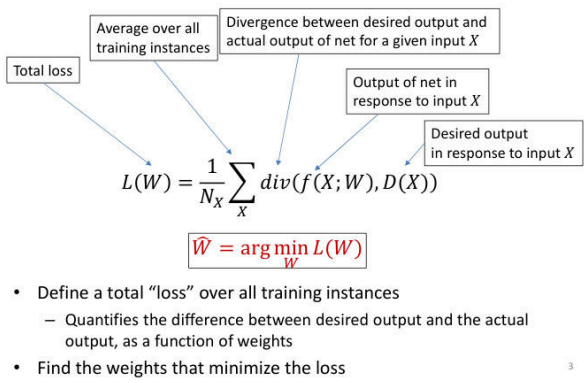
$$\sum_k \eta_k = \infty \qquad \sum_k \eta_k^2 < \infty$$

The first lets you travel any distance; the second makes the jitter die out. $\eta_k = 1/k$ does both: $1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots$ keeps growing without limit (already 2.08 after four terms), while $1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \ldots$ settles at $\pi^2/6 \approx 1.645$.

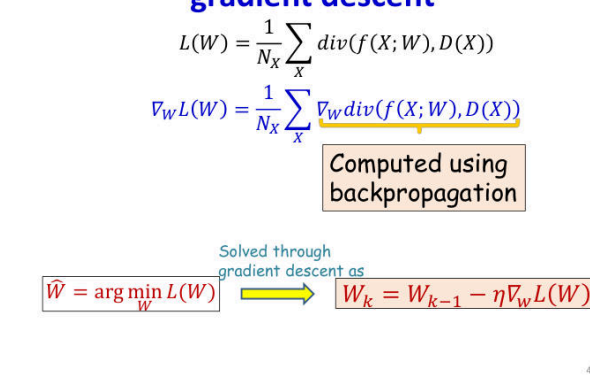
Trend algorithms (p6) use the history of past gradients. **Momentum** and **Nesterov** smooth the *direction* (an averaged gradient). **RMSprop** changes only the *step size* of each weight, dividing by the

root-mean-square of its recent gradients. **Adam** does both. All are usually much faster than plain gradient descent.

Like estimating Cairo's average commute. Batch = survey everyone before saying anything; SGD = ask one random person and update at once (fast, but one person stuck on the Ring Road throws you off); mini-batch = ask 64 at a time. Momentum is then a marble rolling in a bowl: it keeps its direction and isn't knocked off course by every bump. *Where it breaks:* a survey's true answer stays fixed; in training the gradient changes every time the weights move.



L8 slide p3



L8 slide p4

↗ Understand more: d2l, minibatch SGD ↗ d2l, Adam

📖 READ & UNDERSTAND · ⏱ 7 min · Tier A

2 · L2 or KL: the shape of the divergence decides how well descent works

A **divergence** is the per-example score of how wrong the output y is compared with the desired output d; the loss averages it. Gradient descent only sees its slope, so its **shape** matters (L8 p8–10): too flat far from the answer and you crawl; too steep near the answer and you overshoot. The ideal is steep far away and gently bowl-shaped (quadratic) near the minimum.

The two standard choices (L8 p11):

$$\text{L2: Div} = \tfrac{1}{2}\sum_i (y_i - d_i)^2 \qquad \text{KL: Div} = \sum_i d_i \log d_i - \sum_i d_i \log y_i$$

L2 for regression (predicting a number), **KL for classification** (the output is a probability vector, e.g. from a softmax). For one-hot targets d the first KL term is 0, so KL is just cross-entropy $-\log y_{\text{correct}}$. *E.g.* $y = (0.7, 0.2, 0.1)$, $d = (1, 0, 0)$: L2 $= \frac{1}{2}(0.09 + 0.04 + 0.01) = 0.07$; KL $= -\ln 0.7 = 0.357$.

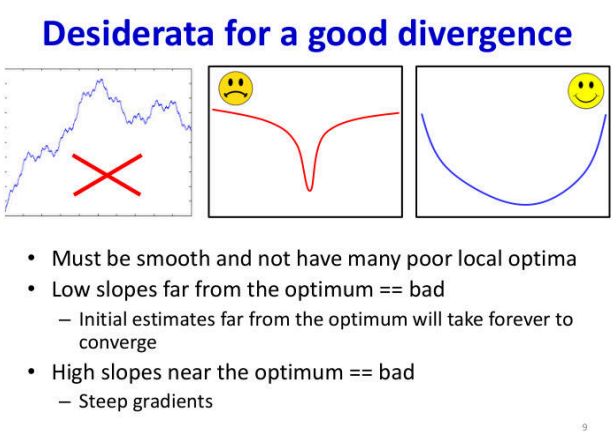
The convexity story (L8 p13–16). A function is **convex** if it is bowl-shaped: any straight line between two points on its graph stays above the graph, so there is one valley and no false dips. As a function of the sigmoid output y, both L2 and KL look convex. But as a function of z, the value *before* the sigmoid ($y = \sigma(z)$), **only KL is convex**. L2 flattens out for large $|z|$: a confidently wrong neuron gets almost no gradient. On a single perceptron's weights, L2 is likewise not convex while KL is (though L2 still has a unique global minimum, L8 p16).

One beautiful fact (L8 p17): with a linear output + L2, *and* with a softmax output + KL, the gradient with respect to the last affine value z is the same:

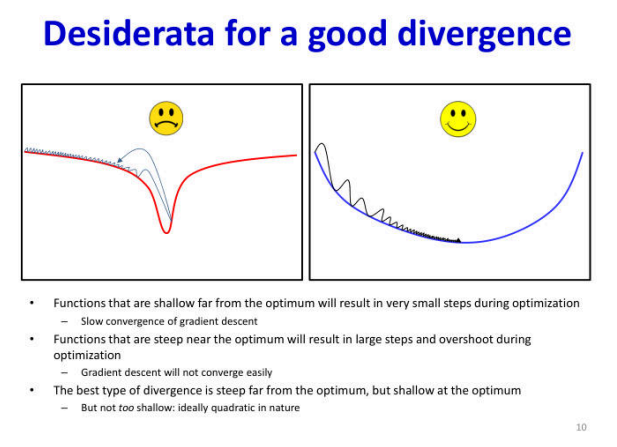
$$\nabla_z \tfrac{1}{2}\|y-d\|^2 = (y-d)^\mathsf{T}, \qquad \nabla_z \mathrm{KL}(y,d) = (y-d)^\mathsf{T}$$

So backprop literally starts by sending the **error** $y - d$ backwards, hence the name "error backpropagation".

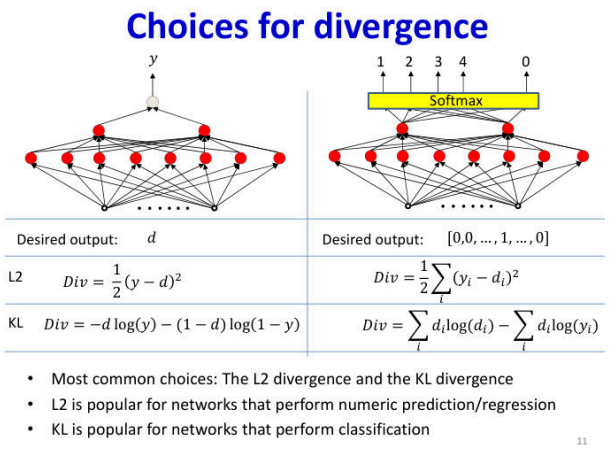
Like the "hot and cold" game. A good divergence shouts "FREEZING!" when you are far away (big slope, you move fast) and whispers "warm… warmer…" near the prize (small slope, you don't overshoot). L2 on a sigmoid is a friend who goes quiet exactly when you are very far off. *Where it breaks:* the game has one hidden prize; a non-convex loss can have several dips that each feel like "warm".



L8 slide p9



L8 slide p10



L8 slide p11

↗ Understand more: guide, the KL divergence ↗ guide, why the gradient is y − d

PEN & PAPER · ⏱ 13 min · Tier A

3 · By hand: L2 goes quiet when a sigmoid neuron is badly wrong, KL does not

Step 1 · What you're being asked, in plain words

One output neuron computes a number z (its weighted sum), then squashes it with the *sigmoid* $y = \sigma(z) = 1/(1+e^{-z})$ into a value between 0 and 1, read as "probability of yes". The right answer d is 1 ("yes") or 0 ("no"). The *divergence* Div scores how wrong y is. Training changes z by $-\eta \times \frac{d\,\mathrm{Div}}{dz}$, so

the size of $d\,\mathrm{Div}/dz$ is how hard training pushes. You compare that push for L2 and for KL (binary KL is $-d\log y-(1-d)\log(1-y)$). Here log means the natural log (ln).

Step 2 · Warm-up by hand, no formulas

Practise with the target $d=0$ ("no"). Then $\mathrm{L2}=\frac{1}{2}(y-0)^2=\frac{1}{2}y^2$ and $\mathrm{KL}=-\ln(1-y)$ (the $d\log y$ term is 0). The neuron is badly wrong at $z=3$ (it says 95 % "yes") and only mildly wrong at $z=1$. Compute the scores, then nudge z up by 0.1 and see how much each score grows.

z	y = σ(z)	L2 = ½y²	KL = −ln(1 − y)
3	0.9526	0.4537	3.0486
3.1	0.9569	0.4578	3.1441
1	0.7311	0.2672	1.3133
1.1	0.7503	0.2814	1.3873

At $z=3$ (badly wrong) the 0.1 nudge raises L2 by only 0.0041 (slope ≈ 0.041) but KL by 0.0955 (slope ≈ 0.95). At $z=1$: L2 slope ≈ 0.14, KL ≈ 0.74. **L2 pushes harder at z = 1 than at z = 3, even though 3 is far worse.**

Step 3 · The one idea

Chain rule: $\frac{d\,\mathrm{Div}}{dz}=\frac{d\,\mathrm{Div}}{dy}\cdot\frac{dy}{dz}$, and $\frac{dy}{dz}=y(1-y)$ is tiny when y is near 0 or 1. L2's first factor is just the error $(y-d)$, never bigger than 1, so the tiny factor wins. KL's first factor has y or $1-y$ in its denominator, which grows exactly fast enough to cancel it.

Step 4 · Build it, step by step (target d = 0)

1. $\sigma'(z)=y(1-y)$.
   Why: differentiating $(1+e^{-z})^{-1}$ gives $e^{-z}/(1+e^{-z})^2=\frac{1}{1+e^{-z}}\cdot\frac{e^{-z}}{1+e^{-z}}=y(1-y)$. At $z=1$: $0.7311\times0.2689=0.1966$.
2. $\mathrm{L2}=\frac{1}{2}y^2$, so $\frac{d\,\mathrm{L2}}{dy}=y\ (=y-d)$.
   Why: the ½ cancels the 2 from the square.
3. L2 through the sigmoid: $\frac{d\,\mathrm{L2}}{dz}=y\cdot y(1-y)=y^2(1-y)$.
   Why: chain rule, multiply steps 2 and 1.
4. $\mathrm{KL}=-\ln(1-y)$, so $\frac{d\,\mathrm{KL}}{dy}=\frac{1}{1-y}$.
   Why: the derivative of $-\ln u$ is $-u'/u$, and $u=1-y$ has $u'=-1$.
5. KL through the sigmoid: $\frac{1}{1-y}\cdot y(1-y)=y=y-d$.
   Why: the $(1-y)$ cancels, leaving just the error.

Step 5 · Check it

At $z=3$, $y=0.9526$: L2 formula $0.9526^2\times0.0474=0.0430$, matching the warm-up's 0.041 ✓; KL formula $y=0.9526$, matching 0.95 ✓. At $z=1$: L2 $0.7311^2\times0.2689=0.1437$ and KL 0.7311, matching the nudges (0.14, 0.74) ✓. **Edge case:** when the neuron is right, $z=-6$, $y=0.0025$: L2 slope ≈

0.000006, KL slope $= 0.0025$, both ≈ 0, as they should be (nothing to fix). So L2 goes quiet at *both* ends; KL only at the right one.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

(The numbers behind L8 p13–17.) One sigmoid output neuron, $y = \sigma(z) = 1/(1+e^{-z})$, target $d = 1$. Compare

$$\text{L2: Div} = \tfrac{1}{2}(y-d)^2 \qquad \text{KL: Div} = -d\log y - (1-d)\log(1-y)$$

(a) Show that $\dfrac{d\,\text{L2}}{dz} = (y-d)\,y(1-y)$ and $\dfrac{d\,\text{KL}}{dz} = y - d$. Use $\sigma'(z) = y(1-y)$. (b) Evaluate both at $z = -4$ (confidently wrong), $z = 0$ and $z = 2$ (nearly right). Which one pushes harder when the neuron is badly wrong?

Do it the same way: steps 4–5 with these numbers (now $d = 1$, so the *other* KL term survives).

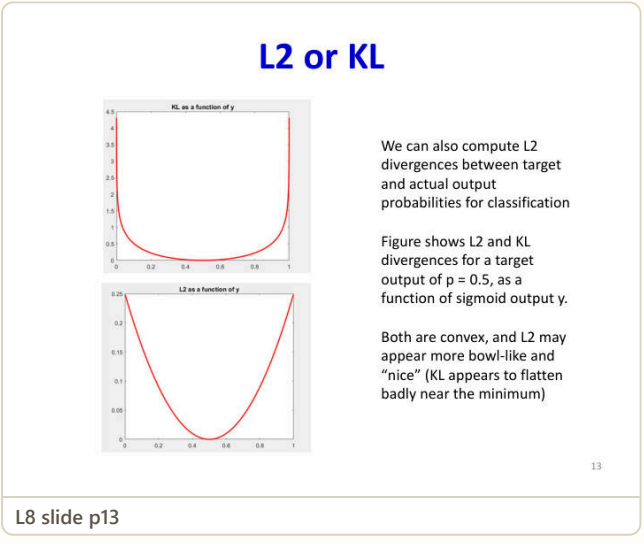
✍ Full worked solution: S8.1 (end of this lecture)

Step 7 · Try alone

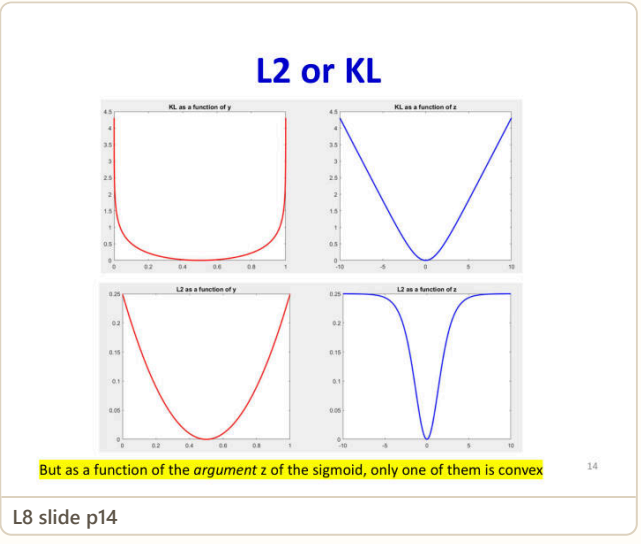
1. Target $d = 1$, $z = -2$ ($\sigma(-2) = 0.1192$). Both slopes?
2. Target $d = 0$ and $z = 5$ ($\sigma(5) = 0.9933$, very confidently wrong). Both slopes, and how many times harder does KL push?
3. Harder: with $d = 1$, at which y (and z) does L2 push hardest? (Maximise $y(1-y)^2$.)

✍ Answers: S8.2 (end of this lecture)

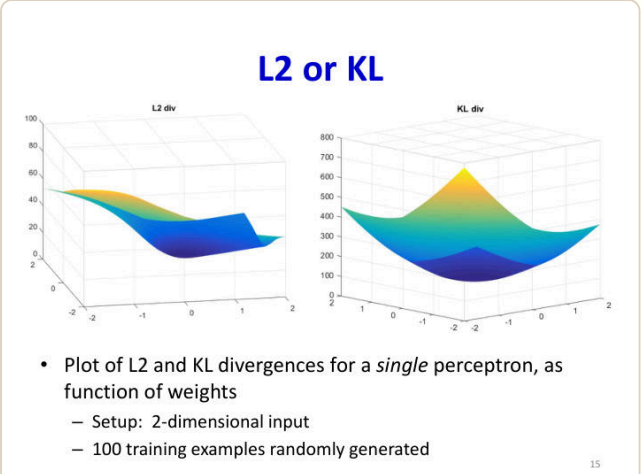
🧭 **Like a smoke alarm whose battery dies exactly when the fire is biggest.** L2 on a sigmoid beeps loudest at medium mistakes and goes almost silent at the worst ones; KL keeps getting louder. *Where it breaks:* L2 never goes fully silent, so the neuron does crawl back eventually, just very slowly.



L8 slide p13



L8 slide p14



L8 slide p15

↗ Understand more: guide, binary KL and its slope ↗ CMU L8 video (slides p8–19)

📖 READ & UNDERSTAND · ⏱ 8 min · Tier A

4 · Covariate shift, and batch normalisation as the fix

Training assumes every mini-batch comes from the same distribution (the same "cloud" of values). In practice each mini-batch can sit in a different place: a **covariate shift** (L8 p21–23). "Covariate" just means the input variables. The shift gets worse inside the network: two batches that are close at layer 1 can end up in entirely different regions by layer 5, because every layer is nonlinear. The weights keep chasing a moving target.

The fix (L8 p24–30): **move every mini-batch to a standard place**, mean 0 and standard deviation 1, then move it to whatever location the network finds most useful. This is **batch normalisation (BN)**.

Where it sits (L8 p33–36): after a neuron's weighted sum $z = \sum_j w_j i_j + b$, **before** the activation f. It is done **separately for each neuron**, over the B values that neuron produced for the B examples of the mini-batch. Four lines:

$$\mu_B = \frac{1}{B}\sum_{i=1}^{B} z_i \qquad \text{(mini-batch mean)}$$

$$\sigma_B^2 = \frac{1}{B}\sum_{i=1}^{B}(z_i - \mu_B)^2 \qquad \text{(mini-batch variance)}$$

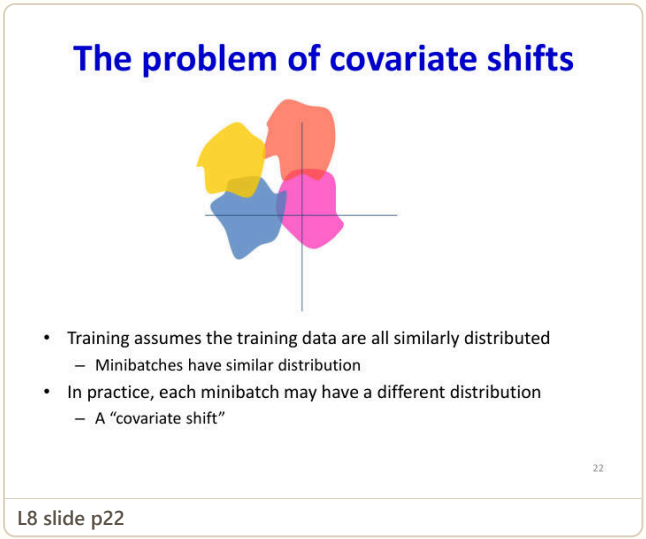
$$u_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad \text{(normalise: mean 0, variance 1)}$$

$$\hat{z}_i = \gamma\, u_i + \beta \qquad \text{(shift to the neuron's own location)}, \qquad y_i = f(\hat{z}_i)$$

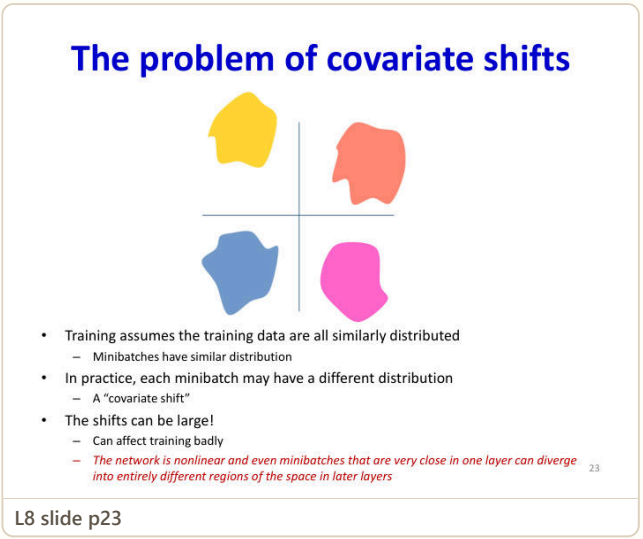
ϵ is a tiny constant so we never divide by zero. γ **(scale) and** β **(shift) are learned parameters**, one pair per neuron, trained by backprop like weights (L8 p45). They let the network undo the normalisation if that is what works best: with $\gamma = \sigma_B$ and $\beta = \mu_B$ you get the original z back.

E.g. a batch of two, $z = (1, 3)$: $\mu_B = 2$, $\sigma_B^2 = 1$, so $u = (-1, 1)$; with $\gamma = 3$, $\beta = 10$ the outputs are $\hat{z} = (7, 13)$.

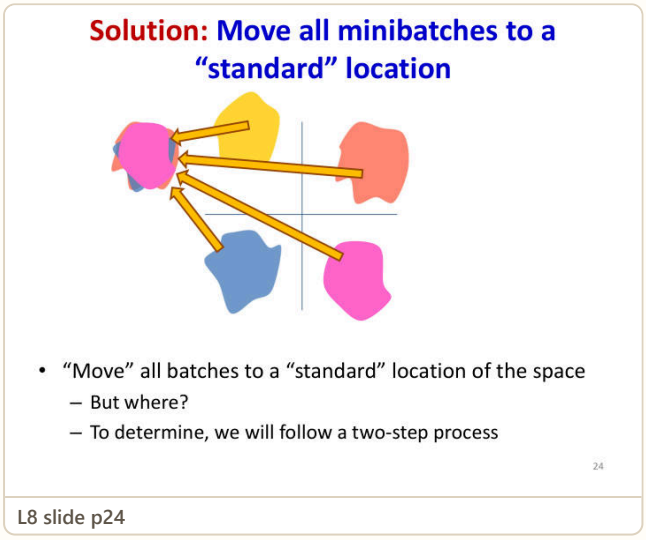
🧭 **Like grading on a curve, then converting to the school's scale.** Each class (mini-batch) had a different exam difficulty, so you first turn raw marks into "how many standard deviations above the class average" (u), then map that onto the scale the school wants ($\gamma u + \beta$). *Where it breaks:* your converted mark now depends on who else sat in your class; in BN the output for one example likewise depends on the other examples in its mini-batch (card 6).



L8 slide p22



L8 slide p23



L8 slide p24

↗ Understand more: d2l, batch normalisation ↗ Wikipedia, Batch normalization ↗ CMU L8 video (slides p21–36)

🖍 **PEN & PAPER · ⏱ 17 min · Tier A**

5 · Batch norm by hand: four numbers through one neuron

Step 1 · What you're being asked, in plain words

One neuron, a mini-batch of B examples: the neuron produces one weighted sum z_i per example. Batch norm (BN) does four things to those B numbers: **subtract their average, divide by their spread, multiply by γ, add β**. The average is the *mean* μ_B. The spread is the *standard deviation*, the square root of the *variance* σ_B^2 (the average squared distance from the mean). γ ("gamma") and β ("beta") are two numbers the network learns. Part (b) asks how the loss L changes when γ or β change: their gradients.

Step 2 · Warm-up by hand, no formulas

Two examples, $z = (1, 3)$, with $\gamma = 3$, $\beta = 10$.

- Average: (1 + 3)/2 = 2.
- Distances from it: −1 and +1. Squared: 1 and 1. Their average: 1. Square root: 1. So the spread is 1.
- Subtract the average, divide by the spread: (1 − 2)/1 = −1 and (3 − 2)/1 = +1.
- Times 3, plus 10: 7 and 13.

The batch went from "centred at 2, spread 1" to "centred at **10**, spread **3**": exactly β and γ.

Step 3 · The one idea

Normalising rewrites each z_i as "how many spreads above the batch average" (u_i); then γ and β choose the new spread and centre. For the gradients: $\hat{z}_i = \gamma u_i + \beta$ is a straight line in γ and β, and the same γ, β serve every example, so their gradients are **sums over the batch**.

Step 4 · Build it, step by step

1. $\mu_B = \frac{1}{B}\sum_i z_i$.
   Why: the batch average (warm-up: 2).
2. $\sigma_B^2 = \frac{1}{B}\sum_i (z_i - \mu_B)^2$, dividing by B, not $B - 1$.
   Why: average squared distance (warm-up: 1).
3. $u_i = (z_i - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, with ϵ a tiny constant against dividing by 0.
   Why: mean 0, spread 1 (warm-up: −1, +1).
4. $\hat{z}_i = \gamma u_i + \beta$.
   Why: the network's chosen spread and centre (warm-up: 7, 13).
5. $\frac{\partial \hat{z}_i}{\partial \beta} = 1$ and $\frac{\partial \hat{z}_i}{\partial \gamma} = u_i$.
   Why: slopes of the straight line in step 4.
6. Chain rule, then add over examples: $\frac{\partial L}{\partial \beta} = \sum_i \frac{\partial L}{\partial \hat{z}_i}$ and $\frac{\partial L}{\partial \gamma} = \sum_i \frac{\partial L}{\partial \hat{z}_i} u_i$.
   Why: γ, β are shared, so each example adds its own contribution. Warm-up with incoming gradients (0.5, −0.2): $\partial L/\partial \beta = 0.3$, $\partial L/\partial \gamma = 0.5(-1) + (-0.2)(1) = -0.7$.

Step 5 · Check it

Warm-up outputs (7, 13): mean 10 = β ✓, distances ±3 so spread 3 = γ ✓. The u's (−1, 1) have mean 0 and mean square 1 ✓. **Edge case:** if every z_i is equal, say (5, 5), the variance is 0. Without ϵ you would divide 0 by 0; with $\epsilon = 10^{-5}$ every $u_i = 0$ and every output is just β.

Step 6 · Now the exam problem — your turn

EXAM PROBLEM

One neuron sees a mini-batch of $B = 4$ examples and produces the weighted sums $z = (1,\ 2,\ 4,\ 5)$. Its BN parameters are $\gamma = 2$, $\beta = 1$. Take $\epsilon = 0$ (the real 10^{-5} changes nothing at four decimals).

(a) Compute μ_B, σ_B^2, the normalised u_i and the outputs $\hat{z}_i$.
(b) Backprop has delivered $\partial L/\partial \hat{z} = (0.1,\ -0.2,\ 0.3,\ 0.4)$ for the four examples (on p44 this is

$\frac{\partial L}{\partial y} f'(\hat{z})$, the usual step back through the activation). Find $\partial L/\partial\beta$ and $\partial L/\partial\gamma$.

Do it the same way: steps 4–5 with these numbers.

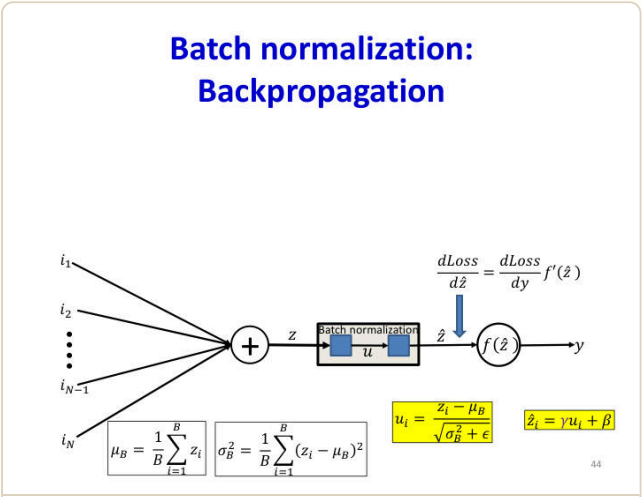
✍ Full worked solution: S8.3 (end of this lecture)

Step 7 · Try alone

1. $z = (0, 4)$, $\gamma = 1$, $\beta = 0$. Find μ_B, σ_B^2, u.
2. $z = (2, 2, 2, 6)$, $\gamma = 2$, $\beta = -1$. Find μ_B, σ_B^2, u and $\hat{z}$ (4 decimals).
3. Same batch as 2, incoming $\partial L/\partial\hat{z} = (1, 1, 1, 1)$. Find $\partial L/\partial\beta$ and $\partial L/\partial\gamma$. Why is the second one what it is?

✍ Answers: S8.4 (end of this lecture)

🧭 **Like a sound engineer's two knobs.** First the compressor brings every track to the same standard loudness (u); then the engineer sets this channel's own level (γ) and offset (β). *Where it breaks:* a compressor works on one track; BN's "standard loudness" is measured across the four examples together.



L8 slide p44

↗ Understand more: d2l, batch normalisation (training mode)

📖 READ & UNDERSTAND · ⏱ 5 min · Tier B

6 · Why BN makes the examples in a mini-batch depend on each other

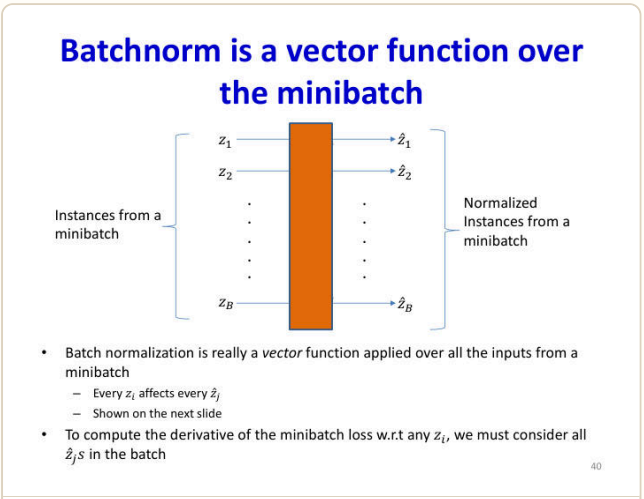
Usually (p37) the mini-batch loss is the average of the per-example divergences, $\frac{1}{B}\sum_t \text{Div}(Y_t, d_t)$, and each example's output depends only on that example. So the mini-batch gradient is just the average of the per-example gradients.

With BN (p38–40) every u_i uses μ_B and σ_B^2, and those are computed from *all* the z_j in the batch. Nudge z_1 and the mean and variance move, so u_1, u_2, u_3, u_4 all change. The training examples in a mini-batch are **no longer independent**: BN at a neuron is a **vector function over the whole mini-batch**, and every z_i affects every $\hat{z}_j$.

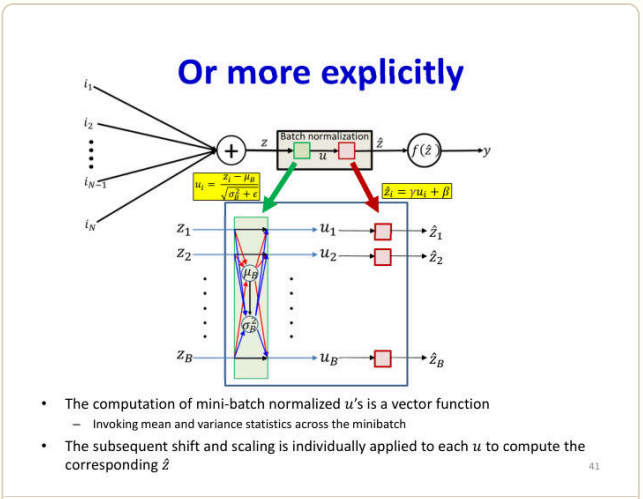
The two stages (p41–42). Stage 1, normalising $z \to u$, is the vector operation (it needs the batch statistics). Stage 2, $\hat{z}_i = \gamma u_i + \beta$, is applied to each example separately. That is why the γ and β

gradients of card 5 are simple sums. Pushing the derivative back through stage 1 is the long derivation of p47 onward (background, not on the exam).

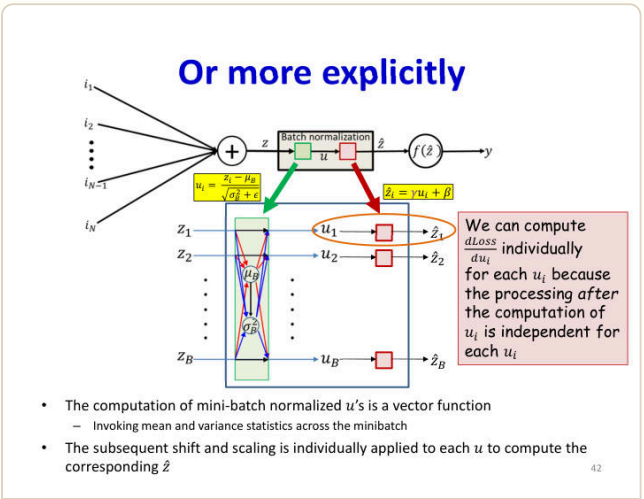
🧭 **Like grading on a curve.** If one student scores one more point, the class average and spread shift a little, which nudges everyone's curved grade, not just theirs. *Where it breaks:* with 4 students the nudge is large; with a batch of 256 it is tiny, but never zero.



L8 slide p40



L8 slide p41



L8 slide p42

↗ Understand more: Wikipedia, Batch normalization ↗ CMU L8 video (slides p37–46)

🔁 **RECALL DRILL · ⏱ 5 min · Tier A**

7 · Class polls and quick recall

Poll 1a (L8 p18). Which loss is convex with respect to the weights of the final softmax layer: KL or L2?

✍️ Answer: S8.5 (end of this lecture)

Poll 1b (L8 p18). For a regression net with linear output + L2, and a classifier with softmax + cross-entropy, the gradient w.r.t. the final affine value z is: the error $y - d$ / the ratio $1/y$ / different for the two networks?

✍️ Answer: S8.6 (end of this lecture)

Poll 2 (L8 p31). Batch norm accounts for covariate shift between: mini-batches / individual training instances / the entire training data?

✍ Answer: S8.7 (end of this lecture)

Quick recall

1. Where exactly does BN sit in a neuron?

   ✍ Answer: S8.8 (end of this lecture)

2. In training, BN's variance divides by ____ .

   ✍ Answer: S8.9 (end of this lecture)

3. Which BN quantities are learned, and how many per neuron?

   ✍ Answer: S8.10 (end of this lecture)

4. What γ, β make BN give back the original z?

   ✍ Answer: S8.11 (end of this lecture)

5. Momentum, RMSprop, Adam: which changes the direction, which the step size?

   ✍ Answer: S8.12 (end of this lecture)

6. Why do SGD and mini-batch need a shrinking learning rate?

   ✍ Answer: S8.13 (end of this lecture)

⚠ Traps

- BN normalises **one neuron across the examples of a mini-batch**, not across the neurons of a layer. And it sits **before** the activation.
- L8 p46 writes $\partial L/\partial \gamma = \frac{\partial L}{\partial z} u$ for one example; for the mini-batch you **add over all examples** (card 5), because γ and β are shared.
- L8 p13 says L2 "may appear more bowl-like" than KL. That's only as a function of the output y. As a function of z (and of the weights) **only KL is convex** (L8 p14, p16).
- **RMSprop does not smooth the gradient direction**; it only rescales each weight's step (L8 p6). Momentum smooths the direction. Adam does both.
- L8 p6 writes "Nestorov"; the name is **Nesterov**.

Solutions and answers — Lecture 8

S8.1 · 3 · By hand: L2 goes quiet when a sigmoid neuron is badly wrong, KL does not — Full worked solution

(a) Step 1, the sigmoid's slope.

$$\frac{dy}{dz} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = y(1-y)$$

Why: $\frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - y$. (The question allows you to quote this.)

L2, slope in y. $\text{Div} = \frac{1}{2}(y-d)^2 \Rightarrow \frac{d\,\text{L2}}{dy} = (y-d)$.

Why: the ½ cancels the 2 from the square.

L2, slope in z. Chain rule through the sigmoid:

$$\frac{d\,\text{L2}}{dz} = \frac{d\,\text{L2}}{dy} \cdot \frac{dy}{dz} = (y-d) \cdot y(1-y)$$

KL with d = 1. The $(1-d)$ term is multiplied by 0 and vanishes:

$$\text{Div} = -1 \cdot \log y - 0 \cdot \log(1-y) = -\log y \;\Rightarrow\; \frac{d\,\text{KL}}{dy} = -\frac{1}{y}$$

Why: the derivative of $\log y$ is $1/y$.

KL, slope in z. Chain rule again:

$$\frac{d\,\text{KL}}{dz} = -\frac{1}{y} \cdot y(1-y) = -(1-y) = y - 1 = y - d$$

Why it is so simple: the y in the denominator cancels the y in $y(1-y)$. (With $d = 0$ the other term survives and $(1-y)$ cancels instead, again giving $y - d$.) This is the "gradient is just the error" fact of L8 p17.

(b) Sigmoid values. $\sigma(-4) = 1/(1+e^4) = 0.0180$, $\sigma(0) = 1/2 = 0.5$, $\sigma(2) = 1/(1+e^{-2}) = 0.8808$.

Plug into both formulas (d = 1, so $y - d = y - 1$):

z	y	y − 1	y(1−y)	dL2/dz = (y − 1)·y(1−y)	dKL/dz = y − 1
−4 (badly wrong)	0.0180	−0.982	0.0177	−0.982 × 0.0177 = −0.0173	−0.982
0	0.5	−0.5	0.25	−0.5 × 0.25 = −0.125	−0.5
2 (nearly right)	0.8808	−0.1192	0.105	−0.1192 × 0.105 = −0.0125	−0.119

Compare. All slopes are negative, so both push z up, towards "yes" ✓. At $z = -4$ the ratio is $0.982/0.0173 \approx 57$: KL pushes about 57 times harder. Worse, L2's push at $z = -4$ (0.0173) is smaller than its own push at $z = 0$ (0.125): the neuron is at its worst and L2 barely moves it. That flat region is why L2 is **not convex in z**. KL's push grows towards 1 in size the more wrong the neuron is.

(a) dL2/dz = (y − d)·y(1 − y); dKL/dz = y − d. (b) L2: −0.0173, −0.125, −0.0125; KL: −0.982, −0.5, −0.119 at z = −4, 0, 2. KL pushes harder when badly wrong (~57× at z = −4).

↩ back to the card

S8.2 · 3 · By hand: L2 goes quiet when a sigmoid neuron is badly wrong, KL does not — Answers

1. $y(1-y) = 0.1050$. L2: $(0.1192 - 1)(0.1050) = -0.0925$. KL: $0.1192 - 1 = -0.881$.
2. KL: $y - d = 0.9933$. L2: $y^2(1-y) = 0.9933^2 \times 0.0067 = 0.0066$. KL pushes about 150× harder.
3. Derivative of $y(1-y)^2$ is $(1-y)(1-3y)$, zero at $y = 1/3$, i.e. $z = \ln(1/2) = -0.693$. The push there is $4/27 = 0.148$; for any more-wrong z L2 pushes *less*.

↩ back to the card

S8.3 · 5 · Batch norm by hand: four numbers through one neuron — Full worked solution

(a) Step 1, mean. $\mu_B = (1+2+4+5)/4 = 12/4 = 3$.
Why: the batch's centre, which BN will move to 0.

Step 2, variance.
Why: the average squared distance from the mean measures the spread.

i	z_i	$z_i - \mu_B$	$(z_i - \mu_B)^2$
1	1	−2	4
2	2	−1	1
3	4	1	1
4	5	2	4

$$\sigma_B^2 = \frac{4+1+1+4}{4} = \frac{10}{4} = 2.5, \qquad \sqrt{\sigma_B^2} = 1.5811$$

Divide by $B = 4$, not $B - 1 = 3$: BN in training uses the plain batch average.

Step 3, normalise. $u_i = (z_i - 3)/1.5811$.
Why: "how many spreads above the batch average".

$$u = \left(\tfrac{-2}{1.5811},\ \tfrac{-1}{1.5811},\ \tfrac{1}{1.5811},\ \tfrac{2}{1.5811}\right) = (-1.2649,\ -0.6325,\ 0.6325,\ 1.2649)$$

Check: they sum to 0, and their squares $(1.6, 0.4, 0.4, 1.6)$ average to 1 ✓.

Step 4, scale and shift. $\hat{z}_i = \gamma u_i + \beta = 2u_i + 1$.
Why: the network's own chosen spread (2) and centre (1).

$$\hat{z} = (2(-1.2649)+1,\ 2(-0.6325)+1,\ 2(0.6325)+1,\ 2(1.2649)+1) = (-1.5298,\ -0.2649,\ 2.2649,\ 3.5298)$$

Check: their mean is $\beta = 1$ and their standard deviation is $\gamma = 2$ ✓. That is all γ and β do: choose the new centre and spread.

(b) Step 5, local slopes (L8 p46). Since $\hat{z}_i = \gamma u_i + \beta$ is a straight line in γ and β: $\partial \hat{z}_i/\partial\beta = 1$ and $\partial \hat{z}_i/\partial\gamma = u_i$.

Step 6, chain rule and sum. *Why sum:* both parameters are shared by all four examples, so each example adds its own contribution:

$$\frac{\partial L}{\partial \beta} = \sum_i \frac{\partial L}{\partial \hat{z}_i} = 0.1 - 0.2 + 0.3 + 0.4 = 0.6$$

$$\frac{\partial L}{\partial \gamma} = \sum_i \frac{\partial L}{\partial \hat{z}_i} u_i = (0.1)(-1.2649) + (-0.2)(-0.6325) + (0.3)(0.6325) + (0.4)(1.2649)$$

$$= -0.1265 + 0.1265 + 0.1897 + 0.5060 = 0.6957$$

Then one gradient-descent step, e.g. with η = 0.1: $\beta \leftarrow 1 - 0.1(0.6) = 0.94$ and $\gamma \leftarrow 2 - 0.1(0.6957) = 1.930$. They are trained exactly like weights.

$\mu_B = 3,\ \sigma_B^2 = 2.5$; $u = (-1.2649, -0.6325, 0.6325, 1.2649)$; $\hat{z} = (-1.5298, -0.2649, 2.2649, 3.5298)$; $\partial L/\partial\beta = 0.6$, $\partial L/\partial\gamma \approx 0.696$.

↩ back to the card

S8.4 · 5 · Batch norm by hand: four numbers through one neuron — Answers

1. $\mu_B = 2$, deviations ±2, $\sigma_B^2 = 4$, spread 2, $u = (-1, 1)$.
2. $\mu_B = 3$; deviations (−1, −1, −1, 3), squares sum to 12, $\sigma_B^2 = 3$, spread 1.7321. $u = (-0.5774, -0.5774, -0.5774, 1.7321)$; $\hat{z} = (-2.1547, -2.1547, -2.1547, 2.4641)$.
3. $\partial L/\partial\beta = 4$. $\partial L/\partial\gamma = \sum u_i = 0$, because the u's always sum to 0: a push that is the same for every example can move the centre (β) but not the spread (γ).

↩ back to the card

S8.5 · 7 · Class polls and quick recall — Answer

KL. L2 on a sigmoid/softmax output flattens for large $|z|$ and is not convex (L8 p14–16, card 3).

↩ back to the card

S8.6 · 7 · Class polls and quick recall — Answer

The error $y - d$, for both (L8 p17).

↩ back to the card

S8.7 · 7 · Class polls and quick recall — Answer

Mini-batches. Each mini-batch is moved to mean 0, variance 1 (L8 p25–30).

↩ back to the card

S8.8 · 7 · Class polls and quick recall — Answer

After the weighted sum z, before the activation f.

↩ back to the card

S8.9 · 7 · Class polls and quick recall — Answer

B, the mini-batch size (not $B - 1$).

↩ back to the card

S8.10 · 7 · Class polls and quick recall — Answer

γ and β, one pair per neuron. μ_B and σ_B^2 are computed from the batch, not learned.

↩ back to the card

S8.11 · 7 · Class polls and quick recall — Answer

$\gamma = \sqrt{\sigma_B^2 + \epsilon}$ ($\approx \sigma_B$) and $\beta = \mu_B$.

↩ back to the card

S8.12 · 7 · Class polls and quick recall — Answer

Momentum (and Nesterov): direction. RMSprop: step size only. Adam: both (L8 p6).

↩ back to the card

S8.13 · 7 · Class polls and quick recall — Answer

Their steps are noisy (higher variance), so with a fixed η the weights keep jittering around the minimum; $\sum \eta_k = \infty, \sum \eta_k^2 < \infty$ (L8 p5).

↩ back to the card

The formula sheet — write it from memory at 11:50

Official scope only. Cover the right column, say each formula out loud, then check. Anything you miss gets a star; redo the starred ones at the end of the session.

Lectures 1–2 · units, gates, what networks can represent	
Threshold unit (perceptron)	$y = 1$ iff $\sum_i w_i x_i \geq T \;\Leftrightarrow\; z = \sum_i w_i x_i + b \geq 0$ with $b = -T$
Universal gates on N literals, L of them plain (weights $+1$ plain, -1 negated)	AND: $T = L$ · OR: $T = L - N + 1$ · at least K true: $T = L - N + K$
XOR	not one unit (not linearly separable); 3 units in 2 layers, or 2 units with a skip connection
Any Boolean function	one hidden layer (one unit per DNF term); worst case (parity/checkerboard) 2^{N-1} hidden units
Parity, built deep	$3(N-1)$ units in $2\log_2 N$ layers — depth buys exponential savings
Convex polygon, N sides	N hidden half-plane units, output fires iff $\sum \geq N$
Lecture 3 · learning	
Empirical risk (the training loss)	$\hat{R}(W) = \frac{1}{N}\sum_{i=1}^{N} \operatorname{div}\big(f(X_i; W), d_i\big),\ \hat{W} = \arg\min_W \hat{R}(W)$
Perceptron rule (labels ± 1)	if $y_i\, W^\top X_i \leq 0$: $W \leftarrow W + y_i X_i$; converges in at most $(R/\gamma)^2$ updates, $R = \max\|X_i\|$, γ = margin
Least squares through the origin	$w^* = \dfrac{\sum_i x_i d_i}{\sum_i x_i^2}$
ADALINE (delta rule; linear $z = W^\top X$)	$W \leftarrow W + \eta\,(d - z)\,X$
Generalised delta rule ($y = f(z)$)	$W \leftarrow W + \eta\,(d - y)\,f'(z)\,X$
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}},\ \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$
Lecture 4 (p1–47) · derivatives and gradient descent	
Gradient	$\nabla f = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]$ — points uphill, steepest; perpendicular to the level curve
Critical point test	$\nabla f = 0$; Hessian $\mathbf{H}$ (second derivatives): all eigenvalues > 0 minimum · all < 0 maximum · mixed signs saddle
Gradient descent	$x^{k+1} = x^k - \eta\,\nabla f(x^k)^\top$, stop when $\lvert f(x^{k+1}) - f(x^k)\rvert$ is tiny
Lecture 5 = Stanford CS231n L4 · backprop on a computational graph	
Chain rule at one node	downstream gradient = local gradient $\times$ upstream gradient: $\dfrac{\partial L}{\partial x} = \dfrac{\partial z}{\partial x}\,\dfrac{\partial L}{\partial z}$
Add gate $z = x + y$	gradient distributor: both inputs get the upstream gradient unchanged
Multiply gate $z = xy$	"swap multiplier": $\partial L/\partial x = y\cdot$ upstream, $\partial L/\partial y = x\cdot$ upstream

Max gate $z = \max(x, y)$	gradient router: the larger input gets the whole upstream gradient, the other gets 0 (background — after slide 91)
Copy gate (one value used twice)	gradient adder: the gradients flowing back from both uses add up (background — after slide 91)
Sigmoid gate	local gradient $\sigma(x)\big(1 - \sigma(x)\big)$ — collapse a whole sub-graph into one gate
Lecture 6 (p36–45, p81–end) · convergence and optimisers	
Quadratic (background)	$E = \frac{1}{2}aw^2 + bw + c$; best fixed step $1/a$ = 1 / second derivative
Scaling the axes	$W \leftarrow W - \eta\, \mathbf{A}^{-1} \nabla_W E^\top$ for $E = \frac{1}{2} W^\top \mathbf{A} W + \mathbf{b}^\top W + c$
Newton's method	$W \leftarrow W - \mathbf{H}^{-1} \nabla_W E^\top$ (exact in one step for a quadratic; the Hessian is expensive and can be non-invertible / not positive definite)
Learning-rate decay	start large, shrink with iterations
RProp (each weight on its own)	same sign of derivative as last time: step $\times\, \alpha$ (1.2); sign flipped: undo, step $\times\, \beta$ (0.5)
QuickProp	Newton with the second derivative estimated from two slopes: $w^{(k+1)} = w^{(k)} - \Delta w^{(k)}$, $\Delta w^{(k)} = \Delta w^{(k-1)} \dfrac{E'(w^{(k)})}{E'(w^{(k)}) - E'(w^{(k-1)})}$
Momentum	$\Delta W^{(k)} = \beta\, \Delta W^{(k-1)} - \eta\, \nabla L\big(W^{(k-1)}\big)^\top$, $W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$, $\beta \approx 0.9$
Nesterov	first jump by $\beta\, \Delta W^{(k-1)}$, then take the gradient *there*, add the two
Lecture 8 (p1–46) · divergences and batch normalisation	
L2 (regression)	$\text{Div} = \frac{1}{2} \lVert \mathbf{y} - \mathbf{d} \rVert^2$
KL (classification)	binary: $-d \log y - (1 - d) \log(1 - y)$ · one-hot: $-\log y_{\text{true class}}$
The "error" gradient	linear output + L2, or softmax output + KL: $\nabla_{\mathbf{z}} \text{Div} = (\mathbf{y} - \mathbf{d})^\top$
Batch norm (per unit, over the mini-batch)	$\mu_B = \frac{1}{B} \sum z_i$, $\sigma_B^2 = \frac{1}{B} \sum (z_i - \mu_B)^2$, $u_i = \dfrac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, $\hat{z}_i = \gamma u_i + \beta$ (γ, β learned)